

Contents

1	Template	4
1.1	Main	4
2	Data Structures	5
2.1	Centroid decomposition	5
2.1.1	Centroid decomposition	5
2.2	DSU	7
2.2.1	DSU(Hassan)	7
2.2.2	DSU	7
2.2.3	DSUOnTree	8
2.2.4	DSU move erase	10
2.2.5	DSU with rollback	10
2.2.6	KRT	11
2.2.7	Offline Dynamic connectivity	14
2.2.8	Partially persistent DSU	15
2.2.9	Small to large	17
2.3	Fenwick Tree	17
2.3.1	2D bit	17
2.3.2	2D bit rangeUpdate rangeQuery	19
2.3.3	Bit rangeUpdate rangeQuery	20
2.3.4	Fenwick tree	22
2.3.5	Lower bound on bit	22
2.4	HLD	24
2.4.1	HLD	24
2.5	Misc	26
2.5.1	CDQ	26
2.5.2	Leftist heap	27
2.5.3	Monotonic queue	28
2.5.4	Monotonic stack	28
2.5.5	ODT	28
2.6	More Data structures	29
2.6.1	Wavelet tree	29
2.7	SQRT decomposition MO	30
2.7.1	MO	30
2.7.2	MO on tree	31
2.7.3	Mo's with updates	33
2.7.4	Rollback mo	36
2.7.5	sqrt decomposition	38
2.7.6	sqrt decomposition rangeUpdate rangeQuery	39
2.8	Segment Tree	40
2.8.1	2D segment tree	40
2.8.2	Dynamic segment tree	42
2.8.3	Iterative segment tree	43
2.8.4	Merge sort tree	45
2.8.5	PST	46
2.8.6	PST Lazy	47
2.8.7	Segment tree	49
2.8.8	Segment tree beats	50
2.8.9	Segment tree lazy	53
2.8.10	Segment tree merging	54
2.9	Sparse Table	56
2.9.1	2D sparse table	56
2.9.2	sparse table	58
2.10	Treap	59
2.10.1	implicit treap	59
2.11	Trie	60
2.11.1	BinaryTrie Using Vector	60
2.11.2	Binary trie	61
2.11.3	Persistent Trie	62
2.11.4	Trie (String)	65
2.11.5	Trie Using Vector	65
2.11.6	Trie	66
3	Graphs	68
3.1	2-Sat	68
3.1.1	2sat	68
3.2	DAG	69
3.2.1	Dag reachability(bitset trick)	69
3.3	Eulerian path	70
3.3.1	Eulerian path	70
3.4	Flows	70
3.4.1	Gomory-Hu Tree	70
3.4.2	MCMF	72
3.4.3	Max flow	73
3.5	Graph Traverse	74
3.5.1	Bipartite Graph	74
3.5.2	Euler tour	74
3.5.3	Top sort	75
3.6	Graph connectivity	75
3.6.1	Articulations	75
3.6.2	SCC	76
3.7	Graph counting	76
3.7.1	BEST	76
3.7.2	Prufer code	77
3.8	LCA	78
3.8.1	LCA(Hassan)	78

3.8.2	LCA	79	4.5.3	Matrix expo Queries	110
3.9	MST	80	4.5.4	Min Path UsingMatrix expo	111
3.9.1	Directed MST	80	4.5.5	Permanent matrix	111
3.9.2	Kirchoff	81	4.6	Optimization	112
3.9.3	Kruskal	82	4.6.1	Simplex	112
3.9.4	Prim	83	4.7	Polynomial	114
3.10	Matching	83	4.7.1	EXP POW SQRT	114
3.10.1	Bipartite matching	83	4.7.2	Lagrange	114
3.10.2	Hungarian algorithm	85	4.7.3	Multipoint evaluation	115
3.10.3	Stable Marriage Problem	87	4.7.4	Poly inv log	116
3.11	Misc	89	4.8	Probability	117
3.11.1	Check Cycle	89	4.8.1	Expected occurence time of substring	117
3.11.2	Degree Sequences	89			
3.11.3	Tree hashing	91	5	Number Theory	118
3.12	Shortest paths	93	5.1	Fibonacci	118
3.12.1	BFS01	93	5.1.1	Fast doubling fibonacci	118
3.12.2	Bellman ford	93	5.1.2	PHI field	118
3.12.3	Dijkstra	93	5.2	Linear diophantine equation	119
3.12.4	Floyd	95	5.2.1	Floor sum	119
3.12.5	Floyd Warshall(Hassan)	96	5.2.2	Generalized floor sum	119
3.12.6	MaxMin Cost Using Dijkstra	97	5.2.3	LDE	120
3.12.7	MinMax Cost Using Dijkstra	97	5.2.4	Pell equation	121
3.13	Trees	98	5.3	Misc	122
3.13.1	Rerooting	98	5.3.1	Min X in Mod range	122
3.13.2	Virtual tree	98	5.3.2	SIMPLEST FRACTION	122
4	Math	100	5.4	Modular arithmetic	122
4.1	Calculus	100	5.4.1	Baby step giant step	122
4.1.1	Differentiation integration	100	5.4.2	Big Mod Power	123
4.2	Convolution	101	5.4.3	CRT	123
4.2.1	FWHT	101	5.4.4	Discrete root	124
4.2.2	GDC LCM Convolution	102	5.4.5	Distinct kth power	124
4.2.3	NTT	103	5.4.6	Linear congruence	125
4.3	Linear algebra	104	5.4.7	MULTIPLICATIVE ORDER CARMICHAEL'S LAMBDA	125
4.3.1	Determinant(composite mod)	104	5.4.8	discrete sqrt	125
4.3.2	Determinant(prime mod)	104	5.5	Multiplicative functions	126
4.3.3	Gaussian elimination	105	5.5.1	Min 25 sieve	126
4.3.4	Gaussian elimination mod2	105	5.5.2	Power tower	126
4.3.5	XOR basis	106	5.5.3	phi	127
4.4	Linear recurrence	107	5.6	Primes And Divisors	128
4.4.1	Berlekamp	107	5.6.1	Basic	128
4.5	Matrix	108	5.6.2	FastSumitionDivisor 1e16	129
4.5.1	Frievald's algorithm	108	5.6.3	Linear sieve	130
4.5.2	Matrix expo	109	5.6.4	Miller pollard	130
			5.6.5	Number of divisors 1e18	131

5.6.6	Prime counting	133	8.6.2	Palindromic tree	161
5.6.7	Sieve 1e9	133	8.7	Suffix Structures	162
5.6.8	segmented sieve	134	8.7.1	Suffix array	162
5.6.9	sum numberOfDivisors	135	8.7.2	Suffix automaton	165
6	Combinatorics	136	9	DP	168
6.1	Main	136	9.1	DP Bitmask	168
6.1.1	Basic	136	9.1.1	Bitmask	168
6.1.2	Pascal triangle	138	9.1.2	SOS	169
6.2	PIE	138	9.1.3	Submask	169
6.2.1	multiples	138	9.1.4	subset convolution	169
6.3	Partition	139	9.2	Digit Dp	170
6.3.1	Integer partition	139	9.2.1	main	170
6.4	nCr	139	9.3	Iterative	170
6.4.1	nCr Mod any mod	139	9.3.1	main	170
6.4.2	nCr over congruence class	141	9.4	Knapsack	171
7	Geometry	141	9.4.1	main	171
7.1	Ball 3D Class	141	9.5	LCS	171
7.2	allGeometry	144	9.5.1	LCS	171
7.3	geometryReferenceSheet	150	9.6	LIS	171
8	String	151	9.6.1	Basic 2D	171
8.1	Aho Corasick	151	9.6.2	LIS1D	171
8.1.1	Aho corasick	151	9.6.3	LIS BS	172
8.2	Hashing	152	9.6.4	LIS seg	172
8.2.1	2D string hashing	152	9.7	Lichao tree	173
8.2.2	Hash(Hassan)	153	9.7.1	lichao tree	173
8.2.3	Hash	154	9.8	Misc	174
8.2.4	MultisetHashing	155	9.8.1	Broken profile	174
8.2.5	XOR hashing	155	9.9	More Tricks	175
8.3	LCS	155	9.9.1	Slope trick	175
8.3.1	Bitset LCS	155	9.10	Optimization	176
8.4	Matching	156	9.10.1	1D1D	176
8.4.1	FFT wildcard matching	156	9.10.2	Aliens	176
8.4.2	KMP	157	9.10.3	DC	177
8.4.3	String matching using bitset	157	9.10.4	Knuth	178
8.4.4	Z	158	9.10.5	Using data structures	178
8.5	Misc	158	10	Misc	180
8.5.1	Debruijn sequence	158	10.1	Big Integer	180
8.5.2	Expression parser	159	10.1.1	Big integers	180
8.5.3	Min cyclic shift	160	10.2	Chess	181
8.6	Palindromes	161	10.2.1	Knight's tour	181
8.6.1	Manacher	161	10.3	Matroids	182
			10.3.1	Matroid intersection	182

10.4	Search	183
10.4.1	Ternary search	183
10.4.2	fraction binary search	184
10.4.3	parallel binary search	184
10.5	Techniques	185
10.5.1	MANHATTAN CHEBYSHEV	185
10.5.2	Meet in the middle	185
10.5.3	Sparse table lsa trick	186
10.6	min plus convolution.cpp	186
10.6.1	shawk	186
10.7	misc	187
10.7.1	Dynamic bitset	187
10.7.2	LGV lemma	188

11 Helpers

11.1	Adhocs	189
11.2	BitMask&Complete Search	190
11.3	Bitmasks(AMR)	191
11.4	Compression	192
11.5	Hashing segment tree	193
11.6	Hashing treap	194
11.7	INT 128	195
11.8	MEX	196
11.9	Ordered set	196
11.10	Power	196
11.11	Query	196
11.12	Solve Equations	196
11.13	Split mix	196
11.14	debug	197
11.15	dist	198
11.16	div	198
11.17	double compare	198
11.18	edge	198
11.19	find last occurrence	199
11.20	inv mod	199
11.21	is prime	199
11.22	kadanes	199
11.23	nCr	200
11.24	next greater	200
11.25	permutation hashing	200
11.26	random	201
11.27	valid	201

1 Template

1.1 Main

```

1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4
5 using namespace std;
6 using namespace __gnu_pbds;
7
8 #define El_Hefnawys \
9     ios_base::sync_with_stdio(false); \
10    cin.tie(nullptr); \
11    cout.tie(nullptr) \
12
13 typedef long long ll;
14 typedef unsigned long long ull;
15
16 #define int ll
17
18 typedef vector<vector<int>> GRAPH;
19
20 const int MOD = 1e9 + 7;
21 const int OO = 1e15;
22 const double PI = acos(-1.0);
23 const double EPS = 1e-9;
24
25 int dx4[4] = {0, 1, 0, -1};
26 int dy4[4] = {1, 0, -1, 0};
27
28 int dx8[8] = {0, 0, 1, -1, 1, 1, -1, -1};
29 int dy8[8] = {1, -1, 0, 0, 1, -1, 1, -1};
30
31 void solve()
32 {
33
34
35
36 }
37
38 signed main()
39 {
40     El_Hefnawys;
41     int t{1};
42     // cin >> t;
43     while (t--)
44     {
45         solve();
46     }
47     return 0;
48 }

```

2 Data Structures

2.1 Centroid decomposition

2.1.1 Centroid decomposition

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: CENTROID DECOMPOSITION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Paths of length K", "Distance between all pairs", "Update node
6  *   color, find closest red node".
7  * - Classic Scenarios: You need to answer heavy path queries on a massive tree
8  *   where standard DFS would Time Limit Exceed (TLE).
9  * - The Magic: We find the "center of gravity" (Centroid) of the tree, make it
10  *   root, and "remove" it.
11  *   This shatters the graph into a forest. We repeat this recursively. The
12  *   resulting "Centroid Tree"
13  *   connects these centers and mathematically guarantees a maximum depth of  $O(\log N)$ .
14  *   You can safely brute-force climbing from any node to the root of the
15  *   Centroid Tree in just ~17 steps!
16  *
17  * 2. HOW TO USE IT (THE BLACK BOX)
18  * - Initialization: Pass the size of the tree (1-indexed) and the adjacency list
19  *   to CentroidDecomposition cd(N, adj);
20  *
21  * - Navigation: The `cd.centroid_parent[u]` array tells you the parent of node
22  *   u in the new tree.
23  * - Distance: Use `cd.get_dist(u, v)` to find the distance between nodes in the
24  *   ORIGINAL tree.
25  *
26  * - Complexity:
27  *   Time:  $O(N \log N)$  to build the tree and the LCA tables.
28  *   Space:  $O(N \log N)$  memory for binary lifting tables.
29  *
30  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
31  * - How to solve a problem with it:
32  *   Instead of answering queries globally, answer them for "all paths passing
33  *   through the current centroid".
34  *   For point updates (e.g., painting a node red), start at the updated node and
35  *   climb up its
36  *   `centroid_parent` path, updating the answer at each ancestor using `get_dist
37  *   (u, ancestor)`.
38  */
39
40 #include <bits/stdc++.h>
41 using namespace std;
42
43 struct CentroidDecomposition
44 {
45     int n;
46     vector<vector<int>>> adj;
47
48     // Variables for the Centroid Tree
49     vector<int> sz;
50     vector<bool> removed;
51     vector<int> centroid_parent;
52
53     // Variables for  $O(1)$  distance queries in the original tree (Binary Lifting)
54     const int LOG = 20;
55     vector<vector<int>>> up;
56     vector<int> depth;
57
58     CentroidDecomposition(int nodes, const vector<vector<int>>> &graph)
59     {
60         n = nodes;
61         adj = graph;
62
63         sz.assign(n + 1, 0);
64         removed.assign(n + 1, false);
65         centroid_parent.assign(n + 1, -1);
66
67         up.assign(n + 1, vector<int>(LOG, 0));
68         depth.assign(n + 1, 0);
69
70         // 1. Precompute LCA for fast distance queries
71         lca_dfs(1, 1);
72
73         // 2. Build the Centroid Tree
74         decompose(1, -1);
75     }
76
77     // Standard Binary Lifting DFS to compute depths and ancestors
78     void lca_dfs(int u, int p)
79     {
80         up[u][0] = p;
81         for (int i = 1; i < LOG; i++)
82         {
83             up[u][i] = up[up[u][i - 1]][i - 1];
84         }
85         for (int v : adj[u])
86         {
87             if (v != p)
88             {
89                 depth[v] = depth[u] + 1;
90                 lca_dfs(v, u);
91             }
92         }
93     }
94
95     int get_lca(int u, int v)
96     {
97         if (depth[u] < depth[v])
98             swap(u, v);
99         int diff = depth[u] - depth[v];
100         for (int i = 0; i < LOG; i++)
```

```

92     {
93         if ((diff >> i) & 1)
94             u = up[u][i];
95     }
96     if (u == v)
97         return u;
98     for (int i = LOG - 1; i >= 0; i--)
99     {
100         if (up[u][i] != up[v][i])
101         {
102             u = up[u][i];
103             v = up[v][i];
104         }
105     }
106     return up[u][0];
107 }
108
109 // Get the shortest path distance between u and v in the ORIGINAL tree
110 int get_dist(int u, int v)
111 {
112     return depth[u] + depth[v] - 2 * depth[get_lca(u, v)];
113 }
114
115 // Your exact dfs function, slightly modified to ignore "removed" nodes!
116 void get_sz(int u, int p)
117 {
118     sz[u] = 1;
119     for (int v : adj[u])
120     {
121         if (v == p || removed[v])
122             continue; // Ignore dead ends
123         get_sz(v, u);
124         sz[u] += sz[v];
125     }
126 }
127
128 // Your exact centroid function, slightly modified to ignore "removed" nodes!
129 int get_centroid(int u, int p, int total_nodes)
130 {
131     for (int v : adj[u])
132     {
133         if (v == p || removed[v])
134             continue; // Ignore dead ends
135
136         if (sz[v] > total_nodes / 2)
137         {
138             return get_centroid(v, u, total_nodes);
139         }
140     }
141     return u;
142 }
143
144 // The master function that shatters the graph and builds the new tree
145 void decompose(int u, int p)
146 {

```

```

147     // 1. Calculate the subtree sizes of the current shattered component
148     get_sz(u, -1);
149     int total_nodes = sz[u];
150
151     // 2. Find the exact center of gravity (Centroid) for this component
152     int c = get_centroid(u, -1, total_nodes);
153
154     // 3. Link it to the Centroid Tree!
155     centroid_parent[c] = p;
156
157     // 4. "Delete" this centroid to shatter the tree into smaller forests
158     removed[c] = true;
159
160     // 5. Recursively find the centroids of the remaining pieces
161     for (int v : adj[c])
162     {
163         if (!removed[v])
164         {
165             decompose(v, c);
166         }
167     }
168 }
169 };
170
171 void solve()
172 {
173     int n, q;
174     // Read N nodes and Q queries
175     if (!(cin >> n >> q))
176         return;
177
178     vector<vector<int>>> adj(n + 1);
179     for (int i = 0; i < n - 1; i++)
180     {
181         int u, v;
182         cin >> u >> v;
183         adj[u].push_back(v);
184         adj[v].push_back(u);
185     }
186
187     // Build the magical O(log N) depth tree
188     CentroidDecomposition cd(n, adj);
189
190     // EXAMPLE QUERY: "Update node U's color, then find closest node with that
191     // color"
192     // To solve this, you would start at U and literally climb the centroid tree!
193     /*
194     int u = 5;
195     int curr = u;
196     while (curr != -1) {
197         int distance_in_original_tree = cd.get_dist(u, curr);
198         // update_answer(curr, distance_in_original_tree);
199         curr = cd.centroid_parent[curr];
200     }
201     */

```

```

201 }
202
203 int main()
204 {
205     // Fast I/O
206     ios_base::sync_with_stdio(false);
207     cin.tie(NULL);
208     solve();
209     return 0;
210 }

```

2.2 DSU

2.2.1 DSU(Hassan)

```

1 class DSU {
2 public:
3     vector<int>parent,sizes,mn,mx;
4     DSU(int n) {
5         parent.resize(n+1);
6         sizes.resize(n+1);
7         mx.resize(n+1);
8         mn.resize(n+1);
9         for (int i = 1; i <= n; i++)
10             parent[i] = mn[i] = mx[i] = i,sizes[i] = 1 ;
11     }
12
13     int findRoot(int u ) {
14         return parent[u] == u ? u : parent[u] = findRoot(parent[u]);
15     }
16
17     bool merge(int u, int v) {
18         int root_u = findRoot(u), root_v = findRoot(v);
19         if (root_u == root_v)
20             return false;
21
22         if (sizes[root_u] > sizes[root_v])
23             swap(root_u, root_v);
24
25         mx[root_v] = max(mx[root_v], mx[root_u]);
26         mn[root_v] = min(mn[root_v], mn[root_u]);
27         parent[root_u] = root_v;
28         sizes[root_v] += sizes[root_u];
29         return true;
30     }
31 };
32
33 void Hassan() {
34     int n,q;cin >>n>>q;
35     DSU d(n);
36     while (q--) {
37         string op;int u,v;cin >> op ;
38         if (op == "get") {

```

```

39             cin >>v;
40             int r = d.findRoot(v);
41             cout << d.mn[r] <<"_" << d.mx[r] <<"_"<< d.sizes[r] <<endl;
42         }else {
43             cin >>u>>v;
44             d.merge(u,v);
45         }
46     }
47 }

```

2.2.2 DSU

```

1 class DSU
2 {
3 private:
4     vector<int> parent, size;
5     int forests{};
6
7     void link(int x, int y)
8     {
9         if (size[x] > size[y])
10             swap(x, y);
11         parent[x] = y;
12         size[y] += size[x];
13     }
14
15 public:
16     DSU(int n)
17     {
18         parent = vector<int>(n), size = vector<int>(n);
19         forests = n;
20         for (int i = 0; i < n; i++)
21             parent[i] = i, size[i] = 1;
22     }
23
24     int find_set(int x)
25     {
26         if (x == parent[x])
27             return x;
28         return parent[x] = find_set(parent[x]);
29     }
30
31     int union_sets(int x, int y)
32     {
33         x = find_set(x), y = find_set(y);
34         if (x != y)
35         {
36             link(x, y);
37             forests--;
38         }
39         return x != y;
40     }
41 }

```

```

42 bool same_set(int x, int y)
43 {
44     x = find_set(x), y = find_set(y);
45     return x == y;
46 }
47
48 int get_size(int x)
49 {
50     return size[find_set(x)];
51 }
52
53 int get_forests() const
54 {
55     return forests;
56 }
57 };

```

2.2.3 DSUOnTree

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DSU ON TREE (SACK / SMALL TO LARGE)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Subtree queries", "Offline queries", "Distinct colors in subtree",
6  *   "Frequencies at depth".
7  * - Classic Scenarios: You have a tree and Q offline queries asking about
8  *   properties within
9  *   the subtree of node U (e.g., "Can we form a palindrome using nodes at depth
10  *   H in U's subtree?").
11  *   A naive DFS for each query gives TLE  $O(N^2)$ .
12  * - The Magic: We use "Small to Large" merging. We find the "heavy" child (
13  *   largest subtree).
14  *   We process light children and ERASE their data, then process the heavy child
15  *   and KEEP its data.
16  *   Finally, we add light children back. Since a node is a "light child" at most
17  *    $O(\log N)$  times,
18  *   the complexity drops to  $O(N \log N)$ .
19  *
20  * 2. HOW TO USE IT (THE BLACK BOX)
21  * - Initialization: Pass the number of nodes, the string of characters, and
22  *   number of queries.
23  *   DSUOnTree dsu_tree(N, s, Q);
24  *
25  * - Updates: Add edges to build the tree, and add queries offline.
26  *   dsu_tree.addEdge(u, v);
27  *   dsu_tree.addQuery(node, depth, query_index);
28  *
29  * - Execution: Call solve(root) to process everything and get the answers.
30  *   vector<string> answers = dsu_tree.process(1);
31  *
32  * - Complexity:
33  *   Time:  $O(N \log N)$  processing time.
34  *   Space:  $O(N)$  memory.

```

```

28 *
29 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
30 * - 0-indexed or 1-indexed?
31 * The template naturally handles 1-indexed nodes. If your string is 0-indexed,
32 *   it automatically
33 *   shifts it to 1-indexed internally for safety.
34 * - Changing the problem condition:
35 * Modify the `modifyState()` function to count frequencies, update maximums,
36 *   or manage a `std::set`.
37 * Then modify the "Answer queries" section in `dfsDSU()` to check your
38 *   specific condition.
39 */
40
41 #include <bits/stdc++.h>
42 using namespace std;
43
44 struct DSUOnTree
45 {
46 private:
47     int n;
48     string s;
49     vector<vector<int>>> adj;
50     vector<vector<pair<int, int>>> queries; // {depth, query_index}
51
52     vector<int> sz;
53     vector<int> lvl;
54     vector<int> mask; // Bitmask to track odd/even frequencies of characters
55     vector<bool> big; // Marks the heavy child
56     vector<string> ans;
57
58     // Step 1: Precalculate subtree sizes and depths
59     void dfsSize(int node, int parent, int level)
60     {
61         lvl[node] = level;
62         sz[node] = 1;
63         for (int child : adj[node])
64         {
65             if (child != parent)
66             {
67                 dfsSize(child, node, level + 1);
68                 sz[node] += sz[child];
69             }
70         }
71     }
72
73     // Step 2: Add or remove a node's contribution from our global state
74     void modifyState(int node, int parent, int x)
75     {
76         // ? THE MAGIC: We toggle the i-th bit to flip the parity of the
77         //   character's frequency
78         mask[lvl[node]] ^= (1 << (s[node] - 'a'));
79
80         for (int child : adj[node])
81         {
82             // ? CRITICAL: Do NOT traverse the heavy child if it's already marked

```

```

79         !
80         if (child != parent && !big[child])
81         {
82             modifyState(child, node, x);
83         }
84     }
85
86     // Step 3: The core DSU on Tree logic
87     void dfsDSU(int node, int parent, bool keep)
88     {
89         int maxSize = -1, bigChild = -1;
90
91         // Find the heavy child
92         for (int child : adj[node])
93         {
94             if (child != parent && sz[child] > maxSize)
95             {
96                 maxSize = sz[child];
97                 bigChild = child;
98             }
99         }
100
101         // Process all light children first, and ERASE their data (keep = 0)
102         for (int child : adj[node])
103         {
104             if (child != parent && child != bigChild)
105             {
106                 dfsDSU(child, node, 0);
107             }
108         }
109
110         // Process the heavy child, and KEEP its data (keep = 1)
111         if (bigChild != -1)
112         {
113             dfsDSU(bigChild, node, 1);
114             big[bigChild] = true; // Mark as heavy so `modifyState` ignores it
115         }
116
117         // Add the current node and its light children back to the state
118         modifyState(node, parent, 1);
119
120         // --- ANSWER QUERIES FOR THIS SUBTREE ---
121         for (auto& q : queries[node])
122         {
123             int target_depth = q.first;
124             int query_id = q.second;
125
126             // ? THE MAGIC: If at most 1 bit is set, the characters can form a
127             //   palindrome.
128             if (__builtin_popcount(mask[target_depth]) <= 1)
129             {
130                 ans[query_id] = "Yes";
131             }
132             else

```

```

132         {
133             ans[query_id] = "No";
134         }
135     }
136
137     // Cleanup: Unmark the heavy child
138     if (bigChild != -1)
139     {
140         big[bigChild] = false;
141     }
142
143     // ? CRITICAL: If this whole subtree was a light child, erase it
144     //   completely!
145     if (keep == 0)
146     {
147         modifyState(node, parent, -1);
148     }
149
150 public:
151     DSUOnTree(int n, string input_s, int q_count)
152     {
153         this->n = n;
154         // Pad the string so it becomes 1-indexed to match the nodes
155         this->s = " " + input_s;
156
157         adj.resize(n + 1);
158         queries.resize(n + 1);
159         sz.assign(n + 1, 0);
160         lvl.assign(n + 1, 0);
161         mask.assign(n + 1, 0);
162         big.assign(n + 1, false);
163         ans.resize(q_count);
164     }
165
166     void addEdge(int u, int v)
167     {
168         adj[u].push_back(v);
169         adj[v].push_back(u);
170     }
171
172     void addQuery(int node, int depth, int query_index)
173     {
174         queries[node].push_back({depth, query_index});
175     }
176
177     vector<string> process(int root = 1)
178     {
179         dfsSize(root, 0, 1);
180         dfsDSU(root, 0, 0);
181         return ans;
182     }
183 };
184
185 void solve()

```

```

186 {
187     int n, q;
188     // Example Input: N nodes, Q queries
189     if (!(cin >> n >> q)) return;
190
191     string s;
192     cin >> s;
193
194     DSUOnTree dsu_tree(n, s, q);
195
196     for (int i = 1; i < n; i++)
197     {
198         int u, v;
199         cin >> u >> v;
200         dsu_tree.addEdge(u, v);
201     }
202
203     for (int i = 0; i < q; i++)
204     {
205         int v, h;
206         cin >> v >> h;
207         // 0-indexed queries
208         dsu_tree.addQuery(v, h, i);
209     }
210
211     // Process all queries offline
212     vector<string> results = dsu_tree.process(1);
213
214     for (const string& res : results)
215     {
216         cout << res << "\n";
217     }
218 }
219
220 int main()
221 {
222     ios_base::sync_with_stdio(false);
223     cin.tie(NULL);
224     // solve();
225     return 0;
226 }

```

2.2.4 DSU move erase

```

1 struct DSU
2 {
3     vector<int> _t;
4
5     DSU(int N) : _t(2 * N, -1)
6     {
7         iota(_t.begin(), _t.begin() + N, N);
8     }
9

```

```

10     int find(int x)
11     {
12         return _t[x] < 0 ? x : _t[x] = find(_t[x]);
13     }
14
15     bool join(int x, int y)
16     {
17         auto fx = find(x), fy = find(y);
18         if (is_del(x) || is_del(y) || fx == fy)
19             return false;
20         if (_t[fx] > _t[fy])
21             swap(fx, fy);
22         _t[fx] += _t[fy];
23         _t[fy] = fx;
24         return true;
25     }
26
27     bool move(int x, int y)
28     {
29         auto fx = find(x), fy = find(y);
30         if (is_del(x) || is_del(y) || fx == fy)
31             return false;
32         _t[fx]++;
33         _t[fy]--;
34         _t[x] = fy;
35         return true;
36     }
37
38     bool erase(int x)
39     {
40         auto fx = find(x);
41         if (is_del(x))
42             return false;
43         _t[fx]++;
44         _t[x] = -1;
45         return true;
46     }
47
48     bool is_del(int x)
49     {
50         return _t[x] == -1;
51     }
52
53     int sz(int x)
54     {
55         return -_t[find(x)];
56     }
57 };

```

2.2.5 DSU with rollback

```

1 class DSU
2 {

```

```

3 private:
4     vector<int> parent, size;
5     stack<int> changes;
6     int forests{};
7
8     void link(int x, int y)
9     {
10         if (size[x] > size[y])
11             swap(x, y);
12         changes.push(x);
13         parent[x] = y;
14         size[y] += size[x];
15     }
16
17 public:
18     DSU(int n)
19     {
20         parent = vector<int>(n), size = vector<int>(n), changes = stack<int>();
21         forests = n;
22         for (int i = 0; i < n; i++)
23             parent[i] = i, size[i] = 1;
24     }
25
26     int find_set(int x)
27     {
28         while (x != parent[x])
29             x = parent[x];
30         return x;
31     }
32
33     int union_sets(int x, int y)
34     {
35         x = find_set(x), y = find_set(y);
36         if (x != y)
37         {
38             link(x, y);
39             forests--;
40         }
41         return x != y;
42     }
43
44     void rollback()
45     {
46         if (changes.empty())
47             return;
48         int x = changes.top();
49         changes.pop();
50         size[parent[x]] -= size[x];
51         parent[x] = x;
52         forests++;
53     }
54
55     bool same_set(int x, int y)
56     {
57         x = find_set(x), y = find_set(y);

```

```

58         return x == y;
59     }
60
61     int get_size(int x)
62     {
63         return size[find_set(x)];
64     }
65
66     int get_forests() const
67     {
68         return forests;
69     }
70 };

```

2.2.6 KRT

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: KRUSKAL RECONSTRUCTION TREE (KRT)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Minimax path", "Reachability under weight limit", "Max edge on
6  *   path".
7  * - Classic Scenarios:
8  *   A) You are asked to find a path between U and V that MINIMIZES the MAXIMUM
9  *   edge weight along the way.
10  *   B) You are at city U. You can only use roads with a weight <= W. How many
11  *   cities can you reach?
12  * - The Magic: During Kruskal's MST algorithm, instead of just merging two
13  *   components, we create a
14  *   BRAND NEW artificial node to represent the edge! We make the roots of the
15  *   two components children
16  *   of this new artificial node. The result is a tree where the "leaves" are the
17  *   original cities, and
18  *   the "internal nodes" are the edges! The weight of the ancestors strictly
19  *   increases as you go up.
20  *
21  * 2. HOW TO USE IT (THE BLACK BOX)
22  * - Initialization: Pass the number of nodes (1-indexed) and a vector of Edges.
23  *   vector<Edge> edges = {{1, 2, 10}, {2, 3, 20}, {1, 3, 5}};
24  *   KRT krt(3, edges);
25  *
26  * - Minimax Path Query:
27  *   // Find the maximum weight edge on the optimal path between U and V
28  *   long long max_edge = krt.getMaxEdgeOnPath(U, V);
29  *
30  * - Reachability Query:
31  *   // From U, how many cities can I reach using ONLY edges with weight <= W
32  *   ?
33  *   int cities = krt.getReachableLeaves(U, W);
34  *
35  * - Complexity:
36  *   Time: O(E log E) to sort edges and build. O(log N) per query!
37  *   Space: O(N log N) memory for Binary Lifting tables.

```

```

30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - Max Spanning Tree instead of Min Spanning Tree?
33 * If you want a path that MAXIMIZES the MINIMUM edge, change the `operator<`
34 * to `return w > other.w;`. Then, in `getReachableLeaves`, change the jump
35 * condition to
36 * `val[p] >= min_w`.
37 * - Disconnected Graphs?
38 * The template natively supports disconnected graphs! `getMaxEdgeOnPath`
39 * returns `-1` if U and V
40 * are completely disconnected.
41 */
42 #include <bits/stdc++.h>
43 using namespace std;
44 struct Edge
45 {
46     int u, v;
47     long long w;
48     // Sort ascending for standard Min-MST KRT
49     bool operator<(const Edge &other) const
50     {
51         return w < other.w;
52     }
53 };
54
55 struct KRT
56 {
57 private:
58     int n, krt_nodes;
59     const int LOG = 21; // Safe for up to 10^6 nodes
60
61     // DSU parent array (crucial that we don't use size/rank here, we MUST link
62     // to the new node)
63     vector<int> dsu_parent;
64
65     // KRT structure
66     vector<long long> val; // The weight of the edge this node represents
67     vector<vector<int>> adj; // Tree edges
68     vector<int> depth; // Depth for LCA
69     vector<vector<int>> up; // Binary Lifting table
70     vector<int> leaves_count; // How many original cities are in this subtree?
71
72     int find(int i)
73     {
74         if (i == dsu_parent[i])
75             return i;
76         // Path compression is perfectly safe here because we only use DSU to
77         // find the current root
78         return dsu_parent[i] = find(dsu_parent[i]);
79     }
80
81     // Standard Binary Lifting DFS
82
83     void dfs(int u, int p)
84     {
85         up[u][0] = p;
86         for (int i = 1; i < LOG; i++)
87         {
88             up[u][i] = up[up[u][i - 1]][i - 1];
89         }
90
91         // Base case: If it has no children, it's an original city (Leaf)
92         if (adj[u].empty())
93         {
94             leaves_count[u] = 1;
95         }
96         else
97         {
98             leaves_count[u] = 0;
99         }
100
101         for (int v : adj[u])
102         {
103             if (v != p)
104             {
105                 depth[v] = depth[u] + 1;
106                 dfs(v, u);
107                 // Bubble up the leaf count
108                 leaves_count[u] += leaves_count[v];
109             }
110         }
111     }
112
113 public:
114     KRT(int n, vector<Edge> &edges)
115     {
116         this->n = n;
117         // The KRT will have exactly N original leaves, and at most N-1
118         // artificial edge-nodes
119         int max_nodes = 2 * n + 5;
120
121         dsu_parent.resize(max_nodes);
122         iota(dsu_parent.begin(), dsu_parent.end(), 0);
123
124         val.assign(max_nodes, 0);
125         adj.resize(max_nodes);
126         leaves_count.assign(max_nodes, 0);
127         depth.assign(max_nodes, 0);
128         up.assign(max_nodes, vector<int>(LOG, 0));
129
130         // 1. Sort the edges to process in Kruskal's order
131         sort(edges.begin(), edges.end());
132
133         krt_nodes = n; // Nodes 1 to N are the original cities
134
135         // 2. Build the Tree
136         for (auto &e : edges)
137         {

```

```

134     int ru = find(e.u);
135     int rv = find(e.v);
136
137     if (ru != rv)
138     {
139         krt_nodes++; // Create a new artificial node for this edge
140         val[krt_nodes] = e.w;
141
142         // Connect the new node to the roots of the two components
143         adj[krt_nodes].push_back(ru);
144         adj[krt_nodes].push_back(rv);
145
146         // ? THE MAGIC: The new node becomes the ultimate DSU root for
147         both components!
148         dsu_parent[ru] = krt_nodes;
149         dsu_parent[rv] = krt_nodes;
150     }
151
152     // 3. Process the DFS for LCA and Subtree sizes
153     // We loop through all nodes in case the graph is a disconnected forest
154     for (int i = 1; i <= krt_nodes; i++)
155     {
156         if (find(i) == i)
157         {
158             // We found a supreme root! Run DFS downwards.
159             dfs(i, 0);
160         }
161     }
162 }
163
164 // Returns the max edge weight on the path between U and V
165 long long getMaxEdgeOnPath(int u, int v)
166 {
167     if (find(u) != find(v))
168         return -1; // They are in disconnected components
169     if (u == v)
170         return 0; // Same node
171
172     // Standard LCA Logic
173     if (depth[u] < depth[v])
174         swap(u, v);
175     int diff = depth[u] - depth[v];
176
177     for (int i = 0; i < LOG; i++)
178     {
179         if ((diff >> i) & 1)
180             u = up[u][i];
181     }
182
183     if (u == v)
184         return val[u]; // One is an ancestor of the other
185
186     for (int i = LOG - 1; i >= 0; i--)
187     {
188         if (up[u][i] != up[v][i])
189         {
190             u = up[u][i];
191             v = up[v][i];
192         }
193     }
194
195     // The LCA node IS the edge that connected them!
196     int lca = up[u][0];
197     return val[lca];
198 }
199
200 // Returns how many original cities can be reached from U using edges <=
201 max_w
202 int getReachableLeaves(int u, long long max_w)
203 {
204     // Jump up the tree as long as the parent edge weight is valid
205     for (int i = LOG - 1; i >= 0; i--)
206     {
207         int p = up[u][i];
208         // Parent 0 means we hit the root boundaries.
209         if (p != 0 && val[p] <= max_w)
210         {
211             u = p; // Safe to jump!
212         }
213     }
214     // Once we reach the highest valid ancestor, its subtree contains ALL
215     reachable cities!
216     return leaves_count[u];
217 }
218
219 void solve()
220 {
221     int n, m, q;
222     // Read N nodes, M edges, Q queries
223     if (!(cin >> n >> m >> q))
224         return;
225
226     vector<Edge> edges(m);
227     for (int i = 0; i < m; i++)
228     {
229         cin >> edges[i].u >> edges[i].v >> edges[i].w;
230     }
231
232     KRT krt(n, edges);
233
234     while (q--)
235     {
236         int type;
237         cin >> type;
238         if (type == 1)
239         {
240             int u, v;
241             cin >> u >> v;

```

```

241     long long ans = krt.getMaxEdgeOnPath(u, v);
242     if (ans == -1)
243         cout << "Disconnected\n";
244     else
245         cout << ans << "\n";
246 }
247 else
248 {
249     int u;
250     long long w;
251     cin >> u >> w;
252     cout << krt.getReachableLeaves(u, w) << "\n";
253 }
254 }
255 }
256
257 int main()
258 {
259     ios_base::sync_with_stdio(false);
260     cin.tie(NULL);
261     // solve();
262     return 0;
263 }

```

2.2.7 Offline Dynamic connectivity

```

1 class DSU
2 {
3 private:
4     vector<int> parent, size;
5     stack<int> changes;
6     int forests{};
7
8     void link(int x, int y)
9     {
10         if (size[x] > size[y])
11             swap(x, y);
12         changes.push(x);
13         parent[x] = y;
14         size[y] += size[x];
15     }
16
17 public:
18     DSU(int n)
19     {
20         parent = vector<int>(n), size = vector<int>(n), changes = stack<int>();
21         forests = n;
22         for (int i = 0; i < n; i++)
23             parent[i] = i, size[i] = 1;
24     }
25
26     int find_set(int x)
27     {

```

```

28         while (x != parent[x])
29             x = parent[x];
30         return x;
31     }
32
33     int union_sets(int x, int y)
34     {
35         x = find_set(x), y = find_set(y);
36         if (x != y)
37         {
38             link(x, y);
39             forests--;
40         }
41         return x != y;
42     }
43
44     void rollback()
45     {
46         if (changes.empty())
47             return;
48         int x = changes.top();
49         changes.pop();
50         size[parent[x]] -= size[x];
51         parent[x] = x;
52         forests++;
53     }
54
55     bool same_set(int x, int y)
56     {
57         x = find_set(x), y = find_set(y);
58         return x == y;
59     }
60
61     int get_size(int x)
62     {
63         return size[find_set(x)];
64     }
65
66     int get_forests() const
67     {
68         return forests;
69     }
70 };
71
72 struct edge
73 {
74     ll from, to;
75
76     edge()
77     {
78     }
79
80     edge(ll from, ll to) : from(from), to(to)
81     {
82     }

```

```

83
84     bool operator==(const edge &e)
85     {
86         return from == e.from && to == e.to;
87     }
88 };
89
90 struct Node
91 {
92     int val;
93
94     Node()
95     {
96         val = 0;
97     }
98
99     Node(int val) : val(val)
100    {
101    }
102
103    void change(int x)
104    {
105        val = x;
106    }
107 };
108
109 struct segmentTree
110 {
111     int sz;
112     vector<Node> seg;
113     vector<vector<edge>> edges;
114     DSU d;
115
116     segmentTree(int n) : d(n)
117     {
118         sz = 1;
119         while (sz < n)
120             sz *= 2;
121         seg.assign(2 * sz, Node());
122         edges.assign(2 * sz, {});
123     }
124
125     void set(int l, int r, int node, int lx, int rx, edge edge)
126     {
127         if (lx >= l && rx <= r)
128         {
129             edges[node].push_back(edge);
130             return;
131         }
132         if (rx <= l || lx >= r)
133             return;
134         int mid = (lx + rx) / 2;
135         set(l, r, 2 * node + 1, lx, mid, edge);
136         set(l, r, 2 * node + 2, mid, rx, edge);
137     }

```

```

138
139     void set(int l, int r, edge edge)
140     {
141         set(l, r, 0, 0, sz, edge);
142     }
143
144     void get(int node, int lx, int rx)
145     {
146         int cnt{};
147         for (auto &[x, y] : edges[node])
148             cnt += d.union_sets(x, y);
149         if (rx - lx == 1)
150         {
151             // answer queries
152         }
153         else
154         {
155             int mid = (lx + rx) / 2;
156             get(2 * node + 1, lx, mid);
157             get(2 * node + 2, mid, rx);
158         }
159         while (cnt-->0)
160             d.rollback();
161     }
162
163     void get()
164     {
165         return get(0, 0, sz);
166     }
167 };

```

2.2.8 Partially persistent DSU

```

1 /**
2  *  THE ULTIMATE CP TEMPLATE BLUEPRINT: PARTIALLY PERSISTENT DSU
3  *
4  *  1.  WHAT PROBLEM DOES THIS SOLVE?
5  *  - Keywords: "At what time did U and V connect?", "Were they connected after K
6    *  - Classic Scenarios: You have a graph where edges are slowly being built over
7    *  - You get queries asking if two cities were connected at a specific year in
8    *  - The Magic: Because we ONLY use Union-by-Size (and strictly ban path
9    *  - naturally forms a tree with a maximum depth of O(log N). By simply recording
10   *  - the `time`
11   *  - an edge was formed on the child node pointing to the parent, we can "time
12   *  - travel".
13   *  - To find the root at time T, we just climb the parent pointers, but we STOP
14   *  - climbing if
15   *  - the edge above us was built AFTER time T!
16   *

```

```

14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Initialization: Pass the number of nodes (1-indexed).
16 *   PartiallyPersistentDSU pp_dsu(N);
17 *
18 * - Updates: Merge U and V at time T. (T must be strictly increasing across
   calls).
19 *   pp_dsu.unite(U, V, T);
20 *
21 * - Queries:
22 *   // Were U and V connected at time T?
23 *   bool connected = pp_dsu.isConnected(U, V, T);
24 *
25 *   // What is the EXACT time U and V became connected?
26 *   int time_joined = pp_dsu.timeConnected(U, V);
27 *
28 * - Complexity:
29 *   Time:  $O(\log N)$  per merge and per query.
30 *   Space:  $O(N)$  memory.
31 *
32 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
33 * - 0-indexed nodes?
34 *   The template naturally allocates  $n + 1$  space. It works flawlessly for both
   0-indexed
35 *   and 1-indexed inputs without changing any code!
36 * - Getting the size of a component at time T?
37 *   This requires a bit more work. You would need to make `sz` a vector of
   vector<pair<int, int>>`
38 *   to store the history of sizes: `sz[root].push_back({time, new_size})`, and
   then binary search it.
39 *   If you just need standard connectivity, this template is perfectly
   lightweight.
40 */
41
42 #include <bits/stdc++.h>
43 using namespace std;
44
45 struct PartiallyPersistentDSU
46 {
47 private:
48     vector<int> parent;
49     vector<int> sz;
50     vector<int> time_joined; // The time this node became a child of its parent
51     const int INF = 2e9 + 7;
52
53 public:
54     PartiallyPersistentDSU(int n)
55     {
56         parent.resize(n + 1);
57         sz.assign(n + 1, 1);
58         time_joined.assign(n + 1, INF);
59         iota(parent.begin(), parent.end(), 0);
60     }
61
62     // ? CRITICAL: Notice there is NO path compression `parent[i] = find(...)`!
63     // Path compression would destroy the historical edges.
64
65     int find(int i, int t)
66     {
67         // We stop climbing if we hit a root, or if the edge above us hasn't been
        // built yet at time `t`
68         if (i == parent[i] || time_joined[i] > t)
69         {
70             return i;
71         }
72         return find(parent[i], t);
73     }
74
75     // Merge u and v at time t. (Assumes t is strictly increasing as you feed
   edges)
76     bool unite(int u, int v, int t)
77     {
78         // When merging, we only care about the present moment, so we pass INF as
        // time
79         int root_u = find(u, INF);
80         int root_v = find(v, INF);
81
82         if (root_u == root_v)
83             return false;
84
85         // Union by Size ensures the tree depth stays  $O(\log N)$ 
86         if (sz[root_u] < sz[root_v])
87         {
88             swap(root_u, root_v);
89         }
90
91         parent[root_v] = root_u;
92         sz[root_u] += sz[root_v];
93
94         // ? THE MAGIC: We record EXACTLY when root_v surrendered to root_u
95         time_joined[root_v] = t;
96
97         return true;
98     }
99
100     // Check if u and v were in the same component at time t
101     bool isConnected(int u, int v, int t)
102     {
103         return find(u, t) == find(v, t);
104     }
105
106     // Find the exact time u and v became connected. Returns INF if they never
   connect.
107     int timeConnected(int u, int v)
108     {
109         if (!isConnected(u, v, INF))
110             return INF; // Never connected
111
112         // Binary search the timeline to find the exact moment of connection!
113         int l = 0, r = INF - 1;
114         int ans = INF;

```

```

115     while (l <= r)
116     {
117         int mid = l + (r - 1) / 2;
118         if (isConnected(u, v, mid))
119         {
120             ans = mid;
121             r = mid - 1; // Try to find an earlier time
122         }
123         else
124         {
125             l = mid + 1; // We need more time
126         }
127     }
128     return ans;
129 }
130 };
131
132 void solve()
133 {
134     int n, m, q;
135     // Example Input: N nodes, M edges added sequentially, Q queries
136     if (!(cin >> n >> m >> q))
137         return;
138
139     PartiallyPersistentDSU pp_dsu(n);
140
141     for (int i = 1; i <= m; i++)
142     {
143         int u, v;
144         cin >> u >> v;
145         // Assume edges are added at time `i`
146         pp_dsu.unite(u, v, i);
147     }
148
149     while (q--)
150     {
151         int type, u, v;
152         cin >> type >> u >> v;
153
154         if (type == 1)
155         {
156             // Were u and v connected at time T?
157             int t;
158             cin >> t;
159             cout << (pp_dsu.isConnected(u, v, t) ? "YES" : "NO") << "\n";
160         }
161         else
162         {
163             // At what time did u and v finally connect?
164             int t = pp_dsu.timeConnected(u, v);
165             if (t > m)
166                 cout << "Not connected\n";
167             else
168                 cout << "Connected at edge " << t << "\n";
169         }

```

```

170     }
171 }
172
173 int main()
174 {
175     ios_base::sync_with_stdio(false);
176     cin.tie(NULL);
177     // solve();
178     return 0;
179 }

```

2.2.9 Small to large

```

1 // Small to large merging
2
3 vector<int> colors;
4 GRAPH graph;
5
6 set<int> dfs(int u, int p)
7 {
8     set<int> st{colors[u]};
9     for (auto &ch : graph[u])
10     {
11         if (ch == p)
12             continue;
13         set<int> child_st = dfs(ch, u);
14         if (int(child_st.size()) > int(st.size()))
15             swap(st, child_st);
16         st.merge(child_st);
17     }
18     return st;
19 }

```

2.3 Fenwick Tree

2.3.1 2D bit

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 2D FENWICK TREE (POINT UPDATE & RANGE
3  * QUERY)
4  *
5  * 1. WHAT PROBLEM DOES THIS SOLVE?
6  * - Keywords: "2D Grid", "Update a single cell", "Sum of subgrid", "Rectangle
7  * Queries".
8  * - Classic Scenarios: You have an  $N \times M$  grid. You need to repeatedly change the
9  * value of
10  * a specific cell  $(X, Y)$ . Then, you need to query the sum of all elements
11  * inside a
12  * subgrid bounded by top-left  $(X1, Y1)$  and bottom-right  $(X2, Y2)$ .
13  * - The Magic: Replaces a bulky, complex 2D Segment Tree. It computes 2D prefix
14  * sums dynamically
15  * in  $O(\log N * \log M)$  time using ridiculously short and fast bitwise loops.

```

```

11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Initialization: Pass the dimensions (rows, cols) to the constructor.
14 * (Note: Fenwick Trees MUST be 1-indexed. The bounds are [1, N] and [1, M]).
15 * FenwickTree2D fenwick(N, M);
16 *
17 * - Point Updates (1-indexed (x, y)):
18 * // Add 'val' to the cell at row X, col Y
19 * fenwick.update(X, Y, val);
20 *
21 * - Range Queries (Inclusive 1-indexed [x1, x2] and [y1, y2]):
22 * // Get the sum of all cells in the rectangle
23 * long long total = fenwick.query(x1, y1, x2, y2);
24 *
25 * - Complexity:
26 * Time:  $O(\log N * \log M)$  per update and query.
27 * Space:  $O(N * M)$  memory.
28 *
29 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
30 * - "Point Set" instead of "Point Add"?
31 * Fenwick Trees can ONLY add values. If the problem asks you to REPLACE the
32 * value at (X, Y)
33 * with `V`, you must maintain a 2D array `grid` of the current values.
34 * Then, do:
35 * long long diff = V - grid[X][Y];
36 * fenwick.update(X, Y, diff);
37 * grid[X][Y] = V;
38 * - 0-Indexed Input?
39 * Simply add +1 to all coordinates before passing them into `update` and `
40 * query`.
41 * NEVER change the internal 1-based indexing of the BIT.
42 */
43 #include <bits/stdc++.h>
44 using namespace std;
45
46 struct FenwickTree2D
47 {
48 private:
49     int n, m;
50     vector<vector<long long>> bit;
51
52     // Get the sum of the rectangle from (1, 1) to (x, y)
53     long long queryPrefix(int x, int y)
54     {
55         long long sum = 0;
56         for (int i = x; i > 0; i -= i & -i)
57         {
58             for (int j = y; j > 0; j -= j & -j)
59             {
60                 sum += bit[i][j];
61             }
62         }
63         return sum;

```

```

64     }
65
66 public:
67     FenwickTree2D(int rows, int cols)
68     {
69         n = rows;
70         m = cols;
71         // 1-indexed, so allocate size + 1
72         bit.assign(n + 1, vector<long long>(m + 1, 0));
73     }
74
75     // Add `val` to the cell at (x, y)
76     void update(int x, int y, long long val)
77     {
78         for (int i = x; i <= n; i += i & -i)
79         {
80             for (int j = y; j <= m; j += j & -j)
81             {
82                 bit[i][j] += val;
83             }
84         }
85     }
86
87     // Get the sum of the rectangle from (x1, y1) to (x2, y2)
88     long long query(int x1, int y1, int x2, int y2)
89     {
90         // Standard 2D Inclusion-Exclusion Principle
91         return queryPrefix(x2, y2) - queryPrefix(x1 - 1, y2) - queryPrefix(x2, y1
92             - 1) + queryPrefix(x1 - 1, y1 - 1);
93     };
94
95     void solve()
96     {
97         int n, m, q;
98         // Example Input Format: N rows, M cols, Q queries
99         if (!(cin >> n >> m >> q))
100             return;
101
102         FenwickTree2D fenwick(n, m);
103
104         // Optional: Maintain original grid if "Point Set" operations are required
105         // vector<vector<long long>> grid(n + 1, vector<long long>(m + 1, 0));
106
107         while (q--)
108         {
109             int type;
110             cin >> type;
111
112             if (type == 1)
113             { // Point Add
114                 int x, y;
115                 long long val;
116                 cin >> x >> y >> val;
117                 fenwick.update(x, y, val);

```

```

118     }
119     else if (type == 2)
120     { // Rectangle Query
121         int x1, y1, x2, y2;
122         cin >> x1 >> y1 >> x2 >> y2;
123         cout << fenwick.query(x1, y1, x2, y2) << "\n";
124     }
125 }
126 }
127
128 int main()
129 {
130     // Fast I/O
131     ios_base::sync_with_stdio(false);
132     cin.tie(NULL);
133     solve();
134     return 0;
135 }

```

2.3.2 2D bit rangeUpdate rangeQuery

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 2D FENWICK TREE (RANGE UPDATE & QUERY)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "2D Grid", "Add X to subgrid", "Sum of subgrid", "Rectangle
6  *   Updates".
7  * - Classic Scenarios: You are given an N x M grid (like a terrain map or matrix
8  *   ).
9  *   You need to repeatedly add a value V to all cells in a rectangle from (X1,
10  *   Y1) to (X2, Y2).
11  *   Then, you need to query the sum of all cells in a rectangle from (X1, Y1) to
12  *   (X2, Y2).
13  * - The Magic: A 2D Segment tree with Lazy Propagation is notoriously difficult
14  *   to code and uses
15  *   gigantic amounts of memory. This 2D Fenwick Tree uses 4 parallel grids to
16  *   mathematically
17  *   track the inclusion-exclusion bounds of the updates. It uses a fraction of
18  *   the memory
19  *   and runs blazingly fast in O(log N * log M) time.
20  *
21  * 2. HOW TO USE IT (THE BLACK BOX)
22  * - Initialization: Pass the dimensions (rows, cols) to the constructor.
23  *   (Note: Fenwick Trees MUST be 1-indexed. The bounds are [1, N] and [1, M]).
24  *   FenwickTree2DRange fenwick(N, M);
25  * - Range Updates (Inclusive 1-indexed [x1, x2] and [y1, y2]):
26  *   // Add 'val' to every cell in the rectangle bounded by top-left (x1, y1)
27  *   and bottom-right (x2, y2)
28  *   fenwick.update(x1, y1, x2, y2, val);
29  * - Range Queries (Inclusive 1-indexed [x1, x2] and [y1, y2]):
30  *   // Get the sum of all cells in the rectangle

```

```

25 *         long long total = fenwick.query(x1, y1, x2, y2);
26 *
27 * - Complexity:
28 *   Time: O(log N * log M) per update and query.
29 *   Space: O(N * M) memory.
30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - 0-Indexed Input?
33 *   If your problem gives coordinates as 0-indexed (e.g., (0, 0) to (N-1, M-1)),
34 *   simply
35 *   add +1 to all coordinates before passing them into `update` and `query`.
36 *   NEVER change the internal 1-based indexing of the BIT, as `index` & `-index`
37 *   fails on 0!
38 * - Modulo Arithmetic?
39 *   If the problem asks for the answer modulo 10^9+7, wrap every `+=` and `-=`
40 *   inside the
41 *   `add` and `queryPrefix` functions with modulo logic: `res = (res % MOD + MOD
42 *   ) % MOD;`
43 */
44 #include <bits/stdc++.h>
45 using namespace std;
46
47 struct FenwickTree2DRange
48 {
49     private:
50         int n, m;
51
52         // We need 4 parallel 2D BITs to mathematically calculate the 2D Prefix Sums
53         // D[i][j], D[i][j] * i, D[i][j] * j, D[i][j] * i * j
54         vector<vector<long long>> bit1, bit2, bit3, bit4;
55
56         // Add value to coordinate (x, y) across all 4 mathematical states
57         void add(int x, int y, long long val)
58         {
59             for (int i = x; i <= n; i += i & -i)
60             {
61                 for (int j = y; j <= m; j += j & -j)
62                 {
63                     bit1[i][j] += val;
64                     bit2[i][j] += val * x;
65                     bit3[i][j] += val * y;
66                     bit4[i][j] += val * x * y;
67                 }
68             }
69         }
70
71         // Get the sum of the rectangle from (1, 1) to (x, y)
72         long long queryPrefix(int x, int y)
73         {
74             long long res = 0;
75             for (int i = x; i > 0; i -= i & -i)
76             {
77                 for (int j = y; j > 0; j -= j & -j)
78                 {

```

```

76         // The mathematical expansion of 2D Difference Array Prefix Sum
77         // Sum = D * (x+1)(y+1) - (D*x) * (y+1) - (D*y) * (x+1) + (D*x*y)
78         res += bit1[i][j] * (x + 1) * (y + 1) - bit2[i][j] * (y + 1) -
              bit3[i][j] * (x + 1) + bit4[i][j];
79     }
80 }
81 return res;
82 }
83
84 public:
85     FenwickTree2DRange(int rows, int cols)
86     {
87         n = rows;
88         m = cols;
89         // 1-indexed, so allocate size + 1
90         bit1.assign(n + 1, vector<long long>(m + 1, 0));
91         bit2.assign(n + 1, vector<long long>(m + 1, 0));
92         bit3.assign(n + 1, vector<long long>(m + 1, 0));
93         bit4.assign(n + 1, vector<long long>(m + 1, 0));
94     }
95
96     // Add `val` to the rectangle from (x1, y1) to (x2, y2)
97     void update(int x1, int y1, int x2, int y2, long long val)
98     {
99         // 2D Difference Array Inclusion-Exclusion
100         add(x1, y1, val);
101         add(x2 + 1, y1, -val);
102         add(x1, y2 + 1, -val);
103         add(x2 + 1, y2 + 1, val);
104     }
105
106     // Get the sum of the rectangle from (x1, y1) to (x2, y2)
107     long long query(int x1, int y1, int x2, int y2)
108     {
109         // 2D Prefix Sum Inclusion-Exclusion
110         return queryPrefix(x2, y2) - queryPrefix(x1 - 1, y2) - queryPrefix(x2, y1
              - 1) + queryPrefix(x1 - 1, y1 - 1);
111     }
112 };
113
114 void solve()
115 {
116     int n, m, q;
117     // Example Input Format: N rows, M cols, Q queries
118     if (!(cin >> n >> m >> q))
119         return;
120
121     FenwickTree2DRange fenwick(n, m);
122
123     while (q--)
124     {
125         int type;
126         cin >> type;
127
128         if (type == 1)

```

```

        { // Range Add
            int x1, y1, x2, y2;
            long long val;
            cin >> x1 >> y1 >> x2 >> y2 >> val;
            fenwick.update(x1, y1, x2, y2, val);
        }
        else if (type == 2)
        { // Range Query
            int x1, y1, x2, y2;
            cin >> x1 >> y1 >> x2 >> y2;
            cout << fenwick.query(x1, y1, x2, y2) << "\n";
        }
    }
}

142 }
143
144 int main()
145 {
146     // Crucial fast I/O setup for dense CP test cases
147     ios_base::sync_with_stdio(false);
148     cin.tie(NULL);
149     solve();
150     return 0;
151 }

```

2.3.3 Bit rangeUpdate rangeQuery

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 1D FENWICK TREE (RANGE UPDATE & QUERY)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Range addition", "Subarray sum", "Add X to range [L, R]".
6  * - Classic Scenarios: You are given an array. You need to repeatedly add a
   value V
7  * to all elements from index L to R. Then, you need to query the sum of all
   elements
8  * from index L to R.
9  * - The Magic: Normally, you need a Lazy Segment Tree for this. However, by
   using
10 * two parallel Fenwick Trees, we can track the difference array and its index-
   multiplier
11 * to mathematically simulate Lazy Propagation. It is ridiculously fast and
   takes zero
12 * extra memory.
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Initialization: Pass the size of the array to the constructor.
16 * (Note: Fenwick Trees MUST be 1-indexed. The bounds are [1, N]).
17 *     FenwickTreeRange fenwick(N);
18 *
19 * - Range Updates (Inclusive 1-indexed [L, R]):
20 *     // Add 'val' to every cell from index L to R
21 *     fenwick.update(L, R, val);
22 *

```

```

23 * - Range Queries (Inclusive 1-indexed [L, R]):
24 *     // Get the sum of all cells from index L to R
25 *     long long total = fenwick.query(L, R);
26 *
27 * - Complexity:
28 *     Time:  $O(\log N)$  per update and query.
29 *     Space:  $O(N)$  memory.
30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - Initial Array Values?
33 *     If the problem gives you an initial array `A`, you don't need a special
34 *     build function.
35 *     Simply loop through the array and call `fenwick.update(i, i, A[i]);` for
36 *     each element.
37 * - 0-Indexed Input?
38 *     If the problem uses 0-based indices, add +1 to L and R before passing them
39 *     to
40 *     `update` or `query`. NEVER change the internal 1-based logic, as `idx & -idx`
41 *     breaks on 0.
42 * - Modulo Arithmetic?
43 *     If the problem asks for the sum modulo  $10^9+7$ , wrap the `+=` and `-=`
44 *     assignments inside
45 *     `add` and `queryPrefix` with modulo logic: `res = (res % MOD + MOD) % MOD;`
46 */
47 #include <bits/stdc++.h>
48 using namespace std;
49 struct FenwickTreeRange
50 {
51 private:
52     int n;
53     // bit1 tracks the standard difference array:  $D[i]$ 
54     // bit2 tracks the difference array multiplied by the index:  $D[i] * i$ 
55     vector<long long> bit1, bit2;
56     // Point add to both parallel BITs
57     void add(int idx, long long val)
58     {
59         // We use the actual `idx` where the value was inserted for the bit2
60         // multiplier
61         for (int i = idx; i <= n; i += i & -i)
62         {
63             bit1[i] += val;
64             bit2[i] += val * idx;
65         }
66     }
67     // Get the sum from index 1 to idx
68     long long queryPrefix(int idx)
69     {
70         long long res = 0;
71         for (int i = idx; i > 0; i -= i & -i)
72             // The magic algebraic expansion:  $Sum = (idx + 1) * D[idx] - (D[idx] * i)$ 
73             res += bit1[i] * (idx + 1) - bit2[i];
74     }
75     return res;
76 }
77
78 public:
79     FenwickTreeRange(int size)
80     {
81         n = size;
82         bit1.assign(n + 1, 0);
83         bit2.assign(n + 1, 0);
84     }
85
86     // Add `val` to the inclusive range [l, r]
87     void update(int l, int r, long long val)
88     {
89         // Standard difference array inclusion-exclusion logic
90         add(l, val);
91         add(r + 1, -val);
92     }
93
94     // Get the sum of the inclusive range [l, r]
95     long long query(int l, int r)
96     {
97         return queryPrefix(r) - queryPrefix(l - 1);
98     }
99 };
100
101 void solve()
102 {
103     int n, q;
104     // Example Input Format: N elements, Q queries
105     if (!(cin >> n >> q))
106         return;
107
108     FenwickTreeRange fenwick(n);
109
110     while (q--)
111     {
112         int type;
113         cin >> type;
114
115         if (type == 1)
116         { // Range Add
117             int l, r;
118             long long val;
119             cin >> l >> r >> val;
120             fenwick.update(l, r, val);
121         }
122         else if (type == 2)
123         { // Range Query
124             int l, r;
125             cin >> l >> r;
126             cout << fenwick.query(l, r) << "\n";

```

```

127     }
128 }
129 }
130
131 int main()
132 {
133     // Fast I/O for competitive programming
134     ios_base::sync_with_stdio(false);
135     cin.tie(NULL);
136     solve();
137     return 0;
138 }

```

2.3.4 Fenwick tree

```

1 template <typename T>
2 struct FenwickTree
3 {
4     int n;
5     vector<T> tree;
6
7     // 1. Initialize an empty tree of size n (all zeros)
8     FenwickTree(int n)
9     {
10         this->n = n;
11         tree.assign(n + 1, 0);
12     }
13
14     // 2. Fast O(N) initialization from an existing array
15     FenwickTree(const vector<T> &a)
16     {
17         n = a.size();
18         tree.assign(n + 1, 0);
19
20         // Step 1: Copy values in (shifting to 1-based indexing)
21         for (int i = 0; i < n; i++)
22         {
23             tree[i + 1] = a[i];
24         }
25
26         // Step 2: Propagate the sums forward to the immediate parent
27         for (int i = 1; i <= n; i++)
28         {
29             int parent = i + (i & -i);
30             if (parent <= n)
31             {
32                 tree[parent] += tree[i];
33             }
34         }
35     }
36
37     void add(int i, T delta)
38     {

```

```

39         for (; i <= n; i += (i & -i))
40         {
41             tree[i] += delta;
42         }
43     }
44
45     T query(int i)
46     {
47         T sum = 0;
48         for (; i > 0; i -= (i & -i))
49         {
50             sum += tree[i];
51         }
52         return sum;
53     }
54
55     T query(int l, int r)
56     {
57         return query(r) - query(l - 1);
58     }
59 };

```

2.3.5 Lower bound on bit

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: LOWER BOUND ON BIT (BINARY LIFTING)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Find the K-th smallest element", "First index with prefix sum >=
6  * X".
7  * - Classic Scenarios: You have an array of frequencies (how many times a number
8  * appears).
9  * You need to dynamically add/remove numbers and frequently ask: "What is the
10 * K-th
11 * smallest number currently in my set?"
12 * - The Magic: Normally, you'd use a C++ PBDS (Policy Based Data Structure) for
13 * this.
14 * If PBDS is banned, or you need raw execution speed, this algorithm does a
15 * binary
16 * search directly ON the Fenwick tree's internal structure. It takes exactly O
17 * (log N)
18 * steps instead of O(log^2 N).
19 *
20 * 2. HOW TO USE IT (THE BLACK BOX)
21 * - Initialization: Pass the size of the array to the constructor.
22 * FenwickTreeLB fenwick(N);
23 *
24 * - Standard Point Updates (1-indexed):
25 * // Add 'val' to index 'idx'
26 * fenwick.update(idx, val);
27 *
28 * - The K-th Element Query (Lower Bound):
29 * // Find the smallest index where the prefix sum is >= V

```

```

24 *      int ans_idx = fenwick.lower_bound(V);
25 *
26 * - Complexity:
27 *      Time:  $O(\log N)$  per update and lower_bound query.
28 *      Space:  $O(N)$  memory.
29 *
30 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
31 * - CRITICAL RULE: MONOTONICITY
32 * This function ONLY works if the original array contains ONLY non-negative
   numbers!
33 * If you add negative numbers, the prefix sums go up and down randomly. Binary
   search
34 * (and therefore this binary lifting trick) will completely fail.
35 * - Out of Bounds Check:
36 * If you search for a value `V` that is larger than the total sum of the
   entire array,
37 * the function will return `N + 1`. Always check `if (ans_idx > N)` if you
   aren't sure
38 * the sum exists!
39 */
40
41 #include <bits/stdc++.h>
42 using namespace std;
43
44 struct FenwickTreeLB
45 {
46 private:
47     int n;
48     int LOGN;
49     vector<long long> bit;
50
51 public:
52     FenwickTreeLB(int size)
53     {
54         n = size;
55         bit.assign(n + 1, 0);
56
57         // Find the highest power of 2 that is <= n
58         LOGN = 0;
59         while ((1 << (LOGN + 1)) <= n)
60         {
61             LOGN++;
62         }
63     }
64
65     // Add `val` to the cell at `idx`
66     void update(int idx, long long val)
67     {
68         for (; idx <= n; idx += idx & -idx)
69         {
70             bit[idx] += val;
71         }
72     }
73
74     // Get standard prefix sum up to `idx`
75
76     long long query(int idx)
77     {
78         long long sum = 0;
79         for (; idx > 0; idx -= idx & -idx)
80         {
81             sum += bit[idx];
82         }
83         return sum;
84     }
85
86     // ? THE MAGIC: Binary Lifting directly on the Fenwick Tree
87     // Returns the 1-based index of the FIRST element whose prefix sum is >= v
88     int lower_bound(long long v)
89     {
90         if (v <= 0)
91             return 1; // Edge case: Target is already reached at the start
92
93         int idx = 0;
94         long long current_sum = 0;
95
96         // Jump down from the highest power of 2 to 1
97         for (int i = LOGN; i >= 0; i--)
98         {
99             int next_idx = idx + (1 << i);
100
101             // If taking this jump keeps us inside the array bounds,
102             // AND the sum AT this jump is still STRICTLY LESS than our target `v`
103             if (next_idx <= n && current_sum + bit[next_idx] < v)
104             {
105                 // We commit to the jump!
106                 idx = next_idx;
107                 current_sum += bit[next_idx];
108             }
109
110             // `idx` is now the largest index where the sum is STRICTLY LESS than `v`
111             // Therefore, `idx + 1` is the first index where the sum is >= `v`!
112             return idx + 1;
113         }
114     };
115
116 void solve()
117 {
118     int n, q;
119     // Example Input Format: Array of size N, Q queries
120     if (!(cin >> n >> q))
121         return;
122
123     FenwickTreeLB fenwick(n);
124
125     while (q--)
126     {
127         int type;

```

```

128     cin >> type;
129
130     if (type == 1)
131     { // Point Add
132         int idx;
133         long long val;
134         cin >> idx >> val;
135         fenwick.update(idx, val);
136     }
137     else if (type == 2)
138     { // Lower Bound Query
139         long long target_sum;
140         cin >> target_sum;
141
142         int ans = fenwick.lower_bound(target_sum);
143
144         if (ans > n)
145         {
146             cout << "Target_sum_not_reached!\n";
147         }
148         else
149         {
150             cout << "Index:_" << ans << "\n";
151         }
152     }
153 }
154 }
155
156 int main()
157 {
158     // Fast I/O
159     ios_base::sync_with_stdio(false);
160     cin.tie(NULL);
161     solve();
162     return 0;
163 }

```

2.4 HLD

2.4.1 HLD

```

1 struct Node
2 {
3     int val;
4
5     Node()
6     {
7         val = -00;
8     }
9
10    Node(int val) : val(val)
11    {
12    }

```

```

13
14    void change(int x)
15    {
16        val = x;
17    }
18 };
19
20 struct segmentTree
21 {
22     int sz;
23     vector<Node> seg;
24
25     segmentTree(int n)
26     {
27         sz = 1;
28         while (sz < n)
29             sz *= 2;
30         seg.assign(2 * sz, Node());
31     }
32
33     void init(vector<int> &arr, int node, int lx, int rx)
34     {
35         if (rx - lx == 1)
36         {
37             if (lx < int(arr.size()))
38                 seg[node] = Node(arr[lx]);
39             return;
40         }
41         int mid = (lx + rx) / 2;
42         init(arr, 2 * node + 1, lx, mid);
43         init(arr, 2 * node + 2, mid, rx);
44         seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
45     }
46
47     void init(vector<int> &arr)
48     {
49         init(arr, 0, 0, sz);
50     }
51
52     Node merge(Node &lf, Node &rt)
53     {
54         Node ans = Node();
55         ans.val = max(lf.val, rt.val);
56         return ans;
57     }
58
59     void set(int idx, int val, int node, int lx, int rx)
60     {
61         if (rx - lx == 1)
62         {
63             seg[node].change(val);
64             return;
65         }
66         int mid = (lx + rx) / 2;
67         if (idx < mid)

```

```

68         set(idcx, val, 2 * node + 1, lx, mid);
69     else
70         set(idcx, val, 2 * node + 2, mid, rx);
71     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
72 }
73
74 void set(int idx, int val)
75 {
76     set(idcx, val, 0, 0, sz);
77 }
78
79 Node get(int l, int r, int node, int lx, int rx)
80 {
81     if (lx >= l && rx <= r)
82         return seg[node];
83     if (rx <= l || lx >= r)
84         return Node();
85     int mid = (lx + rx) / 2;
86     Node lf = get(l, r, 2 * node + 1, lx, mid);
87     Node ri = get(l, r, 2 * node + 2, mid, rx);
88     return merge(lf, ri);
89 }
90
91 int get(int l, int r)
92 {
93     return get(l, r, 0, 0, sz).val;
94 }
95 };
96
97 struct HLD
98 {
99 private:
100     vector<int> sz, par, dep, tin, tout, head;
101     vector<vector<int>> graph;
102     vector<int> a;
103     int timer{1};
104     segmentTree seg;
105
106 public:
107     HLD(int n, GRAPH graph, vector<int> &v) : graph(std::move(graph)), seg(n + 1)
108     {
109         sz = vector<int>(n + 1), par = vector<int>(n + 1), dep = vector<int>(n + 1), tin = vector<int>(n + 1), tout = vector<int>(n + 1), head = vector<int>(n + 1), a = vector<int>(n + 1);
110         dfs1(1, 0);
111         head[1] = 1;
112         dfs2(1);
113         for (int i = 1; i <= n; i++)
114             a[tin[i]] = v[i];
115         seg.init(a);
116     }
117
118     void dfs1(int u, int p)
119     {
120         dep[u] = dep[p] + 1;
121         par[u] = p;
122         sz[u] = 1;
123         if (p != 0)
124             graph[u].erase(find(graph[u].begin(), graph[u].end(), p));
125         for (auto &x : graph[u])
126         {
127             dfs1(x, u);
128             sz[u] += sz[x];
129             if (sz[x] > sz[graph[u][0]])
130                 swap(x, graph[u][0]);
131         }
132     }
133
134     void dfs2(int u)
135     {
136         tin[u] = timer++;
137         for (auto &x : graph[u])
138         {
139             head[x] = x == graph[u][0] ? head[u] : x;
140             dfs2(x);
141         }
142         tout[u] = timer;
143     }
144
145     int lca(int u, int v)
146     {
147         while (head[u] != head[v])
148         {
149             if (dep[head[u]] > dep[head[v]])
150                 u = par[head[u]];
151             else
152                 v = par[head[v]];
153         }
154         if (dep[u] < dep[v])
155             return u;
156         return v;
157     }
158
159     void point_update(int u, int val)
160     {
161         seg.set(tin[u], val);
162     }
163
164     int query_subtree(int u)
165     {
166         return seg.get(tin[u], tout[u]);
167     }
168
169     // now querying for maximum
170     int query_path(int u, int v)
171     {
172         int res{-00}; // TO
173         while (head[u] != head[v])
174         {
175             if (dep[head[u]] < dep[head[v]])

```

```

176         swap(u, v);
177         res = max(res, seg.get(tin[head[u]], tin[u] + 1)); // TO
178         u = par[head[u]];
179     }
180     if (dep[u] < dep[v])
181         swap(u, v);
182     res = max(res, seg.get(tin[v], tin[u] + 1)); // TO
183     return res;
184 }
185 };

```

2.5 Misc

2.5.1 CDQ

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: CDQ DIVIDE AND CONQUER
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "3D Partial Order", "Dynamic 2D Range Queries", "Offline range
   counting".
9  * - Classic Scenarios: You need to count how many points i have (x_i <= X, y_i
   <= Y, z_i <= Z).
10 * - The Magic: CDQ D&C reduces dimensionality.
11 * - Dimension 1 (X): Handled by sorting the array.
12 * - Dimension 2 (Y): Handled by the Divide and Conquer recursion.
13 * - Dimension 3 (Z): Handled by a 1D Fenwick Tree (BIT).
14 *
15 * 2. HOW TO USE IT (THE BLACK BOX)
16 * - Define a struct Element {x, y, z, id}.
17 * - Sort the elements initially by x.
18 * - Call cdq(0, n-1) to process.
19 *
20 * - Complexity:
21 *     Time: O(N log^2 N) where N is the number of elements.
22 *     Space: O(N) memory.
23 */
24
25 struct Element
26 {
27     int x, y, z, id;
28 };
29
30 // 1D Fenwick Tree for the 3rd dimension
31 struct BIT
32 {
33     int n;
34     vector<int> tree;
35     BIT(int n) : n(n), tree(n + 1, 0) {}
36

```

```

37     void update(int i, int delta)
38     {
39         for (; i <= n; i += i & -i)
40             tree[i] += delta;
41     }
42
43     int query(int i)
44     {
45         int sum = 0;
46         for (; i > 0; i -= i & -i)
47             sum += tree[i];
48         return sum;
49     }
50 };
51
52 int n;
53 vector<Element> arr;
54 vector<int> result;
55 BIT bit(1000005); // Adjust size to coordinate range
56
57 void cdq(int l, int r)
58 {
59     if (l >= r)
60         return;
61     int mid = l + (r - l) / 2;
62
63     cdq(l, mid);
64     cdq(mid + 1, r);
65
66     // Sort both halves by Y (Dimension 2)
67     vector<Element> temp;
68     int i = l, j = mid + 1;
69
70     while (i <= mid && j <= r)
71     {
72         if (arr[i].y <= arr[j].y)
73         {
74             bit.update(arr[i].z, 1);
75             temp.push_back(arr[i++]);
76         }
77         else
78         {
79             result[arr[j].id] += bit.query(arr[j].z);
80             temp.push_back(arr[j++]);
81         }
82     }
83
84     // Process remaining
85     while (j <= r)
86     {
87         result[arr[j].id] += bit.query(arr[j].z);
88         temp.push_back(arr[j++]);
89     }
90
91     // Clear BIT before moving on

```

```

92     for (int k = 1; k < i; k++)
93         bit.update(arr[k].z, -1);
94
95     while (i <= mid)
96         temp.push_back(arr[i++]);
97     for (int k = 0; k < temp.size(); k++)
98         arr[l + k] = temp[k];
99 }
100
101 int main()
102 {
103     ios_base::sync_with_stdio(false);
104     cin.tie(NULL);
105
106     // Example: Read N elements with (x, y, z) coordinates
107     // Ensure all coordinates are mapped to 1-indexed values for BIT
108
109     // cdq(0, n - 1);
110
111     return 0;
112 }

```

2.5.2 Leftist heap

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: LEFTIST HEAP (MELDABLE)
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Meldable Heaps", "Priority Queue on Trees", "Greedy Tree DP".
9  * - Classic Scenarios: You need to maintain a set of values for each node in a
   tree
10 * and frequently merge them with children. std::priority_queue cannot merge.
11 * - The Magic: It keeps the tree "left-heavy" to guarantee that the right spine
   is
12 * always short ( $O(\log N)$ ). Merging two heaps is just merging their right
   spines!
13 *
14 * 2. HOW TO USE IT
15 * - Initialization: Root = nullptr.
16 * - Merge: root = merge(root1, root2);
17 * - Complexity:  $O(\log N)$  per merge, push, and pop.
18 */
19
20 struct Node
21 {
22     int val, npl;
23     Node *left, *right;
24
25     Node(int v) : val(v), npl(0), left(nullptr), right(nullptr) {}
26 };

```

```

27
28 int getNpl(Node *n)
29 {
30     return n ? n->npl : -1;
31 }
32
33 // The core MELD operation
34 Node *merge(Node *a, Node *b)
35 {
36     if (!a)
37         return b;
38     if (!b)
39         return a;
40
41     // To maintain a Min-Heap, ensure 'a' has the smaller root
42     if (a->val > b->val)
43         swap(a, b);
44
45     // ! PERSISTENCE NOTE: If you need a fully persistent heap,
46     // uncomment the line below to clone the node before modifying.
47     // a = new Node(*a);
48
49     a->right = merge(a->right, b);
50
51     // Maintain Leftist Property: Left NPL >= Right NPL
52     if (getNpl(a->left) < getNpl(a->right))
53     {
54         swap(a->left, a->right);
55     }
56
57     a->npl = getNpl(a->right) + 1;
58     return a;
59 }
60
61 Node *push(Node *root, int val)
62 {
63     return merge(root, new Node(val));
64 }
65
66 Node *pop(Node *root)
67 {
68     Node *left = root->left;
69     Node *right = root->right;
70     delete root; // Clean up memory
71     return merge(left, right);
72 }
73
74 int top(Node *root)
75 {
76     return root->val;
77 }
78
79 int main()
80 {
81     // Example: Standard Usage

```

```

82     Node *root = nullptr;
83     root = push(root, 10);
84     root = push(root, 5);
85
86     cout << "Top:␣" << top(root) << endl; // 5
87     root = pop(root);
88     cout << "Top␣after␣pop:␣" << top(root) << endl; // 10
89
90     return 0;
91 }

```

2.5.3 Monotonic queue

```

1 class Monotonic_queue
2 {
3     deque<pair<int, int>> dq;
4     int k;
5
6 public:
7     Monotonic_queue(int k) : k(k) {}
8
9     void push(int idx, int val)
10    {
11        if (!dq.empty() && dq.front().second <= idx - k)
12            dq.pop_front();
13        while (!dq.empty() && dq.back().second <= val)
14            dq.pop_back();
15        dq.push_back({idx, val});
16    }
17
18    int max() const
19    {
20        return dq.front().second;
21    }
22 };

```

2.5.4 Monotonic stack

```

1 class MonotonicStack
2 {
3 public:
4     static vector<int> getNextGreater(const vector<int> &nums)
5     {
6         int n = nums.size();
7         vector<int> nextGreater(n, n);
8         stack<int> s;
9         for (int i = 0; i < n; ++i)
10        {
11            while (!s.empty() && nums[i] > nums[s.top()])
12            {
13                nextGreater[s.top()] = i;
14                s.pop();

```

```

15        }
16        s.push(i);
17    }
18    return nextGreater;
19 }
20 };

```

2.5.5 ODT

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: OLD DRIVER TREE (ODT / CHTHOLLY TREE)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Color blocks", "Range assign", "Randomized data", "Interval set".
6  * - Classic Scenarios: You have an array that is mostly composed of long
7  *   segments of identical
8  *   values. You need to perform Range Assignments ( $A[i] = V$  for  $L$  to  $R$ ) and
9  *   other Range Queries.
10  * - The Magic: An ODT stores the array as a set of intervals  $\{l, r, v\}$ .
11  *   The "Assign" operation is the key: it erases all intervals in the set
12  *   between  $[L, R]$ 
13  *   and inserts one single new interval. This keeps the number of intervals very
14  *   small.
15  *
16  * 2. HOW TO USE IT (THE BLACK BOX)
17  * - Initialization: Insert the initial  $[1, N]$  interval into the `odt` set.
18  * - Split: You MUST call `split(l)` and `split(r+1)` before ANY operation to
19  *   ensure
20  *   the boundaries are exact.
21  * - Complexity:
22  *   Time: Amortized  $O(N \log \log N)$  on random data.  $O(N^2)$  on worst-case non-
23  *   random data.
24  *   Space:  $O(N)$  memory.
25  */
26
27 #include <bits/stdc++.h>
28 using namespace std;
29
30 struct Node
31 {
32     int l, r;
33     mutable long long v;
34
35     Node(int l, int r = -1, long long v = 0) : l(l), r(r), v(v) {}
36
37     // Set must be sorted by left endpoint
38     bool operator<(const Node &o) const
39     {
40         return l < o.l;
41     }
42 };
43
44 set<Node> odt;

```

```

39
40 // Splits the interval containing 'pos' into [l, pos-1] and [pos, r]
41 // Returns an iterator to the interval starting at 'pos'
42 auto split(int pos)
43 {
44     auto it = odt.lower_bound({pos});
45     if (it != odt.end() && it->l == pos)
46         return it;
47
48     --it;
49     int l = it->l, r = it->r;
50     long long v = it->v;
51
52     odt.erase(it);
53     odt.insert({l, pos - 1, v});
54     return odt.insert({pos, r, v}).first;
55 }
56
57 // HOW TO ADAPT IT (THE DIALS & SWITCHES)
58 // - If the problem asks for Range Sum: loop through the set from itl to itr and
59 //   sum (r-l+1)*v.
60 // - If the problem asks for K-th smallest or Count: you must keep a secondary
61 //   data
62 //   structure (like a Segment Tree) updated alongside the ODT.
63 // The core efficiency of ODT: replaces range [l, r] with one single interval
64 void assign(int l, int r, long long v)
65 {
66     auto itr = split(r + 1), itl = split(l);
67     odt.erase(itl, itr);
68     odt.insert({l, r, v});
69 }
70
71 void solve()
72 {
73     int n, m;
74     cin >> n >> m;
75
76     // Initially add [1, n] with value 0
77     odt.insert({1, n, 0});
78 }
79
80 int main()
81 {
82     ios_base::sync_with_stdio(false);
83     cin.tie(NULL);
84     solve();
85     return 0;
86 }

```

2.6 More Data structures

2.6.1 Wavelet tree

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: WAVELET TREE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "K-th smallest in range", "Range frequency count", "Count elements
6  *   < K".
7  * - Classic Scenarios: You have an array and you need to answer queries like:
8  *   "Find the k-th smallest element in subarray [L, R]" or "Count occurrences
9  *   of values in [A, B] within index range [L, R]".
10 * - The Magic: It is essentially a Merge Sort Tree, but instead of storing
11 *   sorted vectors (which consume  $O(N \log N)$  memory), it stores a bit-vector
12 *   representation of the partitioning. This allows queries in  $O(\log(\text{MaxVal}))$ 
13 *   time.
14 *
15 * 2. HOW TO USE IT (THE BLACK BOX)
16 * - Initialization:
17 *   // Pass the range of values [min_val, max_val] and the input vector
18 *   WaveletTree wt(1, 1000000, array);
19 *
20 * - Queries:
21 *   // How many numbers < K in subarray [L, R]
22 *   int count = wt.count_less(L, R, K);
23 *   // Find the k-th smallest number in subarray [L, R] (1-indexed k)
24 *   int val = wt.kth_smallest(L, R, k);
25 *
26 * - Complexity:
27 *   Time:  $O(N \log(\text{MaxVal}))$  to build.  $O(\log(\text{MaxVal}))$  per query.
28 *   Space:  $O(N \log(\text{MaxVal}))$  memory.
29 */
30
31 #include <bits/stdc++.h>
32 using namespace std;
33
34 struct WaveletTree
35 {
36     int low, high;
37     WaveletTree *left, *right;
38     vector<int> pref; // Prefix sums of elements going to the left child
39
40     WaveletTree(int l, int r, const vector<int> &a) : low(l), high(r), left(
41         nullptr), right(nullptr)
42     {
43         if (low == high || a.empty())
44             return;
45
46         int mid = low + (high - low) / 2;
47
48         // Define function to check if value goes to left child
49         auto is_left = [&](int x)
50         { return x <= mid; };
51
52         pref.reserve(a.size() + 1);
53         pref.push_back(0);
54
55         vector<int> l_vec, r_vec;

```

```

53     for (int x : a)
54     {
55         if (is_left(x))
56         {
57             l_vec.push_back(x);
58             pref.push_back(pref.back() + 1);
59         }
60         else
61         {
62             r_vec.push_back(x);
63             pref.push_back(pref.back());
64         }
65     }
66
67     left = new WaveletTree(low, mid, l_vec);
68     right = new WaveletTree(mid + 1, high, r_vec);
69 }
70
71 // Count numbers <= k in range [ql, qr] (0-indexed)
72 int count_less_equal(int ql, int qr, int k)
73 {
74     if (ql > qr || low > k)
75         return 0;
76     if (high <= k)
77         return qr - ql + 1;
78
79     int l_count_before = pref[ql];
80     int r_count_before = ql - l_count_before;
81     int l_count_all = pref[qr + 1];
82     int r_count_all = (qr + 1) - l_count_all;
83
84     return left->count_less_equal(l_count_before, l_count_all - 1, k) +
85            right->count_less_equal(r_count_before, r_count_all - 1, k);
86 }
87
88 // Find k-th smallest number in range [ql, qr]
89 int kth_smallest(int ql, int qr, int k)
90 {
91     if (low == high)
92         return low;
93
94     int l_count_before = pref[ql];
95     int r_count_before = ql - l_count_before;
96     int l_count_all = pref[qr + 1];
97     int r_count_all = (qr + 1) - l_count_all;
98
99     int left_size = l_count_all - l_count_before;
100
101     if (k <= left_size)
102         return left->kth_smallest(l_count_before, l_count_all - 1, k);
103     else
104         return right->kth_smallest(r_count_before, r_count_all - 1, k -
105                                     left_size);
106 }

```

2.7 SQRT decomposition MO

2.7.1 MO

```

1
2 // how many distinct values are there in a range [l,r]
3
4 int SQ = 400;
5 struct Query
6 {
7     int l, r, idx;
8     bool operator<(const Query &q) const
9     {
10         if (l / SQ != q.l / SQ)
11         {
12             return l / SQ < q.l / SQ;
13         }
14         return (l / SQ & 1) ? r < q.r : r > q.r;
15     }
16 };
17
18 void solve()
19 {
20     int n, q;
21     cin >> n >> q;
22     vector<int> a(n), frq(n + 10);
23     map<int, int> mp;
24     int id{};
25     for (int i = 0; i < n; i++)
26     {
27         cin >> a[i];
28         if (!mp.count(a[i]))
29         {
30             mp[a[i]] = id++;
31         }
32     }
33     for (int i = 0; i < n; i++)
34         a[i] = mp[a[i]];
35
36     vector<Query> query(q);
37     for (int i = 0; i < q; i++)
38     {
39         cin >> query[i].l >> query[i].r;
40         query[i].l--, query[i].r--;
41         query[i].idx = i;
42     }
43     sort(query.begin(), query.end());
44     int ans = 0;
45     auto add = [&](int i)
46     {
47         // if (frq[a[i]] == 0)
48         // {
49             ans++;
50         // }
51         // frq[a[i]]++;

```

```

52 };
53 auto remove = [&](int i)
54 {
55     // frq[a[i]]--;
56     // if (frq[a[i]] == 0)
57     // {
58     //     ans--;
59     // }
60 };
61
62 vector<int> res(q);
63 int l = 0, r = -1;
64 for (auto [ql, qr, idx] : query)
65 {
66     while (l > ql)
67     {
68         --l;
69         add(l);
70     }
71     while (r < qr)
72     {
73         ++r;
74         add(r);
75     }
76     while (l < ql)
77     {
78         remove(l);
79         ++l;
80     }
81     while (r > qr)
82     {
83         remove(r);
84         --r;
85     }
86     res[idx] = ans;
87 }
88 for (int i = 0; i < q; i++)
89     cout << res[i] << "\n";
90 }

```

2.7.2 MO on tree

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: MO'S ALGORITHM ON TREES
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Path queries", "Distinct values on path", "Offline tree queries"
6  * - Classic Scenarios: You are given a tree where each node has a color. You are
7  *   asked Q queries:
8  *   "How many distinct colors exist on the simple path between node U and node V?"
9  * - The Magic: We flatten the tree using an Euler Tour. Every node appears twice
10  *   in the array:

```

```

9  *   once when we enter it (tin), and once when we leave it (tout).
10 *   If we want the path between U and V (where tin[U] < tin[V]), we query the
11 *   flat array:
12 *   A) If U is an ancestor of V: Query range [tin[U], tin[V]].
13 *   B) If U is NOT an ancestor: Query range [tout[U], tin[V]] PLUS the LCA(U, V)
14 *   .
15 *   Because dead-end branches will appear twice in this range (entry and exit),
16 *   we simply
17 *   TOGGLE their state. 1st appearance = Add. 2nd appearance = Remove.
18 *
19 * 2. HOW TO USE IT (THE BLACK BOX)
20 * - Initialization: Pass your tree edges and node values, then run `dfs(1, 0)`.
21 * - L and R Logic: We maintain an `in_path` boolean array. The `toggle()`
22 *   function handles
23 *   adding OR removing automatically based on the boolean!
24 *
25 * - Complexity:
26 *   Time:  $O(N \log N)$  to setup LCA +  $O((N + Q) * \sqrt{2N})$  for Mo's
27 *   processing.
28 *   Space:  $O(N \log N)$  for LCA lifting table +  $O(N)$  arrays.
29 *
30 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
31 * - Changing the logic:
32 *   Right now, the template counts "Distinct Colors" using a frequency array.
33 *   Modify the `add()` and `remove()` functions inside `toggle()` to track
34 *   whatever
35 *   property your specific problem requires.
36 */
37
38 #include <bits/stdc++.h>
39 using namespace std;
40
41 const int MAXN = 100005;
42 const int LOG = 20;
43
44 int n, q;
45 vector<int> adj[MAXN];
46 long long val[MAXN]; // The color or value of each node
47
48 // Euler Tour Variables
49 int timer_tour = 0;
50 int tin[MAXN], tout[MAXN];
51 int tour[2 * MAXN]; // The flattened 2N array
52
53 // LCA Variables
54 int up[MAXN][LOG];
55 int depth[MAXN];
56
57 void dfs(int u, int p)
58 {
59     // 1. Record Entry Time
60     tin[u] = ++timer_tour;
61     tour[timer_tour] = u;
62
63     // 2. Build Binary Lifting Table for LCA

```

```

58 up[u][0] = p;
59 for (int i = 1; i < LOG; i++)
60 {
61     up[u][i] = up[up[u][i - 1]][i - 1];
62 }
63
64 // 3. Traverse Children
65 for (int v : adj[u])
66 {
67     if (v != p)
68     {
69         depth[v] = depth[u] + 1;
70         dfs(v, u);
71     }
72 }
73
74 // 4. Record Exit Time
75 tout[u] = ++timer_tour;
76 tour[timer_tour] = u;
77 }
78
79 int get_lca(int u, int v)
80 {
81     if (depth[u] < depth[v])
82         swap(u, v);
83
84     int diff = depth[u] - depth[v];
85     for (int i = 0; i < LOG; i++)
86     {
87         if ((diff >> i) & 1)
88             u = up[u][i];
89     }
90
91     if (u == v)
92         return u;
93
94     for (int i = LOG - 1; i >= 0; i--)
95     {
96         if (up[u][i] != up[v][i])
97         {
98             u = up[u][i];
99             v = up[v][i];
100         }
101     }
102     return up[u][0];
103 }
104
105 int BLOCK_SIZE;
106
107 struct Query
108 {
109     int l, r, id, lca;
110
111     bool operator<(const Query &other) const
112     {

```

```

113         int block_own = l / BLOCK_SIZE;
114         int block_other = other.l / BLOCK_SIZE;
115         if (block_own != block_other)
116         {
117             return block_own < block_other;
118         }
119         // Parity optimization for slightly faster R pointer movement
120         return (block_own & 1) ? r < other.r : r > other.r;
121     }
122 };
123
124 bool in_path[MAXN];
125 long long freq[MAXN]; // Example: Frequency array for colors
126 long long distinct_colors = 0;
127
128 // ? THE MAGIC: Toggling a node.
129 // If it's entering our path, we add it. If we hit its exit time, it cancels out
130 // and we remove it!
131 void toggle(int node)
132 {
133     long long c = val[node];
134
135     if (in_path[node])
136     {
137         // It was in the path, so we are removing it
138         freq[c]--;
139         if (freq[c] == 0)
140             distinct_colors--;
141     }
142     else
143     {
144         // It wasn't in the path, so we are adding it
145         if (freq[c] == 0)
146             distinct_colors++;
147         freq[c]++;
148     }
149     in_path[node] ^= 1; // Flip the boolean state
150 }
151
152 void solve()
153 {
154     if (!(cin >> n >> q))
155         return;
156
157     // Reset globals for multiple test cases (if needed)
158     timer_tour = 0;
159     distinct_colors = 0;
160     memset(in_path, 0, sizeof(in_path));
161     memset(freq, 0, sizeof(freq));
162
163     // Coordinate compression for colors might be required here if values > 10^5!
164     for (int i = 1; i <= n; i++)
165         cin >> val[i];
166

```

```

167 for (int i = 0; i < n - 1; i++)
168 {
169     int u, v;
170     cin >> u >> v;
171     adj[u].push_back(v);
172     adj[v].push_back(u);
173 }
174
175 // 1. Flatten the tree!
176 depth[1] = 0;
177 dfs(1, 1);
178
179 // Block size for an array of size 2N
180 BLOCK_SIZE = max(1, (int)sqrt(2 * n));
181
182 vector<Query> queries(q);
183 for (int i = 0; i < q; i++)
184 {
185     int u, v;
186     cin >> u >> v;
187
188     // Ensure u is visited before v in the Euler Tour
189     if (tin[u] > tin[v])
190         swap(u, v);
191
192     int lca = get_lca(u, v);
193
194     if (lca == u)
195     {
196         // SCENARIO A: U is an ancestor of V.
197         // The path is strictly the sequence of entries from U down to V.
198         queries[i].l = tin[u];
199         queries[i].r = tin[v];
200         queries[i].lca = -1; // No extra LCA needed
201     }
202     else
203     {
204         // SCENARIO B: U and V are in different branches.
205         // We take the path from the exit of U, up to the LCA, and down to
206         // the entry of V.
207         queries[i].l = tout[u];
208         queries[i].r = tin[v];
209         queries[i].lca = lca; // We MUST manually include the LCA!
210     }
211     queries[i].id = i;
212 }
213
214 // 2. Sort Queries
215 sort(queries.begin(), queries.end());
216
217 vector<long long> ans(q);
218
219 // 3. Run Mo's Pointers on the 2N Tour Array
220 int L = 1, R = 0;

```

```

221 for (const Query &qry : queries)
222 {
223     while (L > qry.l)
224     {
225         L--;
226         toggle(tour[L]);
227     }
228     while (R < qry.r)
229     {
230         R++;
231         toggle(tour[R]);
232     }
233     while (L < qry.l)
234     {
235         toggle(tour[L]);
236         L++;
237     }
238     while (R > qry.r)
239     {
240         toggle(tour[R]);
241         R--;
242     }
243
244     // If U and V were on different branches, the LCA itself was NOT included
245     // in the [tout[U], tin[V]] range. We must temporarily add it!
246     if (qry.lca != -1)
247         toggle(qry.lca);
248
249     // Record the answer
250     ans[qry.id] = distinct_colors;
251
252     // Revert the temporary LCA addition to keep the L/R pointers clean
253     if (qry.lca != -1)
254         toggle(qry.lca);
255 }
256
257 for (int i = 0; i < q; i++)
258 {
259     cout << ans[i] << "\n";
260 }
261 }
262
263 int main()
264 {
265     ios_base::sync_with_stdio(false);
266     cin.tie(NULL);
267     solve();
268     return 0;
269 }

```

2.7.3 Mo's with updates

1 /**

```

2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 3D MO'S ALGORITHM (WITH UPDATES)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Point Updates", "Range Queries", "Offline", "Distinct elements
    with updates".
6  * - Classic Scenarios: You are given an array of size N. You have two types of
    queries:
7  *   Type 1: Query the number of distinct elements in range [L, R].
8  *   Type 2: Change the element at index X to V.
9  * - The Magic: A standard Segment Tree cannot count "Distinct Elements".
    Standard Mo's
10 *   Algorithm cannot handle "Updates". 3D Mo combines them by adding a "Time"
    pointer (T).
11 *   Queries are sorted into 3D blocks: [L_block, R_block, Time].
12 *
13 * 2. HOW TO USE IT (THE BLACK BOX)
14 * - Initialization:
15 *   As you read the input, split it! If it's an update, push to the `updates`
    array.
16 *   If it's a query, push to the `queries` array and record the CURRENT size of
    the
17 *   `updates` array as its "Time" stamp!
18 *
19 * - Complexity:
20 *   Time:  $O(N^{\frac{5}{3}})$  optimally.
21 *   Space:  $O(N + Q)$  memory.
22 *
23 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
24 * - The Block Size (B):
25 *   Standard Mo uses  $B = N^{\frac{1}{2}}$ . 3D Mo MUST use  $B = N^{\frac{2}{3}}$ .
26 *   If  $N = 100,000$ ,  $N^{\frac{2}{3}}$  is roughly 2154. The template calculates this
    dynamically!
27 * - Coordinate Compression:
28 *   This template assumes array values are  $\leq 10^6$ . If values are up to  $10^9$ ,
    you MUST
29 *   coordinate compress the initial array AND all the update values before
    running Mo!
30 */
31
32 #include <bits/stdc++.h>
33 using namespace std;
34
35 const int MAXN = 200005;
36 const int MAX_VAL = 1000005; // Maximum possible value in the array
37
38 int n, q;
39 int a[MAXN]; // The active array
40 int freq[MAX_VAL]; // Frequency array for distinct element counting
41 int current_ans = 0; // Tracks the current number of distinct elements
42 int BLOCK_SIZE;
43
44 struct Update
45 {
46     int idx;
47     int val; // When an update is applied, this holds the NEW value.
48
49     // When reverted, it automatically swaps to hold the OLD value!
50 };
51 struct Query
52 {
53     int l, r, t, id;
54     int l_block, r_block;
55
56     // Sort by L block, then R block, then Time.
57     // Includes a 3D parity optimization for faster pointer movement!
58     bool operator<(const Query &other) const
59     {
60         if (l_block != other.l_block)
61             return l_block < other.l_block;
62
63         if (r_block != other.r_block)
64             return (l_block & 1) ? r_block < other.r_block : r_block > other.
                r_block;
65
66         return (r_block & 1) ? t < other.t : t > other.t;
67     }
68 };
69
70 vector<Update> updates;
71 vector<Query> queries;
72 vector<int> ans;
73
74 // Add an element at index 'idx' to our active window
75 inline void add(int idx)
76 {
77     if (freq[a[idx]] == 0)
78     {
79         current_ans++;
80     }
81     freq[a[idx]]++;
82 }
83
84 // Remove an element at index 'idx' from our active window
85 inline void remove(int idx)
86 {
87     freq[a[idx]]--;
88     if (freq[a[idx]] == 0)
89     {
90         current_ans--;
91     }
92 }
93
94 // ? THE MAGIC: The Swap Trick
95 // If we move forward in time, we swap the current array value with the update's
    value.
96 // Because we swap, the `updates[t_idx].val` now holds the OLD value!
97 // This means if we need to travel backward in time, we just call the exact same
    function!

```

```

100 inline void apply_time(int t_idx, int L, int R)
101 {
102     int idx = updates[t_idx].idx;
103
104     // If the index being updated is currently INSIDE our active [L, R] window,
105     // we must remove its contribution using the old value.
106     if (L <= idx && idx <= R)
107     {
108         remove(idx);
109     }
110
111     // Swap the array value and the update cache!
112     swap(a[idx], updates[t_idx].val);
113
114     // If the index is in our active window, add its contribution using the new
115     // value!
116     if (L <= idx && idx <= R)
117     {
118         add(idx);
119     }
120 }
121 void solve()
122 {
123     if (!(cin >> n >> q))
124         return;
125
126     for (int i = 1; i <= n; i++)
127     {
128         cin >> a[i];
129     }
130
131     // 1-based indexing for updates is safer for the Time pointer
132     updates.push_back({0, 0}); // Dummy update at T = 0
133
134     // Parse queries and updates
135     for (int i = 0; i < q; i++)
136     {
137         int type;
138         cin >> type;
139         if (type == 1)
140         {
141             // Type 1: Range Query (L, R)
142             int l, r;
143             cin >> l >> r;
144             queries.push_back({l, r, (int)updates.size() - 1, (int)queries.size()
145                             , 0, 0});
146         }
147         else
148         {
149             // Type 2: Point Update (Index X becomes Value V)
150             int idx, val;
151             cin >> idx >> val;
152             updates.push_back({idx, val});
153         }
154     }
155
156     // ? THE MATH: Optimal block size for 3D Mo is  $N^{(2/3)}$ 
157     BLOCK_SIZE = max(1, (int)pow(n, 2.0 / 3.0));
158
159     for (auto &qry : queries)
160     {
161         qry.l_block = qry.l / BLOCK_SIZE;
162         qry.r_block = qry.r / BLOCK_SIZE;
163     }
164
165     sort(queries.begin(), queries.end());
166     ans.resize(queries.size());
167
168     int L = 1, R = 0, T = 0;
169
170     for (const auto &qry : queries)
171     {
172         // Step 1: Expand the spatial window [L, R]
173         while (L > qry.l)
174         {
175             L--;
176             add(L);
177         }
178         while (R < qry.r)
179         {
180             R++;
181             add(R);
182         }
183
184         // Step 2: Shrink the spatial window [L, R]
185         while (L < qry.l)
186         {
187             remove(L);
188             L++;
189         }
190         while (R > qry.r)
191         {
192             remove(R);
193             R--;
194         }
195
196         // Step 3: Fast-forward or Rewind the Time window (T)
197         // We do Time AFTER Space to ensure the active window bounds are correct
198         // for the `apply_time` intersection check!
199         while (T < qry.t)
200         {
201             T++;
202             apply_time(T, L, R);
203         }
204         while (T > qry.t)
205         {
206             apply_time(T, L, R);
207             T--;
208         }
209     }

```

```

208
209     // Record the answer for this specific snapshot in space and time
210     ans[qry.id] = current_ans;
211 }
212
213 for (int i = 0; i < ans.size(); i++)
214 {
215     cout << ans[i] << "\n";
216 }
217 }
218
219 int main()
220 {
221     // Fast I/O is mandatory due to heavy pointer manipulation
222     ios_base::sync_with_stdio(false);
223     cin.tie(NULL);
224     solve();
225     return 0;
226 }

```

2.7.4 Rollback mo

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: ROLLBACK MO'S ALGORITHM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Range queries", "Add is easy, Remove is impossible", "DSU on
   ranges".
6  * - Classic Scenarios: You are given a graph with N edges. You receive queries
   asking:
7  * "If we only use edges from index L to R, how many connected components are
   there?"
8  * - The Magic: Standard Mo's algorithm requires you to slide both the L and R
   pointers
9  * by adding AND removing elements. But you cannot "remove" an edge from a
   standard DSU!
10 * Rollback Mo sorts the queries so that the R pointer ONLY moves forward (add
   only).
11 * The L pointer moves backward temporarily, and we "undo" those L moves using
   a stack
12 * history (Rollback) before moving to the next query. NO `remove()` function
   needed!
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Define your state: You need a data structure that supports `add()`, `
   save_state()`,
16 * and `rollback(state)`. (I have included a Rollback DSU in this template).
17 * - Read Queries: Push all queries into a vector and pass them to the Mo's
   runner.
18 *
19 * - Complexity:
20 * Time:  $O(Q * \sqrt{N} * \text{Cost(Add)})$ . If DSU, it's  $O(Q * \sqrt{N} * \log N)$ .
21 * Space:  $O(N + Q)$  memory.

```

```

22 *
23 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
24 * - The Block Size (B):
25 * The math for Mo's dictates the block size should be roughly  $N / \sqrt{Q}$ .
26 * If the problem has  $N = 10^5$ , setting `B = 350` or `B = 400` is the standard
   sweet spot.
27 * - Changing the Data Structure:
28 * If the problem asks for "Max frequency in range", you don't need DSU. You
   just need
29 * an array tracking frequencies, and a variable `max_freq`. To rollback, you
   just
30 * remember the old `max_freq` and revert the array.
31 */
32
33 #include <bits/stdc++.h>
34 using namespace std;
35
36 // 1. The Rollback Data Structure
37 struct RollbackDSU
38 {
39     vector<int> parent, sz;
40     int components;
41
42     // We store the history of changes: {node_u, node_v, previous_components}
43     struct Change
44     {
45         int u, v, comps;
46     };
47     stack<Change> history;
48
49     RollbackDSU(int n)
50     {
51         parent.resize(n + 1);
52         sz.assign(n + 1, 1);
53         iota(parent.begin(), parent.end(), 0);
54         components = n;
55     }
56
57     // ! CRITICAL: No Path Compression allowed! Only Union by Size.
58     int find(int i)
59     {
60         while (i != parent[i])
61             i = parent[i];
62         return i;
63     }
64
65     void unite(int u, int v)
66     {
67         int root_u = find(u);
68         int root_v = find(v);
69
70         if (root_u == root_v)
71         {
72             // Even if they are already connected, we push a dummy change to keep
73             // the stack height aligned with the number of `add()` calls.

```

```

74     history.push({-1, -1, components});
75     return;
76 }
77
78 if (sz[root_u] < sz[root_v])
79     swap(root_u, root_v);
80
81 // Record the exact state BEFORE we change it
82 history.push({root_u, root_v, components});
83
84 // Apply changes
85 parent[root_v] = root_u;
86 sz[root_u] += sz[root_v];
87 components--;
88 }
89
90 // Returns the current "timestamp" or "height" of the history stack
91 int save()
92 {
93     return history.size();
94 }
95
96 // Undos all operations until the stack shrinks back to `target_time`
97 void rollback(int target_time)
98 {
99     while (history.size() > target_time)
100     {
101         auto [u, v, old_comps] = history.top();
102         history.pop();
103
104         if (u != -1)
105         {
106             sz[u] -= sz[v];
107             parent[v] = v;
108         }
109         components = old_comps;
110     }
111 }
112
113 void reset(int n)
114 {
115     while (!history.empty())
116         history.pop();
117     for (int i = 1; i <= n; i++)
118     {
119         parent[i] = i;
120         sz[i] = 1;
121     }
122     components = n;
123 }
124 };
125
126 // 2. Query Structure and Mo's Sorting logic
127 int BLOCK_SIZE;
128

```

```

129 struct Query
130 {
131     int l, r, id;
132
133     // Sort queries by their Left Block, then by their Right Endpoint
134     bool operator<(const Query &other) const
135     {
136         int block_own = l / BLOCK_SIZE;
137         int block_other = other.l / BLOCK_SIZE;
138         if (block_own != block_other)
139         {
140             return block_own < block_other;
141         }
142         // If in the same block, sort R strictly ascending
143         return r < other.r;
144     }
145 };
146
147 struct Edge
148 {
149     int u, v;
150 };
151
152 void solve()
153 {
154     int n, m, q;
155     // N vertices, M edges, Q queries
156     if (!(cin >> n >> m >> q))
157         return;
158
159     // Standard block size for 10^5 is roughly 350
160     BLOCK_SIZE = max(1, (int)sqrt(m));
161
162     vector<Edge> edges(m + 1); // 1-indexed edges
163     for (int i = 1; i <= m; i++)
164     {
165         cin >> edges[i].u >> edges[i].v;
166     }
167
168     vector<Query> queries(q);
169     for (int i = 0; i < q; i++)
170     {
171         cin >> queries[i].l >> queries[i].r;
172         queries[i].id = i;
173     }
174
175     // 1. Sort the queries
176     sort(queries.begin(), queries.end());
177
178     vector<int> ans(q);
179     RollbackDSU dsu(n);
180
181     int last_block = -1;
182     int current_r = 0;
183

```

```

184 for (const Query &query : queries)
185 {
186     int current_block = query.l / BLOCK_SIZE;
187
188     // If we entered a brand new block, completely wipe the DSU clean!
189     if (current_block != last_block)
190     {
191         dsu.reset(n);
192         // Set our R pointer to the very edge of the current block
193         current_r = min(m, (current_block + 1) * BLOCK_SIZE);
194         last_block = current_block;
195     }
196
197     // SCENARIO 1: The query is incredibly tiny and fits entirely inside one
198     // block
199     // We answer it naively in  $O(\sqrt{N})$  and instantly roll it back.
200     if (query.r < (current_block + 1) * BLOCK_SIZE)
201     {
202         int time_stamp = dsu.save();
203         for (int i = query.l; i <= query.r; i++)
204         {
205             dsu.unite(edges[i].u, edges[i].v);
206         }
207         ans[query.id] = dsu.components;
208         dsu.rollback(time_stamp);
209         continue;
210     }
211
212     // SCENARIO 2: The query stretches outside the block.
213     // Step A: Expand our R pointer forward until it hits the query's R limit.
214     // Notice we NEVER rollback these operations while staying in the same
215     // block!
216     while (current_r <= query.r)
217     {
218         dsu.unite(edges[current_r].u, edges[current_r].v);
219         current_r++;
220     }
221
222     // Step B: We temporarily drag the L pointer backward from the block edge
223     // to cover the left side of the query.
224     int time_stamp = dsu.save();
225     for (int i = query.l; i < (current_block + 1) * BLOCK_SIZE; i++)
226     {
227         dsu.unite(edges[i].u, edges[i].v);
228     }
229
230     // Step C: Record the answer!
231     ans[query.id] = dsu.components;
232
233     // Step D: Rollback the temporary L pointer movements.
234     // The R pointer progress remains saved for the next queries!
235     dsu.rollback(time_stamp);
236 }

```

```

236 for (int i = 0; i < q; i++)
237 {
238     cout << ans[i] << "\n";
239 }
240 }
241
242 int main()
243 {
244     // Fast I/O is mandatory for heavy Mo's algorithm problems
245     ios_base::sync_with_stdio(false);
246     cin.tie(NULL);
247     solve();
248     return 0;

```

2.7.5 sqrt decomposition

```

1
2 struct Sqrt
3 {
4     vector<int> arr, blk;
5     int SQ;
6     Sqrt(vector<int> &a, int sq = 450)
7     {
8         SQ = sq;
9         arr = a;
10        int n = arr.size();
11        blk.resize(n / sq + 5);
12        for (int i = 0; i < n; i++)
13        {
14            blk[i / sq] += arr[i];
15        }
16    }
17    void update(int idx, int val)
18    {
19        int blkNum = idx / SQ;
20        blk[blkNum] -= arr[idx];
21        arr[idx] = val;
22        blk[blkNum] += arr[idx];
23    }
24
25    int query(int r)
26    {
27        int res = 0;
28        int blkNum = r / SQ;
29        for (int i = 0; i < blkNum; i++)
30        {
31            res += blk[i];
32        }
33        for (int i = blkNum * SQ; i <= r; i++)
34        {
35            res += arr[i];
36        }

```

```

37     return res;
38 }
39
40 int query(int l, int r)
41 {
42     return query(r) - query(l - 1);
43 }
44 /*
45 int query(int l, int r) {
46     int res = oo;
47     while (l <= r) {
48         if (l % SQ == 0 && l + SQ <= r) {
49             int blkNum = l / SQ;
50             // res = min(res, blk[blkNum]);
51             l += SQ;
52         }
53         else {
54             // res = min(res, arr[l]);
55             l++;
56         }
57     }
58     return res;
59 }
60 */
61 };

```

2.7.6 sqrt decomposition rangeUpdate rangeQuery

```

1 struct SqrtRangeUpdateRangeQuery
2 {
3     int n, S;
4     vector<long long> a, b, c; // b = lazy, c = sum of block
5
6     SqrtRangeUpdateRangeQuery(int n, const vector<long long> &arr)
7     {
8         this->n = n;
9         S = ceil(sqrt(n));
10        a = arr;
11        b.assign(S + 1, 0);
12        c.assign(S + 1, 0);
13        for (int i = 0; i < n; ++i)
14        {
15            c[i / S] += a[i];
16        }
17    }
18
19    // Helper to get block actual sum safely
20    long long get_block_sum(int k)
21    {
22        // Block sum = original sum + (lazy value * number of elements in block)
23        long long elements_count = min(n - k * S, S);
24        return c[k] + b[k] * elements_count;
25    }

```

```

26
27 // O(sqrt(N))
28 void add(int l, int r, long long delta)
29 {
30     int c_l = l / S, c_r = r / S;
31
32     if (c_l == c_r)
33     {
34         for (int i = l; i <= r; ++i)
35         {
36             a[i] += delta;
37             c[c_l] += delta;
38         }
39     }
40     else
41     {
42         for (int i = l, end = (c_l + 1) * S - 1; i <= end; ++i)
43         {
44             a[i] += delta;
45             c[c_l] += delta;
46         }
47         for (int k = c_l + 1; k <= c_r - 1; ++k)
48         {
49             b[k] += delta; // Lazy update
50         }
51         for (int i = c_r * S; i <= r; ++i)
52         {
53             a[i] += delta;
54             c[c_r] += delta;
55         }
56     }
57 }
58
59 // O(sqrt(N))
60 long long query(int l, int r)
61 {
62     long long sum = 0;
63     int c_l = l / S, c_r = r / S;
64
65     if (c_l == c_r)
66     {
67         for (int i = l; i <= r; ++i)
68             sum += a[i] + b[c_l];
69     }
70     else
71     {
72         for (int i = l, end = (c_l + 1) * S - 1; i <= end; ++i)
73             sum += a[i] + b[c_l];
74
75         for (int k = c_l + 1; k <= c_r - 1; ++k)
76             sum += get_block_sum(k); // Full block sum
77
78         for (int i = c_r * S; i <= r; ++i)
79             sum += a[i] + b[c_r];
80     }

```

```

81     return sum;
82 }
83 };

```

2.8 Segment Tree

2.8.1 2D segment tree

```

1  /**
2   * THE ULTIMATE CP TEMPLATE BLUEPRINT: 2D SEGMENT TREE
3   *
4   * 1. WHAT PROBLEM DOES THIS SOLVE?
5   * - Keywords: "Subgrid queries", "Rectangle Sum/Max/Min", "Update a single cell
6     in a grid".
7   * - Classic Scenarios: You are given an N x M matrix. You need to repeatedly
8     answer
9   * queries about a specific sub-rectangle (from top-left [R1, C1] to bottom-
10    right [R2, C2]).
11   * Between queries, you can update the value of any specific cell [R, C].
12   * - The Magic: A standard Segment Tree handles a 1D array. A 2D Segment Tree is
13     literally
14   * a "Segment Tree of Segment Trees". The outer tree splits the rows, and every
15     single
16   * node in the outer tree contains an inner tree that splits the columns for
17     that row-range.
18   *
19   * 2. HOW TO USE IT (THE BLACK BOX)
20   * - Initialization: Pass the row and column sizes to the constructor.
21   * Then, (optionally) build it using an existing 0-indexed 2D vector.
22   *     vector<vector<long long>> grid = {{1, 2}, {3, 4}};
23   *     SegTree2D st(2, 2);
24   *     st.build(grid);
25   *
26   * - Queries (All 0-indexed, Inclusive [r1, r2] and [c1, c2]):
27   *     // Query the sum of the subgrid from top-left (0, 0) to bottom-right (1
28     1)
29   *     long long total = st.query(0, 0, 1, 1);
30   *
31   * - Updates (All 0-indexed):
32   *     // Change the value at row R, column C to X
33   *     st.update(R, C, X);
34   *
35   * - Complexity:
36   *     Time: O(N * M) to build. O(log N * log M) per update and query.
37   *     Space: O(N * M) memory. (Uses exactly 16 * N * M elements).
38   *
39   * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
40   * - Changing from Range Sum to Range Min/Max?
41   * 1. Change the `NEUTRAL` variable to `1e18` (for Min) or `-1e18` (for Max).
42   * 2. Change the `combine()` function to return `min(a, b)` or `max(a, b)`.
43   *
44   * - Memory Limit Exceeded (MLE) Warning:
45   * If N = 1000 and M = 1000, this tree uses 16,000,000 long longs (~128 MB).

```

```

39  * This easily passes standard 256MB limits.
40  * BUT, if N, M > 2000, you will MLE. If that happens and the problem only asks
41  * for SUM,
42  * you must use a 2D Fenwick Tree (BIT) instead, which uses 1/16th of the
43  * memory!
44  */
45 #include <bits/stdc++.h>
46 using namespace std;
47 struct SegTree2D
48 {
49 private:
50     int n, m;
51     vector<vector<long long>> seg;
52
53     // ? Change these two dials to switch between Sum, Min, Max, GCD, etc.
54     const long long NEUTRAL = 0;
55
56     long long combine(long long a, long long b)
57     {
58         return a + b;
59     }
60
61     void build_y(int vx, int lx, int rx, int vy, int ly, int ry, const vector<
62     vector<long long>> &a)
63     {
64         if (ly == ry)
65         {
66             // We are at a leaf in the column tree.
67             if (lx == rx)
68             {
69                 // We are ALSO at a leaf in the row tree. Base case!
70                 seg[vx][vy] = a[lx][ly];
71             }
72             else
73             {
74                 // We are a combination of two row-segments at the SAME exact
75                 column.
76                 seg[vx][vy] = combine(seg[vx * 2][vy], seg[vx * 2 + 1][vy]);
77             }
78         }
79         else
80         {
81             int my = ly + (ry - ly) / 2;
82             build_y(vx, lx, rx, vy * 2, ly, my, a);
83             build_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, a);
84             // Combine the left and right column-halves for the current row-
85             segment
86             seg[vx][vy] = combine(seg[vx][vy * 2], seg[vx][vy * 2 + 1]);
87         }
88     }
89
90     void build_x(int vx, int lx, int rx, const vector<vector<long long>> &a)
91     {

```

```

89     if (lx != rx)
90     {
91         int mx = lx + (rx - lx) / 2;
92         build_x(vx * 2, lx, mx, a);
93         build_x(vx * 2 + 1, mx + 1, rx, a);
94     }
95     // Build the entire inner column-tree for the current row-segment
96     build_y(vx, lx, rx, 1, 0, m - 1, a);
97 }
98
99 void update_y(int vx, int lx, int rx, int vy, int ly, int ry, int x, int y,
100 long long val)
101 {
102     if (ly == ry)
103     {
104         if (lx == rx)
105         {
106             // Exact cell located!
107             seg[vx][vy] = val;
108         }
109         else
110         {
111             // Update the combined row-segment value for this specific column
112             seg[vx][vy] = combine(seg[vx * 2][vy], seg[vx * 2 + 1][vy]);
113         }
114     }
115     else
116     {
117         int my = ly + (ry - ly) / 2;
118         if (y <= my)
119         {
120             update_y(vx, lx, rx, vy * 2, ly, my, x, y, val);
121         }
122         else
123         {
124             update_y(vx, lx, rx, vy * 2 + 1, my + 1, ry, x, y, val);
125         }
126         seg[vx][vy] = combine(seg[vx][vy * 2], seg[vx][vy * 2 + 1]);
127     }
128 }
129
130 void update_x(int vx, int lx, int rx, int x, int y, long long val)
131 {
132     if (lx != rx)
133     {
134         int mx = lx + (rx - lx) / 2;
135         if (x <= mx)
136         {
137             update_x(vx * 2, lx, mx, x, y, val);
138         }
139         else
140         {
141             update_x(vx * 2 + 1, mx + 1, rx, x, y, val);
142         }
143     }
144 }
145
146 // Force the inner Y-tree to update itself using the new X data
147 update_y(vx, lx, rx, 1, 0, m - 1, x, y, val);
148 }
149
150 long long query_y(int vx, int vy, int ly, int ry, int qly, int qry)
151 {
152     if (qly > ry || qry < ly)
153         return NEUTRAL;
154     if (qly <= ly && ry <= qry)
155         return seg[vx][vy];
156
157     int my = ly + (ry - ly) / 2;
158     return combine(
159         query_y(vx, vy * 2, ly, my, qly, qry),
160         query_y(vx, vy * 2 + 1, my + 1, ry, qly, qry));
161 }
162
163 long long query_x(int vx, int lx, int rx, int qlx, int qrx, int qly, int qry)
164 {
165     if (qlx > rx || qrx < lx)
166         return NEUTRAL;
167
168     // If the row segment is fully inside our query, hand it off to the
169     column query!
170     if (qlx <= lx && rx <= qrx)
171     {
172         return query_y(vx, 1, 0, m - 1, qly, qry);
173     }
174
175     int mx = lx + (rx - lx) / 2;
176     return combine(
177         query_x(vx * 2, lx, mx, qlx, qrx, qly, qry),
178         query_x(vx * 2 + 1, mx + 1, rx, qlx, qrx, qly, qry));
179 }
180
181 public:
182     SegTree2D(int rows, int cols)
183     {
184         n = rows;
185         m = cols;
186         // Allocate 4N rows, each containing a 4M column segment tree
187         seg.assign(4 * n, vector<long long>(4 * m, NEUTRAL));
188     }
189
190     void build(const vector<vector<long long>> &grid)
191     {
192         build_x(1, 0, n - 1, grid);
193     }
194
195     // Update cell at (row, col) to a new exact value
196     void update(int row, int col, long long val)
197     {
198         update_x(1, 0, n - 1, row, col, val);
199     }

```

```

197 // Query inclusive rectangle from top-left (r1, c1) to bottom-right (r2, c2)
198 long long query(int r1, int c1, int r2, int c2)
199 {
200     return query_x(1, 0, n - 1, r1, r2, c1, c2);
201 }
202 };

```

2.8.2 Dynamic segment tree

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DYNAMIC SEGMENT TREE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Massive constraints", "10^9 or 10^18 indices", "Sparse updates",
6  *   "Avoid Coordinate Compression".
7  * - Classic Scenarios: You need a Segment Tree over a massive range of
8  *   coordinates (e.g., from -10^9 to 10^9).
9  *   A standard segment tree would need an array of size 4 * 10^9, which
10  *   guarantees a Memory Limit Exceeded (MLE) crash.
11  * - The Magic: Instead of creating all nodes at the start, we only create nodes
12  *   as we visit them*.
13  *   Initially, only the Root exists. When an update goes down the tree, it
14  *   allocates the exact  $O(\log(\text{Range}))$ 
15  *   children it needs to reach the bottom. Empty space simply doesn't exist in
16  *   memory!
17  *
18  * 2. HOW TO USE IT (THE BLACK BOX)
19  * - Initialization: Pass the absolute minimum and maximum possible coordinates
20  *   to the constructor.
21  *   // Create a tree covering the range [-10^9, 10^9]
22  *   DynamicSegTree st(-1e9, 1e9);
23  *
24  * - Updates (0-indexed point update):
25  *   // Add X to the value at coordinate `idx`
26  *   st.update(idx, X);
27  *
28  * - Queries (Inclusive [L, R]):
29  *   // Query the sum of elements from coordinate L to R
30  *   long long total = st.query(L, R);
31  *
32  * - Complexity:
33  *   Time:  $O(\log(\text{MAX\_VAL} - \text{MIN\_VAL}))$  per update and query
34  *   Space:  $O(Q \log(\text{MAX\_VAL} - \text{MIN\_VAL}))$ . (Creates ~30
35  *   nodes per update for 10^9 ranges).
36  *
37  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
38  * - Changing to Range Min/Max?
39  *   1. Change `Node::sum` to `val` and initialize to `1e18` (Min) or `-1e18` (Max).
40  *   2. Change the `NEUTRAL` variable to `1e18` or `-1e18`.
41  *   3. In `query()`, return `min/max` instead of `+`.
42  * - "My code crashes randomly during updates!":
43  *   Never use references (`Node& curr = seg[node]`) before calling `.push_back`
44  *   or `.Vectors reallocate memory when they
45  *   grow, which instantly invalidates all references to its elements, causing a
46  *   terrifying segmentation fault.
47  *   This template strictly uses indices (`seg[node]`) to completely avoid this
48  *   trap.
49  */
50 #include <bits/stdc++.h>
51 using namespace std;
52
53 struct DynamicSegTree
54 {
55 private:
56     struct Node
57     {
58         long long sum;
59         int lc; // Index of Left Child in the vector
60         int rc; // Index of Right Child in the vector
61
62         Node() : sum(0), lc(-1), rc(-1) {}
63     };
64
65     vector<Node> seg;
66     long long MIN_VAL, MAX_VAL;
67     const long long NEUTRAL = 0; // 0 for Sum. 1e18 for Min. -1e18 for Max.
68
69     // ? Creates children on-demand if they don't exist yet
70     void extend(int node)
71     {
72         if (seg[node].lc == -1)
73         {
74             seg[node].lc = seg.size();
75             seg.emplace_back(); // Safely allocates a new blank node
76         }
77         if (seg[node].rc == -1)
78         {
79             seg[node].rc = seg.size();
80             seg.emplace_back(); // Safely allocates a new blank node
81         }
82     }
83
84     void update(int node, long long lx, long long rx, long long idx, long long
85                 val)
86     {
87         if (lx == rx)
88         {
89             seg[node].sum += val; // Point addition
90             return;
91         }
92
93         // ! We are going deeper, so we MUST ensure the children exist
94         extend(node);
95
96         // ? This safe midpoint calculation gracefully handles negative ranges
97         long long mid = lx + (rx - lx) / 2;

```

```

88     if (idx <= mid)
89     {
90         update(seg[node].lc, lx, mid, idx, val);
91     }
92     else
93     {
94         update(seg[node].rc, mid + 1, rx, idx, val);
95     }
96
97     // Pull up the answer from the children
98     seg[node].sum = seg[seg[node].lc].sum + seg[seg[node].rc].sum;
99 }
100
101 long long query(int node, long long lx, long long rx, long long ql, long long
102 qr)
103 {
104     // ! If this branch doesn't exist, it means nothing was ever added here!
105     Return Neutral.
106     if (node == -1 || ql > rx || qr < lx)
107     {
108         return NEUTRAL;
109     }
110
111     // Total overlap
112     if (ql <= lx && rx <= qr)
113     {
114         return seg[node].sum;
115     }
116
117     long long mid = lx + (rx - lx) / 2;
118
119     // Partial overlap
120     long long left_ans = query(seg[node].lc, lx, mid, ql, qr);
121     long long right_ans = query(seg[node].rc, mid + 1, rx, ql, qr);
122
123     return left_ans + right_ans;
124 }
125
126 public:
127     DynamicSegTree(long long min_v, long long max_v)
128     {
129         MIN_VAL = min_v;
130         MAX_VAL = max_v;
131
132         // Pre-allocate some memory to minimize slow vector reallocations
133         seg.reserve(1000000);
134
135         // Create the root node at index 0
136         seg.emplace_back();
137
138     }
139
140     // Call this to add 'val' to coordinate 'idx'
141     void update(long long idx, long long val)
142     {
143         update(0, MIN_VAL, MAX_VAL, idx, val);

```

```

141     }
142
143     // Call this to get the sum of the range [l, r]
144     long long query(long long l, long long r)
145     {
146         return query(0, MIN_VAL, MAX_VAL, l, r);
147     }
148 };

```

2.8.3 Iterative segment tree

```

1  /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: ITERATIVE SEGMENT TREE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Point Updates", "Range Queries", "Strict Time Limits", "No Lazy
6  *   Propagation".
7  * - Classic Scenarios: You need a Segment Tree, but the time limit is extremely
8  *   tight (e.g., 1.0s for 5*10^5 queries),
9  *   or the memory limit is small. Recursive segment trees have overhead from
10  *   function calls and use 4*N memory.
11  * This iterative version uses EXACTLY 2*N memory and runs in a tight `while`
12  * loop, making it blindingly fast.
13  * - The Magic: The leaves of the tree are stored in indices `N` to `2N-1`. The
14  *   parents are stored in `1` to `N-1`.
15  * To find the parent of node `x`, you just do `x / 2` (or `x >> 1`).
16  * To find the sibling of node `x`, you just do `x ^ 1`.
17  *
18  * 2. HOW TO USE IT (THE BLACK BOX)
19  * - Initialization: Pass a 0-indexed vector to build the tree.
20  *   vector<long long> a = {1, 5, 2, 8, 3};
21  *   IterativeSegTree st(a);
22  *
23  * - Updates (0-indexed point SET):
24  *   // Replace the value at index `idx` with `X`
25  *   st.update(idx, X);
26  *
27  * - Queries (0-indexed, Inclusive [L, R]):
28  *   // Query the combined answer from index L to R
29  *   long long ans = st.query(L, R);
30  *
31  * - Complexity:
32  *   Time: O(N) to build. O(log N) per update and query (with a tiny constant
33  *   factor).
34  *   Space: O(N) memory. Uses exactly 2*N elements.
35  *
36  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
37  * - Changing from Range Sum to Min/Max/GCD?
38  *   1. Change the `NEUTRAL` variable to `1e18` (Min), `-1e18` (Max), or `0` (GCD).
39  *   2. Update the `combine()` function to return `min(a, b)`, `max(a, b)`, or `gcd(a, b)`.
40  * - Changing `update` from "Set" to "Add"?

```

```

35  *   In `update()`, change `seg[idx += n] = val;` to `seg[idx += n] += val;`.
36  */
37
38  #include <bits/stdc++.h>
39  using namespace std;
40
41  struct IterativeSegTree
42  {
43  private:
44      int n;
45      vector<long long> seg;
46
47      // ? Dial 1: The Neutral Element (Identity)
48      // 0 for Sum, 1e18 for Min, -1e18 for Max, 0 for GCD
49      const long long NEUTRAL = 0;
50
51      // ? Dial 2: The Combination Logic
52      long long combine(long long a, long long b)
53      {
54          return a + b;
55      }
56
57  public:
58      IterativeSegTree(const vector<long long> &v)
59      {
60          n = v.size();
61          // Allocate exactly 2 * N memory. The 0-th index is unused dummy space.
62          seg.assign(2 * n, NEUTRAL);
63
64          // 1. Insert the leaf nodes directly into the second half of the array
65          for (int i = 0; i < n; i++)
66          {
67              seg[n + i] = v[i];
68          }
69
70          // 2. Build the parents from bottom to top
71          for (int i = n - 1; i > 0; i--)
72          {
73              // A node `i` has children at `2*i` and `2*i + 1`
74              seg[i] = combine(seg[i << 1], seg[i << 1 | 1]);
75          }
76      }
77
78      // Replace the value at 0-indexed position `idx` with `val`
79      void update(int idx, long long val)
80      {
81          // Jump straight to the leaf node
82          idx += n;
83          seg[idx] = val; // ! Change to += if problem requires Point Add instead
                        // of Point Set
84
85          // Climb to the root, updating parents
86          // `idx > 1` stops us before we update the unused 0-th index
87          for (idx >= 1; idx > 0; idx >= 1)
88          {
89              // `idx << 1` is left child, `idx << 1 | 1` is right child
90              seg[idx] = combine(seg[idx << 1], seg[idx << 1 | 1]);
91          }
92      }
93
94      // Query the inclusive range [l, r]
95      long long query(int l, int r)
96      {
97          long long res_left = NEUTRAL;
98          long long res_right = NEUTRAL;
99
100         // Jump straight to the leaves
101         // Note: We use inclusive [l, r] so our loop condition is l <= r
102         for (l += n, r += n; l <= r; l >>= 1, r >>= 1)
103         {
104             // ? If `l` is odd, it is a right child.
105             // This means its parent covers data OUTSIDE our [l, r] range!
106             // We must process this node independently, then move `l` to the
107             // right.
108             if (l & 1)
109             {
110                 res_left = combine(res_left, seg[l++]);
111             }
112
113             // ? If `r` is even, it is a left child.
114             // This means its parent covers data OUTSIDE our [l, r] range!
115             // We must process this node independently, then move `r` to the left
116             if (!(r & 1))
117             {
118                 res_right = combine(seg[r--], res_right);
119             }
120         }
121
122         // Combine the results from the left sweep and the right sweep.
123         // Doing it this way preserves the strict Left-to-Right order for non-
124         // commutative operations!
125         return combine(res_left, res_right);
126     };
127
128     void solve()
129     {
130         int n, q;
131         if (!(cin >> n >> q))
132             return;
133
134         vector<long long> a(n);
135         for (int i = 0; i < n; i++)
136             cin >> a[i];
137
138         IterativeSegTree st(a);
139
140         while (q--)

```

```

141 {
142     int type;
143     cin >> type;
144     if (type == 1)
145     {
146         int idx;
147         long long val;
148         cin >> idx >> val;
149         st.update(idx, val); // 0-indexed update
150     }
151     else
152     {
153         int l, r;
154         cin >> l >> r;
155         cout << st.query(l, r) << "\n"; // 0-indexed query
156     }
157 }
158 }
159
160 int main()
161 {
162     // Standard fast I/O
163     ios_base::sync_with_stdio(false);
164     cin.tie(NULL);
165     solve();
166     return 0;
167 }

```

2.8.4 Merge sort tree

```

1 struct Node
2 {
3     vector<int> v;
4 };
5
6 struct segTree
7 {
8     vector<Node> seg;
9     int sz;
10
11     segTree(int n)
12     {
13         sz = 1;
14         while (sz < n)
15             sz *= 2;
16         seg.assign(2 * sz, Node());
17     }
18
19     Node merge(Node &l, Node &r)
20     {
21         Node res;
22         res.v.reserve(l.v.size() + r.v.size());
23         std::merge(all(l.v), all(r.v), back_inserter(res.v));

```

```

24         return res;
25     }
26
27     void build(vector<int> &arr, int ni, int lx, int rx)
28     {
29         if (rx - lx == 1)
30         {
31             if (lx < (int)arr.size())
32             {
33                 seg[ni].v = {arr[lx]};
34             }
35             return;
36         }
37         int mid = (lx + rx) / 2;
38         build(arr, ni * 2 + 1, lx, mid);
39         build(arr, ni * 2 + 2, mid, rx);
40         seg[ni] = merge(seg[ni * 2 + 1], seg[ni * 2 + 2]);
41     }
42
43     void build(vector<int> &arr)
44     {
45         build(arr, 0, 0, sz);
46     }
47
48     // 1. Elements Strictly Less Than K (< K)
49     int query_less(int l, int r, int k, int ni, int lx, int rx)
50     {
51         if (lx >= l && rx <= r)
52         {
53             return lower_bound(all(seg[ni].v), k) - seg[ni].v.begin();
54         }
55         if (lx >= r || rx <= l)
56             return 0;
57         int mid = (lx + rx) / 2;
58         return query_less(l, r, k, ni * 2 + 1, lx, mid) + query_less(l, r, k, ni
59                               * 2 + 2, mid, rx);
60     }
61
62     // 2. Elements Less Than or Equal K (<= K)
63     int query_less_equal(int l, int r, int k, int ni, int lx, int rx)
64     {
65         if (lx >= l && rx <= r)
66         {
67             return upper_bound(all(seg[ni].v), k) - seg[ni].v.begin();
68         }
69         if (lx >= r || rx <= l)
70             return 0;
71         int mid = (lx + rx) / 2;
72         return query_less_equal(l, r, k, ni * 2 + 1, lx, mid) + query_less_equal(
73                               l, r, k, ni * 2 + 2, mid, rx);
74     }
75
76     // 3. Elements Strictly Greater Than K (> K)
77     int query_greater(int l, int r, int k, int ni, int lx, int rx)
78     {

```

```

77     if (lx >= 1 && rx <= r)
78     {
79         return seg[ni].v.end() - upper_bound(all(seg[ni].v), k);
80     }
81     if (lx >= r || rx <= 1)
82         return 0;
83     int mid = (lx + rx) / 2;
84     return query_greater(l, r, k, ni * 2 + 1, lx, mid) + query_greater(l, r,
        k, ni * 2 + 2, mid, rx);
85 }
86
87 // 4. Elements Greater Than or Equal K (>= K)
88 int query_greater_equal(int l, int r, int k, int ni, int lx, int rx)
89 {
90     if (lx >= 1 && rx <= r)
91     {
92         return seg[ni].v.end() - lower_bound(all(seg[ni].v), k);
93     }
94     if (lx >= r || rx <= 1)
95         return 0;
96     int mid = (lx + rx) / 2;
97     return query_greater_equal(l, r, k, ni * 2 + 1, lx, mid) +
        query_greater_equal(l, r, k, ni * 2 + 2, mid, rx);
98 }
99 // 5. Elements in Range [X, Y]
100 int query_in_range(int l, int r, int x, int y)
101 {
102     // Total elements <= Y MINUS Total elements < X
103     return get_less_equal(l, r, y) - get_less(l, r, x);
104 }
105
106 // Helpers
107 int get_greater(int l, int r, int k) { return query_greater(l, r, k, 0, 0, sz); }
108 int get_greater_equal(int l, int r, int k) { return query_greater_equal(l, r, k, 0, 0, sz); }
109 int get_less(int l, int r, int k) { return query_less(l, r, k, 0, 0, sz); }
110 int get_less_equal(int l, int r, int k)
111 {
112     return query_less_equal(l, r, k, 0, 0, sz);
113 }
114 };

```

2.8.5 PST

```

1 // Distinct elements in range
2 // kth element
3 // counting or frequency of numbers in range
4
5 struct PST
6 {
7     struct Node
8     {

```

```

9         int l = 0;
10        int r = 0;
11        int val = 0;
12    };
13
14    vector<Node> tree;
15    vector<int> roots;
16    int n;
17    int neutral;
18
19    PST(int n, int neutral = 0) : n(n), neutral(neutral)
20    {
21        tree.reserve(1500000);
22        tree.push_back({0, 0, neutral});
23        roots.push_back(0);
24    }
25
26    int merge(int a, int b)
27    {
28        return a + b;
29    }
30
31    int build(const vector<int> &a, int lx, int rx)
32    {
33        int u = tree.size();
34        tree.push_back({0, 0, neutral});
35
36        if (rx - lx == 1)
37        {
38            if (lx < (int)a.size())
39                tree[u].val = a[lx];
40            return u;
41        }
42
43        int mid = lx + (rx - lx) / 2;
44        tree[u].l = build(a, lx, mid);
45        tree[u].r = build(a, mid, rx);
46        tree[u].val = merge(tree[tree[u].l].val, tree[tree[u].r].val);
47        return u;
48    }
49
50    void build_initial(const vector<int> &a)
51    {
52        roots.push_back(build(a, 0, n));
53    }
54
55    int update(int node, int idx, int val, int lx, int rx)
56    {
57        int u = tree.size();
58        tree.push_back(tree[node]);
59
60        if (rx - lx == 1)
61        {
62            tree[u].val += val;
63            return u;

```

```

64     }
65
66     int mid = lx + (rx - lx) / 2;
67     if (idx < mid)
68     {
69         int nxt = update(tree[node].l, idx, val, lx, mid);
70         tree[u].l = nxt;
71     }
72     else
73     {
74         int nxt = update(tree[node].r, idx, val, mid, rx);
75         tree[u].r = nxt;
76     }
77
78     tree[u].val = merge(tree[tree[u].l].val, tree[tree[u].r].val);
79     return u;
80 }
81
82 int update(int version, int idx, int val)
83 {
84     int new_root = update(roots[version], idx, val, 0, n);
85     roots.push_back(new_root);
86     return (int)roots.size() - 1;
87 }
88
89 int query(int node, int l, int r, int lx, int rx)
90 {
91     if (node == 0)
92         return neutral;
93     if (lx >= r || rx <= l)
94         return neutral;
95     if (lx >= l && rx <= r)
96         return tree[node].val;
97
98     int mid = lx + (rx - lx) / 2;
99     return merge(query(tree[node].l, l, r, lx, mid),
100                 query(tree[node].r, l, r, mid, rx));
101 }
102
103 int query(int version, int l, int r)
104 {
105     return query(roots[version], l, r, 0, n);
106 }
107
108 int findKth(int l, int r, int k, int lx, int rx)
109 {
110     if (rx - lx == 1)
111         return lx;
112     int cnt = tree[tree[r].l].val - tree[tree[l].l].val;
113     int mid = lx + (rx - lx) / 2;
114     if (cnt >= k)
115         return findKth(tree[l].l, tree[r].l, k, lx, mid);
116     return findKth(tree[l].r, tree[r].r, k - cnt, mid, rx);
117 }
118

```

```

119 int findKth(int l, int r, int k)
120 {
121     return findKth(roots[l], roots[r], k, 0, n);
122 }
123 };

```

2.8.6 PST Lazy

```

1  /**
2  *  THE ULTIMATE CP TEMPLATE BLUEPRINT: PERSISTENT LAZY SEGMENT TREE
3  *
4  *  1.  WHAT PROBLEM DOES THIS SOLVE?
5  *  - Keywords: "Time travel", "Historical versions", "Range updates on previous
6  *  states".
7  *  - Classic Scenarios: You are given an array. You need to perform Range
8  *  Additions or Range Sets.
9  *  - Each update creates a new "version" of the array. You must be able to query
10 *  the Range Sum
11 *  of ANY past version of the array efficiently.
12 *  - The Magic: Instead of modifying nodes in place, we create a clone of every
13 *  node we touch.
14 *  This means `update(v, L, R)` leaves version `v` completely untouched and
15 *  returns the root ID
16 *  of the brand new version. Because we only change  $O(\log N)$  nodes per update,
17 *  memory and time
18 *  remain highly efficient!
19 *
20 *  2.  HOW TO USE IT (THE BLACK BOX)
21 *  - Initialization: Pass a 0-indexed vector<long long> to build Version 0.
22 *  vector<long long> a = {1, 5, 2, 8, 3};
23 *  PersistentLazySegTree pst(a);
24 *
25 *  - Tracking Versions: Every update returns a NEW root index. You should store
26 *  these!
27 *  vector<int> versions;
28 *  versions.push_back(pst.roots[0]); // Initial version
29 *
30 *  - Updates (All 0-indexed, Inclusive [L, R]):
31 *  // Add X to range [L, R] on a specific version, and save the new version
32 *  ID
33 *  int new_root = pst.updateAdd(versions.back(), L, R, X);
34 *  versions.push_back(new_root);
35 *
36 *  - Queries (All 0-indexed, Inclusive [L, R]):
37 *  // Query the sum of range [L, R] exactly as it was in version 'V'
38 *  long long sum = pst.querySum(versions[V], L, R);
39 *
40 *  - Complexity:
41 *  Time:  $O(N)$  to build.  $O(\log N)$  per update and query.
42 *  Space:  $O(N + Q \log N)$  memory.
43 *
44 *  3.  HOW TO ADAPT IT (THE DIALS & SWITCHES)
45 *  - Memory Limit Exceeded (MLE)? Persistent Lazy creates ~4 new nodes per update

```

```

38 * If N = 10^5 and Q = 10^5, you will generate roughly 10^5 + 400,000 nodes.
39 * The constructor already calls `seg.reserve(N * 40)` to prevent expensive
    vector reallocations.
40 * Increase this multiplier if Q is exceptionally large.
41 * - Changing from Range Add to Range Set?
42 * Range Add is special because we can just accumulate `lazy` tags during the
    query without
43 * ever modifying the tree. If you want Range Set, you must pass down a `
    lazySet` flag.
44 * In `query`, if you hit a node with an active `lazySet`, simply return `
    lazySet * (r - l + 1)`
45 * immediately, ignoring its children.
46 */
47
48 #include <bits/stdc++.h>
49 using namespace std;
50
51 struct PersistentLazySegTree
52 {
53 private:
54     struct Node
55     {
56         int lc, rc; // Indices of the left and right children
57         long long sum; // The range sum
58         long long lazy; // The lazy tag (Range Add)
59
60         Node() : lc(0), rc(0), sum(0), lazy(0) {}
61         Node(int lc, int rc, long long sum, long long lazy)
62             : lc(lc), rc(rc), sum(sum), lazy(lazy) {}
63     };
64
65     int n;
66     vector<Node> seg;
67
68     // ? Creates a brand new node that is an exact copy of the given node ID
69     int clone(int node)
70     {
71         seg.push_back(seg[node]);
72         return seg.size() - 1;
73     }
74
75     int build(int l, int r, const vector<long long> &a)
76     {
77         int curr = seg.size();
78         seg.emplace_back(); // Allocate new node
79
80         if (l == r)
81         {
82             if (l < (int)a.size())
83             {
84                 seg[curr].sum = a[l];
85             }
86             return curr;
87         }
88

```

```

89         int mid = 1 + (r - l) / 2;
90         int left_child = build(l, mid, a);
91         int right_child = build(mid + 1, r, a);
92
93         seg[curr].lc = left_child;
94         seg[curr].rc = right_child;
95         seg[curr].sum = seg[left_child].sum + seg[right_child].sum;
96
97         return curr;
98     }
99
100 int updateAdd(int node, int l, int r, int ql, int qr, long long val)
101 {
102     // ? Core Persistence Logic: NEVER modify the original node. Always work
        on a clone!
103     int curr = clone(node);
104
105     // 1. Total Overlap
106     if (ql <= l && r <= qr)
107     {
108         seg[curr].sum += val * (r - l + 1);
109         seg[curr].lazy += val;
110         return curr;
111     }
112
113     int mid = 1 + (r - l) / 2;
114
115     // ? Lazy Propagation in Persistence:
116     // Before we branch down, we must push the current node's lazy tag to its
        children.
117     // Because pushing modifies the children, we MUST clone the children
        first!
118     if (seg[curr].lazy != 0)
119     {
120         if (l != r)
121         { // Don't push down if it's a leaf
122             int left_child = clone(seg[curr].lc);
123             seg[left_child].sum += seg[curr].lazy * (mid - l + 1);
124             seg[left_child].lazy += seg[curr].lazy;
125             seg[curr].lc = left_child;
126
127             int right_child = clone(seg[curr].rc);
128             seg[right_child].sum += seg[curr].lazy * (r - mid);
129             seg[right_child].lazy += seg[curr].lazy;
130             seg[curr].rc = right_child;
131         }
132         seg[curr].lazy = 0; // Cleared from the current node
133     }
134
135     // 2. Partial Overlap (Recursion)
136     if (ql <= mid)
137     {
138         seg[curr].lc = updateAdd(seg[curr].lc, l, mid, ql, qr, val);
139     }
140     if (qr > mid)

```

```

141     {
142         seg[curr].rc = updateAdd(seg[curr].rc, mid + 1, r, ql, qr, val);
143     }
144
145     // 3. Update Current Node Sum
146     seg[curr].sum = seg[seg[curr].lc].sum + seg[seg[curr].rc].sum;
147     return curr;
148 }
149
150 // ? IMPORTANT: We CANNOT push down lazy tags during a query. Pushing clones
151 // which would cause a read-only query to consume massive amounts of memory
152 // Instead, we just pass the accumulated lazy value down the recursion.
153 long long querySum(int node, int l, int r, int ql, int qr, long long lazy_acc)
154 {
155     if (!node || ql > r || qr < l)
156         return 0;
157
158     // 1. Total Overlap
159     if (ql <= l && r <= qr)
160     {
161         // Apply all the pending lazy additions from ancestors to this block
162         return seg[node].sum + lazy_acc * (r - l + 1);
163     }
164
165     int mid = l + (r - l) / 2;
166
167     // 2. Add the CURRENT node's lazy tag to the accumulator for its children
168     long long current_lazy = lazy_acc + seg[node].lazy;
169
170     // 3. Partial Overlap
171     long long res = 0;
172     if (ql <= mid)
173     {
174         res += querySum(seg[node].lc, l, mid, ql, qr, current_lazy);
175     }
176     if (qr > mid)
177     {
178         res += querySum(seg[node].rc, mid + 1, r, ql, qr, current_lazy);
179     }
180
181     return res;
182 }
183
184 public:
185     // Keeps track of the root node index for every version
186     vector<int> roots;
187
188     PersistentLazySegTree(const vector<long long> &a)
189     {
190         n = a.size();
191
192         // Pre-allocate memory to prevent expensive vector resizing.
193         // Formula:  $N + (Queries * 4 * \log_2(N))$ . 40 is a safe constant for  $10^5$ .
194         seg.reserve(n * 40);
195
196         // Create a dummy node 0 (Null Node)
197         seg.emplace_back();
198
199         // Build Version 0
200         roots.push_back(build(0, n - 1, a));
201     }
202
203     // Pass the root ID of the version you want to update, get the new root ID
204     back
205     int updateAdd(int root_version, int l, int r, long long val)
206     {
207         return updateAdd(root_version, 0, n - 1, l, r, val);
208     }
209
210     // Pass the root ID of the version you want to query
211     long long querySum(int root_version, int l, int r)
212     {
213         return querySum(root_version, 0, n - 1, l, r, 0LL);
214     }
215 };

```

2.8.7 Segment tree

```

1 struct Node
2 {
3     int val;
4
5     Node()
6     {
7         val = 0;
8     }
9
10    Node(int val) : val(val)
11    {
12    }
13
14    void change(int x)
15    {
16        val = x;
17    }
18 };
19
20 struct segmentTree
21 {
22     int sz;
23     vector<Node> seg;
24
25     segmentTree(int n)
26     {
27         sz = 1;
28         while (sz < n)

```

```

29     sz *= 2;
30     seg.assign(2 * sz, Node());
31 }
32
33 void init(vector<int> &arr, int node, int lx, int rx)
34 {
35     if (rx - lx == 1)
36     {
37         if (lx < int(arr.size()))
38             seg[node] = Node(arr[lx]);
39         return;
40     }
41     int mid = (lx + rx) / 2;
42     init(arr, 2 * node + 1, lx, mid);
43     init(arr, 2 * node + 2, mid, rx);
44     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
45 }
46
47 void init(vector<int> &arr)
48 {
49     init(arr, 0, 0, sz);
50 }
51
52 Node merge(Node &lf, Node &rt)
53 {
54     Node ans = Node();
55     ans.val = lf.val + rt.val;
56     return ans;
57 }
58
59 void set(int idx, int val, int node, int lx, int rx)
60 {
61     if (rx - lx == 1)
62     {
63         seg[node].change(val);
64         return;
65     }
66     int mid = (lx + rx) / 2;
67     if (idx < mid)
68         set(idx, val, 2 * node + 1, lx, mid);
69     else
70         set(idx, val, 2 * node + 2, mid, rx);
71     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
72 }
73
74 void set(int idx, int val)
75 {
76     set(idx, val, 0, 0, sz);
77 }
78
79 Node get(int l, int r, int node, int lx, int rx)
80 {
81     if (lx >= l && rx <= r)
82         return seg[node];
83     if (rx <= l || lx >= r)

```

```

84         return Node();
85     int mid = (lx + rx) / 2;
86     Node lf = get(l, r, 2 * node + 1, lx, mid);
87     Node ri = get(l, r, 2 * node + 2, mid, rx);
88     return merge(lf, ri);
89 }
90
91 int get(int l, int r)
92 {
93     return get(l, r, 0, 0, sz).val;
94 }
95 };

```

2.8.8 Segment tree beats

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SEGMENT TREE BEATS
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Range chmin" ( $A[i] = \min(A[i], X)$ ), "Range chmax", "Range Sum
   with Min/Max updates".
6  * - Classic Scenarios: Standard Segment Trees completely fail if you try to
   perform a range minimum
7  * update (e.g., "for all elements from L to R, replace the element with min(
   element, X)")
8  * while simultaneously asking for the Range Sum.
9  * - The Magic: Segment Tree Beats tracks the *First Maximum* and *Second Maximum
   * in every node.
10 * If your update X is between the First and Second Max, it only affects the
   First Max!
11 * It uses this trick to update the sum mathematically in amortized  $O(\log^2 N)$ 
   time.
12 *
13 * 2. HOW TO USE IT (THE BLACK BOX)
14 * - Initialization: Pass a 0-indexed vector<long long> to the constructor.
15 *     vector<long long> a = {1, 5, 2, 8, 3};
16 *     SegmentTreeBeats seg(a);
17 *
18 * - Queries (All 0-indexed, Inclusive [L, R]):
19 *     seg.updateMin(L, R, X) : Performs  $A[i] = \min(A[i], X)$  for i in [L, R].
20 *     seg.updateMax(L, R, X) : Performs  $A[i] = \max(A[i], X)$  for i in [L, R].
21 *     seg.updateAdd(L, R, X) : Performs  $A[i] = A[i] + X$  for i in [L, R].
22 *     seg.updateSet(L, R, X) : Performs  $A[i] = X$  for i in [L, R].
23 *     seg.querySum(L, R)    : Returns the sum of  $A[i]$  in [L, R].
24 *     seg.queryMin(L, R)    : Returns the Minimum in [L, R].
25 *     seg.queryMax(L, R)    : Returns the Maximum in [L, R].
26 *
27 * - Complexity:
28 *     Time:  $O(N)$  to build. Amortized  $O(\log^2 N)$  per query.
29 *     Space:  $O(N)$  memory.
30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - 1-based indexing? Instead of changing the recursive logic, simply add a
   dummy element `0`

```

```

33 *   to the front of your vector before initializing: `v.insert(v.begin(), 0);`.
34 *   Then query `[1, N]`.
35 * - Numbers larger than 1014? The `INF` value is set to 2 * 1015. If your
36 *   problem features numbers
37 *   larger than that, change the `const long long INF` variable at the top of
38 *   the struct.
39 * - Need more speed? If the problem ONLY asks for `updateMin` and `querySum`,
40 *   you can safely delete
41 *   all variables related to `mn`, `secMn`, and `mnCnt` to cut memory and
42 *   execution time in half.
43 */
44 #include <bits/stdc++.h>
45 using namespace std;
46 // ! Assumes you are using long long to avoid any overflow bugs.
47 struct SegmentTreeBeats
48 {
49 private:
50 // ! FIX: Added strict parentheses to prevent operator precedence bugs in math
51 #define L (2 * node + 1)
52 #define R (2 * node + 2)
53 #define mid ((l + r) >> 1)
54 // ! FIX: Safely typed INF to avoid macro expansion issues
55 const long long INF = 2e15 + 5;
56 struct Node
57 {
58     long long sum, mx, secMx, mxCnt, mn, mnCnt, secMn, lazyAdd, lazySet;
59     Node(long long summ, long long mxx, long long secMxx, long long mxCntt,
60         long long mnn, long long secMnn, long long mnCntt, long long add =
61         0, long long set = -1)
62     : sum(summ), mx(mxx), secMx(secMxx), mxCnt(mxCntt),
63       mn(mnn), secMn(secMnn), mnCnt(mnCntt),
64       lazyAdd(add), lazySet(set)
65     {}
66 };
67 int sz;
68 vector<Node> seg;
69 Node merge(Node &a, Node &b)
70 {
71     Node ret(0, -INF, -INF, 0, INF, INF, 0, 0, -1);
72     ret.sum = a.sum + b.sum;
73     // ? Merge Maximums
74     ret.mx = max(a.mx, b.mx);
75     ret.secMx = max(a.secMx, b.secMx);
76     if (ret.mx == a.mx)
77         ret.mxCnt += a.mxCnt;
78     else

```

```

79         ret.secMx = max(ret.secMx, a.mx);
80     if (ret.mx == b.mx)
81         ret.mxCnt += b.mxCnt;
82     else
83         ret.secMx = max(ret.secMx, b.mx);
84     // ? Merge Minimums
85     ret.mn = min(a.mn, b.mn);
86     ret.secMn = min(a.secMn, b.secMn);
87     if (ret.mn == a.mn)
88         ret.mnCnt += a.mnCnt;
89     else
90         ret.secMn = min(ret.secMn, a.mn);
91     if (ret.mn == b.mn)
92         ret.mnCnt += b.mnCnt;
93     else
94         ret.secMn = min(ret.secMn, b.mn);
95     return ret;
96 }
97 void pushSet(int l, int r, int node, long long val)
98 {
99     seg[node] = Node(val * (r - l + 1), val, -INF, r - l + 1, val, INF, r - l
100     + 1, 0, val);
101 }
102 void pushMin(int l, int r, int node, long long mn)
103 {
104     if (seg[node].mn >= mn)
105     {
106         pushSet(l, r, node, mn);
107         return;
108     }
109     if (seg[node].mx > mn)
110     {
111         if (seg[node].secMn == seg[node].mx)
112             seg[node].secMn = mn;
113         long long val = seg[node].mx - mn;
114         seg[node].sum -= val * seg[node].mxCnt;
115         seg[node].mx = mn;
116     }
117 }
118 void pushMax(int l, int r, int node, long long mx)
119 {
120     if (seg[node].mx <= mx)
121     {
122         pushSet(l, r, node, mx);
123         return;
124     }
125     if (seg[node].mn < mx)
126     {
127         if (seg[node].secMx == seg[node].mn)
128             seg[node].secMx = mx;

```

```

136     long long val = mx - seg[node].mn;
137     seg[node].sum += val * seg[node].mnCnt;
138     seg[node].mn = mx;
139 }
140 }
141
142 void pushAdd(int l, int r, int node, long long val)
143 {
144     if (seg[node].mx == seg[node].mn)
145     {
146         pushSet(l, r, node, seg[node].mn + val);
147         return;
148     }
149     seg[node].mx += val;
150     if (seg[node].secMx != -INF)
151         seg[node].secMx += val;
152     seg[node].mn += val;
153     if (seg[node].secMn != INF)
154         seg[node].secMn += val;
155     seg[node].sum += (r - l + 1) * val;
156     seg[node].lazyAdd += val;
157 }
158
159 void propagate(int l, int r, int node)
160 {
161     if (l == r)
162         return;
163     if (seg[node].lazySet != -1)
164     {
165         pushSet(l, mid, L, seg[node].lazySet);
166         pushSet(mid + 1, r, R, seg[node].lazySet);
167         seg[node].lazySet = -1;
168     }
169     else
170     {
171         pushAdd(l, mid, L, seg[node].lazyAdd);
172         pushAdd(mid + 1, r, R, seg[node].lazyAdd);
173         seg[node].lazyAdd = 0;
174         pushMin(l, mid, L, seg[node].mx);
175         pushMin(mid + 1, r, R, seg[node].mx);
176         pushMax(l, mid, L, seg[node].mn);
177         pushMax(mid + 1, r, R, seg[node].mn);
178     }
179 }
180
181 void build(int l, int r, int node, vector<long long> &v)
182 {
183     if (l == r)
184     {
185         if (l < (int)v.size())
186         {
187             seg[node] = Node(v[l], v[l], -INF, 1, v[l], INF, 1);
188         }
189         return;
190     }

```

```

191     build(l, mid, L, v);
192     build(mid + 1, r, R, v);
193     seg[node] = merge(seg[L], seg[R]);
194 }
195
196 void updateAdd(int l, int r, int node, int lx, int rx, long long val)
197 {
198     if (l > rx || r < lx)
199         return;
200     if (l >= lx && r <= rx)
201     {
202         pushAdd(l, r, node, val);
203         return;
204     }
205     propagate(l, r, node);
206     updateAdd(l, mid, L, lx, rx, val);
207     updateAdd(mid + 1, r, R, lx, rx, val);
208     seg[node] = merge(seg[L], seg[R]);
209 }
210
211 void updateSet(int l, int r, int node, int lx, int rx, long long val)
212 {
213     if (l > rx || r < lx)
214         return;
215     if (l >= lx && r <= rx)
216     {
217         pushSet(l, r, node, val);
218         return;
219     }
220     propagate(l, r, node);
221     updateSet(l, mid, L, lx, rx, val);
222     updateSet(mid + 1, r, R, lx, rx, val);
223     seg[node] = merge(seg[L], seg[R]);
224 }
225
226 void updateMin(int l, int r, int node, int lx, int rx, long long mn)
227 {
228     // ? Core BEATS logic: If the segment max is already <= mn, do nothing.
229     if (r < lx || l > rx || seg[node].mx <= mn)
230         return;
231     // ? If the update affects the mazimums, but NOT the second mazimums, we
232     can apply it safely!
233     if (l >= lx && r <= rx && mn > seg[node].secMx)
234     {
235         pushMin(l, r, node, mn);
236         return;
237     }
238     propagate(l, r, node);
239     updateMin(l, mid, L, lx, rx, mn);
240     updateMin(mid + 1, r, R, lx, rx, mn);
241     seg[node] = merge(seg[L], seg[R]);
242 }
243
244 void updateMax(int l, int r, int node, int lx, int rx, long long mx)
245 {

```

```

245     if (l > rx || r < lx || mx <= seg[node].mn)
246         return;
247     if (l >= lx && r <= rx && mx < seg[node].secMn)
248     {
249         pushMax(l, r, node, mx);
250         return;
251     }
252     propegate(l, r, node);
253     updateMax(l, mid, L, lx, rx, mx);
254     updateMax(mid + 1, r, R, lx, rx, mx);
255     seg[node] = merge(seg[L], seg[R]);
256 }
257
258 long long querySum(int l, int r, int node, int lx, int rx)
259 {
260     if (l > rx || r < lx)
261         return 0;
262     if (l >= lx && r <= rx)
263         return seg[node].sum;
264     propegate(l, r, node);
265     return querySum(l, mid, L, lx, rx) + querySum(mid + 1, r, R, lx, rx);
266 }
267
268 long long queryMin(int l, int r, int node, int lx, int rx)
269 {
270     if (l > rx || r < lx)
271         return INF;
272     if (l >= lx && r <= rx)
273         return seg[node].mn;
274     propegate(l, r, node);
275     return min(queryMin(l, mid, L, lx, rx), queryMin(mid + 1, r, R, lx, rx));
276 }
277
278 long long queryMax(int l, int r, int node, int lx, int rx)
279 {
280     if (l > rx || r < lx)
281         return -INF;
282     if (l >= lx && r <= rx)
283         return seg[node].mx;
284     propegate(l, r, node);
285     return max(queryMax(l, mid, L, lx, rx), queryMax(mid + 1, r, R, lx, rx));
286 }
287
288 #undef L
289 #undef R
290 #undef mid
291
292 public:
293     SegmentTreeBeats(vector<long long> &v)
294     {
295         int n = v.size() + 5;
296         sz = 1;
297         while (sz <= n)
298             sz <<= 1;
299         seg = vector<Node>(sz << 1, Node(0, -INF, -INF, 0, INF, INF, 0, 0, -1));

```

```

300         build(0, sz - 1, 0, v);
301     }
302
303     void updateAdd(int l, int r, long long val) { updateAdd(0, sz - 1, 0, l, r, val); }
304     void updateSet(int l, int r, long long val) { updateSet(0, sz - 1, 0, l, r, val); }
305     void updateMin(int l, int r, long long mn) { updateMin(0, sz - 1, 0, l, r, mn); }
306     void updateMax(int l, int r, long long mx) { updateMax(0, sz - 1, 0, l, r, mx); }
307     long long queryMax(int l, int r) { return queryMax(0, sz - 1, 0, l, r); }
308     long long queryMin(int l, int r) { return queryMin(0, sz - 1, 0, l, r); }
309     long long querySum(int l, int r) { return querySum(0, sz - 1, 0, l, r); }
310 };

```

2.8.9 Segment tree lazy

```

1 struct Node
2 {
3     int val, lazy;
4     bool is_lazy;
5
6     Node()
7     {
8         val = 0, lazy = 0, is_lazy = false;
9     }
10
11     Node(int val) : val(val)
12     {
13         lazy = 0, is_lazy = false;
14     }
15
16     void change(int x, int lx, int rx)
17     {
18         val += x * (rx - lx);
19         lazy += x;
20         is_lazy = true;
21     }
22
23     void reset()
24     {
25         lazy = 0, is_lazy = false;
26     }
27 };
28
29 struct segmentTree
30 {
31     int sz;
32     vector<Node> seg;
33
34     segmentTree(int n)
35     {

```

```

36     sz = 1;
37     while (sz < n)
38         sz *= 2;
39     seg.assign(2 * sz, Node());
40 }
41
42 void init(vector<int> &arr, int node, int lx, int rx)
43 {
44     if (rx - lx == 1)
45     {
46         if (lx < int(arr.size()))
47             seg[node] = Node(arr[lx]);
48         return;
49     }
50     int mid = (lx + rx) / 2;
51     init(arr, 2 * node + 1, lx, mid);
52     init(arr, 2 * node + 2, mid, rx);
53     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
54 }
55
56 void init(vector<int> &arr)
57 {
58     init(arr, 0, 0, sz);
59 }
60
61 Node merge(Node &lf, Node &rt)
62 {
63     Node ans = Node();
64     ans.val = lf.val + rt.val;
65     return ans;
66 }
67
68 void push(int node, int lx, int rx)
69 {
70     if (rx - lx == 1 || !seg[node].is_lazy)
71         return;
72     int mid = (lx + rx) / 2;
73     seg[2 * node + 1].change(seg[node].lazy, lx, mid);
74     seg[2 * node + 2].change(seg[node].lazy, mid, rx);
75     seg[node].reset();
76 }
77
78 void update(int l, int r, int x, int node, int lx, int rx)
79 {
80     if (lx >= r || rx <= l)
81         return;
82     push(node, lx, rx);
83     if (lx >= l && rx <= r)
84     {
85         seg[node].change(x, lx, rx);
86         return;
87     }
88     int mid = (lx + rx) / 2;
89     update(l, r, x, 2 * node + 1, lx, mid);
90     update(l, r, x, 2 * node + 2, mid, rx);

```

```

91
92     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
93 }
94
95 void update(int l, int r, int x)
96 {
97     return update(l, r, x, 0, 0, sz);
98 }
99
100 Node get(int l, int r, int node, int lx, int rx)
101 {
102     if (rx <= l || lx >= r)
103         return Node();
104     push(node, lx, rx);
105     if (lx >= l && rx <= r)
106         return seg[node];
107     int mid = (lx + rx) / 2;
108     Node lf = get(l, r, 2 * node + 1, lx, mid);
109     Node ri = get(l, r, 2 * node + 2, mid, rx);
110     return merge(lf, ri);
111 }
112
113 int get(int l, int r)
114 {
115     return get(l, r, 0, 0, sz).val;
116 }
117 };

```

2.8.10 Segment tree merging

```

1  /**
2  *  THE ULTIMATE CP TEMPLATE BLUEPRINT: SEGMENT TREE MERGING
3  *
4  *  1.  WHAT PROBLEM DOES THIS SOLVE?
5  *  - Keywords: "Subtree queries", "Colors in subtree", "Inversions in subtree", "
6  *  - Classic Scenarios: You are traversing a tree (or graph) bottom-up. Every
7  *  node needs
8  *  to track a frequency array or sums of its subtree.
9  *  - The Magic: Merging two Dynamic Segment Trees takes time proportional to
10 *  their
11 *  *intersecting* nodes. If you merge all children into their parents bottom-up
12 *  ,
13 *  the total time and memory for the entire process is mathematically bounded
14 *  to
15 *   $O(N \log(\text{Range}))$ . It feels like magic because  $10^5$  independent trees merge
16 *  into
17 *  one in a fraction of a second.
18 *
19 *  2.  HOW TO USE IT (THE BLACK BOX)
20 *  - Initialization:
21 *      MergeableSegTree st(-1e9, 1e9);
22 *      vector<int> roots(N + 1, -1); // Tracks the root ID for each node's
23 *      personal tree

```

```

18 *
19 * - Updates:
20 *     // Add 'val' at coordinate 'idx' for node U's personal tree
21 *     roots[U] = st.update(roots[U], idx, val);
22 *
23 * - Merging:
24 *     // Merge the tree of child V into the tree of parent U
25 *     roots[U] = st.merge(roots[U], roots[V]);
26 *
27 * - Queries:
28 *     // Query the sum in range [L, R] inside node U's tree
29 *     long long ans = st.query(roots[U], L, R);
30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - DESTRUCTIVE MERGE WARNING:
33 *     This template performs a "Destructive Merge". When you do `st.merge(roots[U]
34 *     ], roots[V])`,
35 *     the tree of V is absorbed into U. You CANNOT safely query `roots[V]` anymore!
36 *     This is perfectly fine for 99% of Tree DP problems (you process bottom-up
37 *     and don't look back).
38 *     If a problem forces you to query V again later, you must change the `merge`
39 *     function to
40 *     create a clone instead of modifying `u` in place (Warning: clones cause
41 *     massive memory usage).
42 * - Changing to Min/Max:
43 *     Change `seg[u].sum += seg[v].sum` to `= max(seg[u].max, seg[v].max)` in the
44 *     base case of `merge()`.
45 */
46
47 #include <bits/stdc++.h>
48 using namespace std;
49
50 struct MergeableSegTree
51 {
52 private:
53     struct Node
54     {
55         long long sum = 0;
56         int lc = -1; // Left child index
57         int rc = -1; // Right child index
58     };
59     vector<Node> seg;
60     long long MIN_VAL, MAX_VAL;
61
62     int allocate()
63     {
64         seg.emplace_back();
65         return seg.size() - 1;
66     }
67
68     int update(int node, long long lx, long long rx, long long idx, long long val)
69     {
70         // If the node doesn't exist yet, create it!
71         if (node == -1)
72             node = allocate();
73
74         if (lx == rx)
75         {
76             seg[node].sum += val;
77             return node;
78         }
79
80         long long mid = lx + (rx - lx) / 2;
81         if (idx <= mid)
82         {
83             seg[node].lc = update(seg[node].lc, lx, mid, idx, val);
84         }
85         else
86         {
87             seg[node].rc = update(seg[node].rc, mid + 1, rx, idx, val);
88         }
89
90         // Pull up answers from children
91         seg[node].sum = 0;
92         if (seg[node].lc != -1)
93             seg[node].sum += seg[seg[node].lc].sum;
94         if (seg[node].rc != -1)
95             seg[node].sum += seg[seg[node].rc].sum;
96
97         return node;
98     }
99
100     // ? THE MAGIC: The Destructive Merge Function
101     int merge(int u, int v, long long lx, long long rx)
102     {
103         // 1. If one of the branches is empty, just point to the other! (O(1))
104         shortcut
105         if (u == -1)
106             return v;
107         if (v == -1)
108             return u;
109
110         // 2. Base case: Both trees have data at this leaf. Combine them!
111         if (lx == rx)
112         {
113             seg[u].sum += seg[v].sum;
114             return u;
115         }
116
117         // 3. Recursive case: Traverse overlapping paths and merge children
118         long long mid = lx + (rx - lx) / 2;
119         seg[u].lc = merge(seg[u].lc, seg[v].lc, lx, mid);
120         seg[u].rc = merge(seg[u].rc, seg[v].rc, mid + 1, rx);
121
122         // Pull up the new combined sum
123         seg[u].sum = 0;
124         if (seg[u].lc != -1)

```

```

120     seg[u].sum += seg[seg[u].lc].sum;
121     if (seg[u].rc != -1)
122         seg[u].sum += seg[seg[u].rc].sum;
123
124     return u; // Return the absorbed parent node
125 }
126
127 long long query(int node, long long lx, long long rx, long long ql, long long
qr)
128 {
129     if (node == -1 || ql > rx || qr < lx)
130         return 0;
131
132     if (ql <= lx && rx <= qr)
133         return seg[node].sum;
134
135     long long mid = lx + (rx - lx) / 2;
136     return query(seg[node].lc, lx, mid, ql, qr) +
137            query(seg[node].rc, mid + 1, rx, ql, qr);
138 }
139
140 public:
141     MergeableSegTree(long long min_v, long long max_v)
142     {
143         MIN_VAL = min_v;
144         MAX_VAL = max_v;
145         // Pre-allocate to prevent slow resizing.
146         // Formula:  $N * \log(\text{Range})$ . Usually  $40 * N$  is safe for  $10^9$  ranges.
147         seg.reserve(4000000);
148     }
149
150     // Pass the root of the tree you want to update
151     int update(int root, long long idx, long long val)
152     {
153         return update(root, MIN_VAL, MAX_VAL, idx, val);
154     }
155
156     // Pass the roots of the two trees you want to merge
157     int merge(int root_u, int root_v)
158     {
159         return merge(root_u, root_v, MIN_VAL, MAX_VAL);
160     }
161
162     // Pass the root of the tree you want to query
163     long long query(int root, long long l, long long r)
164     {
165         return query(root, MIN_VAL, MAX_VAL, l, r);
166     }
167 };
168
169 void solve()
170 {
171     int n;
172     // ... Read tree structure ...
173     // vector<vector<int>>> adj(n + 1);

```

```

174
175     // Create the global Mergeable Segment Tree
176     MergeableSegTree st(1, 1e9);
177
178     // Array to track the root of the segment tree belonging to each node
179     vector<int> roots(n + 1, -1);
180
181     // Example: Add value '1' to coordinate 'color[u]' for node U's tree
182     // roots[u] = st.update(roots[u], color[u], 1);
183
184     // Example Tree DP logic:
185     // for (int v : adj[u]) {
186     //     dfs(v, u);
187     //     roots[u] = st.merge(roots[u], roots[v]);
188     // }
189
190     // Now roots[u] contains the fully merged frequency array of its entire
191     // subtree!
192 }
193
194 int main()
195 {
196     ios_base::sync_with_stdio(false);
197     cin.tie(NULL);
198     // solve();
199     return 0;

```

2.9 Sparse Table

2.9.1 2D sparse table

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 2D SPARSE TABLE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Static 2D Grid", "Maximum in subgrid", "Minimum in subgrid", "
6  *   Huge number of queries".
7  * - Classic Scenarios: You are given an  $N \times M$  grid of numbers. The grid NEVER
8  *   changes.
9  *   You need to answer  $Q$  queries (up to  $10^6$ ) asking for the max/min in a
10  *   rectangle bounded
11  *   by top-left  $(r1, c1)$  and bottom-right  $(r2, c2)$ .
12  * - The Magic: A 2D Segment tree answers queries in  $O(\log N * \log M)$ , which can
13  *   Time Limit Exceed
14  *   (TLE) if  $Q$  is massive. A 2D Sparse Table builds the answers for all powers
15  *   of 2 in advance,
16  *   allowing it to answer ANY rectangle query in absolute  $O(1)$  time by
17  *   overlapping 4 smaller rectangles!
18  *
19  * 2. HOW TO USE IT (THE BLACK BOX)
20  * - Initialization: Pass a 0-indexed 2D vector to the constructor.
21  *   vector<vector<int>>> grid = {{1, 5}, {2, 8}};

```

```

16 * SparseTable2D st(grid);
17 *
18 * - Queries (All 0-indexed, Inclusive [r1, r2] and [c1, c2]):
19 * // Query the max in the subgrid from top-left (0, 0) to bottom-right (1, 1)
20 * int max_val = st.query(0, 0, 1, 1);
21 *
22 * - Complexity:
23 * Time:  $O(N * M * \log N * \log M)$  to build. Pure  $O(1)$  per query.
24 * Space:  $O(N * M * \log N * \log M)$  memory.
25 *
26 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
27 * - Changing from Max to Min or GCD?
28 * 1. Change the `combine()` function to return `min(a, b)` or `std::gcd(a, b)`
29 * - Memory Limit Exceeded (MLE) Warning:
30 * If  $N = 1000$  and  $M = 1000$ ,  $\log(N) = 10$ . The table requires  $1000 * 1000 * 10^8$ 
31 * ints.
32 * This is ~400 MB of memory. This passes on most platforms (512MB limits), but
33 * if the limit
34 * is strictly 256MB, you will MLE. Always check the memory limit before
35 * choosing this!
36 * - Idempotent Operations ONLY:
37 * Sparse tables ONLY work for operations where  $X + X = X$  (like Min, Max, GCD,
38 * Bitwise OR/AND).
39 * Do NOT use this for Sum queries! (Use a 2D Prefix Sum array for  $O(1)$  sums
40 * instead).
41 */
42
43 #include <bits/stdc++.h>
44 using namespace std;
45
46 // struct SparseTable2D {
47 private:
48 int n, m;
49 int log_n, log_m;
50 // table[ir][ic][r][c]
51 // Represents the rectangle of size  $2^{ir} \times 2^{ic}$  starting at top-left (r, c)
52 vector<vector<vector<vector<int>>>> table;
53
54 // Precomputed logs for pure  $O(1)$  queries
55 vector<int> log_table;
56
57 // ? Dial 1: The Combination Logic (Change to min, gcd, |, &, etc.)
58 inline int combine(int a, int b)
59 {
60 return max(a, b);
61 }
62
63 // public:
64 SparseTable2D(const vector<vector<int>> &grid)
65 {
66 if (grid.empty() || grid[0].empty())
67 return;
68 n = grid.size();

```

```

117         table[ir - 1][ic][i + (1 << (ir - 1))][j]);
118     }
119 }
120 }
121 }
122 }
123 }
124
125 // // Query inclusive rectangle from top-left (r1, c1) to bottom-right (r2, c2)
126 int query(int r1, int c1, int r2, int c2)
127 {
128     // Find the largest power of 2 that fits inside the row and column lengths
129     int k_r = log_table[r2 - r1 + 1];
130     int k_c = log_table[c2 - c1 + 1];
131
132     // ? THE MAGIC: We perfectly cover the requested rectangle using 4 smaller
133     // overlapping
134     // rectangles of size (2^k_r) x (2^k_c).
135     // Because the operation is Idempotent (max/min), overlapping data doesn't
136     // change the answer!
137
138     int top_left = table[k_r][k_c][r1][c1];
139     int top_right = table[k_r][k_c][r1][c2 - (1 << k_c) + 1];
140     int bottom_left = table[k_r][k_c][r2 - (1 << k_r) + 1][c1];
141     int bottom_right = table[k_r][k_c][r2 - (1 << k_r) + 1][c2 - (1 << k_c) + 1];
142
143     return combine(combine(top_left, top_right), combine(bottom_left,
144         bottom_right));
145 }
146 }
147 ;
148
149 // void solve() {
150 int n, m, q;
151 // Example Input: N rows, M cols, Q queries
152 if (!(cin >> n >> m >> q))
153     return;
154
155 vector<vector<int>> grid(n, vector<int>(m));
156 for (int i = 0; i < n; i++)
157 {
158     for (int j = 0; j < m; j++)
159     {
160         cin >> grid[i][j];
161     }
162 }
163 SparseTable2D st(grid);
164 while (q--)
165 {
166     int r1, c1, r2, c2;
167     // Assume 0-indexed coordinates are provided
168     cin >> r1 >> c1 >> r2 >> c2;

```

```

168     cout << st.query(r1, c1, r2, c2) << "\n";
169 }
170 }
171
172 int main()
173 {
174     // Fast I/O is mandatory for heavy 2D queries
175     ios_base::sync_with_stdio(false);
176     cin.tie(NULL);
177     solve();
178     return 0;
179 }

```

2.9.2 sparse table

```

1 struct SparseTable {
2     int n, LG;
3     vector<vector<int>> > T;
4     SparseTable(const vector<int> &arr) {
5         n = arr.size();
6         LG = __lg(n) + 2;
7         T.assign(LG, vector<int>(n));
8         for (int i = 0; i < n; i++) {
9             T[0][i] = arr[i];
10        }
11        for (int pw = 1; (1 << pw) <= n; pw++) {
12            for (int i = 0; i + (1 << pw) <= n; i++) {
13                T[pw][i] = merge(T[pw - 1][i], T[pw - 1][i + (1 << (pw - 1))]);
14            }
15        }
16    }
17
18    int merge(int a, int b) {
19        return min(a, b);
20    }
21
22    int query(int l, int r) {
23        int len = r - l + 1;
24        int pw = __lg(len);
25        return merge(T[pw][l], T[pw][r - (1 << pw) + 1]);
26    }
27
28    int query1(int l, int r) { // o(log)
29        int res = 1e9; // change
30        int sz = r - l + 1;
31        for (int i = 0; l <= r; i++) {
32            if (sz >> i & 1) {
33                res = merge(res, T[i][l]);
34                l += (1 << i);
35            }
36        }
37        return res;
38    }

```

```

39
40 };

2.10 Treap

2.10.1 implicit treap

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: IMPLICIT TREAP (CARTESIAN TREE)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Dynamic Array", "Range Reverse", "Insert/Delete at index", "Cut
   and Paste".
6  * - Classic Scenarios: You have an array and need to perform operations that a
   Segment
7  * Tree CANNOT do, such as:
8  * - Insert/Delete elements at specific indices (shifting the whole array).
9  * - Reversing a whole subsegment [L, R] dynamically.
10 * - Moving a block [L, R] to the front or end.
11 * - The Magic: An Implicit Treap acts like a balanced BST, but instead of
   sorting by
12 * a "Key", it sorts by the "Implicit Index" (the size of the left subtree).
   This allows
13 * us to split and merge arbitrary subarrays in  $O(\log N)$  time.
14 *
15 * 2. HOW TO USE IT (THE BLACK BOX)
16 * - Initialization: Root = nullptr. Use `insert` to populate.
17 * - Range Operations:
18 * // Reverse range [L, R] (0-indexed)
19 * reverse_range(root, L, R);
20 * // Query sum of range [L, R]
21 * long long ans = query_sum(root, L, R);
22 * - Complexity:
23 * Time:  $O(\log N)$  per operation.
24 * Space:  $O(N)$  memory.
25 *
26 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
27 * - Need Range Add? Add a `lazy` field to the Node and push it in the `push()`
   function.
28 * - Need Max/Min instead of Sum? Update `update()` and `get_sum()` to maintain
29 * `max`/`min` fields instead of `sum`.
30 */
31
32 #include <bits/stdc++.h>
33 using namespace std;
34
35 struct Node
36 {
37     int val, priority, size;
38     long long sum;
39     bool rev; // Flag for range reversal (Lazy Propagation)
40     Node *left, *right;
41

```

```

42     Node(int v) : val(v), priority(rand()), size(1), sum(v), rev(false), left(
        nullptr), right(nullptr) {}
43 };
44
45 int get_size(Node *t) { return t ? t->size : 0; }
46 long long get_sum(Node *t) { return t ? t->sum : 0; }
47
48 // Update subtree size and sum based on children
49 void update(Node *t)
50 {
51     if (!t)
52         return;
53     t->size = 1 + get_size(t->left) + get_size(t->right);
54     t->sum = t->val + get_sum(t->left) + get_sum(t->right);
55 }
56
57 // Push lazy propagation (reverse flag) down to children
58 void push(Node *t)
59 {
60     if (t && t->rev)
61     {
62         t->rev = false;
63         swap(t->left, t->right);
64         if (t->left)
65             t->left->rev ^= true;
66         if (t->right)
67             t->right->rev ^= true;
68     }
69 }
70
71 // Split treap t into l (first k nodes) and r (remaining nodes)
72 void split(Node *t, int k, Node *&l, Node *&r)
73 {
74     if (!t)
75     {
76         l = r = nullptr;
77         return;
78     }
79     push(t);
80     int left_size = get_size(t->left);
81     if (left_size < k)
82     {
83         split(t->right, k - left_size - 1, t->right, r);
84         l = t;
85     }
86     else
87     {
88         split(t->left, k, l, t->left);
89         r = t;
90     }
91     update(t);
92 }
93
94 // Merge treap l and treap r
95 Node *merge(Node *l, Node *r)

```

```

96 {
97     push(l);
98     push(r);
99     if (!l || !r)
100         return l ? l : r;
101     if (l->priority > r->priority)
102     {
103         l->right = merge(l->right, r);
104         update(l);
105         return l;
106     }
107     else
108     {
109         r->left = merge(l, r->left);
110         update(r);
111         return r;
112     }
113 }
114
115 // Reverse subarray [l, r] (0-indexed)
116 void reverse_range(Node *&root, int l, int r)
117 {
118     Node *t1, *t2, *t3;
119     split(root, r + 1, t2, t3);
120     split(t2, l, t1, t2);
121     if (t2)
122         t2->rev ^= true;
123     root = merge(t1, merge(t2, t3));
124 }
125
126 // Query sum in subarray [l, r]
127 long long query_sum(Node *&root, int l, int r)
128 {
129     Node *t1, *t2, *t3;
130     split(root, r + 1, t2, t3);
131     split(t2, l, t1, t2);
132     long long res = get_sum(t2);
133     root = merge(t1, merge(t2, t3));
134     return res;
135 }
136
137 // Insert value at index pos
138 void insert(Node *&root, int pos, int val)
139 {
140     Node *t1, *t2;
141     split(root, pos, t1, t2);
142     root = merge(merge(t1, new Node(val)), t2);
143 }
144
145 int main()
146 {
147     srand(time(0));
148     Node *root = nullptr;
149     // Example: Insert values
150     insert(root, 0, 10);

```

```

151     insert(root, 1, 20);
152     insert(root, 2, 30);
153
154     // Example: Sum query [0, 1]
155     // cout << query_sum(root, 0, 1) << endl;
156
157     return 0;
158 }

```

2.11 Trie

2.11.1 BinaryTrie Using Vector

```

1 class BinaryTrie {
2     struct Node {
3         int ch[2];
4         int frq[2];
5         Node() {
6             ch[0] = ch[1] = -1;
7             frq[0] = frq[1] = 0;
8         }
9     };
10    vector<Node> tree;
11 public:
12    BinaryTrie() {
13        tree.emplace_back();
14        insert(0);
15    }
16
17    void insert(int n) {
18        int cur = 0;
19        for (int i = 30; i >= 0; i--) {
20            int idx = (n >> i) & 1;
21
22            if (tree[cur].ch[idx] == -1) {
23                tree[cur].ch[idx] = tree.size();
24                tree.emplace_back();
25            }
26
27            tree[cur].frq[idx]++;
28            cur = tree[cur].ch[idx];
29        }
30    }
31
32    void erase(int n) {
33        int cur = 0;
34        for (int i = 30; i >= 0; i--) {
35            int idx = (n >> i) & 1;
36            tree[cur].frq[idx]--;
37            cur = tree[cur].ch[idx];
38        }
39    }
40 }

```

```

41 bool search(int n) {
42     int cur = 0;
43     for (int i = 30; i >= 0; i--) {
44         int idx = (n >> i) & 1;
45         if (tree[cur].ch[idx] == -1 || tree[cur].frq[idx] == 0) {
46             return false;
47         }
48         cur = tree[cur].ch[idx];
49     }
50     return true;
51 }
52
53 int maxXor(int x) {
54     int cur = 0;
55     int res = 0;
56     for (int i = 30; i >= 0; i--) {
57         int idx = (x >> i) & 1;
58         int target = 1 - idx;
59
60         if (tree[cur].ch[target] != -1 && tree[cur].frq[target] > 0) {
61             res |= (1 << i);
62             cur = tree[cur].ch[target];
63         } else {
64             cur = tree[cur].ch[idx];
65         }
66     }
67     return res;
68 }
69
70 void clear() {
71     tree.clear();
72     tree.emplace_back();
73     insert(0);
74 }
75 };

```

2.11.2 Binary trie

```

1 struct BitTrie
2 {
3 private:
4     BitTrie *child[2];
5     int cnt{};
6
7 public:
8     BitTrie()
9     {
10         child[0] = child[1] = nullptr;
11     }
12
13     ~BitTrie()
14     {
15         delete child[0];

```

```

16         delete child[1];
17     }
18
19     void insert(int num)
20     {
21         BitTrie *curr = this;
22         for (int i = 31; i >= 0; i--)
23         {
24             int bit = num >> i & 1;
25             if (!curr->child[bit])
26                 curr->child[bit] = new BitTrie();
27             curr = curr->child[bit];
28             curr->cnt++;
29         }
30     }
31
32     void erase(int num)
33     {
34         BitTrie *curr = this;
35         for (int i = 31; i >= 0; i--)
36         {
37             int bit = num >> i & 1;
38             curr = curr->child[bit];
39             curr->cnt--;
40         }
41     }
42
43     int getMaxXor(int num)
44     {
45         BitTrie *curr = this;
46         int maxXor = 0;
47         for (int i = 31; i >= 0; i--)
48         {
49             int need = 1 - (num >> i & 1);
50             if (curr->child[need] && curr->child[need]->cnt > 0)
51             {
52                 maxXor |= 1ll << i;
53                 curr = curr->child[need];
54             }
55             else
56                 curr = curr->child[1 - need];
57         }
58         return maxXor;
59     }
60
61     // count Xors with X less than K
62     int countXorLessThan(int x, int k)
63     {
64         int ret{};
65         BitTrie *curr = this;
66         for (int i = 31; i >= 0; i--)
67         {
68             if (!curr)
69                 break;
70             int bitX = x >> i & 1;

```

```

71         int bitK = k >> i & 1;
72         if (bitK == 0)
73             curr = curr->child[bitX];
74         else
75         {
76             if (curr->child[bitX])
77                 ret += curr->child[bitX]->cnt;
78             curr = curr->child[1 - bitX];
79         }
80     }
81     return ret;
82 }
83 };
84
85 // get the Max Xor of two numbers in the array
86 int getMaxXorOfTwoNumbersInArray(vector<int> &v)
87 {
88     if (v.empty())
89         return 0;
90     int n = int(v.size());
91     BitTrie trie;
92     trie.insert(v[0]);
93     int max{};
94     for (int i = 1; i < n; i++)
95     {
96         max = std::max(max, trie.getMaxXor(v[i]));
97         trie.insert(v[i]);
98     }
99     return max;
100 }
101
102 struct Query
103 {
104     int x, a, id;
105
106     bool operator<(const Query &other) const
107     {
108         return a < other.a;
109     }
110 };
111
112 // max xor queries for a number with array with threshold A [The element i pick
113 // from array must be <= A]
114 void getMaxXorOfNumWithArrayLessThanThresholdQueries(vector<int> &v)
115 {
116     int n = int(v.size());
117     sort(v.begin(), v.end());
118     int q;
119     cin >> q;
120     vector<Query> queries(q);
121     for (int i = 0; i < q; i++)
122     {
123         cin >> queries[i].x >> queries[i].a;
124         queries[i].id = i;
125     }

```

```

125     sort(queries.begin(), queries.end());
126     BitTrie trie;
127     int idx{};
128     vector<int> ans(q);
129     for (int i = 0; i < q; i++)
130     {
131         auto [x, a, id] = queries[i];
132         while (idx < n && v[idx] <= a)
133             trie.insert(v[idx++]);
134         if (idx == 0)
135             ans[id] = -1;
136         else
137             ans[id] = trie.getMaxXor(x);
138     }
139     for (int i = 0; i < q; i++)
140         cout << ans[i] << "\n";
141 }
142
143 // Max xor subarray
144 int getMaxXorSubarray(vector<int> &a)
145 {
146     int n = int(a.size());
147     BitTrie trie;
148     trie.insert(0);
149     int pre{}, ret{};
150     for (int i = 0; i < n; i++)
151     {
152         pre ^= a[i];
153         ret = max(ret, trie.getMaxXor(pre));
154         trie.insert(pre);
155     }
156     return ret;
157 }
158
159 int numberOfSubarraysWithXorLessThanK(vector<int> &a, int k)
160 {
161     int n = int(a.size());
162     BitTrie trie;
163     trie.insert(0);
164     int pre{}, ret{};
165     for (int i = 0; i < n; i++)
166     {
167         pre ^= a[i];
168         ret += trie.countXorLessThan(pre, k);
169         trie.insert(pre);
170     }
171     return ret;
172 }

```

2.11.3 Persistent Trie

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PERSISTENT BINARY TRIE (XOR MAXIMIZER)

```

```

3 *
4 * 1. WHAT PROBLEM DOES THIS SOLVE?
5 * - Keywords: "Maximum XOR in a subarray", "Range XOR", "Subsegment XOR with X"
6 * - Classic Scenarios: You are given an array of N integers. You need to answer
  Q queries.
7 * Each query gives you a range [L, R] and a value X. You need to find an
  element Y
8 * in the subarray from L to R such that (Y ^ X) is maximized.
9 * - The Magic: A standard Trie can find the max XOR in a whole array. By making
  it Persistent,
10 * we save a new "version" of the Trie after inserting every single element.
  query the
11 * range [L, R], we take the Trie at version R and mathematically subtract the
  paths from
12 * version L-1. It does this in blindingly fast O(30) time!
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Initialization: Pass a 0-indexed vector to build the historical versions.
  vector<long long> a = {3, 10, 5, 25, 2};
16 * PersistentTrie ptrie(a);
17 *
18 * - Queries (All 0-indexed, Inclusive [L, R]):
19 * // Find the maximum XOR of 7 with any element in the subarray from index
  1 to 3
20 * long long ans = ptrie.queryMaxXor(1, 3, 7);
21 *
22 * - Complexity:
23 * Time: O(N * BITS) to build. O(BITS) per query.
24 * Space: O(N * BITS) memory.
25 *
26 *
27 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
28 * - Dealing with huge numbers (up to 1018)?
29 * Change the `BITS` constant from 30 to 60. (30 is enough for numbers up to
  109).
30 * - Minimum XOR instead of Maximum XOR?
31 * I have included a `queryMinXor` function right below the Max one! It simply
  tries to
32 * take the path of the SAME bit instead of the FLIPPED bit.
33 * - Need to dynamically add elements to the back of the array?
34 * Just call `ptrie.insertBack(X);`. It will add the element and create a new
  version
35 * that you can instantly query.
36 */
37
38 #include <bits/stdc++.h>
39 using namespace std;
40
41 struct PersistentTrie
42 {
43 private:
44     // ? Dial 1: Set to 30 for <= 109. Set to 60 for <= 1018.
45     const int BITS = 30;
46
47     struct Node
48     {
49         int child[2];
50         int cnt; // How many elements exist in this subtree
51
52         Node()
53         {
54             child[0] = child[1] = 0;
55             cnt = 0;
56         }
57     };
58
59     vector<Node> trie;
60
61     // ? Creates a brand new node that is an exact copy of the given node ID
62     int clone(int node)
63     {
64         trie.push_back(trie[node]);
65         return trie.size() - 1;
66     }
67
68     int insert(int prev_root, long long val)
69     {
70         int curr = clone(prev_root);
71         int new_root = curr;
72         int prev = prev_root;
73
74         trie[curr].cnt++; // Increment count for the root of this version
75
76         for (int i = BITS; i >= 0; i--)
77         {
78             int b = (val >> i) & 1;
79
80             // ? We are taking path 'b'. To safely modify it without destroying
81             // the historical version, we MUST clone the 'b' child of the
82             // previous version!
83             trie[curr].child[b] = clone(trie[prev].child[b]);
84
85             // The OTHER child pointer (b ^ 1) was naturally copied over during
86             // the
87             // clone() of the parent, so it still safely points to the historical
88             // branch!
89
90             curr = trie[curr].child[b];
91             prev = trie[prev].child[b];
92
93             trie[curr].cnt++; // Increment the count of elements that pass
94             // through this path
95         }
96
97         return new_root;
98     }
99
100 public:
101     // roots[i] represents the Trie containing all elements from index 0 to i-1.
102     // roots[0] is the empty Trie.
103     vector<int> roots;

```

```

100
101 PersistentTrie(const vector<long long> &a = {})
102 {
103     // Pre-allocate memory to prevent slow vector reallocations.
104     // N elements * roughly (BITS + 1) nodes per element
105     trie.reserve(a.size() * (BITS + 2) + 100);
106
107     // Node 0 is the Dummy/Empty node
108     trie.emplace_back();
109
110     // Version 0 is the empty tree
111     roots.push_back(0);
112
113     for (long long x : a)
114     {
115         insertBack(x);
116     }
117 }
118
119 // Add an element to the end of the array dynamically
120 void insertBack(long long val)
121 {
122     int new_root = insert(roots.back(), val);
123     roots.push_back(new_root);
124 }
125
126 // Find the MAXIMUM value of (val ^ Y) where Y is an element in a[l...r]
127 long long queryMaxXor(int l, int r, long long val)
128 {
129     // Because roots[i] covers [0...i-1], the range [L, R] is extracted by
130     // taking Version (R + 1) and subtracting Version L.
131     int curr_R = roots[r + 1];
132     int curr_L = roots[l];
133     long long ans = 0;
134
135     for (int i = BITS; i >= 0; i--)
136     {
137         int b = (val >> i) & 1;
138         int flipped = b ^ 1; // To MAXIMIZE XOR, we want the opposite bit!
139
140         // How many numbers in our specific range actually take the 'flipped'
141         // path?
142         int count_flipped = trie[trie[curr_R].child[flipped]].cnt -
143             trie[trie[curr_L].child[flipped]].cnt;
144
145         if (count_flipped > 0)
146         {
147             // The path exists! We take it.
148             ans |= (1LL << i);
149             curr_R = trie[curr_R].child[flipped];
150             curr_L = trie[curr_L].child[flipped];
151         }
152         else
153         {
154             // The opposite bit doesn't exist in this range. We are forced
155             // to take the same bit.
156             curr_R = trie[curr_R].child[b];
157             curr_L = trie[curr_L].child[b];
158         }
159     }
160
161     return ans;
162 }
163
164 // Find the MINIMUM value of (val ^ Y) where Y is an element in a[l...r]
165 long long queryMinXor(int l, int r, long long val)
166 {
167     int curr_R = roots[r + 1];
168     int curr_L = roots[l];
169     long long ans = 0;
170
171     for (int i = BITS; i >= 0; i--)
172     {
173         int b = (val >> i) & 1;
174
175         // To MINIMIZE XOR, we want the EXACT SAME bit!
176         int count_same = trie[trie[curr_R].child[b]].cnt -
177             trie[trie[curr_L].child[b]].cnt;
178
179         if (count_same > 0)
180         {
181             // The path exists! We take it. (Ans bit remains 0 because b ^ b
182             // = 0)
183             curr_R = trie[curr_R].child[b];
184             curr_L = trie[curr_L].child[b];
185         }
186         else
187         {
188             // Forced to take the flipped bit, which increases our XOR
189             // penalty.
190             int flipped = b ^ 1;
191             ans |= (1LL << i);
192             curr_R = trie[curr_R].child[flipped];
193             curr_L = trie[curr_L].child[flipped];
194         }
195     }
196
197     return ans;
198 }
199
200 void solve()
201 {
202     int n, q;
203     if (!(cin >> n >> q))
204         return;
205
206     vector<long long> a(n);
207     for (int i = 0; i < n; i++)
208         cin >> a[i];
209
210     PersistentTrie ptrie(a);

```

```

206
207     while (q-->0)
208     {
209         int l, r;
210         long long x;
211         // 0-indexed L and R, and the XOR target X
212         cin >> l >> r >> x;
213
214         cout << ptrie.queryMaxXor(l, r, x) << "\n";
215     }
216 }
217
218 int main()
219 {
220     ios_base::sync_with_stdio(false);
221     cin.tie(NULL);
222     // solve();
223     return 0;
224 }

```

2.11.4 Trie (String)

```

1 class Trie {
2     struct Node {
3         map<string, int> ch;
4         Node() {
5             }
6     };
7     vector<Node> root;
8 public:
9     Trie() {
10         root.emplace_back();
11     }
12     void insert(vector<string> &words) {
13         int cur = 0;
14         for (auto s : words) {
15             if (!root[cur].ch[s]) {
16                 root[cur].ch[s] = root.size();
17                 root.emplace_back();
18             }
19             cur = root[cur].ch[s];
20         }
21     }
22 };

```

2.11.5 Trie Using Vector

```

1 class Trie {
2     struct Node {
3         int ch[26];
4         int prefix, end;
5         Node() {

```

```

6             prefix = end = 0;
7             memset(ch, -1, sizeof(ch));
8         }
9     };
10     vector<Node> tree;
11 public:
12     Trie() {
13         tree.emplace_back();
14     }
15
16     void insert(string &s) {
17         int cur = 0;
18         for (auto i : s) {
19             int idx = i - 'a';
20
21             if (tree[cur].ch[idx] == -1) {
22                 tree[cur].ch[idx] = tree.size();
23                 tree.emplace_back();
24             }
25             cur = tree[cur].ch[idx];
26             tree[cur].prefix++;
27         }
28         tree[cur].end++;
29     }
30
31     bool search(string &s) {
32         int cur = 0;
33         for (auto i : s) {
34             int idx = i - 'a';
35             if (tree[cur].ch[idx] == -1) {
36                 return false;
37             }
38             cur = tree[cur].ch[idx];
39         }
40         return tree[cur].end > 0;
41     }
42
43     int countWords(string &s) {
44         int cur = 0;
45         for (auto i : s) {
46             int idx = i - 'a';
47             if (tree[cur].ch[idx] == -1) {
48                 return 0;
49             }
50             cur = tree[cur].ch[idx];
51         }
52         return tree[cur].end;
53     }
54
55     int countPrefix(string &s) {
56         int cur = 0;
57         for (auto i : s) {
58             int idx = i - 'a';
59             if (tree[cur].ch[idx] == -1) {
60                 return 0;

```

```

61     }
62     cur = tree[cur].ch[idx];
63 }
64 return tree[cur].prefix;
65 }
66
67 // DFS in Trie
68 void dfs(int u) {
69     for (int v = 0; v < 26; v++) {
70         if (root[u].ch[v] != -1) {
71             int x = root[u].ch[v];
72             if (root[u].prefix != root[x].prefix) {
73                 ans+=root[x].prefix;
74             }
75             dfs(x);
76         }
77     }
78 }
79
80 void clear() {
81     tree.clear();
82     tree.emplace_back();
83 }
84 };

```

2.11.6 Trie

```

1 vector<int> ans;
2
3 class Trie
4 {
5 private:
6     Trie *child[26];
7     int word_cnt{}, prefix_cnt{};
8
9 public:
10    Trie()
11    {
12        memset(child, 0, sizeof child);
13    }
14
15    void insert(const string &str, int idx = 0)
16    {
17        prefix_cnt++;
18        if (idx == int(str.size()))
19        {
20            word_cnt++;
21            return;
22        }
23        int curr = str[idx] - 'a';
24        if (!child[curr])
25            child[curr] = new Trie();
26        child[curr]->insert(str, idx + 1);

```

```

27    }
28
29    void erase(const string &str, int idx = 0)
30    {
31        if (idx == 0 && !wordSearch(str))
32            return;
33        prefix_cnt--;
34        if (idx == int(str.size()))
35        {
36            word_cnt--;
37            return;
38        }
39        int curr = str[idx] - 'a';
40        child[curr]->erase(str, idx + 1);
41    }
42
43    void print(int idx = 0)
44    {
45        for (int i = 0; i < 26; i++)
46        {
47            if (!child[i] || !child[i]->prefix_cnt)
48                continue;
49            for (int k = 0; k < idx; k++)
50                cout << " ";
51            cout << char(i + 'a') << "\n";
52            child[i]->print(idx + 1);
53        }
54    }
55
56    // count number of chars needed to be typed by the user for each word [using
57    // auto-completion]
58    // trie.insert("") at the beginning because the user needs to write the first
59    // letter
60    void count(int cnt = 0)
61    {
62        if (word_cnt)
63            ans.push_back(cnt);
64        int num{};
65        for (int i = 0; i < 26; i++)
66        {
67            if (child[i])
68                num++;
69        }
70        for (int i = 0; i < 26; i++)
71        {
72            if (!child[i] || !child[i]->prefix_cnt)
73                continue;
74            if (num == 1 && word_cnt == 0)
75                child[i]->count(cnt);
76            else
77                child[i]->count(cnt + 1);
78        }
79    }
80
81    bool wordSearch(const string &str, int idx = 0)

```

```

80 {
81     if (idx == int(str.size()))
82         return word_cnt;
83     int curr = str[idx] - 'a';
84     if (!child[curr] || child[curr]->prefix_cnt == 0)
85         return false;
86     return child[curr]->wordSearch(str, idx + 1);
87 }
88
89 bool prefixSearch(const string &str, int idx = 0)
90 {
91     if (idx == int(str.size()))
92         return prefix_cnt;
93     int curr = str[idx] - 'a';
94     if (!child[curr] || child[curr]->prefix_cnt == 0)
95         return false;
96     return child[curr]->prefixSearch(str, idx + 1);
97 }
98
99 int countWordsEqualTo(const string &str, int idx = 0)
100 {
101     if (idx == int(str.size()))
102         return word_cnt;
103     int curr = str[idx] - 'a';
104     if (!child[curr] || child[curr]->prefix_cnt == 0)
105         return 0;
106     return child[curr]->countWordsEqualTo(str, idx + 1);
107 }
108
109 int countWordsStartingWith(const string &str, int idx = 0)
110 {
111     if (idx == int(str.size()))
112         return prefix_cnt;
113     int curr = str[idx] - 'a';
114     if (!child[curr] || child[curr]->prefix_cnt == 0)
115         return 0;
116     return child[curr]->countWordsStartingWith(str, idx + 1);
117 }
118
119 void printWordsStartingWith(const string &str, int idx = 0, string res = "")
120 {
121     if (idx < int(str.size()))
122     {
123         int curr = str[idx] - 'a';
124         if (!child[curr] || child[curr]->prefix_cnt == 0)
125             return;
126         child[curr]->printWordsStartingWith(str, idx + 1, res + str[idx]);
127     }
128     else
129     {
130         if (word_cnt && idx != int(str.size()))
131             // ans.push_back(res); //// uncomment this <-----
132             for (int i = 0; i < 26; i++)
133             {
134                 if (!child[i] || child[i]->prefix_cnt == 0)
135                     continue;
136                 child[i]->printWordsStartingWith(str, idx + 1, res + char('a'
137                     + i));
138             }
139     }
140
141     // longest word with all prefixes problem
142     bool checkCompleteness(const string &str, int idx = 0)
143     {
144         if (idx == int(str.size()))
145             return word_cnt > 0;
146         int curr = str[idx] - 'a';
147         if (!child[curr] || child[curr]->word_cnt == 0)
148             return false;
149         return child[curr]->checkCompleteness(str, idx + 1);
150     }
151
152     string longestCompleteString(vector<string> &v)
153     {
154         int n = int(v.size());
155         string longest{};
156         for (int i = 0; i < n; i++)
157         {
158             if (checkCompleteness(v[i]))
159             {
160                 if (v[i].length() > longest.length() || (v[i].length() == longest
161                     .length() && v[i] < longest))
162                     longest = v[i];
163             }
164         }
165         return longest.empty() ? "No Complete String" : longest;
166     }
167
168     int countNumberOfNodes()
169     {
170         int ret{};
171         for (int i = 0; i < 26; i++)
172         {
173             if (child[i])
174                 ret += 1 + child[i]->countNumberOfNodes();
175         }
176         return ret;
177     }
178
179     // count number of distinct substrings
180     int countNumberOfDistinctSubStrings(string &str)
181     {
182         int n = int(str.size());
183         Trie trie;
184         for (int i = 0; i < n; i++)
185             trie.insert(str.substr(i));
186         return trie.countNumberOfNodes();
187     }

```

3 Graphs

3.1 2-Sat

3.1.1 2sat

```
1 const int N = 1e5 + 9;
2
3 int SCC_id[N];
4 GRAPH graph, graph_reverse, cond_graph;
5 int n, m;
6 vector<int> order, vis, vis_reverse, order_reverse;
7
8 void dfs(int node)
9 {
10     vis[node] = true;
11     for (auto &x : graph[node])
12     {
13         if (!vis[x])
14             dfs(x);
15     }
16     order.emplace_back(node);
17 }
18
19 void dfs_reverse(int node)
20 {
21     vis_reverse[node] = true;
22     for (auto &x : graph_reverse[node])
23     {
24         if (!vis_reverse[x])
25             dfs_reverse(x);
26     }
27     order_reverse.emplace_back(node);
28 }
29
30 void SCC()
31 {
32     graph.assign(n + 1, {}), graph_reverse.assign(n + 1, {}), vis.assign(n + 1, {}), vis_reverse.assign(n + 1, {}), cond_graph.assign(n + 1, {});
33     for (int i = 0; i < m; i++)
34     {
35         int x, y;
36         cin >> x >> y;
37         graph[x].emplace_back(y), graph_reverse[y].emplace_back(x);
38     }
39     order.clear();
40     for (int i = 1; i <= n; i++)
41     {
42         if (!vis[i])
43             dfs(i);
44     }
45     reverse(order.begin(), order.end());
46     vector<vector<int>> scc;
47     scc.push_back({});
```

```
48     for (auto &x : order)
49     {
50         if (!vis_reverse[x])
51         {
52             order_reverse.clear();
53             dfs_reverse(x);
54             scc.emplace_back(order_reverse);
55             for (auto &val : order_reverse)
56                 SCC_id[val] = int(scc.size()) - 1;
57         }
58     }
59     for (int u = 1; u <= n; u++)
60     {
61         for (int v : graph[u])
62         {
63             if (SCC_id[u] != SCC_id[v])
64                 cond_graph[SCC_id[u]].emplace_back(SCC_id[v]);
65         }
66     }
67 }
68
69 // Add this helper to add edges easily
70 void add_clause(int i, bool val_i, int j, bool val_j)
71 {
72     // i, j are variable indices (1 to n)
73     // val_i, val_j are the boolean requirements (true for X, false for !X)
74
75     // Logic: (X or Y) is equivalent to (!X -> Y) AND (!Y -> X)
76
77     // Map to 2*i and 2*i+1
78     int u = 2 * i + (val_i ? 0 : 1);
79     int v = 2 * j + (val_j ? 0 : 1);
80
81     // !u is u^1, !v is v^1
82     graph[u ^ 1].emplace_back(v);
83     graph[v ^ 1].emplace_back(u);
84
85     // Keep reversed graph for Kosaraju
86     graph_reverse[v].emplace_back(u ^ 1);
87     graph_reverse[u].emplace_back(v ^ 1);
88 }
89
90 void solve()
91 {
92     // 1. Read input
93     cin >> n >> m;
94     // Remember to adjust N to 2*N+1 if N was 1e5
95
96     // 2. Clear graph from your SCC template
97     // ... (Use your SCC setup code here)
98
99     for (int k = 0; k < m; k++)
100     {
101         int i, j;
102         bool val_i, val_j;
```

```

103 // Example: Reading "1 True 2 False" means (X1 or !X2)
104 cin >> i >> val_i >> j >> val_j;
105 add_clause(i, val_i, j, val_j);
106 }
107
108 // 3. Run Kosaraju
109 SCC();
110
111 // 4. Check for Impossibility
112 vector<int> assignment(n + 1);
113 for (int i = 1; i <= n; i++)
114 {
115     if (SCC_id[2 * i] == SCC_id[2 * i + 1])
116     {
117         cout << "IMPOSSIBLE\n";
118         return;
119     }
120     // 5. The Assignment Logic (The "Higher ID is True" strategy)
121     assignment[i] = (SCC_id[2 * i] > SCC_id[2 * i + 1]);
122 }
123
124 // 6. Print result
125 cout << "POSSIBLE\n";
126 for (int i = 1; i <= n; i++)
127 {
128     cout << assignment[i] << " ";
129 }
130 }

```

3.2 DAG

3.2.1 Dag reachability(bitset trick)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: BITSET REACHABILITY
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Transitive Closure", "DAG Reachability", "Bitset Optimization".
9  * - Classic Scenarios: You have a Directed Acyclic Graph (DAG) and need to
   answer Q
10  * queries of the form "Can I reach node Y from node X?".
11  * - The Magic: Since a DAG has a topological order, we process nodes in REVERSE
12  * topological order. For each node U, its reachability set is the Bitwise OR
13  * of the reachability sets of all its neighbors. Because bitsets are 64x
   faster than
14  * arrays, this brings an  $O(N^3)$  problem down to an  $O(N^3 / 64)$  complexity.
15  *
16  * 2. HOW TO USE IT (THE BLACK BOX)
17  * - Initialization:
18  *     compute_reachability(N); // Run this once after building adjacency list

```

```

19 *
20 * - Query:
21 *     if (reach[start].test(target)) { ... }
22 *
23 * - Complexity:
24 *     Time:  $O(N * (N + M) / 64)$ .
25 *     Space:  $O(N^2 / 8)$  bytes. (For  $N=20,000$ , this is ~50MB).
26 *
27 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
28 * - Memory Limit Warning:
29 *     The current `MAXN = 20005` uses ~50MB. If  $N > 20,000$ , you will exceed
   standard 256MB limits.
30 * If  $N > 20,000$ , you MUST use the "Block Reachability" approach (process bits
   in
31 * chunks of 1024 or 2048) rather than holding the entire  $N \times N$  matrix in
   memory.
32 */
33
34 // ? Dial 1: Adjust MAXN based on memory limits ( $N^2 / 8$  bytes)
35 const int MAXN = 20005;
36
37 // reach[u][v] = 1 if there is a path from u to v
38 bitset<MAXN> reach[MAXN];
39 vector<int> adj[MAXN];
40 vector<int> topo_order;
41 int in_degree[MAXN];
42
43 // Standard Kahn's Algorithm for Topological Sort
44 void compute_topological_sort(int n)
45 {
46     queue<int> q;
47     for (int i = 1; i <= n; i++)
48     {
49         if (in_degree[i] == 0)
50             q.push(i);
51     }
52
53     while (!q.empty())
54     {
55         int u = q.front();
56         q.pop();
57         topo_order.push_back(u);
58
59         for (int v : adj[u])
60         {
61             in_degree[v]--;
62             if (in_degree[v] == 0)
63                 q.push(v);
64         }
65     }
66 }
67
68 void compute_reachability(int n)
69 {
70     compute_topological_sort(n);

```

```

71
72 // ? THE MAGIC: Process in REVERSE topological order
73 // This ensures that when we process node u, all its neighbors v
74 // are already fully computed.
75 for (int i = n - 1; i >= 0; i--)
76 {
77     int u = topo_order[i];
78
79     reach[u].set(u); // A node can always reach itself
80
81     for (int v : adj[u])
82     {
83         // Bitwise OR is the secret sauce:
84         // This ORs all bits of reach[v] into reach[u] at once
85         reach[u] |= reach[v];
86     }
87 }
88 }
89
90 int main()
91 {
92     ios_base::sync_with_stdio(false);
93     cin.tie(NULL);
94
95     int n, m;
96     cin >> n >> m;
97
98     for (int i = 0; i < m; i++)
99     {
100         int u, v;
101         cin >> u >> v;
102         adj[u].push_back(v);
103         in_degree[v]++;
104     }
105
106     compute_reachability(n);
107
108     // Example Query: Can I reach node 'target' from node 'start'?
109     int start, target;
110     cin >> start >> target;
111
112     if (reach[start].test(target))
113     {
114         cout << "Reachable!\n";
115     }
116     else
117     {
118         cout << "Not reachable.\n";
119     }
120
121     return 0;
122 }

```

3.3 Eulerian path

3.3.1 Eulerian path

```

1 const int N = 1e5, M = 1e5;
2
3 struct Edge
4 {
5     int to, id;
6 };
7
8 vector<Edge> graph[N];
9 bool vis[M];
10 vector<int> ans;
11
12 void dfs(int u)
13 {
14     while ((int)graph[u].size() > 0)
15     {
16         auto [v, id] = graph[u].back();
17         graph[u].pop_back();
18         if (!vis[id])
19         {
20             vis[id] = true;
21             dfs(v);
22         }
23     }
24     ans.emplace_back(u); // Eulerian path reversed
25 }

```

3.4 Flows

3.4.1 Gomory-Hu Tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: GOMORY-HU TREE
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "All-pairs Min-Cut", "Minimum cut tree", "Undirected Graph Cut".
9  * - Classic Scenarios: You have an undirected graph and need to know the minimum
10 * cut between *every* possible pair of vertices.
11 * - The Magic: Normally, finding the min-cut between all pairs requires  $O(N^2)$ 
12 * Max Flow calls. The Gomory-Hu Tree (specifically Gusfield's algorithm
13 * implementation)
14 * builds a tree representing all min-cuts using only exactly  $N-1$  Max Flow
15 * calls!
16 * the path between them in this resulting tree.
17 *

```

```

17 * 2. HOW TO USE IT (THE BLACK BOX)
18 * - Initialization:
19 *     GomoryHu gh(N); // N is the number of nodes (0-indexed)
20 * - Add Edge:
21 *     gh.add_edge(u, v, cap); // Undirected edge
22 * - Build:
23 *     gh.build(); // Runs the N-1 Max Flow calls to construct the tree
24 * - Query:
25 *     long long ans = gh.min_cut(u, v);
26 *
27 * - Complexity:
28 *     Time:  $O(N * \text{MaxFlow}(V, E))$  to build.  $O(N)$  per query (or  $O(1)$  if you add LCA).
29 *     Space:  $O(N + E)$ 
30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - Dinic Dependency: This template assumes you have your `Dinic` struct
33 *   in the same file or included. Ensure `Dinic` has `add_edge(u, v, cap)` and
34 *   `max_flow(s, t)`.
35 * - 1-based indexing: If your nodes are 1 to N, instantiate `GomoryHu gh(N + 1)`
36 *   and just ignore index 0.
37 */
38 struct GomoryHu
39 {
40     int n;
41     struct Edge
42     {
43         int u, v;
44         long long cap;
45     };
46     vector<Edge> edges;
47
48     // Parent array for the tree (Gomory-Hu Tree structure)
49     vector<int> p;
50     // Weight array (min-cut value between i and p[i])
51     vector<long long> weight;
52     // Depth array for simple  $O(N)$  queries without complex LCA
53     vector<int> depth;
54
55     GomoryHu(int n) : n(n), p(n), weight(n), depth(n, 0) {}
56
57     void add_edge(int u, int v, long long cap)
58     {
59         edges.push_back({u, v, cap});
60     }
61
62     void build()
63     {
64         // 1. Initialize parents: everyone initially points to node 0
65         fill(p.begin(), p.end(), 0);
66         fill(weight.begin(), weight.end(), 0);
67
68         // 2. Iterate to build the tree (Gusfield's Algorithm)
69         for (int i = 1; i < n; i++)
70         {
71             // We rebuild the graph for Dinic each time.
72             // (Assuming `Dinic` is your standard Dinic template)
73             Dinic dinic(n);
74             for (auto &e : edges)
75             {
76                 dinic.add_edge(e.u, e.v, e.cap);
77                 dinic.add_edge(e.v, e.u, e.cap); // Gomory-Hu requires Undirected edges!
78             }
79
80             // Find min s-t cut between current node `i` and its parent `p[i]`
81             long long flow = dinic.max_flow(i, p[i]);
82             weight[i] = flow;
83
84             // Find which nodes are on the source side of the cut
85             // (reachable from s in the residual graph)
86             vector<bool> reachable(n, false);
87             dfs_residual(i, reachable, dinic);
88
89             // Update parents for next iterations
90             // If any node j > i points to p[i] but is actually on i's side of
91             // the cut,
92             // we redirect its parent to i.
93             for (int j = i + 1; j < n; j++)
94             {
95                 if (reachable[j] && p[j] == p[i])
96                 {
97                     p[j] = i;
98                 }
99             }
100         }
101
102         // 3. Precalculate depths to allow simple climbing during queries
103         for (int i = 1; i < n; i++)
104         {
105             depth[i] = depth[p[i]] + 1;
106         }
107
108     void dfs_residual(int u, vector<bool> &reachable, Dinic &dinic)
109     {
110         reachable[u] = true;
111         for (auto &e : dinic.adj[u])
112         {
113             // If capacity remains (cap - flow > 0), it is not saturated,
114             // so we can walk across it in the residual graph.
115             if (e.cap - e.flow > 0 && !reachable[e.to])
116             {
117                 dfs_residual(e.to, reachable, dinic);
118             }
119         }
120     }

```

```

121
122 // Query min cut between any two nodes
123 long long min_cut(int u, int v)
124 {
125     if (u == v)
126         return 0; // Or INF depending on problem definition
127
128     long long res = 2e18; // Use large INF
129
130     // Walk up the tree from u and v simultaneously to find their LCA,
131     // maintaining the minimum edge weight seen on the path.
132     while (u != v)
133     {
134         if (depth[u] > depth[v])
135         {
136             res = min(res, weight[u]);
137             u = p[u];
138         }
139         else
140         {
141             res = min(res, weight[v]);
142             v = p[v];
143         }
144     }
145     return res;
146 }
147 };

```

3.4.2 MCMF

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: MIN COST MAX FLOW (MCMF)
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Cheapest Flow", "Matching with Costs", "Assignment Problem".
9  * - Classic Scenarios: You have workers and tasks with different costs. You need
10 * to assign every worker a task such that the total cost is minimized.
11 * - The Magic: It works by finding the "shortest" path (where "shortest" is
12 * edge cost) in the residual graph. It keeps augmenting flow along these
13 * shortest
14 * paths until no path from source to sink exists.
15 *
16 * 2. HOW TO USE IT (THE BLACK BOX)
17 * - Initialization:
18 *     MCMF solver(N);
19 * - Add Edge:
20 *     solver.add_edge(u, v, capacity, cost);
21 * - Query:
22 *     pair<long long, long long> result = solver.min_cost_max_flow(source,
23 * sink);

```

```

22 * // result.first = max flow, result.second = min cost
23 *
24 * - Complexity:
25 *     O(F * E * log V) using Dijkstra + Potentials.
26 *     O(F * E * V) using SPFA (included below for simplicity with negative
27 * edges).
28 */
29 const long long INF = 1e18;
30
31 struct Edge
32 {
33     int to;
34     long long cap, flow, cost;
35     int rev; // Reverse edge index
36 };
37
38 struct MCMF
39 {
40     int n;
41     vector<vector<Edge>> adj;
42     vector<long long> dist, parent_edge, parent_node;
43
44     MCMF(int n) : n(n), adj(n), dist(n), parent_edge(n), parent_node(n) {}
45
46     void add_edge(int from, int to, long long cap, long long cost)
47     {
48         adj[from].push_back({to, cap, 0, cost, (int)adj[to].size()});
49         adj[to].push_back({from, 0, 0, -cost, (int)adj[from].size() - 1});
50     }
51
52     // SPFA is used here to handle negative costs.
53     // If all costs are positive, Dijkstra with potentials is faster.
54     bool spfa(int s, int t, long long &flow, long long &cost)
55     {
56         dist.assign(n, INF);
57         parent_node.assign(n, -1);
58         parent_edge.assign(n, -1);
59         vector<bool> in_queue(n, false);
60         queue<int> q;
61
62         dist[s] = 0;
63         q.push(s);
64         in_queue[s] = true;
65
66         while (!q.empty())
67         {
68             int u = q.front();
69             q.pop();
70             in_queue[u] = false;
71
72             for (int i = 0; i < (int)adj[u].size(); i++)
73             {
74                 Edge &e = adj[u][i];
75                 if (e.cap - e.flow > 0 && dist[e.to] > dist[u] + e.cost)

```

```

76     {
77         dist[e.to] = dist[u] + e.cost;
78         parent_node[e.to] = u;
79         parent_edge[e.to] = i;
80         if (!in_queue[e.to])
81         {
82             q.push(e.to);
83             in_queue[e.to] = true;
84         }
85     }
86 }
87
88 if (dist[t] == INF)
89     return false;
90
91 // Push flow
92 long long push = INF;
93 int curr = t;
94 while (curr != s)
95 {
96     int prev = parent_node[curr];
97     int idx = parent_edge[curr];
98     push = min(push, adj[prev][idx].cap - adj[prev][idx].flow);
99     curr = prev;
100 }
101
102 flow += push;
103 cost += push * dist[t];
104 curr = t;
105 while (curr != s)
106 {
107     int prev = parent_node[curr];
108     int idx = parent_edge[curr];
109     adj[prev][idx].flow += push;
110     int rev_idx = adj[prev][idx].rev;
111     adj[curr][rev_idx].flow -= push;
112     curr = prev;
113 }
114
115 return true;
116 }
117
118 pair<long long, long long> min_cost_max_flow(int s, int t)
119 {
120     long long flow = 0, cost = 0;
121     while (spfa(s, t, flow, cost))
122     ;
123     return {flow, cost};
124 }
125
126 };

```

3.4.3 Max flow

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DINIC'S ALGORITHM
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Max Flow", "Min-Cut", "Bipartite Matching", "Maximum Closure".
9  * - Classic Scenarios: You have a network of pipes with capacities and need to
   find
10  * the maximum amount of "stuff" (flow) that can move from a Source to a Sink.
11  * - The Magic:
12  *   a) BFS: Builds a Level Graph (only allows flow from distance d to d+1).
13  *   b) DFS: Pushes flow along the Level Graph.
14  *   c) ptr: The "pointer optimization" ensures we don't re-scan edges that are
   already saturated in the current DFS phase (the secret to  $O(V^2 E)$ ).
15  *
16  *
17  * 2. HOW TO USE IT (THE BLACK BOX)
18  * - Initialization:
19  *   Dinic flow(N);
20  * - Add Edge:
21  *   flow.add_edge(u, v, capacity);
22  * - Query:
23  *   long long max_f = flow.max_flow(source, sink);
24  *
25  * - Complexity:
26  *   General:  $O(V^2 * E)$ .
27  *   Unit Networks (e.g., Bipartite Matching):  $O(E * \text{sqrt}(V))$ .
28  */
29
30 struct Dinic
31 {
32     struct Edge
33     {
34         int to;
35         long long cap, flow;
36         int rev; // Index of the reverse edge in adj[to]
37     };
38
39     int n;
40     vector<vector<Edge>> adj;
41     vector<int> level, ptr;
42
43     Dinic(int n) : n(n), adj(n), level(n), ptr(n) {}
44
45     void add_edge(int from, int to, long long cap)
46     {
47         // Forward edge: capacity 'cap', flow 0
48         adj[from].push_back({to, cap, 0, (int)adj[to].size()});
49         // Backward edge: capacity 0, flow 0
50         adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
51     }
52
53     // BFS to build the Level Graph
54     bool bfs(int s, int t)

```

```

55 {
56     fill(level.begin(), level.end(), -1);
57     level[s] = 0;
58     queue<int> q;
59     q.push(s);
60     while (!q.empty())
61     {
62         int v = q.front();
63         q.pop();
64         for (auto &edge : adj[v])
65         {
66             if (edge.cap - edge.flow > 0 && level[edge.to] == -1)
67             {
68                 level[edge.to] = level[v] + 1;
69                 q.push(edge.to);
70             }
71         }
72     }
73     return level[t] != -1;
74 }
75
76 // DFS to push flow through the Level Graph
77 long long dfs(int v, int t, long long pushed)
78 {
79     if (pushed == 0)
80         return 0;
81     if (v == t)
82         return pushed;
83
84     // ? Dial 1: The Pointer Optimization
85     // ptr[v] ensures we don't look at edges that we already know are dead
86     // ends
87     for (int &cid = ptr[v]; cid < (int)adj[v].size(); cid++)
88     {
89         auto &edge = adj[v][cid];
90         int tr = edge.to;
91         if (level[v] + 1 != level[tr] || edge.cap - edge.flow == 0)
92             continue;
93
94         long long tr_pushed = dfs(tr, t, min(pushed, edge.cap - edge.flow));
95         if (tr_pushed == 0)
96             continue;
97
98         edge.flow += tr_pushed;
99         adj[tr][edge.rev].flow -= tr_pushed;
100        return tr_pushed;
101    }
102    return 0;
103 }
104
105 long long max_flow(int s, int t)
106 {
107     long long flow = 0;
108     while (bfs(s, t))
109     {

```

```

109         fill(ptr.begin(), ptr.end(), 0);
110         while (long long pushed = dfs(s, t, 1e18))
111         {
112             flow += pushed;
113         }
114     }
115     return flow;
116 }
117 };

```

3.5 Graph Traverse

3.5.1 Bipartite Graph

```

1 // Check Odd Cycle
2 // using in problems teams or groups or colors
3 vector<int> teams;
4 Graph adj;
5 int n, m;
6 bool valid_team = true;
7 void bfs(int start) {
8     queue<int> q;
9     q.push(start);
10    if (teams[start] == 0)
11        teams[start] = 1;
12    while (!q.empty())
13    {
14        int cur = q.front(); q.pop();
15        for (auto ch : adj[cur])
16        {
17            if (!teams[ch])
18            {
19                q.push(ch);
20                teams[ch] = 3 - teams[cur];
21            }
22            else if (teams[ch] == teams[cur])
23            {
24                valid_team = false;
25                return;
26            }
27        }
28    }
29 }
30 }

```

3.5.2 Euler tour

```

1 const int N = 1e5 + 5;
2 GRAPH graph(N);
3 int tin[N], tout[N], value[N], flat[N];
4 int timer{1};
5

```

```

6 void dfs(int node, int p)
7 {
8     flat[timer] = value[node];
9     tin[node] = timer++;
10    for (auto &x : graph[node])
11    {
12        if (x != p)
13            dfs(x, node);
14    }
15    tout[node] = timer;
16    // [tin, tout)
17 }

```

3.5.3 Top sort

```

1 vector<int> res;
2
3 void topo_DFS(int node)
4 {
5     vis[node] = true;
6     for (auto &neighbor : graph[node])
7     {
8         if (!vis[neighbor])
9             topo_DFS(neighbor);
10    }
11    res.push_back(node);
12 }
13
14 // Then reverse
15
16 vector<int> topSort()
17 {
18     int n = int(graph.size());
19     vector<int> inDegree(n);
20     for (int i = 0; i < n; i++)
21     {
22         for (auto &neighbor : graph[i])
23             inDegree[neighbor]++;
24     }
25     queue<int> ready;
26     vector<int> ordering;
27     for (int i = 0; i < n; i++)
28     {
29         if (!inDegree[i])
30             ready.push(i);
31     }
32     while (!ready.empty())
33     {
34         int node = ready.front();
35         ready.pop();
36         ordering.push_back(node);
37         for (auto &neighbor : graph[node])
38         {

```

```

39             if (!--inDegree[neighbor])
40                 ready.push(neighbor);
41         }
42     }
43     if ((int)ordering.size() != int(graph.size()))
44         ordering.clear();
45     return ordering;
46 }

```

3.6 Graph connectivity

3.6.1 Articulations

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1e5 + 5;
5 vector<int> adj[MAXN];
6 int disc[MAXN], low[MAXN], timer;
7 vector<pair<int, int>> bridges;
8 set<int> articulation_points;
9
10 void dfs(int u, int p = -1)
11 {
12     disc[u] = low[u] = ++timer;
13     int children = 0; // Needed for the root case
14
15     for (int v : adj[u])
16     {
17         if (v == p)
18             continue; // Don't go back to parent
19
20         if (disc[v])
21         {
22             // Back-edge found!
23             low[u] = min(low[u], disc[v]);
24         }
25         else
26         {
27             // Tree-edge
28             dfs(v, u);
29             low[u] = min(low[u], low[v]);
30
31             // BRIDGE CONDITION
32             if (low[v] > disc[u])
33                 bridges.push_back({u, v});
34
35             // ARTICULATION POINT CONDITION
36             if (low[v] >= disc[u] && p != -1)
37                 articulation_points.insert(u);
38
39             children++;
40         }

```

```

41     }
42
43     // ROOT SPECIAL CASE: If root has > 1 child, it's an AP
44     if (p == -1 && children > 1)
45         articulation_points.insert(u);
46 }

```

3.6.2 SCC

```

1  const int N = 1e5 + 9;
2
3  int SCC_id[N];
4  GRAPH graph, graph_reverse, cond_graph;
5  int n, m;
6  vector<int> order, vis, vis_reverse, order_reverse;
7
8  void dfs(int node)
9  {
10     vis[node] = true;
11     for (auto &x : graph[node])
12     {
13         if (!vis[x])
14             dfs(x);
15     }
16     order.emplace_back(node);
17 }
18
19 void dfs_reverse(int node)
20 {
21     vis_reverse[node] = true;
22     for (auto &x : graph_reverse[node])
23     {
24         if (!vis_reverse[x])
25             dfs_reverse(x);
26     }
27     order_reverse.emplace_back(node);
28 }
29
30 void SCC()
31 {
32     graph.assign(n + 1, {}), graph_reverse.assign(n + 1, {}), vis.assign(n + 1, {}),
33     vis_reverse.assign(n + 1, {}), cond_graph.assign(n + 1, {});
34     for (int i = 0; i < m; i++)
35     {
36         int x, y;
37         cin >> x >> y;
38         graph[x].emplace_back(y), graph_reverse[y].emplace_back(x);
39     }
40     order.clear();
41     for (int i = 1; i <= n; i++)
42     {
43         if (!vis[i])

```

```

44         dfs(i);
45     }
46     reverse(order.begin(), order.end());
47     vector<vector<int>> scc;
48     scc.push_back({});
49     for (auto &x : order)
50     {
51         if (!vis_reverse[x])
52         {
53             order_reverse.clear();
54             dfs_reverse(x);
55             scc.emplace_back(order_reverse);
56             for (auto &val : order_reverse)
57                 SCC_id[val] = int(scc.size()) - 1;
58         }
59     }
60     for (int u = 1; u <= n; u++)
61     {
62         for (int v : graph[u])
63         {
64             if (SCC_id[u] != SCC_id[v])
65                 cond_graph[SCC_id[u]].emplace_back(SCC_id[v]);
66         }
67     }
68 }

```

3.7 Graph counting

3.7.1 BEST

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  /**
5   * THE ULTIMATE CP TEMPLATE BLUEPRINT: BEST THEOREM
6   *
7   * 1. WHAT PROBLEM DOES THIS SOLVE?
8   * - Keywords: "Count Eulerian Circuits", "Directed Graph Cycles".
9   * - Classic Scenarios: You have a directed graph where every node's in-degree
10  *   equals
11  *   its out-degree, and you need to know how many distinct Eulerian circuits it
12  *   has.
13  * - The Magic: The number of Eulerian circuits =  $t_w(G) * \prod ((deg(v) - 1)!)$ 
14  *   where  $t_w(G)$  is the number of arborescences rooted at any vertex  $w$ .
15  */
16
17 const int MOD = 1e9 + 7;
18
19 // Gaussian Elimination for Determinant (MOD)
20 long long power(long long base, long long exp)
21 {
22     long long res = 1;
23     base %= MOD;

```

```

22 while (exp > 0)
23 {
24     if (exp % 2 == 1)
25         res = (res * base) % MOD;
26     base = (base * base) % MOD;
27     exp /= 2;
28 }
29 return res;
30 }
31
32 long long modInverse(long long n) { return power(n, MOD - 2); }
33
34 long long determinant(vector<vector<long long>> &mat, int n)
35 {
36     long long det = 1;
37     for (int i = 0; i < n; i++)
38     {
39         int pivot = i;
40         for (int j = i + 1; j < n; j++)
41             if (abs(mat[j][i]) > abs(mat[pivot][i]))
42                 pivot = j;
43         swap(mat[i], mat[pivot]);
44         if (i != pivot)
45             det = (MOD - det) % MOD;
46         if (mat[i][i] == 0)
47             return 0;
48         det = (det * mat[i][i]) % MOD;
49         long long inv = modInverse(mat[i][i]);
50         for (int j = i + 1; j < n; j++)
51         {
52             long long factor = (mat[j][i] * inv) % MOD;
53             for (int k = i + 1; k < n; k++)
54                 mat[j][k] = (mat[j][k] - factor * mat[i][k] % MOD + MOD) % MOD;
55         }
56     }
57     return det;
58 }
59
60 long long best_theorem(int n, vector<vector<int>> &adj)
61 {
62     vector<int> out_degree(n, 0);
63     vector<vector<long long>> L(n, vector<long long>(n, 0));
64
65     // 1. Build Directed Laplacian
66     for (int u = 0; u < n; ++u)
67     {
68         for (int v : adj[u])
69         {
70             out_degree[u]++;
71             L[u][v]--;
72         }
73     }
74     for (int i = 0; i < n; ++i)
75         L[i][i] += out_degree[i];
76

```

```

77 // 2. Compute arborescences (t_w)
78 // Delete row 0 and col 0 (root at 0)
79 vector<vector<long long>> minor(n - 1, vector<long long>(n - 1));
80 for (int i = 1; i < n; i++)
81     for (int j = 1; j < n; j++)
82         minor[i - 1][j - 1] = (L[i][j] % MOD + MOD) % MOD;
83
84 long long t_w = determinant(minor, n - 1);
85
86 // 3. Multiply by prod((deg(v) - 1)!)
87 long long fact_prod = 1;
88 vector<long long> fact(n + 1, 1);
89 for (int i = 2; i <= n; i++)
90     fact[i] = (fact[i - 1] * i) % MOD;
91
92 for (int i = 0; i < n; i++)
93 {
94     if (out_degree[i] > 0)
95         fact_prod = (fact_prod * fact[out_degree[i] - 1]) % MOD;
96 }
97
98 return (t_w * fact_prod) % MOD;
99 }
100
101 int main()
102 {
103     ios_base::sync_with_stdio(false);
104     cin.tie(NULL);
105     // int n; cin >> n; ... adj list setup ...
106     return 0;
107 }

```

3.7.2 Prufer code

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PRUFER CODE
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Labeled Trees", "Cayley's Formula", "Tree Counting".
9  * - Classic Scenarios: Bijectively mapping a labeled tree to a sequence of
10    length N-2.
11  * - The Magic: It allows you to count trees with specific constraints (like
12    degrees)
13    by counting sequences instead, which is mathematically trivial.
14 */
15 struct Prufer
16 {
17     // Converts a labeled tree (adj list) to a Prufer sequence of length N-2
18     // Complexity: O(N log N)

```

```

18 static vector<int> tree_to_prufer(int n, vector<vector<int>> &adj)
19 {
20     if (n == 2)
21         return {}; // A 2-node tree has no Prufer sequence
22
23     vector<int> degree(n + 1);
24     priority_queue<int, vector<int>, greater<int>> leaves;
25
26     for (int i = 1; i <= n; i++)
27     {
28         degree[i] = adj[i].size();
29         if (degree[i] == 1)
30             leaves.push(i);
31     }
32
33     vector<int> prufer;
34     vector<bool> removed(n + 1, false);
35
36     for (int i = 0; i < n - 2; i++)
37     {
38         int leaf = leaves.top();
39         leaves.pop();
40         removed[leaf] = true;
41
42         // Find the neighbor of this leaf
43         int neighbor = -1;
44         for (int v : adj[leaf])
45         {
46             if (!removed[v])
47             {
48                 neighbor = v;
49                 break;
50             }
51         }
52
53         prufer.push_back(neighbor);
54         degree[neighbor]--;
55         if (degree[neighbor] == 1)
56             leaves.push(neighbor);
57     }
58
59     return prufer;
60 }
61
62 // Converts a Prufer sequence of length N-2 back to an edge list of the tree
63 // Complexity: O(N log N)
64 static vector<pair<int, int>> prufer_to_tree(int n, const vector<int> &prufer)
65 {
66     vector<int> degree(n + 1, 1);
67     for (int x : prufer)
68         degree[x]++;
69
70     priority_queue<int, vector<int>, greater<int>> leaves;
71     for (int i = 1; i <= n; i++)

```

```

72     {
73         if (degree[i] == 1)
74             leaves.push(i);
75     }
76
77     vector<pair<int, int>> edges;
78     for (int x : prufer)
79     {
80         int leaf = leaves.top();
81         leaves.pop();
82         edges.push_back({leaf, x});
83
84         degree[x]--;
85         if (degree[x] == 1)
86             leaves.push(x);
87     }
88
89     // Connect the last two remaining nodes
90     int u = leaves.top();
91     leaves.pop();
92     int v = leaves.top();
93     leaves.pop();
94     edges.push_back({u, v});
95
96     return edges;
97 }
98 };
99
100 void solve()
101 {
102     int n;
103     cin >> n;
104     // Example: Tree to Prufer
105     // vector<vector<int>> adj(n+1);
106     // ... fill adj ...
107     // vector<int> p = Prufer::tree_to_prufer(n, adj);
108 }
109
110 int main()
111 {
112     ios_base::sync_with_stdio(false);
113     cin.tie(NULL);
114     return 0;
115 }

```

3.8 LCA

3.8.1 LCA(Hassan)

```

1 struct LCA {
2     vector<vector<pair<int,int>> > adj;
3     vector<vector<int>> up, cost[2];
4     vector<int> depth;

```

```

5  int LOG = 30;
6  int ign = 1e9;
7  void build(int n, vector<vector<pair<int,int>> > &G, int root = 1) {
8      adj = G;
9      up = vector<vector<int> >(LOG + 5, vector<int>(n + 5));
10     cost[0] = vector<vector<int> >(LOG + 5, vector<int>(n + 5));
11     cost[1] = vector<vector<int> >(LOG + 5, vector<int>(n + 5));
12     depth = vector<int>(n + 5);
13     dfs(root, -1);
14 }
15 void dfs(int u, int p) {
16     for (auto [v,w]: adj[u]) {
17         if (v == p) continue;
18         depth[v] = depth[u] + 1;
19
20         up[0][v] = u;
21         cost[0][0][v] = w;
22         cost[1][0][v] = w;
23         for (int i = 1; i < LOG; i++) {
24             up[i][v] = up[i - 1][up[i - 1][v]];
25             cost[0][i][v] = min(cost[0][i - 1][v], cost[0][i - 1][up[i - 1][v]]);
26             cost[1][i][v] = max(cost[1][i - 1][v], cost[1][i - 1][up[i - 1][v]]);
27         }
28         dfs(v, u);
29     }
30 }
31
32
33 int kth_ancestor(int u, int k) {
34     for (int i = 0; i < LOG; i++) {
35         if (k >> i & 1) {
36             u = up[i][u];
37         }
38     }
39     return u;
40 }
41
42 int getLca(int u, int v) {
43     if (depth[u] < depth[v]) swap(u, v);
44     int k = depth[u] - depth[v];
45     u = kth_ancestor(u, k);
46     if (u == v) return u;
47
48     for (int i = LOG - 1; i >= 0; --i) {
49         if (up[i][u] != up[i][v]) {
50             u = up[i][u];
51             v = up[i][v];
52         }
53     }
54     return up[0][v];
55 }
56 pair<int,int> query(int u,int v) {
57     int mn = ign,mx = -ign;

```

```

58     int lca = getLca(u, v);
59     int k = depth[u] - depth[lca];
60     for (int i=0;i<LOG;i++) {
61         if (k>>i&1) {
62             mn = min(mn,cost[0][i][u]);
63             mx = max(mx,cost[1][i][u]);
64             u = up[i][u];
65         }
66     }
67     k = depth[v] - depth[lca];
68     for (int i=0;i<LOG;i++) {
69         if (k>>i&1) {
70             mn = min(mn,cost[0][i][v]);
71             mx = max(mx,cost[1][i][v]);
72             v = up[i][v];
73         }
74     }
75     return {mn,mx};
76 }
77
78
79 };

```

3.8.2 LCA

```

1  struct LCA
2  {
3      int n, LG, SKIP;
4      vector<vector<int>> T, Cost;
5      vector<int> par, dep;
6
7      LCA(int nodes_count, const GRAPH &graph, int root = 1, int skip_val = 00)
8      {
9          n = nodes_count;
10         LG = __lg(n) + 2;
11         SKIP = skip_val;
12         T.assign(LG, vector<int>(n + 1, -1));
13         Cost.assign(LG, vector<int>(n + 1, SKIP));
14         par.assign(n + 1, -1);
15         dep.assign(n + 1, 0);
16
17         Cost[0][root] = SKIP;
18         dfs(root, -1, 0, graph);
19
20         for (int i = 1; i <= n; i++)
21             T[0][i] = par[i];
22
23         for (int pw = 1; (1 << pw) <= n; pw++)
24         {
25             for (int i = 1; i <= n; i++)
26             {
27                 if (T[pw - 1][i] != -1)
28                     {

```

```

29         T[pw][i] = T[pw - 1][T[pw - 1][i]];
30         Cost[pw][i] = merge(Cost[pw - 1][i], Cost[pw - 1][T[pw - 1][i]]);
31     }
32 }
33 }
34 }
35
36 int merge(int a, int b)
37 {
38     return min(a, b);
39 }
40
41 void dfs(int node, int p, int d, const GRAPH &graph)
42 {
43     dep[node] = d;
44     par[node] = p;
45     for (auto &[i, w] : graph[node])
46     {
47         if (i != p)
48         {
49             Cost[0][i] = w;
50             dfs(i, node, d + 1, graph);
51         }
52     }
53 }
54
55 int get_kth(int x, int k)
56 {
57     while (k)
58     {
59         if (x == -1)
60             break;
61         int pw = __builtin_ctz(k);
62         x = T[pw][x];
63         k &= (k - 1);
64     }
65     return x;
66 }
67
68 int get_lca(int u, int v)
69 {
70     if (dep[u] < dep[v])
71         swap(u, v);
72     u = get_kth(u, dep[u] - dep[v]);
73     if (u == v)
74         return u;
75     for (int j = LG - 1; j >= 0; j--)
76     {
77         if (T[j][u] != T[j][v])
78         {
79             u = T[j][u];
80             v = T[j][v];
81         }
82     }

```

```

83     return par[u];
84 }
85
86 int get_cost(int u, int v)
87 {
88     if (dep[u] < dep[v])
89         swap(u, v);
90     int k = dep[u] - dep[v], x = u;
91     int cost = SKIP;
92     while (k)
93     {
94         if (x == -1)
95             break;
96         int pw = __builtin_ctz(k);
97         cost = merge(cost, Cost[pw][x]);
98         x = T[pw][x];
99         k &= (k - 1);
100     }
101     return cost;
102 }
103
104 int query_path(int u, int v)
105 {
106     int lca = get_lca(u, v);
107     return merge(get_cost(u, lca), get_cost(v, lca));
108 }
109
110 int get_dist(int u, int v)
111 {
112     int lca = get_lca(u, v);
113     return dep[u] + dep[v] - 2 * dep[lca];
114 }
115 };

```

3.9 MST

3.9.1 Directed MST

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DIRECTED MST (CHU-LIU/EDMONDS)
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Minimum Spanning Arborescence", "Directed MST".
9  * - Classic Scenarios: Finding the cheapest way to connect all nodes in a
    DIRECTED graph
10 * rooted at a specific node, such that every node (except the root) has
    exactly 1 incoming edge.
11 * - The Magic: It greedily picks the minimum incoming edge for every node. If
    this creates
12 * cycles, it "contracts" those cycles into a single super-node, updates the
    weights of edges

```

```

13  *   entering the cycle, and recurses.
14  *
15  * 2. HOW TO USE IT
16  * - edge: {u, v, cost}
17  * - roots: The node that acts as the root of the arborescence.
18  * - Complexity:  $O(V * E)$ 
19  */
20
21 struct Edge
22 {
23     int u, v;
24     long long cost;
25 };
26
27 long long edmonds(int n, int root, vector<Edge> &edges)
28 {
29     long long total_cost = 0;
30     vector<int> min_edge(n + 1);
31     vector<int> parent(n + 1);
32     vector<int> id(n + 1);
33     vector<int> vis(n + 1);
34
35     while (true)
36     {
37         fill(min_edge.begin(), min_edge.end(), 2e18); // Use large INF
38         for (const auto &e : edges)
39         {
40             if (e.u != e.v && e.cost < min_edge[e.v])
41             {
42                 min_edge[e.v] = e.cost;
43                 parent[e.v] = e.u;
44             }
45         }
46
47         for (int i = 1; i <= n; i++)
48         {
49             if (i != root && min_edge[i] == 2e18)
50                 return -1; // Impossible
51         }
52
53         int cnt_nodes = 0;
54         fill(id.begin(), id.end(), -1);
55         fill(vis.begin(), vis.end(), -1);
56         min_edge[root] = 0;
57
58         for (int i = 1; i <= n; i++)
59         {
60             total_cost += min_edge[i];
61             int v = i;
62             while (vis[v] != i && id[v] == -1 && v != root)
63             {
64                 vis[v] = i;
65                 v = parent[v];
66             }
67             if (v != root && id[v] == -1)

```

```

68         {
69             for (int u = parent[v]; u != v; u = parent[u])
70                 id[u] = cnt_nodes;
71             id[v] = cnt_nodes++;
72         }
73     }
74
75     if (cnt_nodes == 0)
76         break;
77     for (int i = 1; i <= n; i++)
78     {
79         if (id[i] == -1)
80             id[i] = cnt_nodes++;
81     }
82
83     for (auto &e : edges)
84     {
85         long long v_cost = min_edge[e.v];
86         e.u = id[e.u];
87         e.v = id[e.v];
88         if (e.u != e.v)
89             e.cost -= v_cost;
90     }
91     n = cnt_nodes;
92     root = id[root];
93 }
94 return total_cost;
95 }

```

3.9.2 Kirchoff

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: KIRCHHOFF'S THEOREM
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Count spanning trees", "Matrix Tree Theorem".
9  * - Classic Scenarios: You have a graph and need to know exactly how many
10  *   different
11  *   spanning trees can be formed.
12  * - The Magic: It converts a graph problem into a linear algebra problem.
13  *   It constructs the Laplacian Matrix and computes a determinant using
14  *   Gaussian Elimination.
15  *
16  * 2. HOW TO USE IT
17  * - Build the Laplacian matrix ( $L = \text{Degree} - \text{Adjacency}$ ).
18  * - Remove the last row and last column.
19  * - Compute the determinant modulo MOD.
20  * - Complexity:  $O(N^3)$  (due to Gaussian Elimination).
21  */

```

```

22 const int MOD = 1e9 + 7;
23
24 long long power(long long base, long long exp)
25 {
26     long long res = 1;
27     base %= MOD;
28     while (exp > 0)
29     {
30         if (exp % 2 == 1)
31             res = (res * base) % MOD;
32         base = (base * base) % MOD;
33         exp /= 2;
34     }
35     return res;
36 }
37
38 long long modInverse(long long n)
39 {
40     return power(n, MOD - 2);
41 }
42
43 // Gaussian Elimination to find Determinant
44 long long determinant(vector<vector<long long>> &mat, int n)
45 {
46     long long det = 1;
47     for (int i = 0; i < n; i++)
48     {
49         int pivot = i;
50         for (int j = i + 1; j < n; j++)
51         {
52             if (abs(mat[j][i]) > abs(mat[pivot][i]))
53                 pivot = j;
54         }
55         swap(mat[i], mat[pivot]);
56         if (i != pivot)
57             det = (MOD - det) % MOD;
58         if (mat[i][i] == 0)
59             return 0;
60
61         det = (det * mat[i][i]) % MOD;
62         long long inv = modInverse(mat[i][i]);
63         for (int j = i + 1; j < n; j++)
64         {
65             long long factor = (mat[j][i] * inv) % MOD;
66             for (int k = i + 1; k < n; k++)
67             {
68                 mat[j][k] = (mat[j][k] - factor * mat[i][k] % MOD + MOD) % MOD;
69             }
70         }
71     }
72     return det;
73 }
74
75 void solve()
76 {

```

```

77     int n, m;
78     cin >> n >> m;
79
80     // Laplacian Matrix
81     vector<vector<long long>> L(n, vector<long long>(n, 0));
82
83     for (int i = 0; i < m; i++)
84     {
85         int u, v;
86         cin >> u >> v;
87         --u;
88         --v; // 0-indexed
89
90         L[u][u]++;
91         L[v][v]++;
92         L[u][v]--;
93         L[v][u]--;
94     }
95
96     // Convert to positive modulo
97     for (int i = 0; i < n; i++)
98         for (int j = 0; j < n; j++)
99             L[i][j] = (L[i][j] % MOD + MOD) % MOD;
100
101     // Remove last row and column to get the Minor
102     vector<vector<long long>> minor(n - 1, vector<long long>(n - 1));
103     for (int i = 0; i < n - 1; i++)
104     {
105         for (int j = 0; j < n - 1; j++)
106         {
107             minor[i][j] = L[i][j];
108         }
109     }
110
111     cout << determinant(minor, n - 1) << "\n";
112 }
113
114 int main()
115 {
116     ios_base::sync_with_stdio(false);
117     cin.tie(NULL);
118     solve();
119     return 0;
120 }

```

3.9.3 Kruskal

```

1 class DSU
2 {
3 private:
4     vector<int> parent, rank;
5     int forests{};
6

```

```

7   void link(int x, int y)
8   {
9       if (rank[x] > rank[y])
10          swap(x, y);
11       parent[x] = y;
12       if (rank[x] == rank[y])
13          rank[y]++;
14   }
15
16 public:
17     DSU(int n)
18     {
19         parent = vector<int>(n), rank = vector<int>(n);
20         forests = n;
21         for (int i = 0; i < n; i++)
22             parent[i] = i, rank[i] = 1;
23     }
24
25     int find_set(int x)
26     {
27         if (x == parent[x])
28             return x;
29         return parent[x] = find_set(parent[x]);
30     }
31
32     int union_sets(int x, int y)
33     {
34         x = find_set(x), y = find_set(y);
35         if (x != y)
36         {
37             link(x, y);
38             forests--;
39         }
40         return x != y;
41     }
42 };
43
44 struct edge
45 {
46     int from, to, w;
47
48     edge(int from, int to, int w) : from(from), to(to), w(w)
49     {
50     }
51
52     bool operator<(const edge &e) const
53     {
54         return w < e.w;
55     }
56 };
57
58 int Kruskal(vector<edge> &edgeList, int n)
59 {
60     DSU uf(n + 1);
61     vector<edge> edges;

```

```

62     int mnCost{};
63     sort(edgeList.begin(), edgeList.end());
64     for (auto &[from, to, cost] : edgeList)
65     {
66         if (uf.union_sets(from, to))
67         {
68             edges.emplace_back(from, to, cost);
69             mnCost += cost;
70         }
71     }
72     return mnCost;
73 }

```

3.9.4 Prim

```

1  int n;
2  GRAPH graph;
3
4  void prim(int node)
5  {
6      int total{};
7      priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
8      pq.emplace(0, node);
9      vector<bool> visited(n + 1, 0);
10     vector<int> dist(n + 1, 00);
11     dist[node] = 0;
12     while (!pq.empty())
13     {
14         auto [w, node] = pq.top();
15         pq.pop();
16         if (visited[node])
17             continue;
18         visited[node] = true;
19         total += w;
20         for (auto &[to, c] : graph[node])
21         {
22             if (!visited[to] && dist[to] > c)
23             {
24                 dist[to] = c;
25                 pq.push({dist[to], to});
26             }
27         }
28     }
29 }

```

3.10 Matching

3.10.1 Bipartite matching

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: HOPCROFT-KARP ALGORITHM
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Maximum Bipartite Matching", "Maximum Independent Set", "Minimum
9  * - Classic Scenarios: You have two distinct groups (e.g., Workers and Jobs, or
10 * Each worker can only do specific jobs. You want to pair up as many workers
11 * such that no worker gets two jobs, and no job gets two workers.
12 * - The Magic: It finds the maximum matching in a bipartite graph. It works by
13 * multiple shortest augmenting paths simultaneously, and then uses DFS to push
14 *
15 * 2. HOW TO USE IT (THE BLACK BOX)
16 * - Initialization: Pass the size of the Left partition (N) and Right partition
17 * (M).
18 * HopcroftKarp hk(N, M);
19 *
20 * - Add Edge:
21 * hk.add_edge(u, v); // Worker `u` (1 to N) can do Job `v` (1 to M).
22 *
23 * - Query:
24 * int max_pairs = hk.max_matching();
25 *
26 * - Check Pairs:
27 * // After running max_matching(), pair_u[u] gives the job assigned to
28 * // worker u.
29 * // If pair_u[u] == 0, worker u is unmatched.
30 *
31 * - Complexity:
32 * Time:  $O(E * \sqrt{V})$ . Blazing fast for up to  $10^5$  nodes and edges.
33 * Space:  $O(V + E)$  memory.
34 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
35 * - Knig's Theorem (Min Vertex Cover):
36 * To find the minimum number of nodes needed to "cover" all edges:
37 * 1. Run Max Matching. Min Vertex Cover = Max Matching.
38 * 2. To find the ACTUAL nodes: Run a DFS from all UNMATCHED nodes in the Left
39 * partition
40 * using alternating paths. The Vertex Cover = (Unvisited nodes in Left) +
41 * Visited nodes in Right).
42 *
43 * - Maximum Independent Set:
44 * The maximum number of nodes where no two nodes share an edge.
45 * Formula: Max Independent Set = (Total Nodes) - (Min Vertex Cover).
46 */
47 struct HopcroftKarp
48 {
49     int n; // Size of the Left partition (U)
50
51     int m; // Size of the Right partition (V)
52
53     vector<vector<int>> adj;
54
55     // pair_u[i] = the node in V matched with node i in U
56     // pair_v[j] = the node in U matched with node j in V
57     vector<int> pair_u, pair_v;
58
59     // distance array used by BFS to find shortest augmenting paths
60     vector<int> dist;
61
62     const int INF = 1e9;
63
64     HopcroftKarp(int n, int m) : n(n), m(m)
65     {
66         adj.resize(n + 1);
67         pair_u.assign(n + 1, 0);
68         pair_v.assign(m + 1, 0);
69         dist.assign(n + 1, 0);
70     }
71
72     void add_edge(int u, int v)
73     {
74         adj[u].push_back(v); // u is in Left (1...N), v is in Right (1...M)
75     }
76
77     // BFS builds the level graph, finding the shortest paths to unmatched nodes
78     // in V
79     bool bfs()
80     {
81         queue<int> q;
82
83         for (int u = 1; u <= n; u++)
84         {
85             // If u is unmatched, it's a starting point for an augmenting path
86             if (pair_u[u] == 0)
87             {
88                 dist[u] = 0;
89                 q.push(u);
90             }
91             else
92             {
93                 dist[u] = INF; // Otherwise, mark as unvisited
94             }
95         }
96
97         // Node 0 acts as a dummy node for all unmatched nodes in V
98         dist[0] = INF;
99
100         while (!q.empty())
101         {
102             int u = q.front();
103             q.pop();
104
105             // If this node has a distance less than the dummy node, we can

```

```

103     explore it
104     if (dist[u] < dist[0])
105     {
106         for (int v : adj[u])
107         {
108             // If the node matched to v is unvisited, we can walk there
109             if (dist[pair_v[v]] == INF)
110             {
111                 dist[pair_v[v]] = dist[u] + 1;
112                 q.push(pair_v[v]);
113             }
114         }
115     }
116
117     // If dist[0] is no longer INF, we found at least one augmenting path!
118     return dist[0] != INF;
119 }
120
121 // DFS greedily pushes "flow" along the paths discovered by BFS
122 bool dfs(int u)
123 {
124     if (u != 0)
125     {
126         for (int v : adj[u])
127         {
128             // Ensure we only walk along the shortest path DAG built by BFS
129             if (dist[pair_v[v]] == dist[u] + 1)
130             {
131                 if (dfs(pair_v[v]))
132                 {
133                     // Augment the path! Flip the matching status.
134                     pair_v[v] = u;
135                     pair_u[u] = v;
136                     return true;
137                 }
138             }
139         }
140         // If no augmenting path could be found from u, mark it as dead end
141         dist[u] = INF;
142         return false;
143     }
144     // Base case: Reached the dummy node 0 (an unmatched node in V)
145     return true;
146 }
147
148 // Repeatedly runs BFS and DFS to build the maximum matching
149 int max_matching()
150 {
151     int matching = 0;
152
153     // While there is an augmenting path...
154     while (bfs())
155     {
156         // Try to augment from every unmatched node in U

```

```

157         for (int u = 1; u <= n; u++)
158         {
159             if (pair_u[u] == 0 && dfs(u))
160             {
161                 matching++;
162             }
163         }
164     }
165     return matching;
166 }
167 };
168
169 void solve()
170 {
171     int n, m, edges;
172     // Example: N workers, M jobs, and E compatible worker-job pairs
173     if (!(cin >> n >> m >> edges))
174         return;
175
176     HopcroftKarp hk(n, m);
177
178     for (int i = 0; i < edges; i++)
179     {
180         int u, v;
181         cin >> u >> v;
182         hk.add_edge(u, v);
183     }
184
185     cout << "Maximum Pairs: " << hk.max_matching() << "\n";
186
187     // Print the actual assignments
188     for (int u = 1; u <= n; u++)
189     {
190         if (hk.pair_u[u] != 0)
191         {
192             cout << "Worker " << u << " is assigned Job " << hk.pair_u[u] << "\n"
193                 << "\n";
194         }
195     }
196
197 int main()
198 {
199     ios_base::sync_with_stdio(false);
200     cin.tie(NULL);
201     // solve();
202     return 0;
203 }

```

3.10.2 Hungarian algorithm

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

```

3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: HUNGARIAN ALGORITHM
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Optimal Assignment", "Bipartite Weighted Matching", "Min-Cost
9  *   Perfect Matching".
10 * - Classic Scenarios: You have N workers and M jobs. Every worker-job pair has
11 *   a specific
12 *   cost or time to complete. You want to assign exactly one job to each worker
13 *   such that
14 *   the total cost is minimized.
15 * - The Magic: While MCMF works, Hungarian runs in a strict, highly-optimized
16 *    $O(N * M^2)$  time,
17 *   making it the undisputed king for dense assignment matrices.
18 *
19 * 2. HOW TO USE IT (THE BLACK BOX)
20 * - Initialization:
21 *   // Create a 1-indexed cost matrix of size (N+1) x (M+1)
22 *   // Important: N must be <= M! (Rows <= Cols)
23 *   Hungarian solver(n, m, cost_matrix);
24 *
25 * - Query:
26 *   long long min_cost = solver.solve();
27 *
28 * - Extract Assignments:
29 *   // solver.ans[i] will contain the job (1 to M) assigned to worker i.
30 *
31 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
32 * - Maximum Profit Assignment?
33 *   If you want to MAXIMIZE the score, simply negate all values in your matrix
34 *   cost = -cost)
35 *   before passing it in. Then negate the final answer returned by `solve()`.
36 * - Forbidden Assignments?
37 *   If worker `i` cannot do job `j`, set `cost[i][j] = 1e15` (Infinity).
38 */
39
40 struct Hungarian
41 {
42     int n, m;
43     vector<vector<long long>> a;
44     vector<long long> u, v, minv;
45     vector<int> p, way, ans;
46     const long long INF = 1e15;
47
48     Hungarian(int n, int m, const vector<vector<long long>> &cost_matrix)
49     {
50         this->n = n;
51         this->m = m;
52         // 1-based indexing logic
53         a = cost_matrix;
54         u.assign(n + 1, 0);
55         v.assign(m + 1, 0);
56         p.assign(m + 1, 0);
57         way.assign(m + 1, 0);
58     }
59
60     long long solve()
61     {
62         // ... (Implementation details) ...
63     }
64 }

```

```

53     ans.assign(n + 1, 0);
54 }
55
56 /* STREAMING_CHUNK: Core  $O(N * M^2)$  solver using augmenting paths */
57 long long solve()
58 {
59     // Iterate over every worker (row) to find them a job
60     for (int i = 1; i <= n; ++i)
61     {
62         p[0] = i;
63         int j0 = 0; // Dummy column 0
64
65         minv.assign(m + 1, INF);
66         vector<bool> used(m + 1, false);
67
68         do
69         {
70             used[j0] = true;
71             int i0 = p[j0], j1 = 0;
72             long long delta = INF;
73
74             // Update minv array and find the next column to explore
75             for (int j = 1; j <= m; ++j)
76             {
77                 if (!used[j])
78                 {
79                     long long cur = a[i0][j] - u[i0] - v[j];
80                     if (cur < minv[j])
81                     {
82                         minv[j] = cur;
83                         way[j] = j0;
84                     }
85                     if (minv[j] < delta)
86                     {
87                         delta = minv[j];
88                         j1 = j;
89                     }
90                 }
91             }
92
93             // Update potentials (dual variables)
94             for (int j = 0; j <= m; ++j)
95             {
96                 if (used[j])
97                 {
98                     u[p[j]] += delta;
99                     v[j] -= delta;
100                 }
101                 else
102                 {
103                     minv[j] -= delta;
104                 }
105             }
106             j0 = j1;
107         } while (p[j0] != 0); // Stop when we reach an unmatched column

```

```

108
109         do
110         {
111             int j1 = way[j0];
112             p[j0] = p[j1];
113             j0 = j1;
114         } while (j0 != 0);
115     }
116
117     // The optimal job assigned to worker `i`
118     for (int j = 1; j <= m; ++j)
119     {
120         if (p[j] != 0)
121         {
122             ans[p[j]] = j;
123         }
124     }
125
126     // The final minimum cost is stored negated in the dummy column potential
127     return -v[0];
128 }
129 };
130
131 void example_usage()
132 {
133     int n, m;
134     // Example: N workers, M jobs (ensure N <= M)
135     if (!(cin >> n >> m))
136         return;
137
138     // Build the 1-indexed cost matrix
139     vector<vector<long long>> cost(n + 1, vector<long long>(m + 1, 0));
140     for (int i = 1; i <= n; i++)
141     {
142         for (int j = 1; j <= m; j++)
143         {
144             cin >> cost[i][j];
145         }
146     }
147
148     Hungarian solver(n, m, cost);
149     cout << "Minimum Total Cost: " << solver.solve() << "\n";
150
151     for (int i = 1; i <= n; i++)
152     {
153         cout << "Worker " << i << " assigned to Job " << solver.ans[i] << "\n";
154     }
155 }

```

3.10.3 Stable Marriage Problem

```
1 #include <bits/stdc++.h>
```

```

2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: STABLE MARRIAGE (GALE-SHAPLEY)
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Stable Matching", "Preference Lists", "Gale-Shapley".
9  * - Classic Scenarios: Matching N men to N women, or N students to N colleges,
10 * based on ranked preference lists. A matching is "stable" if there is no
11 * pair (Man A, Woman B) who both prefer each other over their current partners
12 * - The Magic: The Gale-Shapley algorithm guarantees a stable matching in  $O(N^2)$ 
13 * time.
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Initialization:
16 *     StableMarriage sm(N);
17 *     sm.set_preferences(men_prefs, women_prefs);
18 * - Query:
19 *     sm.solve();
20 * - Extract Assignments:
21 *     // match_proposer[i] contains the receiver assigned to proposer i.
22 *     // match_receiver[j] contains the proposer assigned to receiver j.
23 *
24 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
25 * - The Proposer Advantage:
26 *     Gale-Shapley strongly favors the group doing the proposing!
27 *     If you want the optimal outcome for Group X, pass Group X as the "Proposers"
28 *     (the first argument in set_preferences).
29 * - Unequal sizes: If Proposers > Receivers, add "dummy" receivers that rank
30 *     everyone equally at the bottom.
31 */
32
33 struct StableMarriage
34 {
35     int n;
36
37     // pref_proposer[i][k] = the k-th preferred receiver by proposer i
38     vector<vector<int>> pref_proposer;
39
40     // To make rejection  $O(1)$ , we invert the receiver's preferences.
41     // rank_receiver[j][i] = the rank position (1 to N) of proposer i in receiver
42     // j's list.
43     // Lower rank number means highly preferred!
44     vector<vector<int>> rank_receiver;
45
46     // The current matches. 0 means unassigned.
47     vector<int> match_proposer;
48     vector<int> match_receiver;
49
50     StableMarriage(int n) : n(n)
51     {
52         pref_proposer.assign(n + 1, vector<int>(n + 1, 0));
53         rank_receiver.assign(n + 1, vector<int>(n + 1, 0));
54         match_proposer.assign(n + 1, 0);

```

```

54     match_receiver.assign(n + 1, 0);
55 }
56
57 // Both 2D vectors should be 1-indexed: [1..n][1..n]
58 void set_preferences(const vector<vector<int>> &proposers, const vector<
    vector<int>> &receivers)
59 {
60     pref_proposer = proposers;
61
62     // Invert the receiver's preference list for O(1) comparison later
63     for (int j = 1; j <= n; ++j)
64     {
65         for (int rank = 1; rank <= n; ++rank)
66         {
67             int proposer_id = receivers[j][rank];
68             rank_receiver[j][proposer_id] = rank;
69         }
70     }
71 }
72
73 void solve()
74 {
75     queue<int> free_proposers;
76     // next_proposal[i] tracks the index of the next receiver that proposer
    will ask
77     vector<int> next_proposal(n + 1, 1);
78
79     for (int i = 1; i <= n; ++i)
80     {
81         free_proposers.push(i);
82     }
83
84     // Loop until every proposer is matched
85     while (!free_proposers.empty())
86     {
87         int p = free_proposers.front();
88
89         // The receiver this proposer is going to ask next
90         int r = pref_proposer[p][next_proposal[p]];
91         next_proposal[p]++; // Increment pointer so he doesn't ask her again
    if rejected
92
93         if (match_receiver[r] == 0)
94         {
95             // Scenario 1: The receiver is entirely free. She happily accepts
    !
96             match_receiver[r] = p;
97             match_proposer[p] = r;
98             free_proposers.pop(); // The proposer is no longer free
99         }
100         else
101         {
102             // Scenario 2: The receiver is already engaged to someone else
    current_p).
103             int current_p = match_receiver[r];
104
105             // Does she prefer the new proposer (p) over her current partner
    (current_p)?
106             if (rank_receiver[r][p] < rank_receiver[r][current_p])
107             {
108                 // She dumps her current partner!
109                 match_proposer[current_p] = 0;
110                 free_proposers.push(current_p); // The dumped partner is now
    free and goes back in line
111
112                 // She accepts the new proposer
113                 match_receiver[r] = p;
114                 match_proposer[p] = r;
115                 free_proposers.pop();
116             }
117             // If she prefers her current partner, she rejects 'p', and 'p'
    stays in the queue.
118         }
119     }
120 }
121 };
122
123 void solve_example()
124 {
125     int n;
126     // Read N pairs
127     if (!(cin >> n))
128         return;
129
130     // 1-indexed preference lists
131     vector<vector<int>> men_prefs(n + 1, vector<int>(n + 1));
132     vector<vector<int>> women_prefs(n + 1, vector<int>(n + 1));
133
134     // Read Men's preferences (Proposers)
135     for (int i = 1; i <= n; ++i)
136     {
137         for (int j = 1; j <= n; ++j)
138         {
139             cin >> men_prefs[i][j];
140         }
141     }
142
143     // Read Women's preferences (Receivers)
144     for (int i = 1; i <= n; ++i)
145     {
146         for (int j = 1; j <= n; ++j)
147         {
148             cin >> women_prefs[i][j];
149         }
150     }
151
152     StableMarriage sm(n);
153     sm.set_preferences(men_prefs, women_prefs);
154     sm.solve();
155 }

```

```

156     cout << "Stable_Matches_(Man->Woman):\n";
157     for (int i = 1; i <= n; ++i)
158     {
159         cout << "Man_" << i << "_matches_with_Woman_" << sm.match_proposer[i] <<
            "\n";
160     }
161 }
162
163 int main()
164 {
165     ios_base::sync_with_stdio(false);
166     cin.tie(NULL);
167     // solve_example();
168     return 0;
169 }

```

3.11 Misc

3.11.1 Check Cycle

```

1 // Check Cycle in Undirected
2 Graph graph ;
3 vector<bool> vis;
4 bool isCycle = 0;
5 void dfs(int u,int prv )
6 {
7     vis[u] = 1;
8     for (auto& n : graph[u])
9     {
10         if (n == prv) continue;
11         if (!vis[n])
12             dfs(n, u);
13         else
14             isCycle = 1;
15     }
16 }
17
18 // Check Cycle in directed
19 Graph graph ;
20 vector<int> vis;
21 bool isCycle = 0;
22 void dfs(int u )
23 {
24     vis[u] = 2; // mark node that exist in recursion stack
25     for (auto& n : graph[u])
26     {
27         if (!vis[n]) // if node not vis & not exist inf recursion stack
28             dfs(n);
29         else if (vis[n] == 2) // node is visited and exist in recursion stack
30             isCycle = 1;
31     }
32
33     vis[u] = 1; // mark node that visited and not exist in recursion stack

```

3.11.2 Degree Sequences

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DEGREE SEQUENCES
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Given an array of degrees", "Can we form a simple graph", "
   Construct a graph".
9  * - Classic Scenarios: You are given N integers representing the number of
   friends each person has.
10 * You need to verify if this network is mathematically possible, and if so,
   pair them up.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Validation Only (Fast):
14 *     bool possible = DegreeSequence::erdos_gallai(degrees);
15 *     // Complexity: O(N log N). Use this when N = 10^5 and you only need YES/
   NO.
16 *
17 * - Construction (Slower, but gives edges):
18 *     vector<pair<int, int>> edges = DegreeSequence::havel_hakimi(degrees);
19 *     // Complexity: O(N^2 log N). Use this when you must PRINT the graph.
20 *     // Returns an empty vector if construction is impossible.
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - 0-based vs 1-based indexing for Construction:
24 * The `havel_hakimi` function returns edges using 1-based indexing by default.
25 * If you need 0-based indexing, change `nodes.push_back({d[i], i + 1});` to `i
   `.`
26 */
27
28 struct DegreeSequence
29 {
30
31     static bool erdos_gallai(vector<long long> d)
32     {
33         int n = d.size();
34         long long sum = 0;
35
36         // A valid graph MUST have an even sum of degrees (Handshaking Lemma)
37         for (long long x : d)
38             sum += x;
39         if (sum % 2 != 0)
40             return false;
41
42         // Sort degrees in descending order: d_1 >= d_2 >= ... >= d_n
43         sort(d.begin(), d.end(), greater<long long>());

```

```

44
45 // Build 1-indexed prefix sums for O(1) range queries
46 vector<long long> pref(n + 1, 0);
47 for (int i = 0; i < n; i++)
48 {
49     pref[i + 1] = pref[i] + d[i];
50 }
51
52 // Evaluate the Erds-Gallai inequality for every k from 1 to n:
53 // Sum(i=1 to k) d_i <= k(k-1) + Sum(i=k+1 to n) min(d_i, k)
54 for (int k = 1; k <= n; k++)
55 {
56     long long left_sum = pref[k];
57
58     // Binary search to find the split point in the suffix where d_i
59     // drops below k.
60     // This avoids an O(N) loop and keeps the overall complexity at O(N log N).
61     int p = max(k, (int)(lower_bound(d.begin(), d.end(), k, greater<long
62     long>()) - d.begin()));
63
64     long long right_sum = 1LL * k * (k - 1);
65
66     // Elements from k to p-1 are >= k, so min(d_i, k) is always exactly k
67     right_sum += 1LL * k * (p - k);
68
69     // Elements from p to n-1 are < k, so min(d_i, k) is just d_i
70     right_sum += (pref[n] - pref[p]);
71
72     if (left_sum > right_sum)
73     {
74         return false; // Mathematical contradiction found!
75     }
76
77     return true;
78 }
79
80 static vector<pair<int, int>> havel_hakimi(const vector<int> &d)
81 {
82     int n = d.size();
83     long long sum = 0;
84     for (int x : d)
85         sum += x;
86     if (sum % 2 != 0)
87         return {}; // Fails Handshaking Lemma
88
89     // Store {degree, original_node_id}
90     vector<pair<int, int>> nodes;
91     for (int i = 0; i < n; i++)
92     {
93         if (d[i] > 0)
94             nodes.push_back({d[i], i + 1}); // 1-based indexing for nodes
95
96     }
97
98     vector<pair<int, int>> edges;
99
100     while (!nodes.empty())
101     {
102         // Sort descending by degree
103         sort(nodes.begin(), nodes.end(), greater<pair<int, int>>());
104
105         // The node with the highest degree demands to be connected
106         int current_degree = nodes[0].first;
107         int u = nodes[0].second;
108
109         // Remove it from the pool of available nodes
110         nodes.erase(nodes.begin());
111
112         // If it demands more connections than there are nodes left, it's
113         // impossible
114         if (current_degree > nodes.size())
115             return {};
116
117         // Connect 'u' greedily to the next 'current_degree' highest nodes
118         for (int i = 0; i < current_degree; i++)
119         {
120             edges.push_back({u, nodes[i].second});
121             nodes[i].first--; // They used up one of their connections
122
123             // If a degree drops below 0, it means we forced a connection
124             // that broke the graph
125             if (nodes[i].first < 0)
126                 return {};
127         }
128
129         // Purge fully satisfied nodes (degree == 0) to speed up sorting
130         vector<pair<int, int>> remaining_nodes;
131         for (auto &node : nodes)
132         {
133             if (node.first > 0)
134             {
135                 remaining_nodes.push_back(node);
136             }
137         }
138         nodes = remaining_nodes;
139     }
140     return edges;
141 };
142
143 void solve()
144 {
145     int n;
146     // Example: Read N, then N degrees
147     if (!(cin >> n))

```

```

148     return;
149
150     vector<int> d_int(n);
151     vector<long long> d_ll(n);
152     for (int i = 0; i < n; i++)
153     {
154         cin >> d_int[i];
155         d_ll[i] = d_int[i];
156     }
157
158     // 1.  $O(N \log N)$  Check (Best for large  $N$ )
159     if (DegreeSequence::erdos_gallai(d_ll))
160     {
161         cout << "YES\n";
162
163         // 2.  $O(N^2 \log N)$  Construction (Only run if you actually need the edges
164         // In CF/ICPC, if  $N > 10^4$ , they usually don't ask you to print the edges
165         vector<pair<int, int>> edges = DegreeSequence::havel_hakimi(d_int);
166         for (auto &edge : edges)
167         {
168             cout << edge.first << " " << edge.second << "\n";
169         }
170     }
171     else
172     {
173         cout << "NO\n";
174     }
175 }
176
177 int main()
178 {
179     ios_base::sync_with_stdio(false);
180     cin.tie(NULL);
181     // solve();
182     return 0;
183 }

```

```

13 {
14     // Generate a random seed once per program execution to prevent anti-hash
15     // tests
16     static uint64_t get_seed()
17     {
18         mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
19         return rng();
20     }
21
22     // A highly robust non-linear permutation function (SplitMix64)
23     static uint64_t shift_hash(uint64_t x)
24     {
25         static const uint64_t SEED = get_seed();
26         x += SEED;
27         x ^= (x >> 33);
28         x *= 0xff51afd7ed558ccdULL;
29         x ^= (x >> 33);
30         x *= 0xc4ceb9fe1a85ec53ULL;
31         x ^= (x >> 33);
32         return x;
33     }
34
35     static vector<int> get_centers(int n, const vector<vector<int>> &adj)
36     {
37         vector<int> degree(n + 1);
38         queue<int> q;
39
40         for (int i = 1; i <= n; i++)
41         {
42             degree[i] = adj[i].size();
43             // Leaves have degree 1 (or 0 if n=1)
44             if (degree[i] <= 1)
45             {
46                 q.push(i);
47             }
48         }
49
50         int remaining_nodes = n;
51
52         // Peel away the leaves layer by layer like an onion
53         while (remaining_nodes > 2)
54         {
55             int layer_size = q.size();
56             remaining_nodes -= layer_size;
57
58             for (int i = 0; i < layer_size; i++)
59             {
60                 int u = q.front();
61                 q.pop();
62
63                 for (int v : adj[u])
64                 {
65                     if (--degree[v] == 1)
66                     {

```

3.11.3 Tree hashing

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: TREE ISOMORPHISM (HASHING)
6  *
7  * Solves: Checks if two unrooted trees are structurally identical.
8  * Time Complexity:  $O(N)$ 
9  * Space Complexity:  $O(N)$ 
10 */
11
12 struct TreeIsomorphism

```

```

66

```

```

67         }
68     }
69 }
70
71 // The remaining 1 or 2 nodes are the absolute centers
72 vector<int> centers;
73 while (!q.empty())
74 {
75     centers.push_back(q.front());
76     q.pop();
77 }
78 return centers;
79 }
80
81 static uint64_t compute_hash(int u, int p, const vector<vector<int>> &adj)
82 {
83     uint64_t current_hash = 1; // Base value for a node
84
85     for (int v : adj[u])
86     {
87         if (v != p)
88         {
89             // To combine children, we hash the child's subtree, run it
90             // through
91             // our non-linear shift function, and sum them up.
92             // Using SUM guarantees that the order of children does not
93             // matter!
94             current_hash += shift_hash(compute_hash(v, u, adj));
95         }
96     }
97     return current_hash;
98 }
99
100 static bool are_isomorphic(int n, const vector<vector<int>> &adj1, const
101 vector<vector<int>> &adj2)
102 {
103     if (n == 1)
104         return true; // Trivial case
105
106     // 1. Find centers of both trees
107     vector<int> centers1 = get_centers(n, adj1);
108     vector<int> centers2 = get_centers(n, adj2);
109
110     // If the number of centers doesn't match, they physically cannot be
111     // isomorphic
112     if (centers1.size() != centers2.size())
113     {
114         return false;
115     }
116
117     // 2. Hash Tree 1 from its first center
118     uint64_t hash1 = compute_hash(centers1[0], 0, adj1);
119
120     // 3. Check Tree 2 against Tree 1's hash

```

```

118     for (int c2 : centers2)
119     {
120         uint64_t hash2 = compute_hash(c2, 0, adj2);
121         if (hash1 == hash2)
122         {
123             return true; // Match found!
124         }
125     }
126     return false;
127 }
128 };
129 };
130
131 void solve()
132 {
133     int n;
134     // Example: Read N, then N-1 edges for Tree 1, then N-1 edges for Tree 2
135     if (!(cin >> n))
136         return;
137
138     vector<vector<int>> tree1(n + 1);
139     for (int i = 0; i < n - 1; i++)
140     {
141         int u, v;
142         cin >> u >> v;
143         tree1[u].push_back(v);
144         tree1[v].push_back(u);
145     }
146
147     vector<vector<int>> tree2(n + 1);
148     for (int i = 0; i < n - 1; i++)
149     {
150         int u, v;
151         cin >> u >> v;
152         tree2[u].push_back(v);
153         tree2[v].push_back(u);
154     }
155
156     if (TreeIsomorphism::are_isomorphic(n, tree1, tree2))
157     {
158         cout << "Isomorphic_They_are_the_exact_same_tree\n";
159     }
160     else
161     {
162         cout << "Not_Isomorphic\n";
163     }
164 }
165
166 int main()
167 {
168     ios_base::sync_with_stdio(false);
169     cin.tie(NULL);
170     // solve();
171     return 0;
172 }

```

3.12 Shortest paths

3.12.1 BFS01

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void zero_one_bfs(int start, int n, vector<vector<pair<int, int>>> &adj, vector<
    int> &dist)
5 {
6     dist.assign(n, 1e9);
7     deque<int> dq;
8     dist[start] = 0;
9     dq.push_front(start);
10
11     while (!dq.empty())
12     {
13         int u = dq.front();
14         dq.pop_front();
15
16         for (auto &edge : adj[u])
17         {
18             int v = edge.first;
19             int w = edge.second;
20             if (dist[v] > dist[u] + w)
21             {
22                 dist[v] = dist[u] + w;
23                 if (w == 0)
24                     dq.push_front(v);
25                 else
26                     dq.push_back(v);
27             }
28         }
29     }
30 }
```

3.12.2 Bellman ford

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long INF = 1e18;
5
6 struct Edge
7 {
8     int u, v, w;
9 };
10
11 bool bellman_ford(int n, int start, vector<Edge> &edges, vector<long long> &dist)
12 {
13     dist.assign(n, INF);
14     dist[start] = 0;
15
```

```
16     for (int i = 0; i < n - 1; ++i)
17     {
18         for (auto &e : edges)
19         {
20             if (dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w)
21                 dist[e.v] = dist[e.u] + e.w;
22         }
23     }
24
25     for (auto &e : edges)
26     {
27         if (dist[e.u] != INF && dist[e.v] > dist[e.u] + e.w)
28             return true;
29     }
30     return false;
31 }
```

3.12.3 Dijkstra

```
1 ll n;
2 GRAPH graph;
3
4 vector<ll> dijkstra(ll x)
5 {
6     vector<ll> dist(n + 1, 00);
7     dist[x] = 0;
8     priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
9     pq.push({0, x});
10    while (!pq.empty())
11    {
12        auto [l, node] = pq.top();
13        pq.pop();
14        if (l > dist[node])
15            continue;
16        for (auto &[to, w] : graph[node])
17        {
18            if (dist[to] > dist[node] + w)
19            {
20                dist[to] = dist[node] + w;
21                pq.push({dist[to], to});
22            }
23        }
24    }
25    return dist;
26 }
27
28 // Path
29 ll n;
30 GRAPH graph;
31 vector<int> parent;
32 deque<int> path;
33
34
```

```

35 vector<ll> dijkstra(ll x)
36 {
37     parent.assign(n + 1, -1);
38     vector<ll> dist(n + 1, 00);
39     dist[x] = 0;
40     priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
41     pq.push({0, x});
42     while (!pq.empty())
43     {
44         auto [l, node] = pq.top();
45         pq.pop();
46         if (l > dist[node])
47             continue;
48         for (auto &[to, w] : graph[node])
49         {
50             if (dist[to] > dist[node] + w)
51             {
52                 dist[to] = dist[node] + w;
53                 parent[to] = node;
54                 pq.push({dist[to], to});
55             }
56         }
57     }
58     return dist;
59 }
60
61 void build(int node)
62 {
63     path.push_front(node);
64     if (parent[node] == -1)
65         return;
66     build(parent[node]);
67 }
68
69 // Destination
70 ll n;
71 GRAPH graph;
72
73 vector<ll> dijkstra(ll x, ll d)
74 {
75     vector<ll> dist(n + 1, 00);
76     dist[x] = 0;
77     priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
78     pq.push({0, x});
79     while (!pq.empty())
80     {
81         auto [l, node] = pq.top();
82         pq.pop();
83         if (node == d)
84             break;
85         if (l > dist[node])
86             continue;
87         for (auto &[to, w] : graph[node])
88         {
89             if (dist[to] > dist[node] + w)
90             {
91                 dist[to] = dist[node] + w;
92                 pq.push({dist[to], to});
93             }
94         }
95     }
96     return dist;
97 }
98
99
100 // Dijkstra indexed PQ
101 #include <bits/stdc++.h>
102
103 #include <ext/pb_ds/priority_queue.hpp>
104 using namespace std;
105
106 using namespace __gnu_pbds;
107 typedef long long ll;
108 const ll 00 = 1e18;
109 typedef vector<vector<pair<ll, ll>>> GRAPH;
110 typedef __gnu_pbds::priority_queue<pair<ll, ll>, greater<pair<ll, ll>>,
    pairing_heap_tag> indexedPQ;
111
112 ll n;
113 GRAPH graph;
114
115 vector<ll> dijkstra(ll x)
116 {
117     vector<ll> dist(n + 1, 00);
118     indexedPQ pq;
119     vector<indexedPQ::point_iterator> key(n + 1, nullptr);
120     dist[x] = 0;
121     key[x] = pq.push({0, x});
122     while (!pq.empty())
123     {
124         auto [l, node] = pq.top();
125         pq.pop();
126
127         for (auto &[to, w] : graph[node])
128         {
129             if (dist[to] > dist[node] + w)
130             {
131                 dist[to] = dist[node] + w;
132                 if (key[to] == nullptr)
133                     key[to] = pq.push({dist[to], to});
134                 else
135                     pq.modify(key[to], {dist[to], to});
136             }
137         }
138     }
139     return dist;
140 }
141
142 // Dijkstra remove
143 ll n;

```

```

144 GRAPH graph;
145
146 vector<ll> dijkstra(ll x)
147 {
148     vector<ll> dist(n + 1, 00);
149     dist[x] = 0;
150     set<pair<ll, ll>> s;
151     s.insert({0, x});
152     while (!s.empty())
153     {
154         auto [l, node] = *s.begin();
155         s.erase(s.begin());
156         if (l > dist[node])
157             continue;
158         for (auto &[to, w] : graph[node])
159         {
160             if (dist[to] > dist[node] + w)
161             {
162                 dist[to] = dist[node] + w;
163                 s.insert({dist[to], to});
164             }
165         }
166     }
167     return dist;
168 }

```

3.12.4 Floyd

```

1 int n;
2 GRAPH graph;
3
4 vector<vector<int>> floyd()
5 {
6     vector<vector<int>> dist = graph;
7     for (int k = 0; k < n; k++)
8     {
9         for (int i = 0; i < n; i++)
10        {
11            for (int j = 0; j < n; j++)
12            {
13                if (dist[i][k] < 00 && dist[k][j] < 00)
14                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
15            }
16        }
17    }
18    return dist;
19 }
20
21 // real
22 int n;
23 GRAPH graph;
24
25 vector<vector<double>> floyd()

```

```

26 {
27     vector<vector<double>> dist = graph;
28     for (int k = 0; k < n; k++)
29     {
30         for (int i = 0; i < n; i++)
31         {
32             for (int j = 0; j < n; j++)
33             {
34                 if (dist[i][k] < 00 && dist[k][j] < 00)
35                 {
36                     if (dist[i][k] + dist[k][j] < dist[i][j] - DBL_EPSILON)
37                         dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
38                 }
39             }
40         }
41     }
42     return dist;
43 }
44
45 // path
46 int n;
47 GRAPH graph;
48 vector<int> path;
49 vector<vector<int>> phase;
50
51 vector<vector<int>> floyd()
52 {
53     phase.assign(n + 1, vector<int>(n + 1, -1));
54     vector<vector<int>> dist = graph;
55     for (int k = 0; k < n; k++)
56     {
57         for (int i = 0; i < n; i++)
58         {
59             for (int j = 0; j < n; j++)
60             {
61                 if (dist[i][k] < 00 && dist[k][j] < 00)
62                 {
63                     if (dist[i][j] > dist[i][k] + dist[k][j])
64                     {
65                         dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
66                         phase[i][j] = k;
67                     }
68                 }
69             }
70         }
71     }
72     return dist;
73 }
74
75 // push the start node manually
76 void build(int i, int j)
77 {
78     if (phase[i][j] == -1)
79     {
80         path.push_back(j);

```

```

81     return;
82 }
83 int k = phase[i][j];
84 build(i, k);
85 build(k, j);
86 }

```

3.12.5 Floyd Warshall(Hassan)

```

1 void hassan() {
2     int n,m;cin >>n>>m;
3     vector<vector<int>> dist(n+1, vector<int>(n+1, oo)); // oo >= n * maxCost
4
5     for (int i=0;i<m;i++) {
6         int u,v,w;cin>>u>>v>>w;
7         dist[u][v] = min(dist[u][v], w);
8     }
9     for (int i=1;i<=n;i++) {
10        dist[i][i] = 0;
11    }
12
13    for (int k = 1; k <= n; k++) {
14        for (int from = 1; from <= n; from++) {
15            for (int to = 1; to <= n; to++) {
16                if (dist[from][k] < oo && dist[k][to] < oo) {
17                    dist[from][to] = min(dist[from][to], dist[from][k] + dist[k]
18                                           to]);
19                }
20            }
21        }
22        for (int i=1;i<=n;i++) {
23            for (int j=1;j<=n;j++) {
24                cout << "from_" << i << "_to_" << j << "_Dist_" <<dist[i][j] << "\n"
25            }
26        }
27 }
28
29 void reachability(int n, int m) {
30     vector<vector<bool>> canReach(n+1, vector<bool>(n+1, false));
31     // ... input u, v ... canReach[u][v] = true;
32     for (int i=1; i<=n; i++) canReach[i][i] = true;
33
34     for (int k=1; k<=n; k++) {
35         for (int i=1; i<=n; i++) {
36             for (int j=1; j<=n; j++) {
37                 canReach[i][j] = canReach[i][j] || (canReach[i][k] && canReach[k]
38                                                         j[j]);
39             }
40         }
41     }
42

```

```

43 /// Find Minimax Or Maxmini Path //////////
44 void hassan() {
45     int n,m;cin >>n>>m;
46     vector<vector<int>> dist(n+1, vector<int>(n+1, oo)); // oo >= n * maxCost
47     for (int i=1;i<=n;i++) {
48         dist[i][i] = 0;
49     }
50     for (int i=0;i<m;i++) {
51         int u,v,w;cin>>u>>v>>w;
52         dist[u][v] = min(dist[u][v], w);
53     }
54
55     for (int k = 1; k <= n; k++) {
56         for (int from = 1; from <= n; from++) {
57             for (int to = 1; to <= n; to++) {
58                 if (dist[from][k] < oo && dist[k][to] < oo) {
59                     dist[from][to] = min(dist[from][to], max(dist[from][k],dist[k]
60                                                                ] [to]));
61                     //dist[from][to] = max(dist[from][to], min(dist[from][k],dist
62                                                                [k][to]));
63                 }
64             }
65         }
66         for (int i=1;i<=n;i++) {
67             for (int j=1;j<=n;j++) {
68                 cout << "from_" << i << "_to_" << j << "_Dist_" <<dist[i][j] << "
69                 \n";
70             }
71         }
72     }
73 }
74
75 /// Counting Shortest Paths //////////
76 void hassan_counting() {
77     int n, m; cin >> n >> m;
78     long long oo = 1e15;
79
80     vector<vector<long long>> dist(n + 1, vector<long long>(n + 1, oo));
81     vector<vector<long long>> pathsCount(n + 1, vector<long long>(n + 1, 0));
82
83     for (int i = 1; i <= n; i++) {
84         dist[i][i] = 0;
85         pathsCount[i][i] = 1;
86     }
87
88     for (int i = 0; i < m; i++) {
89         int u, v, w; cin >> u >> v >> w;
90         if (w < dist[u][v]) {
91             dist[u][v] = w;
92             pathsCount[u][v] = 1;
93         } else if (w == dist[u][v]) {
94             pathsCount[u][v]++;
95         }
96     }
97 }

```

```

95     for (int k = 1; k <= n; k++) {
96         for (int i = 1; i <= n; i++) {
97             for (int j = 1; j <= n; j++) {
98                 if (dist[i][k] + dist[k][j] < dist[i][j]) {
99                     dist[i][j] = dist[i][k] + dist[k][j];
100                     pathsCount[i][j] = pathsCount[i][k] * pathsCount[k][j];
101                 }
102                 else if (dist[i][k] + dist[k][j] == dist[i][j] && i != k && k != j) {
103                     pathsCount[i][j] += (pathsCount[i][k] * pathsCount[k][j]);
104                 }
105             }
106         }
107     }
108
109     for (int i = 1; i <= n; i++) {
110         for (int j = 1; j <= n; j++) {
111             if (dist[i][j] == oo) continue;
112             cout << "From_" << i << "_to_" << j << " :_Min_Dist_=" << dist[i][j]
113                 << "_|_Ways_=" << pathsCount[i][j] << "\n";
114         }
115     }
116 }
117
118
119 ////////////// Diameter and Radius //////////////////
120 int diameter = 0;
121 int radius = oo;
122
123 for (int i=1; i<=n; i++) {
124     int max_dist_for_i = 0;
125     for (int j=1; j<=n; j++) {
126         if (dist[i][j] != oo)
127             max_dist_for_i = max(max_dist_for_i, dist[i][j]);
128     }
129     // Diameter:  " "
130     diameter = max(diameter, max_dist_for_i);
131     // Radius:  " "
132     radius = min(radius, max_dist_for_i);
133 }

```

3.12.6 MaxMin Cost Using Dijkstra

```

1 /*
2  * Problem Description:
3  * The goal is to find a path from the source node (Node 1) to all other nodes in
   the graph
4  * such that the minimum edge weight along this path is maximized (Maximize the
   minimum weight).
5 */
6 struct node {
7     int u,w;
8

```

```

9     bool operator<(const node &b) const {
10         return w > b.w;
11     }
12 };
13
14 void hassan() {
15     int n, m;
16     cin >> n >> m;
17     vector<vector<pair<int, int> > > adj(n + 1);
18     for (int i = 1; i <= m; i++) {
19         int u, v, w;
20         cin >> u >> v >> w;
21         adj[u].push_back({v, w});
22         adj[v].push_back({u, w});
23     }
24     vector<int> maxMin(n + 1, -1);
25     priority_queue<node> pq;
26     pq.push({1, oo});
27     maxMin[1] = oo;
28     while (!pq.empty()) {
29         auto [u,w] = pq.top();
30         pq.pop();
31
32         if (w < maxMin[u]) continue;
33
34         for (auto [v,c]: adj[u]) {
35             if (min(maxMin[u], c) > maxMin[v]) {
36                 maxMin[v] = min(maxMin[u], c);
37                 pq.push({v, maxMin[v]});
38             }
39         }
40     }
41     for (int i = 1; i <= n; i++) {
42         cout << maxMin[i] << "_";
43     }
44 }

```

3.12.7 MinMax Cost Using Dijkstra

```

1 /*
2  * Problem Description:
3  * The goal is to find a path from the source node (Node 1) to all other nodes in
   the graph
4  * such that the Maximum edge weight along this path is Minimized (Minimize the
   maximum weight).
5 */
6 struct node {
7     int u, w;
8
9     bool operator<(const node &b) const {
10         return w > b.w;
11     }
12 };

```

```

13
14 void hassan() {
15     int n, m;
16     cin >> n >> m;
17     vector<vector<pair<int, int> > > adj(n + 1);
18     for (int i = 1; i <= m; i++) {
19         int u, v, w;
20         cin >> u >> v >> w;
21         adj[u].push_back({v, w});
22         adj[v].push_back({u, w});
23     }
24     priority_queue<node> pq;
25     pq.push({1, 0});
26     vector<int> minMax(n + 1, oo);
27     minMax[1] = 0;
28     while (!pq.empty()) {
29         auto [u,w] = pq.top();
30         pq.pop();
31
32         if (w > minMax[u]) continue;
33
34         for (auto [v,c]: adj[u]) {
35             if (max(minMax[u], c) < minMax[v]) {
36                 minMax[v] = max(minMax[u], c);
37                 pq.push({v, minMax[v]});
38             }
39         }
40     }
41     for (int i = 1; i <= n; i++) {
42         cout << minMax[i] << "□";
43     }
44 }

```

3.13 Trees

3.13.1 Rerooting

```

1 int n;
2 vector<int> sz, dist, ans;
3 GRAPH graph;
4
5 void dfs(int u, int p)
6 {
7     sz[u] = 1;
8     for (auto &v : graph[u])
9     {
10         if (v == p)
11             continue;
12         dfs(v, u);
13         sz[u] += sz[v];
14         dist[u] += dist[v] + sz[v];
15     }
16 }

```

```

17
18 void dfs2(int u, int p)
19 {
20     for (auto &v : graph[u])
21     {
22         if (v == p)
23             continue;
24         ans[v] = ans[u] - sz[v] + (n - sz[v]);
25         dfs2(v, u);
26     }
27 }
28
29 void solve()
30 {
31     cin >> n;
32     sz.assign(n + 1, {}), dist.assign(n + 1, {}), graph.assign(n + 1, {}), ans.
33         assign(n + 1, {});
34     for (int i = 1; i < n; i++)
35     {
36         int a, b;
37         cin >> a >> b;
38         graph[a].emplace_back(b), graph[b].emplace_back(a);
39     }
40     dfs(1, 0);
41     ans[1] = dist[1];
42     dfs2(1, 0);
43     for (int i = 1; i <= n; i++)
44         cout << ans[i] << "□";
45 }

```

3.13.2 Virtual tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /**
5  * THE ULTIMATE CP TEMPLATE BLUEPRINT: VIRTUAL TREE (AUXILIARY TREE)
6  *
7  * 1. WHAT PROBLEM DOES THIS SOLVE?
8  * - Keywords: "Paths between subset nodes", "DP on subset of tree", "Sparse
9  *   subset queries".
10 * - Classic Scenarios: A tree has  $N=10^5$  nodes. You have  $Q$  queries. Each query
11 *   gives you a subset
12 *   of  $K$  nodes. You need to perform a Tree DP or path calculation on these  $K$ 
13 *   nodes, but running
14 *   an  $O(N)$  DFS per query is impossible.
15 * - The Magic: The Virtual Tree builds a smaller tree containing only the  $K$ 
16 *   subset nodes
17 *   AND their necessary LCAs to maintain tree structure. This new tree has  $O(K)$ 
18 *   nodes.
19 *
20 * 2. HOW TO USE IT (THE BLACK BOX)
21 * - Initialization:

```

```

17 *      1. Build the full tree using standard DFS.
18 *      2. For each query, collect the subset of K nodes.
19 *      3. Call `build_virtual_tree(subset)`.
20 *      4. Perform your DP on `virtual_adj` (the edges of the virtual tree).
21 * - Complexity:
22 *      Time:  $O(K \log K)$  per query to build.
23 *      Space:  $O(N)$  to store the full tree +  $O(K)$  for the virtual tree.
24 */
25
26 const int MAXN = 2e5 + 5;
27 const int LOG = 20;
28
29 // adj: Original tree, virtual_adj: The sparse tree we build for the subset
30 vector<int> adj[MAXN], virtual_adj[MAXN];
31 int tin[MAXN], tout[MAXN], depth[MAXN];
32 int up[MAXN][LOG];
33 int timer;
34
35 // Standard DFS to precalculate depths, entry times, and binary lifting table
36 void dfs(int u, int p)
37 {
38     tin[u] = ++timer;
39     depth[u] = depth[p] + 1;
40     up[u][0] = p;
41     for (int i = 1; i < LOG; i++)
42         up[u][i] = up[up[u][i - 1]][i - 1];
43
44     for (int v : adj[u])
45     {
46         if (v != p)
47             dfs(v, u);
48     }
49     tout[u] = timer;
50 }
51
52 // Find LCA using binary lifting
53 int get_lca(int u, int v)
54 {
55     if (depth[u] < depth[v])
56         swap(u, v);
57     for (int i = LOG - 1; i >= 0; i--)
58     {
59         if (depth[u] - (1 << i) >= depth[v])
60             u = up[u][i];
61     }
62     if (u == v)
63         return u;
64     for (int i = LOG - 1; i >= 0; i--)
65     {
66         if (up[u][i] != up[v][i])
67         {
68             u = up[u][i], v = up[v][i];
69         }
70     }
71     return up[u][0];

```

```

72 }
73
74 // Ancestor check using Entry/Exit times
75 bool is_ancestor(int u, int v)
76 {
77     return tin[u] <= tin[v] && tout[u] >= tout[v];
78 }
79
80 // Main function to build the virtual tree from a subset of nodes
81 void build_virtual_tree(vector<int> &nodes)
82 {
83     // 1. Sort by entry time so we can process the tree in DFS order
84     sort(nodes.begin(), nodes.end(), [](int a, int b)
85         { return tin[a] < tin[b]; });
86
87     // 2. Inject LCAs: To keep the tree connected, we must add the LCA of every
88     // adjacent pair in our sorted list.
89     int sz = nodes.size();
90     for (int i = 0; i < sz - 1; i++)
91     {
92         nodes.push_back(get_lca(nodes[i], nodes[i + 1]));
93     }
94
95     // 3. Sort again by entry time and remove duplicates (LCAs might have been
96     // added multiple times)
97     sort(nodes.begin(), nodes.end(), [](int a, int b)
98         { return tin[a] < tin[b]; });
99     nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
100
101     // 4. Connect the nodes using a Stack
102     // We maintain a stack representing the current path from the root.
103     vector<int> st;
104     st.push_back(nodes[0]);
105
106     for (size_t i = 1; i < nodes.size(); i++)
107     {
108         // Pop nodes that are not ancestors of the current node
109         while (st.size() > 1 && !is_ancestor(st.back(), nodes[i]))
110         {
111             virtual_adj[st[st.size() - 2]].push_back(st.back());
112             st.pop_back();
113         }
114         st.push_back(nodes[i]);
115     }
116     // Connect remaining nodes in the stack
117     while (st.size() > 1)
118     {
119         virtual_adj[st[st.size() - 2]].push_back(st.back());
120         st.pop_back();
121     }
122
123 void solve()
124 {
125     int n;

```

```

126     cin >> n;
127
128     // Clear graph before building for multiple test cases
129     for (int i = 0; i <= n; ++i)
130     {
131         adj[i].clear();
132         virtual_adj[i].clear();
133     }
134
135     // Example: Reading N-1 edges
136     for (int i = 0; i < n - 1; ++i)
137     {
138         int u, v;
139         cin >> u >> v;
140         adj[u].push_back(v);
141         adj[v].push_back(u);
142     }
143
144     // Precompute full tree properties
145     timer = 0;
146     dfs(1, 1);
147
148     int k;
149     cin >> k;
150     vector<int> subset(k);
151     for (int &x : subset)
152         cin >> x;
153
154     // Build the virtual tree for the K nodes
155     build_virtual_tree(subset);
156
157     // Now perform DP on `virtual_adj`!
158     // subset now contains all nodes in the virtual tree.
159 }
160
161 int main()
162 {
163     ios_base::sync_with_stdio(false);
164     cin.tie(NULL);
165     solve();
166     return 0;
167 }

```

4 Math

4.1 Calculus

4.1.1 Differentiation integration

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: ADAPTIVE SIMPSON's RULE
3  *

```

```

4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Area under curve", "Evaluate integral", "Continuous probability".
6  * - Classic Scenarios: You are given a mathematical function  $f(x)$  and need the
7  *   integral from A to B with a precision of  $10^{-6}$ .
8  * - The Magic: Instead of chopping the interval into  $10^6$  fixed pieces (which
9  *   can
10  *   TLE or lose precision), Adaptive Simpson's evaluates the error dynamically.
11  *   It recursively splits the interval only in areas where the curve is highly
12  *   irregular, guaranteeing extreme precision in logarithmic time.
13  * 2. HOW TO USE IT (THE BLACK BOX)
14  * - Query:
15  *   double area = integrate(A, B, 1e-7, my_function);
16  *
17  * - Complexity:
18  *   Time:  $O(\text{Depends on curve complexity, practically instant})$ .
19  *   Space:  $O(\text{Recursion depth})$ .
20  */
21
22 double simpson(double l, double r, double f_l, double f_r, double f_mid)
23 {
24     return (f_l + 4 * f_mid + f_r) * (r - l) / 6.0;
25 }
26
27 double adaptive_simpson(double l, double r, double f_l, double f_r, double f_mid,
28     double S, double eps, const function<double(double)>& f)
29 {
30     double mid = (l + r) / 2.0;
31     double l_mid = (l + mid) / 2.0;
32     double r_mid = (mid + r) / 2.0;
33
34     double f_l_mid = f(l_mid);
35     double f_r_mid = f(r_mid);
36
37     double left_S = simpson(l, mid, f_l, f_mid, f_l_mid);
38     double right_S = simpson(mid, r, f_mid, f_r, f_r_mid);
39
40     if (abs(left_S + right_S - S) <= 15.0 * eps)
41     {
42         return left_S + right_S + (left_S + right_S - S) / 15.0;
43     }
44
45     return adaptive_simpson(l, mid, f_l, f_mid, f_l_mid, left_S, eps / 2.0, f) +
46         adaptive_simpson(mid, r, f_mid, f_r, f_r_mid, right_S, eps / 2.0, f);
47 }
48 double integrate(double a, double b, double eps, const function<double(double)>&
49     f)
50 {
51     double f_a = f(a);
52     double f_b = f(b);
53     double f_mid = f((a + b) / 2.0);
54     double S = simpson(a, b, f_a, f_b, f_mid);
55     return adaptive_simpson(a, b, f_a, f_b, f_mid, S, eps, f);
56 }

```

4.2 Convolution

4.2.1 FWHT

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FAST WALSH-HADAMARD TRANSFORM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Bitwise convolution", "XOR sum", "AND sum", "OR sum".
6  * - Classic Scenarios: Find array C where  $C[k] = \sum(A[i] * B[j])$  for all
7  *   pairs (i, j) such that  $(i \text{ XOR } j) == k$ .
8  * - The Magic: It alters the NTT butterfly structure to map binary bitwise
9  *   operations into addition/subtraction, resolving the convolution in  $O(N \log N)$ .
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *   vector<long long> C = fwht_xor(A, B);
14 *   // Swap 'fwht_xor' for 'fwht_and' or 'fwht_or' as needed.
15 *
16 * - Complexity:
17 *   Time:  $O(N \log N)$ . Note that N MUST be a power of 2.
18 *   Space:  $O(N)$ .
19 *
20 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21 * - Padding: The input arrays MUST be padded with zeros until their size
22 *   is exactly a power of 2 before passing them into these functions.
23 */
24
25 using ll = long long;
26 const ll MOD = 998244353;
27
28 ll binpow_fwht(ll a, ll b)
29 {
30     ll res = 1;
31     a %= MOD;
32     while (b > 0)
33     {
34         if (b & 1)
35             res = res * a % MOD;
36         a = a * a % MOD;
37         b >>= 1;
38     }
39     return res;
40 }
41
42 ll modInv(ll n) { return binpow_fwht(n, MOD - 2); }
43
44 void fwht_xor_base(vector<ll> &a, bool inv)
45 {
46     int n = a.size();
47     for (int len = 1; 2 * len <= n; len <= 1)
48     {
49         for (int i = 0; i < n; i += 2 * len)
50         {
```

```
51             for (int j = 0; j < len; j++)
52             {
53                 ll u = a[i + j];
54                 ll v = a[i + len + j];
55                 a[i + j] = (u + v) % MOD;
56                 a[i + len + j] = (u - v + MOD) % MOD;
57             }
58         }
59     }
60     if (inv)
61     {
62         ll invN = modInv(n);
63         for (ll &x : a)
64             x = x * invN % MOD;
65     }
66 }
67
68 void fwht_and_base(vector<ll> &a, bool inv)
69 {
70     int n = a.size();
71     for (int len = 1; 2 * len <= n; len <= 1)
72     {
73         for (int i = 0; i < n; i += 2 * len)
74         {
75             for (int j = 0; j < len; j++)
76             {
77                 ll u = a[i + j];
78                 ll v = a[i + len + j];
79                 if (!inv)
80                 {
81                     a[i + j] = (u + v) % MOD;
82                 }
83                 else
84                 {
85                     a[i + j] = (u - v + MOD) % MOD;
86                 }
87             }
88         }
89     }
90 }
91
92 void fwht_or_base(vector<ll> &a, bool inv)
93 {
94     int n = a.size();
95     for (int len = 1; 2 * len <= n; len <= 1)
96     {
97         for (int i = 0; i < n; i += 2 * len)
98         {
99             for (int j = 0; j < len; j++)
100             {
101                 ll u = a[i + j];
102                 ll v = a[i + len + j];
103                 if (!inv)
104                 {
105                     a[i + len + j] = (u + v) % MOD;
```

```

106         }
107         else
108         {
109             a[i + len + j] = (v - u + MOD) % MOD;
110         }
111     }
112 }
113 }
114 }
115
116 vector<ll> fwht_xor(vector<ll> a, vector<ll> b)
117 {
118     fwht_xor_base(a, false);
119     fwht_xor_base(b, false);
120     for (size_t i = 0; i < a.size(); i++)
121         a[i] = a[i] * b[i] % MOD;
122     fwht_xor_base(a, true);
123     return a;
124 }
125
126 vector<ll> fwht_and(vector<ll> a, vector<ll> b)
127 {
128     fwht_and_base(a, false);
129     fwht_and_base(b, false);
130     for (size_t i = 0; i < a.size(); i++)
131         a[i] = a[i] * b[i] % MOD;
132     fwht_and_base(a, true);
133     return a;
134 }
135
136 vector<ll> fwht_or(vector<ll> a, vector<ll> b)
137 {
138     fwht_or_base(a, false);
139     fwht_or_base(b, false);
140     for (size_t i = 0; i < a.size(); i++)
141         a[i] = a[i] * b[i] % MOD;
142     fwht_or_base(a, true);
143     return a;
144 }

```

4.2.2 GDC LCM Convolution

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: GCD & LCM CONVOLUTION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "gcd(i, j) = k", "lcm(i, j) = k", "Divisor transformation".
6  * - Classic Scenarios: Find array C where C[k] = sum(A[i] * B[j]) over all pairs
7  *   (i, j) such that gcd(i, j) == k (or lcm(i, j) == k).
8  * - The Magic: Bypasses polynomial rings entirely. It transforms the arrays into
9  *   prefix sums over their multiples (for GCD) or divisors (for LCM), multiplies
10  *   the point values, and reverses the transformation using Mobius-style
   subtraction.

```

```

11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     vector<long long> C_gcd = gcd_convolution(A, B);
15 *     vector<long long> C_lcm = lcm_convolution(A, B);
16 *
17 * - Complexity:
18 *     Time: O(N log log N) utilizing a prime sieve optimization.
19 *     Space: O(N).
20 *
21 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22 * - 1-Based Indexing: These arrays MUST be 1-indexed (i.e., A[1] is the first
23 *   element, A[0] is ignored). Ensure A and B are the same size before passing.
24 */
25
26 using ll = long long;
27 const ll MOD = 998244353;
28
29 vector<int> get_primes(int n)
30 {
31     vector<bool> is_prime(n + 1, true);
32     vector<int> primes;
33     for (int p = 2; p <= n; p++)
34     {
35         if (is_prime[p])
36         {
37             primes.push_back(p);
38             for (int i = p * 2; i <= n; i += p)
39                 is_prime[i] = false;
40         }
41     }
42     return primes;
43 }
44
45 vector<ll> gcd_convolution(vector<ll> a, vector<ll> b)
46 {
47     int n = a.size() - 1;
48     vector<int> primes = get_primes(n);
49
50     for (int p : primes)
51     {
52         for (int i = n / p; i > 0; i--)
53         {
54             a[i] = (a[i] + a[i * p]) % MOD;
55             b[i] = (b[i] + b[i * p]) % MOD;
56         }
57     }
58
59     vector<ll> c(n + 1);
60     for (int i = 1; i <= n; i++)
61     {
62         c[i] = a[i] * b[i] % MOD;
63     }
64
65     for (int p : primes)

```

```

66 {
67     for (int i = 1; i * p <= n; i++)
68     {
69         c[i] = (c[i] - c[i * p] + MOD) % MOD;
70     }
71 }
72 return c;
73 }
74
75 vector<ll> lcm_convolution(vector<ll> a, vector<ll> b)
76 {
77     int n = a.size() - 1;
78     vector<int> primes = get_primes(n);
79
80     for (int p : primes)
81     {
82         for (int i = 1; i * p <= n; i++)
83         {
84             a[i * p] = (a[i * p] + a[i]) % MOD;
85             b[i * p] = (b[i * p] + b[i]) % MOD;
86         }
87     }
88
89     vector<ll> c(n + 1);
90     for (int i = 1; i <= n; i++)
91     {
92         c[i] = a[i] * b[i] % MOD;
93     }
94
95     for (int p : primes)
96     {
97         for (int i = n / p; i > 0; i--)
98         {
99             c[i * p] = (c[i * p] - c[i] + MOD) % MOD;
100         }
101     }
102     return c;
103 }

```

4.2.3 NTT

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: NUMBER THEORETIC TRANSFORM (NTT)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Polynomial multiplication", "Convolution", "Sum of pairs".
6  * - Classic Scenarios: You are given two arrays A and B and need to find C
7  *   where  $C[k] = \sum(A[i] * B[k-i])$  modulo 998244353.
8  * - The Magic: A naive double loop takes  $O(N^2)$  and TLEs for  $N = 10^5$ .
9  *   NTT evaluates the polynomials at primitive roots of unity, multiplies
10  *   the point values in  $O(N)$ , and interpolates them back in  $O(N \log N)$ .
11  *
12  * 2. HOW TO USE IT (THE BLACK BOX)

```

```

13 * - Query:
14 *     vector<long long> C = multiply(A, B);
15 *
16 * - Complexity:
17 *     Time:  $O(N \log N)$ 
18 *     Space:  $O(N)$ 
19 *
20 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21 * - Modulo: This template is strictly hardcoded for MOD = 998244353 and
22 *   its primitive root 3. If the problem has a different modulo, you MUST
23 *   use an Arbitrary Modulo NTT (MTT) or find the primitive root of the new
24 *   modulo.
25
26 using ll = long long;
27 const ll MOD = 998244353;
28 const ll ROOT = 3;
29
30 ll binpow(ll a, ll b)
31 {
32     ll res = 1;
33     a %= MOD;
34     while (b > 0)
35     {
36         if (b & 1)
37             res = res * a % MOD;
38         a = a * a % MOD;
39         b >>= 1;
40     }
41     return res;
42 }
43
44 ll modInverse(ll n)
45 {
46     return binpow(n, MOD - 2);
47 }
48
49 void ntt(vector<ll> &a, bool invert)
50 {
51     int n = a.size();
52     for (int i = 1, j = 0; i < n; i++)
53     {
54         int bit = n >> 1;
55         for (; j & bit; bit >>= 1)
56             j ^= bit;
57         j ^= bit;
58         if (i < j)
59             swap(a[i], a[j]);
60     }
61     for (int len = 2; len <= n; len <= 1)
62     {
63         ll wlen = binpow(ROOT, (MOD - 1) / len);
64         if (invert)
65             wlen = modInverse(wlen);
66         for (int i = 0; i < n; i += len)

```

```

67     {
68         ll w = 1;
69         for (int j = 0; j < len / 2; j++)
70         {
71             ll u = a[i + j];
72             ll v = a[i + j + len / 2] * w % MOD;
73             a[i + j] = (u + v < MOD ? u + v : u + v - MOD);
74             a[i + j + len / 2] = (u - v >= 0 ? u - v : u - v + MOD);
75             w = w * wlen % MOD;
76         }
77     }
78 }
79 if (invert)
80 {
81     ll n_inv = modInverse(n);
82     for (ll &x : a)
83         x = x * n_inv % MOD;
84 }
85 }
86
87 vector<ll> multiply(vector<ll> const &a, vector<ll> const &b)
88 {
89     vector<ll> fa(a.begin(), a.end()), fb(b.begin(), b.end());
90     int n = 1;
91     while (n < a.size() + b.size())
92         n <<= 1;
93     fa.resize(n);
94     fb.resize(n);
95     ntt(fa, false);
96     ntt(fb, false);
97     for (int i = 0; i < n; i++)
98         fa[i] = fa[i] * fb[i] % MOD;
99     ntt(fa, true);
100    while (fa.size() > 1 && fa.back() == 0)
101        fa.pop_back();
102    return fa;
103 }

```

4.3 Linear algebra

4.3.1 Determinant(composite mod)

4.3.2 Determinant(prime mod)

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DETERMINANT (PRIME MODULO)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Matrix Tree Theorem", "Number of Spanning Trees".
6  * - Classic Scenarios: You have an N x N matrix and need its determinant modulo
7  *   a prime number like 1e9+7 or 998244353.

```

```

8  * - The Magic: Standard Gaussian elimination. It tracks row swaps because every
9  *   swap multiplies the final determinant by -1.
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *     long long ans = det_prime(matrix, MOD);
14 *
15 * - Complexity:
16 *     Time: O(N^3)
17 *     Space: O(N^2)
18 */
19
20 using ll = long long;
21
22 ll binpow_det(ll a, ll b, ll mod)
23 {
24     ll res = 1;
25     a %= mod;
26     while (b > 0)
27     {
28         if (b & 1)
29             res = res * a % mod;
30         a = a * a % mod;
31         b >>= 1;
32     }
33     return res;
34 }
35
36 ll modInverse_det(ll n, ll mod)
37 {
38     return binpow_det(n, mod - 2, mod);
39 }
40
41 ll det_prime(vector<vector<ll>> a, ll mod)
42 {
43     int n = a.size();
44     ll ans = 1;
45     for (int i = 0; i < n; ++i)
46     {
47         int pivot = i;
48         for (int j = i + 1; j < n; ++j)
49         {
50             if (a[j][i] > a[pivot][i])
51                 pivot = j;
52         }
53         if (a[pivot][i] == 0)
54             return 0;
55         if (i != pivot)
56         {
57             swap(a[i], a[pivot]);
58             ans = (mod - ans) % mod;
59         }
60         ans = ans * a[i][i] % mod;
61         ll inv = modInverse_det(a[i][i], mod);
62         for (int j = i + 1; j < n; ++j)

```

```

63     {
64         ll factor = a[j][i] * inv % mod;
65         for (int k = i; k < n; ++k)
66         {
67             a[j][k] = (a[j][k] - factor * a[i][k]) % mod;
68             if (a[j][k] < 0)
69                 a[j][k] += mod;
70         }
71     }
72 }
73 return ans;
74 }

```

4.3.3 Gaussian elimination

```

1  /**
2  *  THE ULTIMATE CP TEMPLATE BLUEPRINT: GAUSSIAN ELIMINATION
3  *
4  *  1.  WHAT PROBLEM DOES THIS SOLVE?
5  *  - Keywords: "System of equations", "Probabilities", "Markov Chains".
6  *  - Classic Scenarios: Solving  $Ax = B$  for floating-point values.
7  *  - The Magic: It reduces the matrix to row-echelon form to find the exact
8  *    values of the variables.
9  *
10 *  2.  HOW TO USE IT (THE BLACK BOX)
11 *  - Query:
12 *    vector<double> ans;
13 *    int solutions = gauss(matrix, ans);
14 *  - Result Breakdown:
15 *    Returns 1 if there is exactly one solution (stored in 'ans').
16 *    Returns 0 if there are no solutions.
17 *    Returns 2 if there are infinitely many solutions.
18 *
19 *  - Complexity:
20 *    Time:  $O(N^3)$ 
21 *    Space:  $O(N^2)$ 
22 */
23
24 const double EPS = 1e-9;
25
26 int gauss(vector<vector<double>> a, vector<double> &ans)
27 {
28     int n = a.size();
29     if (n == 0)
30         return 0;
31     int m = a[0].size() - 1;
32     vector<int> where(m, -1);
33
34     for (int col = 0, row = 0; col < m && row < n; ++col)
35     {
36         int sel = row;
37         for (int i = row; i < n; ++i)
38         {

```

```

39             if (abs(a[i][col]) > abs(a[sel][col]))
40                 sel = i;
41         }
42         if (abs(a[sel][col]) < EPS)
43             continue;
44
45         for (int i = col; i <= m; ++i)
46             swap(a[sel][i], a[row][i]);
47         where[col] = row;
48
49         for (int i = 0; i < n; ++i)
50         {
51             if (i != row)
52             {
53                 double c = a[i][col] / a[row][col];
54                 for (int j = col; j <= m; ++j)
55                 {
56                     a[i][j] -= a[row][j] * c;
57                 }
58             }
59         }
60         ++row;
61     }
62
63     ans.assign(m, 0);
64     for (int i = 0; i < m; ++i)
65     {
66         if (where[i] != -1)
67             ans[i] = a[where[i]][m] / a[where[i]][i];
68     }
69
70     for (int i = 0; i < n; ++i)
71     {
72         double sum = 0;
73         for (int j = 0; j < m; ++j)
74         {
75             sum += ans[j] * a[i][j];
76         }
77         if (abs(sum - a[i][m]) > EPS)
78             return 0;
79     }
80
81     for (int i = 0; i < m; ++i)
82     {
83         if (where[i] == -1)
84             return 2;
85     }
86     return 1;
87 }

```

4.3.4 Gaussian elimination mod2

```

1  /**

```

```

2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: GAUSSIAN ELIMINATION MOD 2
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "XOR subset", "Boolean equations", "Lights out game".
6  * - Classic Scenarios: Finding if a specific XOR sum can be formed from an array
7  *   , or solving  $Ax = B$  where all values and operations are modulo 2 (XOR).
8  * - The Magic: Replaces standard arrays with std::bitset, shrinking the time
9  *   constant by a factor of 64. Instead of division and subtraction, it simply
10 *   uses XOR.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *   bitset<2005> ans;
15 *   int solutions = gauss_mod2(matrix, N, M, ans);
16 *
17 * - Complexity:
18 *   Time:  $O(N^3 / 64)$ 
19 *   Space:  $O(N^2 / 64)$ 
20 */
21
22 const int MAX_V = 2005;
23
24 int gauss_mod2(vector<bitset<MAX_V>> a, int n, int m, bitset<MAX_V> &ans)
25 {
26     vector<int> where(m, -1);
27     for (int col = 0, row = 0; col < m && row < n; ++col)
28     {
29         for (int i = row; i < n; ++i)
30         {
31             if (a[i][col])
32             {
33                 swap(a[i], a[row]);
34                 break;
35             }
36         }
37         if (!a[row][col])
38             continue;
39         where[col] = row;
40         for (int i = 0; i < n; ++i)
41         {
42             if (i != row && a[i][col])
43             {
44                 a[i] ^= a[row];
45             }
46         }
47         ++row;
48     }
49
50     ans.reset();
51     for (int i = 0; i < m; ++i)
52     {
53         if (where[i] != -1)
54             ans[i] = a[where[i]][m];
55     }

```

```

56
57     for (int i = 0; i < n; ++i)
58     {
59         int sum = 0;
60         for (int j = 0; j < m; ++j)
61         {
62             if (ans[j] && a[i][j])
63                 sum ^= 1;
64         }
65         if (sum != a[i][m])
66             return 0;
67     }
68
69     for (int i = 0; i < m; ++i)
70     {
71         if (where[i] == -1)
72             return 2;
73     }
74     return 1;
75 }

```

4.3.5 XOR basis

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: XOR BASIS (LINEAR BASIS)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Maximum XOR subset", "K-th XOR sum", "Can we form X".
6  * - Classic Scenarios: You are given an array of size  $10^5$  and need to find the
7  *   maximum possible XOR sum of any subset.
8  * - The Magic: It reduces the entire array into a set of at most B "basis"
9  *   vectors
10 *   (where B is the number of bits, usually 60 for long long). These basis
11 *   vectors
12 *   span the exact same XOR space as the original array.
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Initialization:
16 *   XORBasis basis;
17 *   for (ll x : arr) basis.insert(x);
18 * - Queries:
19 *   long long max_val = basis.max_xor();
20 *   bool exists = basis.contains(X);
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Reduced Row Echelon Form (RREF) / K-th Smallest: To answer "K-th smallest
24 *   XOR sum"
25 *   (Topic 383/384), you MUST call basis.build_rref() first. Then use basis.kth(K).
26 *   (Note: K is 1-indexed. If K=1, it returns the smallest possible XOR sum > 0)
27 *
28 */

```

```

26 using ll = long long;
27
28 struct XORBasis
29 {
30     static const int BITS = 60; // Max bits for long long
31     vector<ll> basis;
32     vector<ll> rref; // For K-th queries
33     int sz;
34     bool is_rref_built;
35
36     XORBasis()
37     {
38         basis.assign(BITS, 0);
39         sz = 0;
40         is_rref_built = false;
41     }
42
43     bool insert(ll x)
44     {
45         for (int i = BITS - 1; i >= 0; i--)
46         {
47             if ((x >> i) & 1)
48             {
49                 if (!basis[i])
50                 {
51                     basis[i] = x;
52                     sz++;
53                     is_rref_built = false;
54                     return true;
55                 }
56                 x ^= basis[i];
57             }
58         }
59         return false;
60     }
61
62     bool contains(ll x)
63     {
64         for (int i = BITS - 1; i >= 0; i--)
65         {
66             if ((x >> i) & 1)
67             {
68                 if (!basis[i])
69                     return false;
70                 x ^= basis[i];
71             }
72         }
73         return true;
74     }
75
76     ll max_xor(ll current_xor = 0)
77     {
78         ll res = current_xor;
79         for (int i = BITS - 1; i >= 0; i--)
80         {

```

```

81             res = max(res, res ^ basis[i]);
82         }
83         return res;
84     }
85
86     // Call this strictly BEFORE querying kth()
87     void build_rref()
88     {
89         if (is_rref_built)
90             return;
91         rref.clear();
92         vector<ll> temp = basis;
93         for (int i = BITS - 1; i >= 0; i--)
94         {
95             if (!temp[i])
96                 continue;
97             for (int j = i + 1; j < BITS; j++)
98             {
99                 if ((temp[j] >> i) & 1)
100                     temp[j] ^= temp[i];
101             }
102         }
103         for (int i = 0; i < BITS; i++)
104         {
105             if (temp[i])
106                 rref.push_back(temp[i]);
107         }
108         is_rref_built = true;
109     }
110
111     // Returns K-th smallest XOR sum (1-indexed, excluding 0)
112     // Returns -1 if K is out of bounds
113     ll kth(ll k)
114     {
115         build_rref();
116         if (k > (1LL << sz) - 1)
117             return -1;
118         ll res = 0;
119         for (int i = 0; i < rref.size(); i++)
120         {
121             if ((k >> i) & 1)
122                 res ^= rref[i];
123         }
124         return res;
125     }
126 };

```

4.4 Linear recurrence

4.4.1 Berlekamp

```

1 /**
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: BERLEKAMP-MASSEY

```

```

3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Find the recurrence", "Sequence guesser".
6  * - Classic Scenarios: You brute-forced the first 50 terms of a DP or counting
7  *   problem, but you cannot figure out the actual mathematical recurrence
8  *   relation.
9  * - The Magic: It iteratively builds a polynomial that perfectly models the
10 *   input sequence. If the true recurrence has length N, giving BM the first
11 *   2N terms guarantees it will perfectly reconstruct the formula.
12 *
13 * 2. HOW TO USE IT (THE BLACK BOX)
14 * - Query:
15 *   vector<ll> C = berlekamp_massey(S, MOD);
16 *   // S is the sequence you brute-forced.
17 *   // C contains the coefficients: S[i] = C[0]*S[i-1] + C[1]*S[i-2] + ...
18 *
19 * - Complexity:
20 *   Time: O(|S|^2)
21 *   Space: O(|S|)
22 */
23 using ll = long long;
24
25 ll binpow_bm(ll a, ll b, ll mod)
26 {
27     ll res = 1;
28     a %= mod;
29     while (b > 0)
30     {
31         if (b & 1)
32             res = res * a % mod;
33         a = a * a % mod;
34         b >>= 1;
35     }
36     return res;
37 }
38
39 ll modInverse_bm(ll n, ll mod)
40 {
41     return binpow_bm(n, mod - 2, mod);
42 }
43
44 vector<ll> berlekamp_massey(const vector<ll> &s, ll mod)
45 {
46     vector<ll> c = {1}, b = {1};
47     ll L = 0, m = 1, b_val = 1;
48     for (int i = 0; i < s.size(); i++)
49     {
50         ll d = 0;
51         for (int j = 0; j <= L; j++)
52         {
53             d = (d + c[j] * s[i - j]) % mod;
54         }
55         if (d == 0)
56         {

```

```

57             m++;
58         }
59         else
60         {
61             vector<ll> temp = c;
62             ll c_val = (d * modInverse_bm(b_val, mod)) % mod;
63             while (c.size() <= b.size() + m)
64                 c.push_back(0);
65             for (int j = 0; j < b.size(); j++)
66             {
67                 c[j + m] = (c[j + m] - c_val * b[j] % mod + mod) % mod;
68             }
69             if (2 * L <= i)
70             {
71                 L = i + 1 - L;
72                 b = temp;
73                 b_val = d;
74                 m = 1;
75             }
76             else
77             {
78                 m++;
79             }
80         }
81     }
82     c.erase(c.begin());
83     for (ll &x : c)
84         x = (mod - x) % mod;
85     return c;
86 }

```

4.5 Matrix

4.5.1 Frievald's algorithm

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FREIVALD'S ALGORITHM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Verify Matrix Multiplication", "A * B = C".
6  * - Classic Scenarios: You are given three massive N x N matrices (A, B, and C).
7  *   You need to determine if A * B equals C without TLEing.
8  * - The Magic: A full multiplication takes O(N^3). Freivald's generates a random
9  *   1D vector (r) of 0s and 1s, and checks if A * (B * r) == C * r.
10 *   Since matrix-vector multiplication is O(N^2), this is drastically faster.
11 *   Repeating it K times drops the probability of a false positive to (1/2)^K.
12 *
13 * 2. HOW TO USE IT (THE BLACK BOX)
14 * - Query:
15 *   bool is_correct = freivalds(A, B, C, 15); // 15 iterations is perfectly
16 *   safe
17 * - Complexity:

```

```

18 *      Time:  $O(K * N^2)$ 
19 *      Space:  $O(N)$ 
20 */
21
22 #include <random>
23 #include <chrono>
24
25 using ll = long long;
26 const ll MOD = 1e9 + 7;
27
28 vector<ll> multiply_mat_vec(const vector<vector<ll>> &mat, const vector<ll> &vec)
29 {
30     int n = mat.size();
31     vector<ll> res(n, 0);
32     for (int i = 0; i < n; i++)
33     {
34         for (int j = 0; j < n; j++)
35         {
36             res[i] = (res[i] + mat[i][j] * vec[j]) % MOD;
37         }
38     }
39     return res;
40 }
41
42 bool freivalds(const vector<vector<ll>> &a, const vector<vector<ll>> &b, const
vector<vector<ll>> &c, int k_iterations = 15)
43 {
44     int n = a.size();
45     mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
46     uniform_int_distribution<int> dist(0, 1);
47
48     for (int k = 0; k < k_iterations; k++)
49     {
50         vector<ll> r(n);
51         for (int i = 0; i < n; i++)
52         {
53             r[i] = dist(rng);
54         }
55
56         vector<ll> br = multiply_mat_vec(b, r);
57         vector<ll> abr = multiply_mat_vec(a, br);
58         vector<ll> cr = multiply_mat_vec(c, r);
59
60         for (int i = 0; i < n; i++)
61         {
62             if (abr[i] != cr[i])
63             {
64                 return false;
65             }
66         }
67     }
68     return true;
69 }

```

4.5.2 Matrix expo

```

1 typedef vector<vector<ll>> Matrix;
2
3 Matrix matrix_mul(Matrix &A, Matrix &B)
4 {
5     Matrix C;
6     ll n = ll(A.size());
7     ll m = ll(A[0].size());
8     ll l = ll(B[0].size());
9     C.resize(n, vector<ll>(l));
10    for (ll i = 0; i < n; i++)
11    {
12        for (ll k = 0; k < l; k++)
13        {
14            for (ll j = 0; j < m; j++)
15            {
16                C[i][k] += A[i][j] % MOD * B[j][k] % MOD;
17                C[i][k] %= MOD;
18            }
19        }
20    }
21    return C;
22 }
23
24 Matrix matrix_expo(Matrix &matrix, ll pow)
25 {
26     ll n = ll(matrix.size());
27     Matrix res;
28     res.resize(n, vector<ll>(n));
29     for (int i = 0; i < n; i++)
30         res[i][i] = 1;
31     while (pow)
32     {
33         if (pow & 1)
34             res = matrix_mul(matrix, res);
35         matrix = matrix_mul(matrix, matrix);
36         pow >>= 1;
37     }
38     return res;
39 }
40
41 Matrix I(int n)
42 {
43     Matrix T(n, vector<int>(n));
44     for (int i = 0; i < n; i++)
45         T[i][i] = 1;
46     return T;
47 }
48
49 // 2d Grid represent as graph
50 // count paths
51 short dx[8] = {1,1,-1,-1,2,2,-2,-2};
52 short dy[8] = {2,-2,2,-2,1,-1,1,-1};
53 void solve() {

```

```

54     int k;cin >>k;
55     Matrix T(65,Row(65));
56     for (int i = 0;i<8;i++) {
57         for (int j = 0;j<8;j++) {
58             int cur = i*8 + j;
59             for (int k = 0;k<8;k++) {
60                 short ni = i+dx[k];
61                 short nj = j+dy[k];
62                 if (ni < 0 || nj < 0 || ni > 7 || nj>7) continue;
63                 int nCur = ni*8+nj;
64                 T[nCur][cur] = 1;
65             }
66         }
67     }
68     for (int i = 0;i<65;i++) {
69         T[i][64] = 1;
70     }
71     T = fastPower(T,k+1);
72     int ans = T[0][64];
73     cout << ans;
74 }

```

4.5.3 Matrix expo Queries

```

1
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: FAST PATH COUNTING (PRECOMPUTED MATRIX
  EXPONENTIATION)
3 * 1. WHAT PROBLEM DOES THIS SOLVE?
4 * - Keywords: "Number_of_paths_of_length_K", "Multiple_Queries(Q)", "State_
  Vector_multiplication".
5 * - Classic Scenarios: You need to find the number of valid paths of exactly
  length K between
6 * node U and node V in a directed graph, modulo some number. BUT, you have Q
  queries!
7 * Standard Matrix Exponentiation per query takes  $O(Q * N^3 * \log K)$ , which
  will Time Limit Exceed (TLE).
8 * - The Magic: "The_Vector-Matrix_Trick". We precompute the binary powers of the
  adjacency
9 * matrix  $A^{(2^0)}$ ,  $A^{(2^1)}$ , ...,  $A^{(2^{29})}$  ONCE in  $O(N^3 * \log K)$ .
10 * Then, for each query, we don't multiply  $N \times N$  matrices together. Instead, we
  start with
11  $a_{1 \times N}$  state_vector (where only the starting node U is 1) and multiply this
  VECTOR
12 by the precomputed  $N \times N$  matrices. This slashes the query time down to just
 $O(N^2 * \log K)$ !
13
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Base State: Build your adjacency matrix `t`. If there is a directed edge u->v,
  `t[u][v]=1`.
16 * - Precomputation:
17  $pre[0]=t$ ;
18  $pre[i]=mul(pre[i-1],pre[i-1]);$  // Computes  $A^{(2^i)}$ 
19 * - Queries:

```

```

20  $a_{1 \times N}$  Create  $a_{1 \times N}$  matrix `s` (state_vector). Set `s[0][u]=
21  $1$  For each set bit i in K, update `s=s*mul(s,pre[i])`.
22 * The answer is simply `s[0][v]`.
23
24 * Complexity:
25  $Time: Precomputation: O(N^3 * \log K) + Queries: O(Q * N^2 * \log K)$ .
26  $Space: O(\log K * N^2)$  to store the 30 precomputed matrices.
27
28 const int mod=1e9+7;
29 #define Row_vector<int>
30 #define mat_vector<Row>
31 mat_mul(mat&a,mat&b){
32     int n=a.size(),m=b[0].size(),k=a[0].size();
33     mat_res(n,Row(m));
34     for(int i=0;i<n;i++){
35         for(int j=0;j<m;j++){
36             for(int o=0;o<k;o++){
37                 res[i][j]+=a[i][o]*b[o][j]%mod;
38                 res[i][j]%=mod;
39             }
40         }
41     }
42     return res;
43 }
44
45 mat_power(mat a,int b){
46     int n=a.size();
47     mat_res(n,Row(n));
48     for(int i=0;i<n;i++){
49         res[i][i]=1;
50     }
51     if(b&1){
52         res=mul(res,a);
53     }
54     b>>=1;
55     a=mul(a,a);
56     return res;
57 }
58
59 void solve(){
60     int n,m,q;cin>>n>>m>>q;
61     mat t(n,Row(n));
62     for(int i=0;i<m;i++){
63         int u,v;cin>>u>>v;
64         u--;v--;
65         t[u][v]=1;
66     }
67     vector<mat>pre(30,mat(n,Row(n)));
68     pre[0]=t;
69     for(int i=1;i<30;i++){
70         pre[i]=mul(pre[i-1],pre[i-1]);
71     }
72     while(q--){
73         int u,v,k;cin>>u>>v>>k;--u,--v;
74         mat s(1,Row(n));

```

```

75         for(int i=0; i<30; i++){
76             if(1<=i&k){
77                 s=mul(s,pre[i]);
78             }
79         }
80         cout<<s[0][v]<<"\n";
81     }
82 }

```

4.5.4 Min Path Using Matrix expo

```

1 * THE ULTIMATE CP TEMPLATE BLUEPRINT: MIN-PLUS MATRIX EXPONENTIATION
2 * 1. WHAT PROBLEM DOES THIS SOLVE?
3 * - Keywords: "Shortest path of exactly K edges", "Paths of length K", "K is up
   to 10^9 or 10^18", "Min-Plus Algebra".
4 * - Classic Scenarios: You are given a graph and need to find the minimum cost
   to travel between
5 * nodes using EXACTLY (or at most) K edges. Since K is huge, standard
   algorithms like Dijkstra
6 * or Bellman-Ford will Time Limit Exceed (TLE).
7 * - The Magic: We treat the adjacency matrix as paths of length 1. If we
   multiply the matrix by itself,
8 * we get paths of length 2. But instead of standard multiplication (+ and *),
   we use the
9 * Min-Plus Semiring (min instead of +, and + instead of *).
10 * Using binary exponentiation, we can compute A^K to find paths of length K in
    just O(N^3 log K)!
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Initialization: Create an N x N matrix filled with infinity (`oo`).
14 *     Matrix t(n, Row(n, oo));
15 * - Build Base State: Set the weight of directed edges.
16 *     t[u][v] = w; // Cost of 1 edge from u to v
17 * - Execution: Call `fastPower` to raise the matrix to the power of K.
18 *     t = fastPower(t, k);
19
20
21 #define Row vector<int>
22 #define Matrix vector<Row>
23 const int mod = 1e9 + 7;
24 const int oo = 2e18+5;
25
26 Matrix mul(Matrix &a, Matrix &b) {
27     int n = a.size();
28     int m = b[0].size();
29     int sz = a[0].size();
30     Matrix res(n, Row(m, oo));
31     for (int i = 0; i < n; i++) {
32         for (int j = 0; j < m; j++) {
33             for (int o = 0; o < sz; o++) {
34                 res[i][j] = min(res[i][j], a[i][o] + b[o][j]);
35             }
36         }
37     }

```

```

37     }
38     return res;
39 }
40
41 Matrix fastPower(Matrix &a, int b) {
42     int n = a.size();
43
44     Matrix res(n, Row(n, oo));
45     for (int i = 0; i < n; i++) {
46         res[i][i] = 0;
47     }
48
49     while (b) {
50         if (b & 1)
51             res = mul(res, a);
52         a = mul(a, a);
53         b /= 2;
54     }
55     return res;
56 }
57
58 void solve() {
59     int n, m, k;
60     cin >> n >> m >> k;
61     Matrix t(n, Row(n, oo));
62     for (int i = 0; i < m; i++) {
63         int u, v, w;
64         cin >> u >> v >> w;
65         --u;
66         --v;
67         t[u][v] = w;
68     }
69     t = fastPower(t, k);
70
71     int ans = oo;
72     for (int i = 0; i < n; i++) {
73         for (int j = 0; j < n; j++) {
74             ans = min(ans, t[i][j]);
75         }
76     }
77     if (ans > 1e18) {
78         cout << "IMPOSSIBLE\n";
79     } else {
80         cout << ans << "\n";
81     }
82 }

```

4.5.5 Permanent matrix

```

1 /**
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: PERMANENT OF A MATRIX (RYSER'S FORMULA)
3 *
4 * 1. WHAT PROBLEM DOES THIS SOLVE?

```

```

5  * - Keywords: "Number of perfect matchings", "Bipartite graph matchings".
6  * - Classic Scenarios: Given an  $N \times N$  adjacency matrix of a bipartite graph,
7  *   find exactly how many distinct perfect matchings exist.
8  * - The Magic: Uses Inclusion-Exclusion (Ryser's Formula) to bypass the
9  *   factorial complexity of permutations, solving it in exactly  $O(N * 2^N)$ .
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *   long long ways = permanent(adj_matrix);
14 *
15 * - Complexity:
16 *   Time:  $O(N * 2^N)$  Generally safe for  $N \leq 20$ .
17 *   Space:  $O(N)$ 
18 */
19
20 using ll = long long;
21 const ll MOD = 1e9 + 7;
22
23 ll permanent(const vector<vector<ll>> &a)
24 {
25     int n = a.size();
26     if (n == 0)
27         return 0;
28
29     ll total_perm = 0;
30     int max_mask = 1 << n;
31
32     for (int mask = 1; mask < max_mask; mask++)
33     {
34         ll current_prod = 1;
35         int set_bits = 0;
36
37         for (int i = 0; i < n; i++)
38         {
39             ll row_sum = 0;
40             int bit_count = 0;
41             for (int j = 0; j < n; j++)
42             {
43                 if ((mask >> j) & 1)
44                 {
45                     row_sum = (row_sum + a[i][j]) % MOD;
46                     if (i == 0)
47                         bit_count++;
48                 }
49             }
50             if (i == 0)
51                 set_bits = bit_count;
52             current_prod = (current_prod * row_sum) % MOD;
53         }
54
55         if ((n - set_bits) % 2 == 1)
56         {
57             total_perm = (total_perm - current_prod + MOD) % MOD;
58         }
59         else

```

```

60         {
61             total_perm = (total_perm + current_prod) % MOD;
62         }
63     }
64     return total_perm;
65 }

```

4.6 Optimization

4.6.1 Simplex

```

1  /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SIMPLEX ALGORITHM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Maximize profit", "Minimize cost", "Subject to constraints".
6  * - Classic Scenarios: A generic resource allocation problem where you need to
7  *   maximize an objective function  $C^T X$  subject to a system of inequalities  $AX \leq B$ .
8  * - The Magic: Bypasses the need for complex min-cost max-flow graph modeling
9  *   or impossible greedy logic. You literally just pass the coefficients of your
10 *   equations into the matrix.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *   vector<double> ans_vars;
15 *   double max_profit = simplex(A, b, c, ans_vars);
16 * - Result Breakdown:
17 *   Returns numeric_limits<double>::infinity() if the problem is unbounded.
18 *   Returns -numeric_limits<double>::infinity() if there is no feasible
19 *   solution.
20 *   Otherwise, returns the maximum possible value of the objective function,
21 *   and populates `ans_vars` with the optimal values for variables  $X_1$  to
22 *    $X_n$ .
23 *
24 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
25 * - Handling  $\geq$  constraints: If an equation is  $5x + 3y \geq 10$ , multiply the
26 *   entire
27 *   row by  $-1$  to make it  $-5x - 3y \leq -10$  before pushing it to matrix  $A$  and
28 *   vector  $b$ .
29 * - Handling  $=$  constraints: Split it into two inequalities:
30 *    $(5x + 3y \leq 10)$  AND  $(-5x - 3y \leq -10)$ .
31 */
32 const double EPS = 1e-9;
33 const double INF = numeric_limits<double>::infinity();
34
35 double simplex(vector<vector<double>> A, vector<double> b, vector<double> c,
36               vector<double> &x)
37 {
38     int m = b.size(), n = c.size();
39     vector<int> N(n + 1), B(m);
40     vector<vector<double>> D(m + 2, vector<double>(n + 2));

```

```

37
38 for (int i = 0; i < m; i++)
39 {
40     for (int j = 0; j < n; j++)
41         D[i][j] = A[i][j];
42     B[i] = n + i;
43     D[i][n] = -1;
44     D[i][n + 1] = b[i];
45 }
46 for (int j = 0; j < n; j++)
47 {
48     N[j] = j;
49     D[m][j] = -c[j];
50 }
51 N[n] = -1;
52 D[m + 1][n] = 1;
53
54 auto pivot = [&](int r, int s)
55 {
56     double inv = 1.0 / D[r][s];
57     for (int i = 0; i < m + 2; i++)
58     {
59         if (i != r && abs(D[i][s]) > EPS)
60         {
61             double inv2 = D[i][s] * inv;
62             for (int j = 0; j < n + 2; j++)
63             {
64                 D[i][j] -= D[r][j] * inv2;
65             }
66             D[i][s] = D[r][s] * inv2;
67         }
68     }
69     for (int j = 0; j < n + 2; j++)
70     {
71         if (j != s)
72             D[r][j] *= inv;
73     }
74     D[r][s] = inv;
75     swap(B[r], N[s]);
76 };
77
78 auto phase = [&](int row)
79 {
80     while (true)
81     {
82         int s = -1;
83         for (int j = 0; j <= n; j++)
84         {
85             if (N[j] != -1 && (s == -1 || D[row][j] < D[row][s] - EPS))
86             {
87                 s = j;
88             }
89         }
90         if (D[row][s] > -EPS)
91             return true;

```

```

92
93     int r = -1;
94     for (int i = 0; i < m; i++)
95     {
96         if (D[i][s] > EPS && (r == -1 || D[i][n + 1] / D[i][s] < D[r][n + 1] / D[r][s] - EPS))
97         {
98             r = i;
99         }
100     }
101     if (r == -1)
102         return false;
103     pivot(r, s);
104 }
105 };
106
107 int r = 0;
108 for (int i = 1; i < m; i++)
109 {
110     if (D[i][n + 1] < D[r][n + 1])
111         r = i;
112 }
113
114 if (D[r][n + 1] < -EPS)
115 {
116     pivot(r, n);
117     if (!phase(m + 1) || D[m + 1][n + 1] < -EPS)
118         return -INF;
119     for (int i = 0; i < m; i++)
120     {
121         if (B[i] == -1)
122         {
123             int s = -1;
124             for (int j = 0; j <= n; j++)
125             {
126                 if (N[j] != -1 && (s == -1 || D[i][j] < D[i][s] - EPS))
127                     s = j;
128             }
129             pivot(i, s);
130         }
131     }
132 }
133
134 if (!phase(m))
135     return INF;
136
137 x.assign(n, 0);
138 for (int i = 0; i < m; i++)
139 {
140     if (B[i] < n)
141         x[B[i]] = D[i][n + 1];
142 }
143
144 return D[m][n + 1];
145 }

```

4.7 Polynomial

4.7.1 EXP POW Sqrt

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: POLYNOMIAL EXP, POW, & Sqrt
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Exponentiate Generating Function", " $e^{A(x)}$ ", " $A(x)^K$ ", " $\sqrt{A(x)}$ ".
6  * - Classic Scenarios: Solving combinatorial problems involving unlabelled sets
7  *   or Euler transforms (Topic 335, 337, 338).
8  * - The Magic: Polynomial Exp uses Newton's method on logarithms:
9  *    $F_{k+1} = F_k * (1 - \ln(F_k) + A)$ . Polynomial Pow uses the identity
10 *    $A^k = \exp(k * \ln(A))$ .
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *   vector<ll> expA = poly_exp(A, N);
15 *   vector<ll> powA = poly_pow(A, K, N);
16 *   vector<ll> sqrtA = poly_sqrt(A, N);
17 *
18 * - Complexity:
19 *   Time:  $O(N \log N)$ 
20 *   Space:  $O(N)$ 
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Dependencies: Requires all functions from FPS Toolkit Part 1.
24 * - Constant Term Strictness:
25 *   - poly_exp: A[0] MUST be 0.
26 *   - poly_pow: A[0] MUST be 1 (if not, factor out the lowest degree term
27 *     first).
28 *   - poly_sqrt: A[0] MUST be 1.
29 */
30 vector<ll> poly_exp(vector<ll> a, int deg)
31 {
32     if (deg == 1)
33         return {1};
34     vector<ll> b = poly_exp(a, (deg + 1) / 2);
35     b.resize(deg);
36     vector<ll> lnb = poly_log(b, deg);
37     vector<ll> diff(deg);
38     for (int i = 0; i < deg; i++)
39     {
40         diff[i] = (a.size() > i ? a[i] : 0) - lnb[i];
41         if (diff[i] < 0)
42             diff[i] += MOD;
43     }
44     diff[0] = (diff[0] + 1) % MOD;
45     vector<ll> res = multiply(b, diff);
46     res.resize(deg);
47     return res;
48 }
49
```

```
50 vector<ll> poly_pow(vector<ll> a, ll k, int deg)
51 {
52     vector<ll> lna = poly_log(a, deg);
53     for (int i = 0; i < lna.size(); i++)
54     {
55         lna[i] = lna[i] * (k % MOD) % MOD;
56     }
57     return poly_exp(lna, deg);
58 }
59
60 vector<ll> poly_sqrt(vector<ll> a, int deg)
61 {
62     if (deg == 1)
63         return {1};
64     vector<ll> b = poly_sqrt(a, (deg + 1) / 2);
65     b.resize(deg);
66     vector<ll> inv_b = poly_inv(b, deg);
67     vector<ll> res = multiply(a, inv_b);
68     ll inv2 = modInverse(2);
69     res.resize(deg);
70     for (int i = 0; i < deg; i++)
71     {
72         res[i] = (res[i] + b[i]) % MOD * inv2 % MOD;
73     }
74     return res;
75 }
```

4.7.2 Lagrange

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: LAGRANGE INTERPOLATION  $O(K)$ 
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "N-th term of a sequence", "Sum of  $i^p$ ", "Polynomial fitting".
6  * - Classic Scenarios: You know a sequence is generated by a polynomial of
7  *   degree K.
8  *   You need the exact value at a massive index X (e.g.,  $X = 10^{18}$ ), but you can
9  *   only easily compute the first K+1 values (at  $x = 0, 1, 2, \dots, K$ ).
10 * - The Magic: Standard Lagrange takes  $O(K^2)$ . Because our evaluation points are
11 *   strictly contiguous integers (0 to K), the products in the numerator and
12 *   denominator become factorials and prefix/suffix products. This collapses the
13 *   complexity to pure  $O(K)$ , allowing you to evaluate degree  $10^6$  polynomials in
14 *   less than a second.
15 *
16 * 2. HOW TO USE IT (THE BLACK BOX)
17 * - Query:
18 *   long long ans = lagrange_eval(Y, X);
19 *   // Y is a vector of size K+1 containing P(0), P(1), ..., P(K).
20 *   // X is the massive point you want to evaluate at.
21 *
22 * - Complexity:
23 *   Time:  $O(K)$  where K is the degree of the polynomial (size of Y - 1).
24 *   Space:  $O(K)$  for prefix/suffix and factorial arrays.
```

```

24 *
25 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
26 * - Modulo: Adjust the global MOD variable if the problem uses 1e9+7 instead
27 *   of 998244353.
28 */
29
30 using ll = long long;
31 const ll MOD = 998244353;
32
33 ll binpow(ll base, ll exp)
34 {
35     ll res = 1;
36     base %= MOD;
37     while (exp > 0)
38     {
39         if (exp % 2 == 1)
40             res = (res * base) % MOD;
41         base = (base * base) % MOD;
42         exp /= 2;
43     }
44     return res;
45 }
46
47 ll modInverse(ll n)
48 {
49     return binpow(n, MOD - 2);
50 }
51
52 ll lagrange_eval(const vector<ll> &y, ll x)
53 {
54     int k = y.size() - 1;
55     if (x <= k)
56         return y[x];
57
58     vector<ll> pref(k + 1, 1), suff(k + 1, 1);
59     vector<ll> fact(k + 1, 1), invFact(k + 1, 1);
60
61     for (int i = 1; i <= k; i++)
62         fact[i] = fact[i - 1] * i % MOD;
63     invFact[k] = modInverse(fact[k]);
64     for (int i = k - 1; i >= 0; i--)
65         invFact[i] = invFact[i + 1] * (i + 1) % MOD;
66
67     pref[0] = x % MOD;
68     for (int i = 1; i <= k; i++)
69         pref[i] = pref[i - 1] * (x - i) % MOD;
70
71     suff[k] = (x - k) % MOD;
72     for (int i = k - 1; i >= 0; i--)
73         suff[i] = suff[i + 1] * (x - i) % MOD;
74
75     ll ans = 0;
76     for (int i = 0; i <= k; i++)
77     {
78         ll num = (i == 0 ? 1 : pref[i - 1]) * (i == k ? 1 : suff[i + 1]) % MOD;

```

```

79         ll den = invFact[i] * invFact[k - i] % MOD;
80         ll term = y[i] * num % MOD * den % MOD;
81         if ((k - i) % 2 == 1)
82         {
83             ans = (ans - term + MOD) % MOD;
84         }
85         else
86         {
87             ans = (ans + term) % MOD;
88         }
89     }
90     return ans;
91 }

```

4.7.3 Multipoint evaluation

```

1 /**
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: MULTIPOINT EVALUATION
3 *
4 * 1. WHAT PROBLEM DOES THIS SOLVE?
5 * - Keywords: "Evaluate polynomial at multiple points", " $P(x_i)$ ".
6 * - Classic Scenarios: You have a massive polynomial  $P(x)$  of degree  $N$ , and you
   need
7 * to find its exact value at  $M$  different points (Topic 340).
8 * - The Magic: Horner's method takes  $O(N * M)$  which TLEs. This uses a Subproduct
   Tree
9 * (a segment tree of polynomials) and polynomial division with remainder to
   process
10 * all queries simultaneously in  $O(N \log^2 N)$ .
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     vector<ll> results = multipoint_eval(P, X);
15 *     // results[i] contains  $P(X[i]) \bmod 998244353$ .
16 *
17 * - Complexity:
18 *     Time:  $O(N \log^2 N)$ 
19 *     Space:  $O(N \log N)$ 
20 *
21 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22 * - Polynomial Modulo: This struct includes poly_mod(A, B) which finds the
   remainder
23 * of  $A(x) / B(x)$ . This is an incredibly powerful tool on its own.
24 */
25
26 vector<ll> poly_rev(vector<ll> a)
27 {
28     reverse(a.begin(), a.end());
29     return a;
30 }
31
32 vector<ll> poly_mod(vector<ll> a, vector<ll> b)
33 {

```

```

34 while (a.size() > 1 && a.back() == 0)
35     a.pop_back();
36 while (b.size() > 1 && b.back() == 0)
37     b.pop_back();
38 if (a.size() < b.size())
39     return a;
40
41 int n = a.size(), m = b.size();
42 vector<ll> rev_a = poly_rev(a);
43 vector<ll> rev_b = poly_rev(b);
44
45 rev_a.resize(n - m + 1);
46 vector<ll> inv_rev_b = poly_inv(rev_b, n - m + 1);
47 vector<ll> q = multiply(rev_a, inv_rev_b);
48 q.resize(n - m + 1);
49 q = poly_rev(q);
50
51 vector<ll> qb = multiply(q, b);
52 vector<ll> r(m - 1, 0);
53 for (int i = 0; i < m - 1; i++)
54 {
55     r[i] = (a[i] - (i < qb.size() ? qb[i] : 0)) % MOD;
56     if (r[i] < 0)
57         r[i] += MOD;
58 }
59 while (r.size() > 1 && r.back() == 0)
60     r.pop_back();
61 return r;
62 }
63
64 vector<vector<ll>> tree;
65
66 void build_subproduct_tree(int node, int l, int r, const vector<ll> &x)
67 {
68     if (l == r)
69     {
70         tree[node] = {(MOD - x[l]) % MOD, 1};
71         return;
72     }
73     int mid = l + (r - l) / 2;
74     build_subproduct_tree(2 * node, l, mid, x);
75     build_subproduct_tree(2 * node + 1, mid + 1, r, x);
76     tree[node] = multiply(tree[2 * node], tree[2 * node + 1]);
77 }
78
79 void eval_subproduct_tree(int node, int l, int r, vector<ll> p, vector<ll> &ans)
80 {
81     p = poly_mod(p, tree[node]);
82     if (l == r)
83     {
84         ans[l] = p.empty() ? 0 : p[0];
85         return;
86     }
87     int mid = l + (r - l) / 2;
88     eval_subproduct_tree(2 * node, l, mid, p, ans);

```

```

89     eval_subproduct_tree(2 * node + 1, mid + 1, r, p, ans);
90 }
91
92 vector<ll> multipoint_eval(const vector<ll> &p, const vector<ll> &x)
93 {
94     if (x.empty())
95         return {};
96     int m = x.size();
97     tree.resize(4 * m);
98     build_subproduct_tree(1, 0, m - 1, x);
99     vector<ll> ans(m);
100    eval_subproduct_tree(1, 0, m - 1, p, ans);
101    return ans;
102 }

```

4.7.4 Poly inv log

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: POLYNOMIAL INV & LOG
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Generating Functions", "Formal Power Series (FPS)", "Divide
6  * Polynomials".
7  * - Classic Scenarios: You have a generating function expressed as a fraction  $A(x)/B(x)$ 
8  * and need the first  $N$  coefficients.
9  * - The Magic: Polynomial Inverse (Topic 334) uses Newton's Method to find  $B^{-1}(x)$ 
10  * such that  $B(x) * B^{-1}(x) = 1 \bmod x^n$  in  $O(N \log N)$ . Polynomial Log (Topic
11  * 336)
12  * uses calculus:  $\ln(A(x)) = \text{integral}(A'(x) * A^{-1}(x))$ .
13  *
14  * 2. HOW TO USE IT (THE BLACK BOX)
15  * - Query:
16  *     vector<ll> invA = poly_inv(A, N);
17  *     vector<ll> logA = poly_log(A, N);
18  *
19  * - Complexity:
20  *     Time:  $O(N \log N)$ 
21  *     Space:  $O(N)$ 
22  *
23  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
24  * - Dependencies: Requires `ntt`, `multiply`, and `modInverse` from your FFT/NTT
25  * snippet.
26  * - Constant Term: For poly_inv,  $A[0]$  MUST NOT be 0. For poly_log,  $A[0]$  MUST be
27  * 1.
28  */
29 using ll = long long;
30 const ll MOD = 998244353;
31
32 vector<ll> poly_inv(vector<ll> a, int deg)
33 {

```

```

31     if (deg == 1)
32         return {modInverse(a[0])};
33     vector<ll> b = poly_inv(a, (deg + 1) / 2);
34     int n = 1;
35     while (n < deg * 2)
36         n <= 1;
37     vector<ll> fa(a.begin(), a.begin() + min((int)a.size(), deg));
38     fa.resize(n);
39     vector<ll> fb = b;
40     fb.resize(n);
41     ntt(fa, false);
42     ntt(fb, false);
43     for (int i = 0; i < n; i++)
44     {
45         fb[i] = fb[i] * (2 - fa[i] * fb[i] % MOD + MOD) % MOD;
46     }
47     ntt(fb, true);
48     fb.resize(deg);
49     return fb;
50 }
51
52 vector<ll> poly_deriv(const vector<ll> &a)
53 {
54     if (a.empty())
55         return {};
56     vector<ll> res(max(1, (int)a.size() - 1));
57     for (int i = 1; i < a.size(); i++)
58     {
59         res[i - 1] = a[i] * i % MOD;
60     }
61     return res;
62 }
63
64 vector<ll> poly_integr(const vector<ll> &a)
65 {
66     if (a.empty())
67         return {0};
68     vector<ll> res(a.size() + 1, 0);
69     for (int i = 0; i < a.size(); i++)
70     {
71         res[i + 1] = a[i] * modInverse(i + 1) % MOD;
72     }
73     return res;
74 }
75
76 vector<ll> poly_log(vector<ll> a, int deg)
77 {
78     vector<ll> deriv = poly_deriv(a);
79     vector<ll> inv = poly_inv(a, deg);
80     vector<ll> res = poly_integr(multiply(deriv, inv));
81     res.resize(deg);
82     return res;
83 }

```

4.8 Probability

4.8.1 Expected occurrence time of substring

```

1  /**
2  *   THE ULTIMATE CP TEMPLATE BLUEPRINT: EXPECTED OCCURRENCE TIME OF SUBSTRING
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Expected number of coin flips", "Typing monkeys", "Expected time
   to see string".
6  * - Classic Scenarios: A random generator picks characters from an alphabet of
   size A.
7  * What is the expected number of generated characters until string S appears?
8  * - The Magic: Bypasses  $O(N^3)$  Gaussian elimination. By setting up a fair casino
9  * Martingale where gamblers bet on consecutive characters, the expected time
   simplifies
10 * to a summation over the borders (prefix == suffix) of the string.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     long long expected_time = expected_string_occurrence(S, ALPHABET_SIZE,
   MOD);
15 *
16 * - Complexity:
17 *     Time:  $O(|S|)$ 
18 *     Space:  $O(|S|)$ 
19 *
20 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21 * - Alphabet Size: If flipping a coin, alphabet_size = 2. If rolling a die, 6.
22 * - Array Form: This strictly uses the KMP prefix function. The condition `j ==
   pi[j-1]`
23 * efficiently finds all prefix-suffix borders of the entire string.
24 */
25
26 using ll = long long;
27
28 vector<int> compute_pi(const string &s)
29 {
30     int n = s.length();
31     vector<int> pi(n);
32     for (int i = 1; i < n; i++)
33     {
34         int j = pi[i - 1];
35         while (j > 0 && s[i] != s[j])
36             j = pi[j - 1];
37         if (s[i] == s[j])
38             j++;
39         pi[i] = j;
40     }
41     return pi;
42 }
43
44 ll expected_string_occurrence(const string &s, ll alphabet_size, ll mod)
45 {
46     vector<int> pi = compute_pi(s);

```

```

47 int n = s.length();
48
49 auto binpow = [&](ll base, ll exp)
50 {
51     ll res = 1;
52     base %= mod;
53     while (exp > 0)
54     {
55         if (exp % 2 == 1)
56             res = (res * base) % mod;
57         base = (base * base) % mod;
58         exp /= 2;
59     }
60     return res;
61 };
62
63 ll expected_time = 0;
64 int current_border = n;
65
66 while (current_border > 0)
67 {
68     expected_time = (expected_time + binpow(alphabet_size, current_border)) %
69         mod;
70     current_border = pi[current_border - 1];
71 }
72
73 return expected_time;
74 }

```

5 Number Theory

5.1 Fibonacci

5.1.1 Fast doubling fibonacci

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FAST DOUBLING FIBONACCI
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "N-th Fibonacci", "Massive N".
6  * - Classic Scenarios: You need  $F_N$  modulo  $M$  where  $N = 10^{18}$ .
7  * - The Magic: Derives  $F_{2k}$  and  $F_{2k+1}$  directly from  $F_k$  and  $F_{k+1}$ .
8  * It completely sidesteps matrix overhead, making it drastically faster
9  * and much shorter to type than a robust matrix exponentiation struct.
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *     pair<ll, ll> ans = fast_doubling(N, M);
14 * - Result Breakdown:
15 *     ans.first =  $F_N$ 
16 *     ans.second =  $F_{N+1}$ 
17 *

```

```

18 * - Complexity:
19 *     Time: Strictly  $O(\log N)$ .
20 *     Space:  $O(\log N)$  for the recursion stack.
21 */
22
23 using ll = long long;
24
25 pair<ll, ll> fast_doubling(ll n, ll mod)
26 {
27     if (n == 0)
28         return {0, 1};
29     auto p = fast_doubling(n >> 1, mod);
30     ll c = p.first * (2 * p.second - p.first + mod) % mod;
31     ll d = (p.first * p.first + p.second * p.second) % mod;
32     if (n & 1)
33         return {d, (c + d) % mod};
34     return {c, d};
35 }

```

5.1.2 PHI field

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PHI FIELD ( $\mathbb{Z}[\sqrt{5}]$ )
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Fibonacci sum of powers", "Binet's formula modulo  $M$ ".
6  * - Classic Scenarios: You need to evaluate algebraic expressions involving
7  * Fibonacci numbers, such as  $\sum (F_i)^K \bmod M$ .
8  * - The Magic: It operates exactly like complex numbers, but instead of  $i^2 = -1$ ,
9  * it maps  $a + b\sqrt{5}$  under modulo  $M$ . This allows you to directly use Binet's
10 * formula natively even if 5 is a quadratic non-residue modulo  $M$ !
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     Phi res = power_phi({A, B}, exp, MOD);
15 *
16 * - Complexity:
17 *     Time:  $O(\log(\text{exp}))$  per multiplication.
18 *     Space:  $O(1)$ 
19 */
20
21 using ll = long long;
22
23 struct Phi
24 {
25     ll a, b;
26 };
27
28 Phi mult_phi(Phi x, Phi y, ll mod)
29 {
30     return {

```

```

31         (x.a * y.a + 5 * x.b % mod * y.b) % mod,
32         (x.a * y.b + x.b * y.a) % mod};
33 }
34
35 Phi power_phi(Phi base, ll exp, ll mod)
36 {
37     Phi res = {1, 0};
38     base.a %= mod;
39     base.b %= mod;
40     while (exp > 0)
41     {
42         if (exp % 2 == 1)
43             res = mult_phi(res, base, mod);
44         base = mult_phi(base, base, mod);
45         exp /= 2;
46     }
47     return res;
48 }

```

5.2 Linear diophantine equation

5.2.1 Floor sum

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FLOOR SUM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "ax + by <= c", "Lattice points under line", "Sum of arithmetic
   progression mod M".
6  * - Classic Scenarios: Compute  $\sum_{i=0}^{n-1} \text{floor}((a * i + b) / m)$ .
7  * - The Magic: It operates exactly like the Euclidean algorithm, trading
   dimensions
8  * and folding the geometry to evaluate the entire sum logarithmically.
9  *
10 * 2. HOW TO USE IT (THE BLACK BOX)
11 * - Query:
12 *     long long ans = floor_sum(n, m, a, b);
13 *
14 * - Complexity:
15 *     Time:  $O(\log(\min(m, a)))$ 
16 *     Space:  $O(\log(\min(m, a)))$  due to recursion.
17 *
18 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
19 * - Equivalent to  $ax + by \leq c$ : For a line equation in the first quadrant,
   rearrange
20 * it to  $y \leq (c - ax) / b$ , which directly translates to summing  $\text{floor}((c - a * x) / b)$ 
21 * over all valid x.
22 */
23
24 using ll = long long;
25
26 ll floor_sum(ll n, ll m, ll a, ll b)

```

```

27 {
28     ll ans = 0;
29     if (a >= m)
30     {
31         ans += (n - 1) * n / 2 * (a / m);
32         a %= m;
33     }
34     if (b >= m)
35     {
36         ans += n * (b / m);
37         b %= m;
38     }
39     ll y_max = (a * n + b) / m;
40     ll x_max = y_max * m - b;
41     if (y_max == 0)
42         return ans;
43     ans += (n - (x_max + a - 1) / a) * y_max;
44     ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
45     return ans;
46 }

```

5.2.2 Generalized floor sum

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: GENERALIZED FLOOR SUM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Sum of i * floor", "Sum of squares of floors".
6  * - Classic Scenarios: A step above standard floor sum, used when evaluating
   complex grid summations or polynomial constraints under a modulo line.
7  * - The Magic: Through a complex series of algebraic expansions and Euclidean
   domain folding, it simultaneously resolves three interconnected sums in
8  * pure logarithmic time.
9  *
10 * 2. HOW TO USE IT (THE BLACK BOX)
11 * - Query:
12 *     FSR ans = gen_floor_sum(n, a, b, c);
13 * - Result Breakdown:
14 *     ans.f =  $\sum_{i=0}^{n-1} \text{floor}((a*i+b)/c)$ 
15 *     ans.g =  $\sum_{i=0}^{n-1} i * \text{floor}((a*i+b)/c)$ 
16 *     ans.h =  $\sum_{i=0}^{n-1} \text{floor}((a*i+b)/c)^2$ 
17 *
18 * - Complexity:
19 *     Time:  $O(\log(\min(a, c)))$ 
20 *     Space:  $O(\log(\min(a, c)))$ 
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Inclusive Bounds: This specifically calculates for  $i = 0$  to  $N$  (inclusive).
24 * - If your problem is 1-based, evaluate for  $N$  and subtract the  $i=0$  term, or
25 * use  $N-1$  if the upper bound is exclusive.
26 */
27
28 using ll = long long;
29
30 ll gen_floor_sum(ll n, ll m, ll a, ll b, ll c)

```

```

31
32 struct FSR
33 {
34     ll f, g, h;
35 };
36
37 FSR gen_floor_sum(ll n, ll a, ll b, ll c)
38 {
39     ll ac = a / c, bc = b / c;
40     if (a == 0)
41     {
42         ll f = (n + 1) * bc;
43         ll g = bc * n * (n + 1) / 2;
44         ll h = (n + 1) * bc * bc;
45         return {f, g, h};
46     }
47     if (a >= c || b >= c)
48     {
49         FSR res = gen_floor_sum(n, a % c, b % c, c);
50         ll f = res.f + n * (n + 1) / 2 * ac + (n + 1) * bc;
51         ll g = res.g + ac * n * (n + 1) * (2 * n + 1) / 6 + bc * n * (n + 1) / 2;
52         ll h = res.h + ac * ac * n * (n + 1) * (2 * n + 1) / 6 + bc * bc * (n + 1) + 2 * ac * bc * n * (n + 1) / 2 + 2 * ac * res.g + 2 * bc * res.f;
53         return {f, g, h};
54     }
55     ll m = (a * n + b) / c;
56     FSR res = gen_floor_sum(m - 1, c, c - b - 1, a);
57     ll f = n * m - res.f;
58     ll g = (m * n * (n + 1) - res.f - res.h) / 2;
59     ll h = n * m * m - 2 * res.g - res.f;
60     return {f, g, h};
61 }

```

5.2.3 LDE

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: BOUNDED LDE2
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Solutions in range", " $ax + by = c$ ", "Bounded variables".
6  * - Classic Scenarios: You need the exact count of integer pairs  $(x, y)$ 
   satisfying
7  *  $ax + by = c$  such that  $\min_x \leq x \leq \max_x$  and  $\min_y \leq y \leq \max_y$ .
8  * - The Magic: It finds a base solution using Extended Euclidean Algorithm, then
9  * mathematically shifts the solution using  $(b/g)$  and  $(a/g)$  to find the
   absolute
10 * minimum and maximum valid boundaries, intersecting them safely.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     long long total = count_solutions(a, b, c, min_x, max_x, min_y, max_y);
15 *
16 * - Complexity:
17 *     Time:  $O(\log(\min(a, b)))$ 
18 *     Space:  $O(1)$ 
19 *
20 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21 * - Zero Handling: Safely handles edge cases where  $a = 0$  or  $b = 0$ , which often
22 * crash naive implementations.
23 */
24
25 using ll = long long;
26
27 ll extgcd(ll a, ll b, ll &x, ll &y)
28 {
29     if (b == 0)
30     {
31         x = 1;
32         y = 0;
33         return a;
34     }
35     ll x1, y1;
36     ll d = extgcd(b, a % b, x1, y1);
37     x = y1;
38     y = x1 - y1 * (a / b);
39     return d;
40 }
41
42 void shift_solution(ll &x, ll &y, ll a, ll b, ll cnt)
43 {
44     x += cnt * b;
45     y -= cnt * a;
46 }
47
48 ll count_solutions(ll a, ll b, ll c, ll min_x, ll max_x, ll min_y, ll max_y)
49 {
50     if (a == 0 && b == 0)
51     {
52         if (c == 0)
53             return (max_x - min_x + 1) * (max_y - min_y + 1);
54         return 0;
55     }
56     if (a == 0)
57     {
58         if (c % b == 0 && c / b >= min_y && c / b <= max_y)
59             return max_x - min_x + 1;
60         return 0;
61     }
62     if (b == 0)
63     {
64         if (c % a == 0 && c / a >= min_x && c / a <= max_x)
65             return max_y - min_y + 1;
66         return 0;
67     }
68
69     ll x, y;
70     ll g = extgcd(abs(a), abs(b), x, y);

```

```

71     if (c % g != 0)
72         return 0;
73
74     x *= c / g;
75     y *= c / g;
76     if (a < 0)
77         x = -x;
78     if (b < 0)
79         y = -y;
80
81     ll a_g = a / g, b_g = b / g;
82     ll sign_a = a > 0 ? +1 : -1;
83     ll sign_b = b > 0 ? +1 : -1;
84
85     shift_solution(x, y, a_g, b_g, (min_x - x) / b_g);
86     if (x < min_x)
87         shift_solution(x, y, a_g, b_g, sign_b);
88     if (x > max_x)
89         return 0;
90     ll lx1 = x;
91
92     shift_solution(x, y, a_g, b_g, (max_x - x) / b_g);
93     if (x > max_x)
94         shift_solution(x, y, a_g, b_g, -sign_b);
95     ll rx1 = x;
96
97     shift_solution(x, y, a_g, b_g, -(min_y - y) / a_g);
98     if (y < min_y)
99         shift_solution(x, y, a_g, b_g, -sign_a);
100    if (y > max_y)
101        return 0;
102    ll lx2 = x;
103
104    shift_solution(x, y, a_g, b_g, -(max_y - y) / a_g);
105    if (y > max_y)
106        shift_solution(x, y, a_g, b_g, sign_a);
107    ll rx2 = x;
108
109    if (lx2 > rx2)
110        swap(lx2, rx2);
111    ll lx = max(lx1, lx2);
112    ll rx = min(rx1, rx2);
113
114    if (lx > rx)
115        return 0;
116    return (rx - lx) / abs(b_g) + 1;
117 }

```

5.2.4 Pell equation

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PELL'S EQUATION
3  *

```

```

4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: " $x^2 - d \cdot y^2 = 1$ ", "Fundamental Solution".
6  * - Classic Scenarios: Find the smallest positive integer solution (x, y)
7  *   satisfying Pell's equation where d is a non-square positive integer.
8  * - The Magic: It generates the continued fraction expansion of  $\sqrt{d}$ .
9  *   The convergents ( $h_i / k_i$ ) of this fraction are guaranteed to eventually
10 *   satisfy the equation.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     pair<__int128, __int128> ans = solve_pell(d);
15 *     // Returns {x, y}. If d is a perfect square, returns {-1, -1}.
16 *
17 * - Complexity:
18 *     Time:  $O(\sqrt{d} \cdot \log(d))$ 
19 *     Space:  $O(1)$ 
20 *
21 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22 * - 128-bit Return Type: Fundamental solutions to Pell's equation grow
23 *   exponentially fast. Even for small d (e.g.,  $d = 61$ ), x and y can easily
24 *   overflow 64-bit integers. __int128 is highly recommended here.
25 * - Generating More Solutions: If (x1, y1) is the fundamental solution,
26 *   the next solution (x_k, y_k) is  $(x_1 + y_1 \cdot \sqrt{d})^k$ .
27 */
28
29 using u128 = __int128;
30 using ll = long long;
31
32 pair<u128, u128> solve_pell(ll d)
33 {
34     ll sq = sqrt(d);
35     if (sq * sq == d)
36         return {-1, -1};
37
38     ll m = 0, d_den = 1, a = sq;
39     u128 num1 = 1, num0 = a;
40     u128 den1 = 0, den0 = 1;
41
42     while (num0 * num0 - (u128)d * den0 * den0 != 1)
43     {
44         m = d_den * a - m;
45         d_den = (d - m * m) / d_den;
46         a = (sq + m) / d_den;
47
48         u128 num2 = num1;
49         num1 = num0;
50         num0 = (u128)a * num1 + num2;
51
52         u128 den2 = den1;
53         den1 = den0;
54         den0 = (u128)a * den1 + den2;
55     }
56     return {num0, den0};
57 }

```

5.3 Misc

5.3.1 Min X in Mod range

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: MINIMUM X IN MODULO RANGE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Smallest x", "L <= ax mod m <= R", "Min of Mod of AP".
6  * - Classic Scenarios: Finding the first index where an arithmetic progression
7  * wraps around a modulo into a specific target window.
8  * - The Magic: It uses a recursive Euclidean-style domain folding. Instead
9  * of simulating the progression, it inverts the bounds and steps to find
10 * the exact minimum x in O(log M) time.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     long long x = min_x_in_mod_range(A, M, L, R);
15 *     // Returns -1 if no such x exists.
16 *
17 * - Complexity:
18 *     Time: O(log M)
19 *     Space: O(log M) for recursion.
20 *
21 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22 * - Min of Mod of AP (Topic 295): If you just need the absolute minimum of
23 * (A*x) mod M for x in [0, N], you can binary search the answer V in [0, M-1]
24 * and use this function to check if a valid x <= N exists for L=0, R=V.
25 */
26
27 using ll = long long;
28
29 ll min_x_in_mod_range(ll a, ll m, ll l, ll r)
30 {
31     if (l == 0)
32         return 0;
33     if (a == 0)
34         return -1;
35     ll step = (l + a - 1) / a;
36     if (step * a <= r)
37         return step;
38     ll b = m % a;
39     ll res = min_x_in_mod_range(b, a, a - r % a, a - l % a);
40     if (res == -1)
41         return -1;
42     return (l + res * b + a - 1) / a;
43 }
```

5.3.2 SIMPLEST FRACTION

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: STERN-BROCOT TREE (SIMPLEST FRACTION)
3  *
```

```
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Stern-Brocot", "Simplest Fraction", "Rational Approximation".
6  * - Classic Scenarios: Find the fraction P/Q with the absolute smallest
7  * denominator Q that strictly lies between two other fractions A/B and C/D.
8  * - The Magic: Instead of simulating the binary search descent down the
9  * Stern-Brocot tree (which can TLE in O(max(P, Q))), it uses a continued
10 * fraction / Euclidean acceleration to jump down the branches.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     pair<long long, long long> ans = simplest_fraction(A, B, C, D);
15 *     // Returns {P, Q} such that A/B < P/Q < C/D.
16 *
17 * - Complexity:
18 *     Time: O(log(max(A, B, C, D)))
19 *     Space: O(log(max)) for recursion.
20 *
21 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22 * - Fraction format: Ensure your input fractions A/B and C/D are simplified
23 * and that A/B is strictly less than C/D before passing them in.
24 */
25
26 using ll = long long;
27
28 pair<ll, ll> simplest_fraction(ll a, ll b, ll c, ll d)
29 {
30     if (a / b < c / d)
31         return {a / b + 1, 1};
32     if (a == 0)
33         return {1, d / c + 1};
34     auto res = simplest_fraction(d, c % d, b, a % b);
35     return {res.second + (a / b) * res.first, res.first};
36 }
```

5.4 Modular arithmetic

5.4.1 Baby step giant step

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: BABY-STEP GIANT-STEP (DISCRETE LOG)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Discrete Logarithm", "Find the exponent", "A^x = B mod M".
6  * - Classic Scenarios: You are given A, B, and M, and need to find the minimum
7  * non-negative integer x such that A^x ≡ B (mod M).
8  * - The Magic: It trades space for time. By rewriting x = n*p - q (where n =
9  * sqrt(M)),
10 * the equation becomes A^(n*p) ≡ B * A^q (mod M). We precompute the right side
11 * (Baby Steps) and store them in a hash map, then calculate the left side
12 * (Giant Steps) and check for collisions.
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Query:
```

```

15 *      long long ans = solve_bsgs(A, B, M);
16 *      // Returns -1 if no solution exists.
17 *
18 * - Complexity:
19 *      Time:  $O(\sqrt{M} * \log(\sqrt{M}))$  due to the map.  $O(\sqrt{M})$  if using
        unordered_map.
20 *      Space:  $O(\sqrt{M})$  to store the baby steps.
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Coprime Requirement: This standard version assumes  $\gcd(A, M) == 1$ . If A and M
        are not coprime, you need the Extended BSGS algorithm (which involves
        dividing
25 * out the GCDs until they are coprime).
26 */
27
28 using ll = long long;
29
30 ll solve_bsgs(ll a, ll b, ll m)
31 {
32     a %= m, b %= m;
33     if (b == 1 || m == 1)
34         return 0;
35
36     ll n = sqrt(m) + 1;
37     map<ll, ll> vals;
38     ll cur = b;
39     for (ll q = 0; q <= n; ++q)
40     {
41         vals[cur] = q;
42         cur = (cur * a) % m;
43     }
44
45     ll a_n = 1;
46     for (ll i = 0; i < n; ++i)
47     {
48         a_n = (a_n * a) % m;
49     }
50
51     cur = 1;
52     for (ll p = 1; p <= n; ++p)
53     {
54         cur = (cur * a_n) % m;
55         if (vals.count(cur))
56         {
57             ll ans = n * p - vals[cur];
58             return ans;
59         }
60     }
61     return -1;
62 }

```

5.4.2 Big Mod Power

```

1 /// if we need to calculate  $(n \% m)$  if EX:  $n = 10^{10000}$  and  $m = 10^9$ 
2 ll Big_Mod_Power(string n, ll m)
3 {
4     ll x = 0;
5     for(ll i = 0; i < n.size(); i++)
6     {
7         x = ((x * 10) + (n[i] - '0'));
8         x %= m;
9     }
10    return x;    /// return  $n \% m$ 
11 }

```

5.4.3 CRT

```

1 /**
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: CHINESE REMAINDER THEOREM (GENERAL)
3 *
4 * 1. WHAT PROBLEM DOES THIS SOLVE?
5 * - Keywords: "System of congruences", " $x = a_1 \bmod m_1, x = a_2 \bmod m_2$ ".
6 * - The Magic: Unlike basic CRT which fails if the moduli are not coprime,
7 * this robust version iteratively combines the equations using the Extended
8 * Euclidean algorithm. It safely catches impossible systems.
9 *
10 * 2. HOW TO USE IT (THE BLACK BOX)
11 * - Query:
12 *      pair<ll, ll> ans = crt(A, M);
13 * - Result: Returns {remainder, modulo}. If no solution exists, returns {-1,
        -1}.
14 */
15
16 using ll = long long;
17
18 ll extgcd_crt(ll a, ll b, ll &x, ll &y)
19 {
20     if (b == 0)
21     {
22         x = 1;
23         y = 0;
24         return a;
25     }
26     ll x1, y1;
27     ll d = extgcd_crt(b, a % b, x1, y1);
28     x = y1;
29     y = x1 - y1 * (a / b);
30     return d;
31 }
32
33 pair<ll, ll> crt(const vector<ll> &a, const vector<ll> &m)
34 {
35     if (a.empty())
36         return {0, 1};
37     ll rem = a[0], mod = m[0];
38     for (size_t i = 1; i < a.size(); i++)

```

```

39 {
40     ll x, y;
41     ll g = extgcd_crt(mod, m[i], x, y);
42     if ((a[i] - rem) % g != 0)
43         return {-1, -1};
44     ll target = (a[i] - rem) / g;
45     ll m_i_g = m[i] / g;
46     x = (x % m_i_g + m_i_g) % m_i_g;
47     x = (x * (target % m_i_g + m_i_g)) % m_i_g;
48     rem += mod * x;
49     mod *= m_i_g;
50     rem = (rem % mod + mod) % mod;
51 }
52 return {rem, mod};
53 }

```

5.4.4 Discrete root

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DISCRETE ROOT (K-TH ROOT)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "K-th Root Modulo P", " $x^K = A \pmod P$ ".
6  * - Classic Scenarios: You need to reverse an exponentiation where the modulo is
   prime.
7  * - The Magic: It finds a primitive root g of P. Since x can be represented as  $g^y$ ,
   the equation becomes  $(g^y)^K \equiv A \pmod P \Rightarrow (g^K)^y \equiv A \pmod P$ . This
   perfectly
9  * reduces the problem into a Discrete Logarithm (BSGS) problem!
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *     long long root = discrete_root(K, A, P);
14 *     // Returns -1 if no root exists.
15 *
16 * - Complexity:
17 *     Time:  $O(\sqrt{P} \cdot \log(\sqrt{P}))$ .
18 *     Space:  $O(\sqrt{P})$ .
19 *
20 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21 * - Dependencies: This snippet requires both `custom_pow` and `solve_bsgs` to be
22 * available in your code (from the snippets above).
23 */
24
25 using ll = long long;
26
27 ll primitive_root(ll p)
28 {
29     vector<ll> fact;
30     ll phi = p - 1, n = phi;
31     for (ll i = 2; i * i <= n; ++i)
32     {

```

```

33         if (n % i == 0)
34         {
35             fact.push_back(i);
36             while (n % i == 0)
37                 n /= i;
38         }
39     }
40     if (n > 1)
41         fact.push_back(n);
42
43     for (ll res = 2; res <= p; ++res)
44     {
45         bool ok = true;
46         for (size_t i = 0; i < fact.size() && ok; ++i)
47         {
48             ok &= custom_pow(res, phi / fact[i], p) != 1;
49         }
50         if (ok)
51             return res;
52     }
53     return -1;
54 }
55
56 ll discrete_root(ll k, ll a, ll p)
57 {
58     if (a == 0)
59         return 0;
60     if (k == 0)
61         return (a == 1) ? 1 : -1;
62
63     ll g = primitive_root(p);
64     if (g == -1)
65         return -1;
66
67     ll base = custom_pow(g, k, p);
68     ll y = solve_bsgs(base, a, p);
69     if (y == -1)
70         return -1;
71
72     return custom_pow(g, y, p);
73 }

```

5.4.5 Distinct kth power

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DISTINCT K-TH POWERS &  $x^2 = 1 \pmod M$ 
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "How many perfect K-th powers modulo P", "Square roots of 1 modulo
   M".
6  * - The Magic: These are closed-form formula generators. Calculating the number
7  * of valid values for  $x^2 = 1 \pmod M$  depends heavily on the prime factorization
8  * of M (specifically treating the powers of 2 differently than odd primes).

```

```

9  *
10 * 2. HOW TO USE IT (THE BLACK BOX)
11 * - Query:
12 *      ll ans1 = distinct_kth_powers_prime_mod(K, P); // Requires P to be a
           prime
13 *      ll ans2 = solutions_x2_1_mod_m(M);
14 */
15
16 using ll = long long;
17
18 ll distinct_kth_powers_prime_mod(ll k, ll p)
19 {
20     return (p - 1) / __gcd(k, p - 1) + 1;
21 }
22
23 ll solutions_x2_1_mod_m(ll m)
24 {
25     if (m <= 1)
26         return 1;
27     ll count = 1;
28     ll temp = m;
29     int e2 = 0;
30     while (temp % 2 == 0)
31     {
32         e2++;
33         temp /= 2;
34     }
35     if (e2 == 2)
36         count *= 2;
37     else if (e2 >= 3)
38         count *= 4;
39
40     for (ll i = 3; i * i <= temp; i += 2)
41     {
42         if (temp % i == 0)
43         {
44             count *= 2;
45             while (temp % i == 0)
46                 temp /= i;
47         }
48     }
49     if (temp > 1)
50         count *= 2;
51     return count;
52 }

```

5.4.6 Linear congruence

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: LINEAR CONGRUENCE EQUATION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Solve  $ax = b \pmod m$ ".

```

```

6  * - Classic Scenarios: You are given  $a$ ,  $b$ , and  $m$ , and need to find  $x$ .
7  * - The Magic: It uses the Extended Euclidean Algorithm to solve  $ax + my = b$ .
8  * It handles cases where  $\gcd(a, m) \neq 1$ . If a solution exists, it returns
9  * the base solution  $x_0$  and the number of distinct solutions modulo  $m$ .
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *      ll x, num_solutions;
14 *      bool possible = linear_congruence(a, b, m, x, num_solutions);
15 */
16
17 using ll = long long;
18
19 ll extgcd(ll a, ll b, ll &x, ll &y)
20 {
21     if (b == 0)
22     {
23         x = 1;
24         y = 0;
25         return a;
26     }
27     ll x1, y1;
28     ll d = extgcd(b, a % b, x1, y1);
29     x = y1;
30     y = x1 - y1 * (a / b);
31     return d;
32 }
33
34 bool linear_congruence(ll a, ll b, ll m, ll &x, ll &num_solutions)
35 {
36     ll x0, y0;
37     ll g = extgcd(abs(a), m, x0, y0);
38     if (b % g != 0)
39     {
40         x = 0;
41         num_solutions = 0;
42         return false;
43     }
44     x0 = (x0 % (m / g) + (m / g)) % (m / g);
45     x = (x0 * (b / g)) % (m / g);
46     num_solutions = g;
47     return true;
48 }

```

5.4.7 MULTIPLICATIVE ORDER CARMICHAEL'S LAMBDA

5.4.8 discrete sqrt

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: TONELLI-SHANKS ALGORITHM
3  *

```

```

4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Quadratic Residue", "Modular Square Root", " $x^2 = N \bmod P$ ".
6  * - Classic Scenarios: You are given an integer  $N$  and an odd prime  $P$ , and you
7  *   need to find  $x$  such that  $x^2 \equiv N \pmod{P}$ .
8  * - The Magic: It factors  $P-1$  as  $Q \cdot 2^S$ . By finding a quadratic non-residue  $Z$ ,
9  *   it iteratively manipulates the exponents to isolate the square root. It is
10 *   a beautiful but mathematically dense algorithm that you do not want to debug
    live.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *   long long root = tonelli_shanks(N, P);
15 *   // Returns -1 if no square root exists.
16 *   // If 'root' is a solution, 'P - root' is the other valid solution.
17 *
18 * - Complexity:
19 *   Time:  $O(\log^2 P)$  worst case,  $O(\log P)$  on average. Extremely fast.
20 *   Space:  $O(1)$ 
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Prime Modulo Only: This specifically requires  $P$  to be an odd prime. If you
24 *   need to find roots modulo a composite number, you must use Tonelli-Shanks
25 *   on its prime factors and combine the results using the Chinese Remainder
    Theorem (CRT).
26 */
27
28 using ll = long long;
29
30 ll custom_pow(ll base, ll exp, ll mod)
31 {
32     ll res = 1;
33     base %= mod;
34     while (exp > 0)
35     {
36         if (exp % 2 == 1)
37             res = (res * base) % mod;
38         base = (base * base) % mod;
39         exp /= 2;
40     }
41     return res;
42 }
43
44 ll tonelli_shanks(ll n, ll p)
45 {
46     if (n == 0)
47         return 0;
48     if (custom_pow(n, (p - 1) / 2, p) != 1)
49         return -1;
50
51     ll q = p - 1, s = 0;
52     while (q % 2 == 0)
53     {
54         s++;
55         q /= 2;
56     }
57
58     if (s == 1)
59         return custom_pow(n, (p + 1) / 4, p);
60
61     ll z = 2;
62     while (custom_pow(z, (p - 1) / 2, p) != p - 1)
63     {
64         z++;
65     }
66
67     ll c = custom_pow(z, q, p);
68     ll r = custom_pow(n, (q + 1) / 2, p);
69     ll t = custom_pow(n, q, p);
70     ll m = s;
71
72     while (t != 1)
73     {
74         ll t2 = (t * t) % p;
75         ll i = 1;
76         for (; i < m; i++)
77         {
78             if (t2 == 1)
79                 break;
80             t2 = (t2 * t2) % p;
81         }
82
83         ll b = c;
84         for (int j = 0; j < m - i - 1; j++)
85         {
86             b = (b * b) % p;
87         }
88
89         r = (r * b) % p;
90         c = (b * b) % p;
91         t = (t * c) % p;
92         m = i;
93     }
94     return r;
95 }

```

5.5 Multiplicative functions

5.5.1 Min 25 sieve

5.5.2 Power tower

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: POWER TOWER (GENERALIZED EULER)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Power Tower", " $A^B{}^C{}^D \bmod M$ ".

```

```

6  * - Classic Scenarios: You are given an array of numbers and asked to compute
   their
7  * power tower (a[0] ^ (a[1] ^ (a[2] ^ ... ))) modulo M.
8  * - The Magic: It applies the Generalized Euler Theorem to safely reduce
   exponents
9  * using the Phi function, effectively capping the recursion depth at O(log M)
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *     long long ans = power_tower(arr, 0, M);
14 *
15 * - Complexity:
16 *     Time: O(log M * sqrt(M))
17 *     Space: O(log M) for the recursion stack.
18 */
19
20 using ll = long long;
21
22 ll phi(ll n)
23 {
24     ll res = n;
25     for (ll i = 2; i * i <= n; i++)
26     {
27         if (n % i == 0)
28         {
29             while (n % i == 0)
30                 n /= i;
31             res -= res / i;
32         }
33     }
34     if (n > 1)
35         res -= res / n;
36     return res;
37 }
38
39 ll custom_pow(ll base, ll exp, ll mod)
40 {
41     ll res = 1;
42     base = (base >= mod) ? (base % mod + mod) : base;
43     while (exp > 0)
44     {
45         if (exp % 2 == 1)
46         {
47             res *= base;
48             res = (res >= mod) ? (res % mod + mod) : res;
49         }
50         base *= base;
51         base = (base >= mod) ? (base % mod + mod) : base;
52         exp /= 2;
53     }
54     return res;
55 }
56
57 ll power_tower(const vector<ll> &arr, int idx, ll mod)
58 {

```

```

59     if (idx == arr.size() || mod == 1)
60         return 1;
61     if (arr[idx] == 1)
62         return 1;
63     ll next_mod = phi(mod);
64     ll exponent = power_tower(arr, idx + 1, next_mod);
65     return custom_pow(arr[idx], exponent, mod);
66 }

```

5.5.3 phi

```

1 // computes the number of coprimes number with n up to n
2
3 const int N = 5000006;
4 ll phi[N];
5
6 void sieve()
7 {
8     for (int i = 0; i < N; i++)
9     {
10         phi[i] = i;
11     }
12     for (int i = 2; i < N; i++)
13     {
14         if (phi[i] == i)
15         {
16             for (int j = i; j < N; j += i)
17             {
18                 phi[j] -= phi[j] / i;
19             }
20         }
21     }
22 }
23
24 ll phi(ll n)
25 {
26     ll ret = n;
27     for (ll i = 2; i * i <= n; i++)
28     {
29         if (n % i == 0)
30         {
31             while (n % i == 0)
32                 n /= i;
33             ret -= ret / i;
34         }
35     }
36     if (n > 1)
37         ret -= ret / n;
38     return ret;
39 }

```

5.6 Primes And Divisors

5.6.1 Basic

```
1 struct NumberTheory {
2     int n;
3     vector<int> p, spf, phi_arr, fac, inv, finv;
4     vector<bool> isPrime;
5     vector<vector<int>> divisor;
6
7     NumberTheory(int max_n = 1000000, bool pre_divs = false) {
8         n = max_n;
9         isPrime.assign(n + 1, true);
10        spf.assign(n + 1, 0);
11        phi_arr.assign(n + 1, 0);
12        isPrime[0] = isPrime[1] = false;
13
14        for (int i = 0; i <= n; i++) phi_arr[i] = i;
15
16        for (int i = 2; i <= n; i++) {
17            if (isPrime[i]) {
18                p.push_back(i);
19                spf[i] = i;
20                phi_arr[i] -= phi_arr[i] / i;
21            }
22            for (int j = 0; j < (int) p.size() && i * p[j] <= n; j++) {
23                isPrime[i * p[j]] = false;
24                spf[i * p[j]] = p[j];
25                if (i % p[j] == 0) {
26                    phi_arr[i * p[j]] = phi_arr[i] * p[j];
27                    break;
28                } else {
29                    phi_arr[i * p[j]] = phi_arr[i] * phi_arr[p[j]];
30                }
31            }
32        }
33
34        fac.assign(n + 1, 1);
35        inv.assign(n + 1, 1);
36        finv.assign(n + 1, 1);
37        for (int i = 1; i <= n; i++) fac[i] = mul(fac[i - 1], i);
38        for (int i = 2; i <= n; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] %
39            MOD) % MOD;
40        for (int i = 2; i <= n; i++) finv[i] = mul(finv[i - 1], inv[i]);
41
42        if (pre_divs) {
43            divisor.resize(n + 1);
44            for (int i = 1; i <= n; i++) {
45                for (int j = i; j <= n; j += i) {
46                    divisor[j].push_back(i);
47                }
48            }
49        }
50    }
```

```
51    int add(int a, int b) { return ((a % MOD) + (b % MOD)) % MOD; }
52    int sub(int a, int b) { return (((a - b) % MOD) + MOD) % MOD; }
53    int mul(int a, int b) { return ((a % MOD) * (b % MOD)) % MOD; }
54
55    int power(int b, int pow) {
56        b %= MOD;
57        int res = 1;
58        while (pow) {
59            if (pow & 1) res = mul(res, b);
60            b = mul(b, b);
61            pow >>= 1;
62        }
63        return res;
64    }
65
66    int modInv(int x) { return power(x, MOD - 2); }
67    int divide(int a, int b) { return mul(a, modInv(b)); }
68
69    int nCr(int x, int y) {
70        if (x < 0 || y > x || y < 0) return 0;
71        return mul(fac[x], mul(finv[y], finv[x - y]));
72    }
73
74    int nPr(int x, int y) {
75        if (x < 0 || y > x || y < 0) return 0;
76        return mul(fac[x], finv[x - y]);
77    }
78
79    int catalan(int x) { return mul(nCr(2 * x, x), modInv(x + 1)); }
80    int StarsAndPars(int x, int k) { return nCr(x + k - 1, k - 1); }
81
82    int nCr_no_mod(int x, int r) {
83        if (r < 0 || x < 0 || r > x) return 0;
84        int ret = 1;
85        for (int i = 1; i <= min(r, x - r); i++) {
86            ret *= x - i + 1;
87            ret /= i;
88        }
89        return ret;
90    }
91
92    int nPr_no_mod(int x, int r) {
93        if (r < 0 || x < 0 || r > x) return 0;
94        int ret = 1;
95        for (int i = 0; i < r; i++) ret *= (x - i);
96        return ret;
97    }
98
99    int numberOfDivisors1ToN(int limit) {
100        int sum = 0;
101        for (int i = 1; i <= limit; i++) sum += limit / i;
102        return sum;
103    }
104
105    bool is_prime(int x) {
```

```

106     if (x <= 1) return false;
107     if (x % 2 == 0) return x == 2;
108     for (int i = 3; i * i <= x; i += 2) {
109         if (x % i == 0) return false;
110     }
111     return true;
112 }
113
114 vector<int> factorization(int x) {
115     vector<int> primes_factors;
116     for (int i = 2; i * i <= x; i++) {
117         while (x % i == 0) {
118             primes_factors.push_back(i);
119             x /= i;
120         }
121     }
122     if (x > 1) primes_factors.push_back(x);
123     return primes_factors;
124 }
125
126 vector<int> spf_factorization(int x) {
127     vector<int> primes_factors;
128     while (x != 1) {
129         primes_factors.push_back(spf[x]);
130         x /= spf[x];
131     }
132     return primes_factors;
133 }
134
135 map<int, int> spf_factorization_freq(int x) {
136     map<int, int> frq;
137     while (x != 1) {
138         frq[spf[x]]++;
139         x /= spf[x];
140     }
141     return frq;
142 }
143
144 int phi(int x) {
145     int ret = x;
146     for (int i = 2; i * i <= x; i++) {
147         if (x % i == 0) {
148             while (x % i == 0) x /= i;
149             ret -= ret / i;
150         }
151     }
152     if (x > 1) ret -= ret / x;
153     return ret;
154 }
155 };

```

5.6.2 FastSumitionDivisor 1e16

```

1 * THE ULTIMATE CP TEMPLATE BLUEPRINT: FAST SUM OF PROPER DIVISORS
2 *
3 * 1. WHAT PROBLEM DOES THIS SOLVE?
4 * - Keywords: "Sum_of_divisors", "Proper_divisors", "Aliquot_sum", "N_up_to_
   10^16".
5 * - Classic Scenarios: You are given a massive number N (up to 10^16) and you
   need to find
6 * the sum of all its divisors EXCEPT the number itself.
7
8 vector<int> Primes;
9
10 void Sieve(int num)
11 {
12     vector<bool> is_prime(num + 1, true);
13     is_prime[0], is_prime[1] = false;
14     for (int i = 2; i * i <= num; i++)
15     {
16         if (is_prime[i])
17         {
18             for (int j = i * i; j <= num; j += i)
19                 is_prime[j] = false;
20         }
21     }
22     Primes.push_back(2);
23     for (int i = 3; i <= num; i += 2)
24     {
25         if (is_prime[i])
26             Primes.push_back(i);
27     }
28 }
29
30 void Hassan ()
31 {
32     //one integer between 1 and 1e16 inclusive
33     ll n; cin >> n;
34     ll ans = 1;
35     ll copy = n;
36     for (auto p : Primes)
37     {
38         if (1LL * p * p > n)
39             break;
40         if (n % p == 0)
41         {
42             ll sum = 1, pow = 1;
43             while (n % p == 0)
44             {
45                 pow *= p;
46                 sum += pow;
47                 n /= p;
48             }
49             ans *= sum;
50         }
51     }
52     if (n > 1) ans *= (n + 1);
53 }

```

```

54     cout << ans - copy << '\n'; /// note
55 }

```

5.6.3 Linear sieve

```

1  const int N = 1e5 + 5;
2  vector<bool> isPrime(N, true);
3  vector<int> p;
4
5  void linear_sieve()
6  {
7      isPrime[0] = isPrime[1] = 0;
8      for (ll i = 2; i < N; i++)
9      {
10         if (isPrime[i] == 1)
11             p.push_back(i);
12         for (ll o = 0; o < p.size(); o++)
13         {
14             if (i * p[o] >= N)
15                 break;
16             isPrime[i * p[o]] = 0;
17             if (i % p[o] == 0)
18                 break;
19         }
20     }
21 }

```

5.6.4 Miller pollard

```

1  /**
2  *  THE ULTIMATE CP TEMPLATE BLUEPRINT: MILLER-RABIN & POLLARD'S RHO
3  *
4  *  1.  WHAT PROBLEM DOES THIS SOLVE?
5  *  - Keywords: "Primality Test", "Prime Factorization", "Count Divisors", "
    Massive N".
6  *  - Classic Scenarios: You need to check if N is prime, find all prime factors
    of N,
7  *  or count the total number of divisors where N is up to 1018. Standard O(
    sqrt(N))
8  *  trial division will result in a Time Limit Exceeded (TLE) verdict.
9  *  - The Magic: Miller-Rabin deterministically checks primality for 64-bit
    integers
10 *  in O(log N) time using 7 specific mathematical bases. If it's composite,
    Pollard's Rho
11 *  steps in to find a prime factor in expected O(N(1/4)) time. Together, they
    instantly
12 *  break down 1018 numbers.
13 *
14 *  2.  HOW TO USE IT (THE BLACK BOX)
15 *  - Primality Check:
16 *      bool is_prime = MillerRabin(n);
17 *  - Prime Factorization:

```

```

18 *      map<uint64_t, int> prime_factors;
19 *      factorize(n, prime_factors); // Populates the map with {prime, frequency
    }
20 *  - Count Total Divisors:
21 *      uint64_t total = count_divisors(n);
22 *
23 *  - Complexity:
24 *      Time: O(log N) for Miller-Rabin. Expected O(N(1/4)) for Pollard's Rho.
25 *      Space: O(log N) for recursion depth and the frequency map.
26 *
27 *  3.  HOW TO ADAPT IT (THE DIALS & SWITCHES)
28 *  - Type Aliases: This template strictly uses `u64` (uint64_t) and `u128` (
    __uint128_t)
29 *  to prevent silent overflows during modular multiplication of massive numbers
    .
30 *  Do not change these to standard `long long`.
31 *  - Randomness: `rand()` is used for the Rho polynomial `x^2 + c`. This is
    usually
32 *  sufficient, but if you hit a malicious anti-hash test case, you can swap it
33 *  for a faster, custom `mt19937_64` implementation.
34 */
35
36 using u64 = uint64_t;
37 using u128 = __uint128_t;
38
39 u64 binpower(u64 base, u64 e, u64 mod)
40 {
41     u64 result = 1;
42     base %= mod;
43     while (e)
44     {
45         if (e & 1)
46             result = (u128)result * base % mod;
47         base = (u128)base * base % mod;
48         e >>= 1;
49     }
50     return result;
51 }
52
53 bool check_composite(u64 n, u64 a, u64 d, int s)
54 {
55     u64 x = binpower(a, d, n);
56     if (x == 1 || x == n - 1)
57         return false;
58     for (int r = 1; r < s; r++)
59     {
60         x = (u128)x * x % n;
61         if (x == n - 1)
62             return false;
63     }
64     return true;
65 }
66
67 bool MillerRabin(u64 n)
68 {

```

```

69     if (n < 2)
70         return false;
71     int r = 0;
72     u64 d = n - 1;
73     while ((d & 1) == 0)
74     {
75         d >>= 1;
76         r++;
77     }
78     // These 7 bases are deterministically sufficient for all  $n < 2^{64}$ 
79     for (u64 a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022})
80     {
81         if (n == a)
82             return true;
83         if (check_composite(n, a, d, r))
84             return false;
85     }
86     return true;
87 }
88
89 u64 pollard_rho(u64 n)
90 {
91     if (n % 2 == 0)
92         return 2;
93     if (MillerRabin(n))
94         return n;
95
96     u64 x = 2, y = 2, d = 1, c = 1;
97     auto f = [&](u64 x, u64 n, u64 c)
98     {
99         return (u64)(((u128)x * x + c) % n);
100     };
101
102     while (d == 1)
103     {
104         x = f(x, n, c);
105         y = f(f(y, n, c), n, c);
106         d = __gcd(x > y ? x - y : y - x, n);
107         if (d == n)
108         {
109             x = rand() % (n - 2) + 2;
110             y = x;
111             c = rand() % (n - 1) + 1;
112             d = 1;
113         }
114     }
115     return d;
116 }
117
118 void factorize(u64 n, map<u64, int> &prime_factors)
119 {
120     if (n == 1)
121         return;
122     if (MillerRabin(n))
123     {

```

```

124         prime_factors[n]++;
125         return;
126     }
127     u64 divisor = pollard_rho(n);
128     factorize(divisor, prime_factors);
129     factorize(n / divisor, prime_factors);
130 }
131
132 u64 count_divisors(u64 n)
133 {
134     map<u64, int> prime_factors;
135     factorize(n, prime_factors);
136     u64 total_divisors = 1;
137     for (auto &pf : prime_factors)
138     {
139         total_divisors *= (pf.second + 1);
140     }
141     return total_divisors;
142 }

```

5.6.5 Number of divisors 1e18

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FAST DIVISOR COUNTING (WHEEL
3  * FACTORIZATION)
4  *
5  * 1. WHAT PROBLEM DOES THIS SOLVE?
6  * - Keywords: "Number of divisors of N", "N up to  $10^{18}$ ", "Count divisors
7  * without full factorization".
8  * - Classic Scenarios: You are given a massive number N (up to  $10^{14}$  or  $10^{18}$ )
9  * and you need to
10 * find exactly how many divisors it has, but  $O(\sqrt{N})$  is too slow, and you
11 * can't build a Sieve
12 * because N is too large.
13 * - The Magic: "Wheel Factorization". Instead of checking every number up to
14 *  $\sqrt{N}$ , we manually
15 * factor out the first 8 primes (2, 3, 5, 7, 11, 13, 17, 19). The product of
16 * these primes is ~9.6 million.
17 * For the remaining part of N, we ONLY check divisors that are coprime to
18 * these 8 primes.
19 * This eliminates about ~83% of the useless checks in the  $O(\sqrt{N})$  loop!
20 *
21 * 2. HOW TO USE IT (THE BLACK BOX)
22 * - Initialization: Create an instance globally or in `main`. It will precompute
23 * the "wheel spokes" ONCE.
24 * FastDivisorCounter fdc;
25 *
26 * - Queries: Pass your number N (up to  $10^{18}$ ) to get the number of divisors.
27 * ll total_divisors = fdc.count(N);
28 *
29 * - Complexity:
30 * Time (Initialization): Precomputing the wheel takes  $O(P)$  where P is ~9.6
31 * *  $10^6$  (Done once in ~15ms).

```

```

23 *      Time (Per Query): roughly  $O(\sqrt{N} / 9.6 \cdot 10^6)$ . Lightning fast for huge
      queries!
24 *      Space: ~6 MB for the precomputed wheel.
25 *
26 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
27 * - If Memory Limit is EXTREMELY tight (<10 MB), you can remove the `spokes`
      vector and calculate the
28 * coprime numbers inside the `count` function loop like your original code,
      but only if  $Q=1$ .
29 * For  $Q > 1$ , the precomputed `spokes` vector is vastly superior.
30 */
31
32 #include <bits/stdc++.h>
33 using namespace std;
34 #define ll long long
35 struct FastDivisorCounter
36 {
37 private:
38     vector<int> primes;
39     ll wheel_size;
40     vector<ll> spokes; // Stores numbers from 1 to wheel_size that are coprime to
      the base primes
41 public:
42     FastDivisorCounter()
43     {
44         primes = {2, 3, 5, 7, 11, 13, 17, 19};
45         wheel_size = 1;
46
47         for (int p : primes)
48         {
49             wheel_size *= p;
50         }
51         // ? PRECOMPUTATION MAGIC: We find all valid "spokes" ONCE.
52         // There are exactly 1,658,880 such numbers. By saving them, we skip
      millions
53         // of useless iterations in future queries.
54         for (ll i = 1; i < wheel_size; i++)
55         {
56             bool is_coprime = true;
57             for (int p : primes)
58             {
59                 if (i % p == 0)
60                 {
61                     is_coprime = false;
62                     break;
63                 }
64             }
65             if (is_coprime)
66             {
67                 spokes.push_back(i);
68             }
69         }
70     }
71     ll count(ll n)
72     {

```

```

11 ans = 1;
// Step 1: Remove all factors of the base primes and count their
      contribution
for (int p : primes)
{
    int c = 0;
    while (n % p == 0)
    {
        n /= p;
        c++;
    }
    // Standard divisor formula: multiply by (power + 1)
    ans *= (c + 1);
}
// Step 2: The remaining part of 'n' is now completely coprime to our
      base primes.
// We only check potential divisors using our precomputed wheel spokes.
ll all = 0;
for (ll spoke : spokes)
{
    ll o = spoke;
    // ? THE SPEED JUMP: We jump by `wheel_size` (9,699,690) instead of
      +1 or +2!
    for (; o * o < n; o += wheel_size)
    {
        if (n % o == 0)
        {
            all += 2; // Found pair of divisors: `o` and `n/o`
        }
    }
    // Check perfect square
    if (o * o == n)
    {
        all++;
    }
}
// The total divisors of the original N is the product of the divisors
// of the base-prime part and the divisors of the remaining coprime part.
return ans * all;
}
111 };
112 void solve()
113 {
114     int t;
115     if (!(cin >> t))
116         return;
117     // Initialize OUTSIDE the query loop so the wheel is built only once!
118     FastDivisorCounter fdc;
119     while (t--)
120     {
121         ll n;
122         cin >> n;
123         cout << fdc.count(n) << "\n";
124     }

```

5.6.6 Prime counting

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PRIME COUNTING FUNCTION (LUCY-HEDGEHOG)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Count primes up to N", "Pi(x)", "Lucy-Hedgehog", "Min-25 Sieve".
6  * - Classic Scenarios: You are given N up to 1011 and asked exactly how many
7  * prime numbers are less than or equal to N.
8  * - The Magic: A standard sieve up to 1011 will instantly crash with Memory
   Limit
9  * Exceeded and Time Limit Exceeded. The Lucy-Hedgehog dynamic programming
   approach
10 * computes this mathematically without iterating over or storing the actual
   primes!
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     uint64_t total_primes = prime_count(N);
15 *
16 * - Complexity:
17 *     Time: O(N(3/4)) Runs almost instantly for N = 1010, and easily
18 *     passes 1-second limits for N = 1011.
19 *     Space: O(sqrt(N)) Extremely memory-efficient.
20 *
21 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22 * - Array Sizes: It only creates vectors of size sqrt(N). For N = 1011, this is
23 * just an array of size 3.16 * 105, taking mere megabytes of RAM.
24 */
25
26 using u64 = uint64_t;
27
28 u64 prime_count(u64 N)
29 {
30     if (N <= 1)
31         return 0;
32     u64 v = sqrt(N);
33     vector<u64> S0(v + 1), S1(v + 1);
34     for (u64 i = 1; i <= v; i++)
35     {
36         S0[i] = i - 1;
37         S1[i] = N / i - 1;
38     }
39     for (u64 p = 2; p <= v; p++)
40     {
41         if (S0[p] == S0[p - 1])
42             continue;
43         u64 p_cnt = S0[p - 1];
44         u64 q = p * p;
45         u64 end = min(v, N / q);
46         for (u64 i = 1; i <= end; i++)

```

```

47     {
48         u64 d = i * p;
49         if (d <= v)
50             S1[i] -= S1[d] - p_cnt;
51         else
52             S1[i] -= S0[N / d] - p_cnt;
53     }
54     for (u64 i = v; i >= q; i--)
55     {
56         S0[i] -= S0[i / p] - p_cnt;
57     }
58 }
59 return S1[1];
60 }

```

5.6.7 Sieve 1e9

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // credit: min_25
5 // takes 0.5s for n = 1e9
6 vector<int> sieve(const int N, const int Q = 17, const int L = 1 << 15)
7 {
8     static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
9     struct P
10     {
11         P(int p) : p(p)
12         {
13         }
14
15         int p;
16         int pos[8];
17     };
18     auto approx_prime_count = [](const int N) -> int
19     {
20         return N > 60184
21             ? N / (log(N) - 1.1)
22             : max(1., N / (log(N) - 1.11)) + 1;
23     };
24
25     const int v = sqrt(N), vv = sqrt(v);
26     vector<bool> isp(v + 1, true);
27     for (int i = 2; i <= vv; ++i)
28         if (isp[i])
29         {
30             for (int j = i * i; j <= v; j += i)
31                 isp[j] = false;
32         }
33
34     const int rsize = approx_prime_count(N + 30);
35     vector<int> primes = {2, 3, 5};
36     int psize = 3;

```

```

37 primes.resize(rsize);
38
39 vector<P> sprimes;
40 size_t pbeg = 0;
41 int prod = 1;
42 for (int p = 7; p <= v; ++p)
43 {
44     if (!isp[p])
45         continue;
46     if (p <= Q)
47         prod *= p, ++pbeg, primes[psize++] = p;
48     auto pp = P(p);
49     for (int t = 0; t < 8; ++t)
50     {
51         int j = (p <= Q) ? p : p * p;
52         while (j % 30 != rs[t])
53             j += p << 1;
54         pp.pos[t] = j / 30;
55     }
56     sprimes.push_back(pp);
57 }
58
59 vector<unsigned char> pre(prod, 0xFF);
60 for (size_t pi = 0; pi < pbeg; ++pi)
61 {
62     auto pp = sprimes[pi];
63     const int p = pp.p;
64     for (int t = 0; t < 8; ++t)
65     {
66         const unsigned char m = ~(1 << t);
67         for (int i = pp.pos[t]; i < prod; i += p)
68             pre[i] &= m;
69     }
70 }
71
72 const int block_size = (L + prod - 1) / prod * prod;
73 vector<unsigned char> block(block_size);
74 unsigned char *pblock = block.data();
75 const int M = (N + 29) / 30;
76
77 for (int beg = 0; beg < M; beg += block_size, pblock -= block_size)
78 {
79     int end = min(M, beg + block_size);
80     for (int i = beg; i < end; i += prod)
81     {
82         copy(pre.begin(), pre.end(), pblock + i);
83     }
84     if (beg == 0)
85         pblock[0] &= 0xFE;
86     for (size_t pi = pbeg; pi < sprimes.size(); ++pi)
87     {
88         auto &pp = sprimes[pi];
89         const int p = pp.p;
90         for (int t = 0; t < 8; ++t)
91         {

```

```

92             int i = pp.pos[t];
93             const unsigned char m = ~(1 << t);
94             for (; i < end; i += p)
95                 pblock[i] &= m;
96             pp.pos[t] = i;
97         }
98     }
99     for (int i = beg; i < end; ++i)
100     {
101         for (int m = pblock[i]; m > 0; m &= m - 1)
102         {
103             primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
104         }
105     }
106 }
107 assert(psize <= rsize);
108 while (psize > 0 && primes[psize - 1] > N)
109     --psize;
110 primes.resize(psize);
111 return primes;
112 }
113
114 int32_t main()
115 {
116     ios_base::sync_with_stdio(0);
117     cin.tie(0);
118     int n, a, b;
119     cin >> n >> a >> b;
120     auto primes = sieve(n);
121     vector<int> ans;
122     for (int i = b; i < primes.size() && primes[i] <= n; i += a)
123         ans.push_back(primes[i]);
124     cout << primes.size() << ' ' << ans.size() << '\n';
125     for (auto x : ans)
126         cout << x << ' ';
127     cout << '\n';
128     return 0;
129 }

```

5.6.8 segmented sieve

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SEGMENTED SIEVE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Primes in Range [L, R]", "Massive Bounds".
6  * - Classic Scenarios: Find all primes or the number of primes between L and R.
7  * L and R can be as massive as 1012, but the difference (R - L) is small (<= 106).
8  * - The Magic: A regular sieve cannot allocate an array up to 1012. The segmented
9  * sieve only sieves up to sqrt(R), caches those base primes, and then "shifts"
10 * the sieving logic directly to the target window [L, R], ignoring the void before L.

```

```

11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     vector<uint64_t> primes = segmented_sieve(L, R);
15 *
16 * - Complexity:
17 *     Time:  $O((R - L + \sqrt{R}) * \log(\log(R)))$ 
18 *     Space:  $O(\sqrt{R} + (R - L))$ 
19 *
20 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21 * - Edge Cases Guarded: Safely handles constraints where  $L = 0$  or  $L = 1$ 
22 *   (which are not prime but often trip up basic segmented implementations).
23 */
24
25 using u64 = uint64_t;
26
27 vector<u64> segmented_sieve(u64 L, u64 R)
28 {
29     if (L > R)
30         return {};
31     u64 lim = sqrt(R);
32     vector<bool> mark(lim + 1, false);
33     vector<u64> primes;
34     for (u64 i = 2; i <= lim; ++i)
35     {
36         if (!mark[i])
37         {
38             primes.push_back(i);
39             for (u64 j = i * i; j <= lim; j += i)
40                 mark[j] = true;
41         }
42     }
43
44     vector<bool> isPrime(R - L + 1, true);
45     for (u64 p : primes)
46     {
47         u64 start = max(p * p, (L + p - 1) / p * p);
48         for (u64 j = start; j <= R; j += p)
49             isPrime[j - L] = false;
50     }
51
52     if (L == 0)
53     {
54         isPrime[0] = false;
55         if (R >= 1)
56             isPrime[1] = false;
57     }
58     else if (L == 1)
59     {
60         isPrime[0] = false;
61     }
62
63     vector<u64> range_primes;
64     for (u64 i = 0; i <= R - L; ++i)
65     {

```

```

66         if (isPrime[i])
67             range_primes.push_back(L + i);
68     }
69     return range_primes;
70 }

```

5.6.9 sum numberOfDivisors

```

1 /**
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: CUBIC ROOT DIVISOR SUM
3 *
4 * 1. WHAT PROBLEM DOES THIS SOLVE?
5 * - Keywords: "Sum of Divisors", "Sigma0(i)", "Sum of floor(N/i)".
6 * - Classic Scenarios: You are asked to compute  $\sum_{i=1}^N d(i)$ , which is
7 *   mathematically equivalent to  $\sum_{i=1}^N \text{floor}(N / i)$ , for a staggering  $N$ 
8 *   =  $10^{18}$ .
9 * - The Magic: The naive  $O(N)$  loop crashes. The standard Dirichlet hyperbola
10 *   optimization takes  $O(\sqrt{N})$ , which runs in ~1 second for  $10^{12}$  but fails
11 *   at  $10^{18}$ .
12 *   This extremely advanced algorithm uses Stern-Brocot tree fraction
13 *   approximation
14 *   to step through the hyperbola in blocks, reducing the time to roughly  $O(N^{1/3})$ !
15 *
16 * 2. HOW TO USE IT (THE BLACK BOX)
17 * - Query:
18 *     __uint128_t ans = sum_sigma0(N);
19 *     // Remember to write a custom print function for __uint128_t if
20 *     outputting directly.
21 *
22 * - Complexity:
23 *     Time:  $\sim O(N^{1/3})$ . Easily executes  $N = 10^{18}$  in < 100 milliseconds.
24 *     Space:  $O(\log N)$  for the stack depth.
25 *
26 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
27 * - 128-bit Types: Be very careful if modifying the inner loop logic. The
28 *   condition
29 *    $\text{`uint128}(x) * x * dy \geq \text{uint128}(n) * dx\text{'}$  uses 128-bit integers
30 *   because cubing boundaries near  $10^{18}$  guarantees overflow in standard
31 *    $\text{uint64\_t}$ .
32 */
33
34 using uint32 = unsigned int;
35 using uint64 = unsigned long long;
36 using uint128 = __uint128_t;
37
38 uint128 sum_sigma0(uint64 n)
39 {
40     auto out = [n](uint64 x, uint32 y)
41     { return x * y > n; };
42     auto cut = [n](uint64 x, uint32 dx, uint32 dy)
43     { return uint128(x) * x * dy >= uint128(n) * dx; };
44     const uint64 sn = sqrtl(n);

```

```

39 const uint64 cn = pow(n, 0.34);
40 uint64 x = n / sn;
41 uint32 y = n / x + 1;
42 uint128 ret = 0;
43 stack<pair<uint32, uint32>> st;
44 st.emplace(1, 0);
45 st.emplace(1, 1);
46 while (true)
47 {
48     uint32 lx, ly;
49     tie(lx, ly) = st.top();
50     st.pop();
51     while (out(x + lx, y - ly))
52     {
53         ret += x * ly + uint64(ly + 1) * (lx - 1) / 2;
54         x += lx, y -= ly;
55     }
56     if (y <= cn)
57         break;
58     uint32 rx = lx, ry = ly;
59     while (true)
60     {
61         tie(lx, ly) = st.top();
62         if (out(x + lx, y - ly))
63             break;
64         rx = lx, ry = ly;
65         st.pop();
66     }
67     while (true)
68     {
69         uint32 mx = lx + rx, my = ly + ry;
70         if (out(x + mx, y - my))
71         {
72             st.emplace(lx = mx, ly = my);
73         }
74         else
75         {
76             if (cut(x + mx, lx, ly))
77                 break;
78             rx = mx, ry = my;
79         }
80     }
81 }
82 for (--y; y > 0; --y)
83     ret += n / y;
84 return ret * 2 - sn * sn;
85 }

```

6 Combinatorics

6.1 Main

6.1.1 Basic

```

1 // =====
2 // MODULE 1: MODULAR ARITHMETIC & CORE COMBINATORICS
3 // =====
4 struct Combinatorics
5 {
6     int N;
7     vector<ll> fact, invFact;
8
9     // Fast Modular Exponentiation: Computes (base^exp) % MOD in O(log exp)
10    ll power(ll base, ll exp)
11    {
12        ll res = 1;
13        base %= MOD;
14        while (exp > 0)
15        {
16            if (exp % 2 == 1)
17                res = (res * base) % MOD;
18            base = (base * base) % MOD;
19            exp /= 2;
20        }
21        return res;
22    }
23
24    // Modular Multiplicative Inverse: Computes (1 / n) % MOD in O(log MOD)
25    // Uses Fermat's Little Theorem (Requires MOD to be prime)
26    ll modInverse(ll n)
27    {
28        return power(n, MOD - 2);
29    }
30
31    // Initialize Factorials and Inverse Factorials in O(N)
32    // CALL THIS ONCE IN MAIN BEFORE SOLVING TEST CASES!
33    void init(int n)
34    {
35        N = n;
36        fact.resize(N + 1);
37        invFact.resize(N + 1);
38
39        fact[0] = 1;
40        invFact[0] = 1;
41        for (int i = 1; i <= N; i++)
42        {
43            fact[i] = (fact[i - 1] * i) % MOD;
44        }
45
46        // Compute the inverse of the largest factorial, then work backwards
47        invFact[N] = modInverse(fact[N]);
48        for (int i = N - 1; i >= 1; i--)

```

```

49     {
50         invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
51     }
52 }
53
54 // Choose: Number of ways to pick r items from n items. O(1)
55 ll nCr(int n, int r)
56 {
57     if (r < 0 || r > n)
58         return 0;
59     return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
60 }
61
62 // Permutation: Number of ways to arrange r items out of n items. O(1)
63 ll nPr(int n, int r)
64 {
65     if (r < 0 || r > n)
66         return 0;
67     return fact[n] * invFact[n - r] % MOD;
68 }
69
70 // Stars and Bars: Ways to distribute n identical items into k distinct bins.
71 // O(1)
72 // Example: Solutions to  $x_1 + x_2 + \dots + x_k = n$  (where  $x_i \geq 0$ )
73 ll starsAndBars(int n, int k)
74 {
75     if (n == 0 && k == 0)
76         return 1;
77     return nCr(n + k - 1, k - 1);
78 }
79
80 // Stars and Bars (Positive): Distribute n identical items into k distinct
81 // bins,
82 // such that every bin gets AT LEAST ONE item. O(1)
83 // Example: Solutions to  $x_1 + x_2 + \dots + x_k = n$  (where  $x_i \geq 1$ )
84 ll starsAndBarsPositive(int n, int k)
85 {
86     if (n < k)
87         return 0;
88     return nCr(n - 1, k - 1);
89 }
90
91 // Catalan Numbers: O(1)
92 // Solves: Valid parenthesis expressions, binary trees, polygon
93 // triangulations.
94 // Formula:  $C(n) = (1 / (n + 1)) * (2n C n)$ 
95 ll catalan(int n)
96 {
97     if (n < 0)
98         return 0;
99     ll res = nCr(2 * n, n);
100     res = (res * modInverse(n + 1)) % MOD;
101     return res;
102 }

```

```

101
102 // =====
103 // MODULE 2: ADVANCED COUNTING (DERANGEMENTS, STIRLING, BELL)
104 // =====
105 struct AdvancedCombinatorics
106 {
107     // Derangements: Number of permutations of N elements where NO element is in
108     // its original position.
109     // O(N) precomputation. Formula:  $D(n) = (n-1) * (D(n-1) + D(n-2))$ 
110     vector<ll> derangement;
111
112     void initDerangements(int n)
113     {
114         derangement.assign(n + 1, 0);
115         if (n >= 0)
116             derangement[0] = 1;
117         if (n >= 1)
118             derangement[1] = 0;
119         for (int i = 2; i <= n; i++)
120         {
121             derangement[i] = ((i - 1) * (derangement[i - 1] + derangement[i - 2])
122                             % MOD) % MOD;
123         }
124     }
125
126     // Stirling Numbers of the First Kind:  $S1[n][k]$ 
127     // Solves: Number of permutations of 'n' elements that contain exactly 'k'
128     // disjoint cycles.
129     // O(N^2) precomputation.
130     vector<vector<ll>> S1;
131
132     void initStirling1(int n)
133     {
134         S1.assign(n + 1, vector<ll>(n + 1, 0));
135         S1[0][0] = 1;
136         for (int i = 1; i <= n; i++)
137         {
138             for (int j = 1; j <= i; j++)
139             {
140                 S1[i][j] = (S1[i - 1][j - 1] + (i - 1) * S1[i - 1][j]) % MOD;
141             }
142         }
143     }
144
145     // Stirling Numbers of the Second Kind:  $S2[n][k]$ 
146     // Solves: Number of ways to partition a set of 'n' DISTINCT elements into
147     // exactly 'k' non-empty indistinguishable subsets.
148     // O(N^2) precomputation.
149     vector<vector<ll>> S2;
150
151     void initStirling2(int n)
152     {
153         S2.assign(n + 1, vector<ll>(n + 1, 0));
154         S2[0][0] = 1;
155         for (int i = 1; i <= n; i++)

```

```

152     {
153         for (int j = 1; j <= i; j++)
154         {
155             S2[i][j] = (S2[i - 1][j - 1] + j * S2[i - 1][j]) % MOD;
156         }
157     }
158 }
159
160 // Bell Numbers: Bell[n]
161 // Solves: Total number of ways to partition a set of 'n' elements into ANY
// number of non-empty subsets.
162 // O(N^2) precomputation (Sum of Stirling2(n, k) for k = 0 to n).
163 vector<ll> bell;
164
165 void initBell(int n)
166 {
167     if (S2.empty())
168         initStirling2(n);
169     bell.assign(n + 1, 0);
170     for (int i = 0; i <= n; i++)
171     {
172         for (int j = 0; j <= i; j++)
173         {
174             bell[i] = (bell[i] + S2[i][j]) % MOD;
175         }
176     }
177 }
178 };
179
180 // =====
181 // MODULE 3: LUCAS THEOREM (LARGE N, SMALL PRIME MODULO)
182 // =====
183 // Solves: nCr % P when N and R are massive (up to 10^18) but P is small (up to
10^5) and P is PRIME.
184 // Note: This requires a separate Combinatorics instance initialized up to P.
185 ll lucas_nCr(ll n, ll r, int p, Combinatorics &combo)
186 {
187     if (r < 0 || r > n)
188         return 0;
189     if (r == 0)
190         return 1;
191
192     // Split n and r into base P representation
193     ll ni = n % p;
194     ll ri = r % p;
195
196     if (ni < ri)
197         return 0; // Optimization: P divides nCr
198
199     // nCr = Product of (n_i C r_i) mod P
200     return (combo.nCr(ni, ri) * lucas_nCr(n / p, r / p, p, combo)) % p;
201 }

```

6.1.2 Pascal triangle

```

1 ll c[n][n]={};
2 c[0][0]=1;
3 for(int i=1;i<n;i++)
4 {
5     c[i][0]=c[i][i]=1;
6     for(int o=1;o<i;o++)
7     {
8         c[i][o]=c[i-1][o-1]+c[i-1][o];
9     }
10 }

```

6.2 PIE

6.2.1 multiples

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: INCLUSION-EXCLUSION (PIE) ON MULTIPLES
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Divisible by any", "Coprime to array", "Multiples below X".
6  * - Classic Scenarios: You are given a target number X, and an array of K
// integers.
7  * You need to find how many numbers from 1 to X are divisible by AT LEAST ONE
8  * of the integers in the array.
9  * - The Magic: It systematically adds numbers divisible by 1 element, subtracts
10 * those divisible by 2 elements, adds for 3, etc. A naive bitmask loop goes
// out
11 * of bounds (LCM overflow) quickly. This DFS safely prunes branches if the LCM
12 * ever exceeds X, making it insanely fast and overflow-proof.
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Query:
16 *     long long ans = count_divisible(X, A);
17 *     // To find numbers COPRIME to the array: ans = X - count_divisible(X, A)
// ;
18 *
19 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
20 * - Array Cleaning: For maximum speed, it is highly recommended to sort the
// array `A`,
21 * remove duplicates, and remove any number that is a multiple of another
// number
22 * in the array before calling this function.
23 */
24
25 using ll = long long;
26
27 ll pie_dfs(int idx, ll current_lcm, int elements_chosen, ll max_val, const vector
<ll> &a)
28 {
29     if (idx == a.size())
30     {

```

```

31     if (elements_chosen == 0)
32         return 0;
33     if (elements_chosen % 2 == 1)
34         return max_val / current_lcm;
35     return -(max_val / current_lcm);
36 }
37
38 ll ans = pie_dfs(idx + 1, current_lcm, elements_chosen, max_val, a);
39
40 ll g = __gcd(current_lcm, a[idx]);
41 if (current_lcm / g <= max_val / a[idx])
42 {
43     ans += pie_dfs(idx + 1, (current_lcm / g) * a[idx], elements_chosen + 1,
44         max_val, a);
45 }
46 return ans;
47 }
48
49 ll count_divisible(ll max_val, const vector<ll> &a)
50 {
51     return pie_dfs(0, 1, 0, max_val, a);
52 }

```

6.3 Partition

6.3.1 Integer partition

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: INTEGER PARTITION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Ways to sum to N", "Unordered integer partitions".
6  * - Classic Scenarios: Find the number of distinct ways to represent the integer
7  *   N
8  *   as a sum of positive integers (e.g., 4 = 4, 3+1, 2+2, 2+1+1, 1+1+1+1).
9  * - The Magic: A standard coin-change DP takes  $O(N^2)$ , which crashes for  $N = 10^5$ .
10 *   Euler's Pentagonal Number Theorem computes the partition function  $P(N)$  in  $O(\sqrt{N})$ 
11 *   by subtracting and adding previous  $P()$  values at specifically spaced
12 *   pentagonal
13 *   intervals:  $P(k) = P(k-1) + P(k-2) - P(k-5) - P(k-7) + P(k-12) + \dots$ 
14 *
15 * 2. HOW TO USE IT (THE BLACK BOX)
16 * - Query:
17 *     vector<long long> P = partition_function(N, MOD);
18 *     // P[i] contains the number of partitions for integer i.
19 *
20 * - Complexity:
21 *     Time:  $O(N * \sqrt{N})$ 
22 *     Space:  $O(N)$ 

```

```

22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Array Precomputation: It returns the entire array up to N. If the problem
24 *   has
25 *   multiple test cases querying different N's, run this function ONCE globally
26 *   for
27 *   the maximum possible N, then answer each query in  $O(1)$ .
28 */
29 using ll = long long;
30 vector<ll> partition_function(int n, ll mod)
31 {
32     vector<ll> p(n + 1, 0);
33     p[0] = 1;
34     for (int i = 1; i <= n; i++)
35     {
36         for (int k = 1;; k++)
37         {
38             int p1 = k * (3 * k - 1) / 2;
39             int p2 = k * (3 * k + 1) / 2;
40             if (p1 > i && p2 > i)
41                 break;
42
43             ll sign = (k % 2 == 1) ? 1 : -1;
44
45             if (p1 <= i)
46                 p[i] = (p[i] + sign * p[i - p1]) % mod;
47             if (p2 <= i)
48                 p[i] = (p[i] + sign * p[i - p2]) % mod;
49         }
50         p[i] = (p[i] + mod) % mod;
51     }
52     return p;
53 }

```

6.4 nCr

6.4.1 nCr Mod any mod

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: nCr MODULO ANY MOD
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "nCr", "Composite Modulo", "No Modular Inverse".
6  * - Classic Scenarios: Compute  $nCr(N, K)$  modulo M where M is NOT a prime number
7  *   (e.g.,  $M = 10^9$ , or some random composite).
8  * - The Magic: Since M is composite, you cannot divide by factorials (no inverse).
9  *   This algorithm factors M into prime powers  $p^k$ . It then counts the exact
10 *   number of times p divides  $N!$ ,  $K!$ , and  $(N-K)!$ , isolates the coprime parts,
11 *   calculates the binomial coefficient modulo  $p^k$ , and finally uses the Chinese
12 *   Remainder Theorem (CRT) to stitch them all back together into a single
13 *   answer.

```

```

13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Query:
16 *     long long ans = nCr_arbitrary_mod(N, K, M);
17 *
18 * - Complexity:
19 *     Time:  $O(\sum(p^k) + \log N)$ . Extremely fast for  $M \leq 10^6$ .
20 *     Space:  $O(\sum(p^k))$  to store factorials modulo  $p^k$ .
21 *
22 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
23 * - Dependencies: This heavily relies on your existing `extgcd` and `crt`
    templates
24 * from your Modular Arithmetic section. Make sure they are included above this
    .
25 */
26
27 using ll = long long;
28
29 ll custom_pow(ll base, ll exp, ll mod)
30 {
31     ll res = 1;
32     base %= mod;
33     while (exp > 0)
34     {
35         if (exp % 2 == 1)
36             res = (res * base) % mod;
37         base = (base * base) % mod;
38         exp /= 2;
39     }
40     return res;
41 }
42
43 ll extgcd(ll a, ll b, ll &x, ll &y);
44 pair<ll, ll> crt(const vector<ll> &a, const vector<ll> &m);
45
46 ll modInverse(ll a, ll m)
47 {
48     ll x, y;
49     extgcd(a, m, x, y);
50     return (x % m + m) % m;
51 }
52
53 ll fact_mod_pk(ll n, ll p, ll pk, int &e)
54 {
55     e = 0;
56     if (n == 0)
57         return 1;
58     ll res = 1;
59     for (ll i = 2; i <= pk; i++)
60     {
61         if (i % p != 0)
62             res = (res * i) % pk;
63     }
64     res = custom_pow(res, n / pk, pk);
65     for (ll i = 2; i <= n % pk; i++)

```

```

66     {
67         if (i % p != 0)
68             res = (res * i) % pk;
69     }
70     int next_e;
71     res = (res * fact_mod_pk(n / p, p, pk, next_e)) % pk;
72     e += n / p + next_e;
73     return res;
74 }
75
76 ll nCr_pk(ll n, ll r, ll p, ll pk)
77 {
78     if (n < r || r < 0)
79         return 0;
80     int e1, e2, e3;
81     ll num = fact_mod_pk(n, p, pk, e1);
82     ll den1 = fact_mod_pk(r, p, pk, e2);
83     ll den2 = fact_mod_pk(n - r, p, pk, e3);
84     ll p_pow = custom_pow(p, e1 - e2 - e3, pk);
85     if (p_pow == 0)
86         return 0;
87     ll ans = (num * modInverse((den1 * den2) % pk, pk)) % pk;
88     return (ans * p_pow) % pk;
89 }
90
91 ll nCr_arbitrary_mod(ll n, ll r, ll m)
92 {
93     if (n < r || r < 0)
94         return 0;
95     if (m == 1)
96         return 0;
97     vector<ll> rem, mod;
98     ll temp = m;
99     for (ll i = 2; i * i <= temp; i++)
100     {
101         if (temp % i == 0)
102         {
103             ll pk = 1;
104             while (temp % i == 0)
105             {
106                 pk *= i;
107                 temp /= i;
108             }
109             rem.push_back(nCr_pk(n, r, i, pk));
110             mod.push_back(pk);
111         }
112     }
113     if (temp > 1)
114     {
115         rem.push_back(nCr_pk(n, r, temp, temp));
116         mod.push_back(temp);
117     }
118     return crt(rem, mod).first;
119 }

```

6.4.2 nCr over congruence class

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: nCr OVER CONGRUENCE CLASS
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Sum of nCr where r mod M = K".
6  * - Classic Scenarios: You need to evaluate sum(nCr(N, i)) for all i where
7  *   i % M == K. N can be up to 1018, but M is small (M <= 100).
8  * - The Magic: It maps the combinatorial states into a matrix. If you have an
9  *   array tracking the sum mod M, multiplying it by a transition matrix
10  *   simulates
11  *   increasing N by 1. By exponentiating the matrix to the power of N in
12  *   O(M3 log N), you instantly compute the exact sum across the entire boundary.
13  *
14  * 2. HOW TO USE IT (THE BLACK BOX)
15  * - Query:
16  *   long long ans = nCr_congruence(N, K, M, MOD);
17  * - Complexity:
18  *   Time: O(M3 * log N)
19  *   Space: O(M2) for the matrix.
20  */
21
22 using ll = long long;
23
24 using Matrix = vector<vector<ll>>;
25
26 Matrix multiply_matrix(const Matrix &A, const Matrix &B, ll mod)
27 {
28     int n = A.size();
29     Matrix C(n, vector<ll>(n, 0));
30     for (int i = 0; i < n; i++)
31     {
32         for (int k = 0; k < n; k++)
33         {
34             for (int j = 0; j < n; j++)
35             {
36                 C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod;
37             }
38         }
39     }
40     return C;
41 }
42
43 Matrix power_matrix(Matrix A, ll exp, ll mod)
44 {
45     int n = A.size();
46     Matrix res(n, vector<ll>(n, 0));
47     for (int i = 0; i < n; i++)
48         res[i][i] = 1;
49     while (exp > 0)
50     {
51         if (exp % 2 == 1)
```

```
52         res = multiply_matrix(res, A, mod);
53         A = multiply_matrix(A, A, mod);
54         exp /= 2;
55     }
56     return res;
57 }
58
59 ll nCr_congruence(ll n, int k, int m, ll mod)
60 {
61     if (m == 1)
62         return custom_pow(2, n, mod);
63     Matrix T(m, vector<ll>(m, 0));
64     for (int i = 0; i < m; i++)
65     {
66         T[i][i] = 1;
67         T[i][(i + m - 1) % m] = 1;
68     }
69     T = power_matrix(T, n, mod);
70     return T[k][0];
71 }
```

7 Geometry

7.1 Ball 3D Class

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 3D CLOSEST PAIR OF POINTS (DIVIDE &
3  *   CONQUER)
4  *
5  * 1. WHAT PROBLEM DOES THIS SOLVE?
6  * - Keywords: "Minimum Euclidean distance in 3D", "Closest points in space", "N
7  *   up to 105".
8  * - Classic Scenarios: You are given N points in a 3D coordinate system. Finding
9  *   the closest
10  *   pair naively takes O(N2), which will Time Limit Exceed (TLE) for N > 104.
11  * - The Magic: Divide and Conquer! We sort the points by the X-axis and
12  *   recursively divide the
13  *   space into halves. The tricky part in 3D is merging the halves (the "strip")
14  *   . If we just
15  *   check all points in the strip naively, it can still degrade to O(N2).
16  *   To fix this, the algorithm groups the strip into 1D "columns" along the Z-
17  *   axis using the
18  *   current minimum distance `d` as the column width. This brilliant trick
19  *   guarantees that each
20  *   point only checks a strict O(1) number of neighboring points. The overall
21  *   complexity
22  *   beautifully drops to O(N log N).
23  *
24  * 2. HOW TO USE IT (THE BLACK BOX)
25  * - Initialization: Create a `BallManager` object. It automatically reads `N`
26  *   from `std::cin`,
27  *   followed by the `X Y Z` coordinates of the `N` points.
```

```

19 *      BallManager bm;
20 *
21 * - Execution: Call the main solver function to get the minimum distance.
22 *      long double min_distance = bm.ClosestPair();
23 *
24 * - Complexity:
25 *      Time:  $O(N \log N)$  The X and Y sorting takes  $O(N \log N)$ , and the
26 *      recursive strip
27 *      processing takes  $O(N)$  per level.
28 *      Space:  $O(N)$  to store the objects, vectors, and recursion stack.
29 *
30 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
31 * - Decoupling I/O: The constructor currently reads input directly. For better
32 *   flexibility,
33 *   you can modify the constructor to accept a `vector<vector<ll>>` or `vector<
34 *   Ball>` instead
35 *   of using `cin` inside the class.
36 * - Precision Issues: The algorithm uses `long double` and `sqrtl`. While this
37 *   is highly accurate,
38 *   some problems require exact integer squared distances. You can return the
39 *   squared distance
40 *   at the very end by squaring `finalMinDist`, but do NOT remove `sqrtl` from
41 *   DistanceTo`,
42 *   as the `SearchStrip` bucketing logic mathematically relies on the true
43 *   linear distance.
44 */
45 #include <iostream>
46 #include <vector>
47 #include <cmath>
48 #include <algorithm>
49 #include <limits>
50
51 using namespace std;
52 #define ll long long
53 #define ld long double
54
55 class Ball
56 {
57 public:
58     ll posX, posY, posZ; //Position values in 3D space
59     ll columnNumber[3]; //ID number of the column which ball belongs to (left,
60     //middle, right)
61     ll columnIndex[3]; //Position inside the column (left, middle, right)
62
63     // Time Complexity:  $O(1)$ 
64     //
65     ld DistanceTo(Ball* targetBall)
66     {
67         ll squareDistX = (posX - targetBall->posX) * (posX - targetBall->posX);
68         ll squareDistY = (posY - targetBall->posY) * (posY - targetBall->posY);
69         ll squareDistZ = (posZ - targetBall->posZ) * (posZ - targetBall->posZ);
70         return sqrtl(squareDistX + squareDistY + squareDistZ);
71     }
72 };
73
74 class BallManager
75 {
76 public:
77     ll ballCount = 0; //Total number of balls
78     Ball* ballArray; //Dynamically allocated ball array which holds all balls
79     //inside
80
81     // Time Complexity:  $O(1)$ 
82     static bool CompareX(Ball ball1, Ball ball2)
83     {
84         return (ball1.posX < ball2.posX);
85     }
86
87     // Time Complexity:  $O(1)$ 
88     static bool CompareY(Ball* ball1, Ball* ball2)
89     {
90         return (ball1->posY < ball2->posY);
91     }
92
93     // Time Complexity:  $O(N^2)$ 
94     // N (balls.size()).
95     ld NaiveMethod(std::vector<Ball*> balls)
96     {
97         int size = balls.size();
98         ld minDist = std::numeric_limits<double>::max();
99
100         //Take balls one by one
101         for (int i = 0; i < size - 1; ++i)
102         {
103             //Check other remaining balls
104             for (int j = i + 1; j < size; ++j)
105             {
106                 ld dist = balls[i]->DistanceTo(balls[j]);
107                 if (dist < minDist)
108                     minDist = dist;
109             }
110         }
111         return minDist;
112     }
113
114     // Time Complexity:  $O(S)$ 
115     // S (stripSize).
116     // (Columns) Z
117     // ( ).
118     ld SearchStrip(std::vector<Ball*> strip, ld stripDist)
119     {
120         //Save the strip distance value as minimum distance known
121         ld minDist = stripDist;
122
123         //If ball count in strip is less than 2, return strip distance
124         int stripSize = strip.size();
125         if (stripSize < 2)
126             return minDist;
127
128         //If ball count in strip is 2 or 3, find distance with naive method

```

```

120     if (stripSize < 4)
121     {
122         ld newDist = NaiveMethod(strip);
123         if (newDist < minDist)
124             minDist = newDist;
125         return minDist;
126     }
127
128     //If ball count is more than 3, start processing of creating columns
129     //Find the smallest z value among balls -> O(S)
130     int smallestZ = strip[0]->posZ;
131     for (int i = 1; i < stripSize; i++)
132         if (strip[i]->posZ < smallestZ)
133             smallestZ = strip[i]->posZ;
134
135     //Find the largest z value among balls -> O(S)
136     int largestZ = strip[0]->posZ;
137     for (int i = 1; i < stripSize; i++)
138         if (strip[i]->posZ > largestZ)
139             largestZ = strip[i]->posZ;
140
141     //Find range between z values and column count
142     int rangeZ = largestZ - smallestZ;
143     int columnCount = (int)(rangeZ / stripDist) + 1;
144
145     //Create an array of ball pointer vectors to put pointers of balls into
146     //proper columns
147     std::vector<Ball*>* columns = new std::vector<Ball*>[columnCount];
148
149     //Put all balls into proper columns (Phase 1) -> O(S)
150     //Start picking balls one by one with the smallest Y first
151     for (int i = 0; i < stripSize; i++)
152     {
153         //Find middle(main) column of ball, put ball into it, save index
154         strip[i]->columnNumber[1] = (11)((1d)(strip[i]->posZ - smallestZ) /
155             stripDist);
156         columns[strip[i]->columnNumber[1]].push_back(strip[i]);
157         strip[i]->columnIndex[1] = columns[strip[i]->columnNumber[1]].size()
158             - 1;
159
160         //Find and save left and right columns of ball
161         strip[i]->columnNumber[0] = strip[i]->columnNumber[1] - 1;
162         strip[i]->columnNumber[2] = strip[i]->columnNumber[1] + 1;
163
164         //There is no column at left of leftmost column, and right of
165         //rightmost column
166         //Save indices in left and right columns
167         if (strip[i]->columnNumber[1] != 0) //If not leftmost column
168             strip[i]->columnIndex[0] = columns[strip[i]->columnNumber[0]].
169                 size() - 1;
170
171         if (strip[i]->columnNumber[1] != columnCount - 1) //If not rightmos
172             strip[i]->columnIndex[2] = columns[strip[i]->columnNumber[2]].
173                 size() - 1;
174
175         //Find if such lesser distance exists between any pair of balls inside
176         //strip (Phase 2)
177         // O(S) overall. The inner loops run a constant number of times O(1) per
178         // ball.
179         //Start picking balls one by one with the smallest Y first
180         for (int i = 0; i < stripSize; i++)
181         {
182             //For left, middle and right columns of the selected ball
183             for (int j = 0; j < 3; j++)
184             {
185                 //Skip invalid out of bounds columns
186                 if ((strip[i]->columnNumber[j] == -1) || (strip[i]->columnNumber[
187                     j] == columnCount))
188                     continue;
189
190                 //Look through increasing Y value = O(1)
191                 for (int k = strip[i]->columnIndex[j] + 1; k < (11)columns[strip[
192                     i]->columnNumber[j]].size(); k++)
193                 {
194                     //If Y distance is already more than strip distance, no need
195                     //to check remaning balls in column
196                     if (columns[strip[i]->columnNumber[j]][k]->posY > strip[i]->
197                         posY + stripDist)
198                         break;
199
200                     //Check distance and if it is smaller than current minimum
201                     //distance, save it as new minimum distance
202                     ld newDist = strip[i]->DistanceTo(columns[strip[i]->
203                         columnNumber[j]][k]);
204                     if (newDist < minDist)
205                         minDist = newDist;
206                 }
207             }
208         }
209
210         //Delete dynamically allocated columns array
211         delete[] columns;
212
213         //Return minimum distance found in strip or initially given strip
214         //distance
215         return minDist;
216     }
217
218     // Time Complexity:  $T(N) = 2*T(N/2) + O(N) \Rightarrow O(N \log N)$ 
219     // N (sortedY.size()). O(N).
220     ld RecursiveCall(int sortedXPos, std::vector<Ball*> sortedY)
221     {
222         //If size is small enough, find distance with naive method -> O(1)
223         int size = sortedY.size();
224         if (size < 4)
225             return NaiveMethod(sortedY);
226
227         //Get middle ball for X axis

```



```

16
17 typedef long double ld;
18 typedef ld T;
19 typedef complex<T> pt;
20
21 const T EPS = 1e-9;
22 const long double PI = acos(-1.0);
23
24 #define x real()
25 #define y imag()
26
27 bool operator==(pt a, pt b) { return fabs(a.x - b.x) < EPS && fabs(a.y - b.y) <
    EPS; }
28 bool operator!=(pt a, pt b) { return !(a == b); }
29
30 int sgn(T val) {
31     if (val > EPS) return 1;
32     if (val < -EPS) return -1;
33     return 0;
34 }
35
36 T sq(pt p) { return p.x * p.x + p.y * p.y; }
37 ld abs(pt p) { return sqrt(sq(p)); }
38
39 pt perp(pt p) { return {-p.y, p.x}; }
40
41 T dot(pt v, pt w) { return v.x * w.x + v.y * w.y; }
42 T cross(pt v, pt w) { return v.x * w.y - v.y * w.x; }
43
44 bool isPerp(pt v, pt w) { return fabs(dot(v, w)) < EPS; }
45
46 pt unitVector(pt p) {
47     ld len = abs(p);
48     if (len < EPS) return {0, 0};
49     return p / len;
50 }
51
52 // =====
53 //                                TRANSFORMATIONS
54 // =====
55
56 pt translate(pt v, pt p) { return p + v; }
57
58 // Scale point p by a factor around center c
59 pt scale(pt c, T factor, pt p) {
60     return c + (p - c) * factor;
61 }
62
63 // Rotate point p by angle a around center c
64 pt rot(pt p, pt c, ld a) {
65     pt v = p - c;
66     pt rotate = {cos(a), sin(a)};
67     return c + rotate * v;
68 }
69

```

```

70 // If point p maps to fp and q maps to fq, find the image of r
71 pt linearTransfo(pt p, pt q, pt r, pt fp, pt fq) {
72     return fp + (r - p) * (fq - fp) / (q - p);
73 }
74
75 // =====
76 //                                ANGLES
77 // =====
78
79 // (AB X AC) --> relative to AB: if (C left) return pos, else if (C right) return
    neg
80 T orient(pt a, pt b, pt c) { return cross(b - a, c - a); }
81
82 bool inAngle(pt a, pt b, pt c, pt p) {
83     T abp = orient(a, b, p), acp = orient(a, c, p), abc = orient(a, b, c);
84     if (abc < 0) swap(abp, acp);
85     return (abp >= 0 && acp <= 0) ^ (abc < 0);
86 }
87
88 T angle(pt v, pt w) {
89     return atan2(fabs(cross(v, w)), dot(v, w));
90 }
91
92 ld orientedAngle(pt a, pt b, pt c) {
93     if (orient(a, b, c) >= 0)
94         return angle(b - a, c - a);
95     else
96         return 2 * PI - angle(b - a, c - a);
97 }
98
99 ld angleTravelled(pt a, pt p, pt q) {
100     ld ampli = angle(p - a, q - a);
101     if (orient(a, p, q) > 0) return ampli;
102     else return -ampli;
103 }
104
105 bool half(pt p) {
106     return p.y > EPS || (fabs(p.y) <= EPS && p.x < -EPS);
107 }
108
109 // =====
110 //                                LINES
111 // =====
112
113 struct line {
114     pt v; T c;
115
116     // From direction vector v and offset c
117     line(pt v, T c) : v(v), c(c) {}
118
119     // From equation ax+by=c
120     line(T a, T b, T _c) {
121         v = {b, -a};
122         c = _c;
123     }

```

```

124
125 // From points P and Q
126 line(pt p, pt q) {
127     v = q - p;
128     c = cross(v, p);
129 }
130
131 T side(pt p) { return cross(v, p) - c; }
132 double dist(pt p) { return abs(side(p)) / abs(v); }
133 double sqDist(pt p) { return side(p) * side(p) / (T)sq(v); }
134 line perpThrough(pt p) { return {p, p + perp(v)}; }
135 bool cmpProj(pt p, pt q) { return dot(v, p) < dot(v, q); }
136 line translate(pt t) { return {v, c + cross(v, t)}; }
137 line shiftLeft(double dist) { return {v, c + dist * abs(v)}; }
138 pt proj(pt p) { return p - perp(v) * side(p) / sq(v); }
139 pt refl(pt p) { return p - perp(v) * (T)2.0 * side(p) / sq(v); }
140 };
141
142 bool inter(line l1, line l2, pt &out) {
143     T d = cross(l1.v, l2.v);
144     if (fabs(d) <= EPS) return false;
145     out = (l2.v * l1.c - l1.v * l2.c) / d;
146     return true;
147 }
148
149 line bisector(line l1, line l2, bool interior) {
150     assert(cross(l1.v, l2.v) != 0); // Cannot be parallel
151     T sign = interior ? 1 : -1;
152     return {l2.v / (T)abs(l2.v) + l1.v / (T)abs(l1.v) * sign,
153           l2.c / abs(l2.v) + l1.c / abs(l1.v) * sign};
154 }
155
156 // =====
157 //                               SEGMENTS
158 // =====
159
160 bool inDisk(pt a, pt b, pt p) {
161     return dot(a - p, b - p) <= EPS;
162 }
163
164 bool onSegment(pt a, pt b, pt p) {
165     return fabs(orient(a, b, p)) <= EPS && inDisk(a, b, p);
166 }
167
168 bool properInter(pt a, pt b, pt c, pt d, pt &out) {
169     T oa = orient(c, d, a), ob = orient(c, d, b);
170     T oc = orient(a, b, c), od = orient(a, b, d);
171     if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0) {
172         out = (a * ob - b * oa) / (ob - oa);
173         return true;
174     }
175     return false;
176 }
177
178 set<pair<ld, ld>> inters(pt a, pt b, pt c, pt d) {

```

```

179     set<pair<ld, ld>> s;
180     pt out;
181     if (a == c || a == d) s.insert({a.x, a.y});
182     if (b == c || b == d) s.insert({b.x, b.y});
183     if (s.size()) return s;
184
185     if (properInter(a, b, c, d, out)) return {{out.x, out.y}};
186     if (onSegment(c, d, a)) s.insert({a.x, a.y});
187     if (onSegment(c, d, b)) s.insert({b.x, b.y});
188     if (onSegment(a, b, c)) s.insert({c.x, c.y});
189     if (onSegment(a, b, d)) s.insert({d.x, d.y});
190
191     return s;
192 }
193
194 ld segPoint(pt a, pt b, pt p) {
195     if (a != b) {
196         line l(a, b);
197         if (l.cmpProj(a, p) && l.cmpProj(p, b)) return l.dist(p);
198     }
199     return min(abs(p - a), abs(p - b));
200 }
201
202 ld rayPoint(pt a, pt b, pt p) {
203     pt ab = b - a, ap = p - a;
204     if (dot(ab, ap) < 0) return abs(ap);
205     line l(a, b);
206     return l.dist(p);
207 }
208
209 ld segSeg(pt a, pt b, pt c, pt d) {
210     pt dummy;
211     if (properInter(a, b, c, d, dummy)) return 0;
212     return min({segPoint(a, b, c), segPoint(a, b, d),
213               segPoint(c, d, a), segPoint(c, d, b)});
214 }
215
216 // =====
217 //                               CIRCLES
218 // =====
219
220 pair<pt, T> circumCircle(pt a, pt b, pt c) {
221     b = b - a, c = c - a;
222     assert(cross(b, c) != 0); // No circumcircle if aligned
223     return {a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / (T)2,
224           abs(perp(b * sq(c) - c * sq(b)) / cross(b, c) / (T)2)};
225 }
226
227 int circleLine(pt o, double r, line l, pair<pt, pt> &out) {
228     double h2 = r * r - l.sqDist(o);
229     if (h2 >= 0) {
230         pt p = l.proj(o);
231         pt h = l.v * (T)(sqrt(h2) / abs(l.v));
232         out = {p - h, p + h};
233     }

```

```

234     return 1 + sgn(h2);
235 }
236
237 int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt> &out) {
238     pt d = o2 - o1; T d2 = sq(d);
239     if (d2 == 0) { assert(r1 != r2); return 0; }
240     T pd = (d2 + r1 * r1 - r2 * r2) / 2;
241     T h2 = r1 * r1 - pd * pd / d2;
242     if (h2 >= 0) {
243         pt p = o1 + d * pd / d2, h = perp(d) * sqrt(h2 / d2);
244         out = {p - h, p + h};
245     }
246     return 1 + sgn(h2);
247 }
248
249 int tangents(pt o1, T r1, pt o2, T r2, bool inner, vector<pair<pt, pt>> &out) {
250     if (inner) r2 = -r2;
251     pt d = o2 - o1;
252     T dr = r1 - r2, d2 = sq(d), h2 = d2 - dr * dr;
253     if (d2 == 0 || h2 < 0) { assert(h2 != 0); return 0; }
254     for (T sign : {-1, 1}) {
255         pt v = (d * dr + perp(d) * sqrt(h2) * sign) / d2;
256         out.push_back({o1 + v * r1, o2 + v * r2});
257     }
258     return 1 + (h2 > 0);
259 }
260
261 // =====
262 //                      BASIC POLYGONS
263 // =====
264
265 ld areaTriangle(pt a, pt b, pt c) { return abs(cross(b - a, c - a)) / 2.0; }
266
267 ld areaPolygon(vector<pt> p) {
268     ld area = 0.0;
269     for (int i = 0, n = p.size(); i < n; i++)
270         area += cross(p[i], p[(i + 1) % n]);
271     return abs(area) / 2.0;
272 }
273
274 bool isConvex(vector<pt> p) {
275     bool hasPos = false, hasNeg = false;
276     for (int i = 0, n = p.size(); i < n; i++) {
277         int o = sgn(orient(p[i], p[(i + 1) % n], p[(i + 2) % n]));
278         if (o > 0) hasPos = true;
279         if (o < 0) hasNeg = true;
280     }
281     return !(hasPos && hasNeg);
282 }
283
284 bool above(pt a, pt p) { return p.y >= a.y; }
285
286 bool crossesRay(pt a, pt p, pt q) {
287     return (above(a, q) - above(a, p)) * orient(a, p, q) > 0;
288 }

```

```

289
290 bool inPolygon(vector<pt> p, pt a, bool strict = true) {
291     int numCrossings = 0;
292     for (int i = 0, n = p.size(); i < n; i++) {
293         if (onSegment(p[i], p[(i + 1) % n], a)) return !strict;
294         numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
295     }
296     return numCrossings & 1;
297 }
298
299 bool onPolygonSide(vector<pt> p, pt a) {
300     for (int i = 0, n = p.size(); i < n; i++) {
301         if (onSegment(p[i], p[(i + 1) % n], a)) return true;
302     }
303     return false;
304 }
305
306 pt polygonCentroid(const vector<pt>& p) {
307     int n = p.size();
308     ld area = 0.0, cx = 0.0, cy = 0.0;
309     for (int i = 0; i < n; i++) {
310         int next = (i + 1) % n;
311         ld cross_val = cross(p[i], p[next]);
312         area += cross_val;
313         cx += (p[i].x + p[next].x) * cross_val;
314         cy += (p[i].y + p[next].y) * cross_val;
315     }
316     area /= 2.0;
317     return {cx / (6.0 * area), cy / (6.0 * area)};
318 }
319
320 // =====
321 //                      PICK'S THEOREM
322 // =====
323
324 int boundaryPoints(const vector<pt>& p) {
325     int b = 0, n = p.size();
326     for (int i = 0; i < n; i++) {
327         int next = (i + 1) % n;
328         int dx = abs(p[i].x - p[next].x);
329         int dy = abs(p[i].y - p[next].y);
330         b += std::gcd(dx, dy);
331     }
332     return b;
333 }
334
335 int doubleAreaPolygon(const vector<pt>& p) {
336     int doubleArea = 0, n = p.size();
337     for (int i = 0; i < n; i++) {
338         int next = (i + 1) % n;
339         doubleArea += cross(p[i], p[next]);
340     }
341     return abs(doubleArea);
342 }
343

```

```

344 int interiorPoints(const vector<pt>& p) {
345     return (doubleAreaPolygon(p) - boundaryPoints(p) + 2) / 2;
346 }
347
348 // =====
349 //                               ADVANCED POLYGONS
350 // =====
351
352 bool cw(pt a, pt b, pt c, bool include_collinear) {
353     int o = sgn(orient(a, b, c));
354     return o < 0 || (include_collinear && o == 0);
355 }
356
357 bool collinear(pt a, pt b, pt c) { return sgn(orient(a, b, c)) == 0; }
358
359 void convex_hull(vector<pt>& a, bool include_collinear = false) {
360     if (a.empty()) return;
361     pt p0 = *min_element(a.begin(), a.end(), [](pt a, pt b) {
362         return make_pair(a.y, a.x) < make_pair(b.y, b.x);
363     });
364     sort(a.begin(), a.end(), [&p0](const pt& a, const pt& b) {
365         int o = sgn(orient(p0, a, b));
366         if (o == 0) return sq(p0 - a) < sq(p0 - b);
367         return o < 0;
368     });
369     if (include_collinear) {
370         int i = (int)a.size() - 1;
371         while (i >= 0 && collinear(p0, a[i], a.back())) i--;
372         reverse(a.begin() + i + 1, a.end());
373     }
374     vector<pt> st;
375     for (int i = 0; i < (int)a.size(); i++) {
376         while (st.size() > 1 && !cw(st[st.size() - 2], st.back(), a[i],
377             include_collinear))
378             st.pop_back();
379         if (st.empty() || a[i] != st.back())
380             st.push_back(a[i]);
381     }
382     if (!include_collinear && st.size() == 2 && st[0] == st[1])
383         st.pop_back();
384     a = st;
385 }
386
387 void reorder_polygon(vector<pt> & P) {
388     size_t pos = 0;
389     for (size_t i = 1; i < P.size(); i++) {
390         if (P[i].y < P[pos].y || (P[i].y == P[pos].y && P[i].x < P[pos].x))
391             pos = i;
392     }
393     rotate(P.begin(), P.begin() + pos, P.end());
394 }
395 // P and Q must be counter clockwise
396 vector<pt> minkowski(vector<pt> P, vector<pt> Q) {
397     reorder_polygon(P);
398     reorder_polygon(Q);
399     P.push_back(P[0]); P.push_back(P[1]);
400     Q.push_back(Q[0]); Q.push_back(Q[1]);
401
402     vector<pt> result;
403     size_t i = 0, j = 0;
404     while (i < P.size() - 2 || j < Q.size() - 2) {
405         result.push_back(P[i] + Q[j]);
406         auto crs = cross(P[i + 1] - P[i], Q[j + 1] - Q[j]);
407         if (crs >= 0 && i < P.size() - 2) ++i;
408         if (crs <= 0 && j < Q.size() - 2) ++j;
409     }
410     return result;
411 }
412
413 double maximum_dist_from_polygon_to_polygon(vector<pt> &u, vector<pt> &v) {
414     int n = (int)u.size(), m = (int)v.size();
415     double ans = 0;
416     if (n < 3 || m < 3) {
417         for (int i = 0; i < n; i++)
418             for (int j = 0; j < m; j++)
419                 ans = max(ans, (double)sq(u[i] - v[j]));
420     }
421     return sqrt(ans);
422 }
423
424 if (u[0].x > v[0].x) { swap(n, m); swap(u, v); }
425
426 int i = 0, j = 0, step = n + m + 10;
427 while (j + 1 < m && v[j].x < v[j + 1].x) j++;
428 while (step-- > 0) {
429     if (cross(u[(i + 1) % n] - u[i], v[(j + 1) % m] - v[j]) >= 0) j = (j + 1) % m;
430     else i = (i + 1) % n;
431     ans = max(ans, (double)sq(u[i] - v[j]));
432 }
433 return sqrt(ans);
434 // =====
435 //                               HALF-PLANE INTERSECTION
436 // =====
437
438 struct Halfplane {
439     pt p, pq;
440     long double angle;
441
442     Halfplane() {}
443     Halfplane(const pt& a, const pt& b) : p(a), pq(b - a) {
444         angle = atan2l(pq.y, pq.x);
445     }
446
447     bool out(const pt& r) {
448         return cross(pq, r - p) < -EPS;
449     }
450
451     bool operator < (const Halfplane& e) const { return angle < e.angle; }

```

```

452
453     friend pt inter(const Halfplane& s, const Halfplane& t) {
454         long double alpha = cross((t.p - s.p), t.pq) / cross(s.pq, t.pq);
455         return s.p + (s.pq * alpha);
456     }
457 };
458
459 vector<pt> hp_intersect(vector<Halfplane>& H) {
460     const int inf = 1e9;
461     pt box[4] = { pt(inf, inf), pt(-inf, inf), pt(-inf, -inf), pt(inf, -inf) };
462     for (int i = 0; i < 4; i++) {
463         Halfplane aux(box[i], box[(i + 1) % 4]);
464         H.push_back(aux);
465     }
466
467     sort(H.begin(), H.end());
468     deque<Halfplane> dq;
469     int len = 0;
470
471     for (int i = 0; i < int(H.size()); i++) {
472         while (len > 1 && H[i].out(inter(dq[len - 1], dq[len - 2]))) {
473             dq.pop_back(); --len;
474         }
475         while (len > 1 && H[i].out(inter(dq[0], dq[1]))) {
476             dq.pop_front(); --len;
477         }
478
479         if (len > 0 && fabs1(cross(H[i].pq, dq[len - 1].pq)) < EPS) {
480             if (dot(H[i].pq, dq[len - 1].pq) < 0.0) return vector<pt>();
481             if (H[i].out(dq[len - 1].p)) {
482                 dq.pop_back(); --len;
483             } else continue;
484         }
485         dq.push_back(H[i]); ++len;
486     }
487
488     while (len > 2 && dq[0].out(inter(dq[len - 1], dq[len - 2]))) {
489         dq.pop_back(); --len;
490     }
491     while (len > 2 && dq[len - 1].out(inter(dq[0], dq[1]))) {
492         dq.pop_front(); --len;
493     }
494
495     if (len < 3) return vector<pt>();
496
497     vector<pt> ret(len);
498     for (int i = 0; i + 1 < len; i++) {
499         ret[i] = inter(dq[i], dq[i + 1]);
500     }
501     ret.back() = inter(dq[len - 1], dq[0]);
502     return ret;
503 }
504
505 // =====
506 //                                O(LOG N) CONVEX INSIDE

```

```

507 // =====
508
509 bool inConvex(vector<pt> &p, pt a) {
510     if (p.size() < 3) return false;
511     int n = p.size(), l = 1, r = n - 2, mid, ans = 1;
512     while (l <= r) {
513         mid = (l + r) / 2;
514         if (sgn(orient(p[0], p[mid], a)) > 0) {
515             l = mid + 1;
516             ans = mid;
517         } else {
518             r = mid - 1;
519         }
520     }
521     return inPolygon({p[0], p[ans], p[ans + 1]}, a, false);
522 }
523
524 bool StrictInConvex(vector<pt> &p, pt a) {
525     if (p.size() < 3) return false;
526     int n = p.size(), l = 1, r = n - 2, mid, ans = 1;
527     while (l <= r) {
528         mid = (l + r) / 2;
529         if (sgn(orient(p[0], p[mid], a)) > 0) {
530             l = mid + 1;
531             ans = mid;
532         } else {
533             r = mid - 1;
534         }
535     }
536     bool f = true;
537     if (ans == 1) f = !onSegment(p[1], p[2], a) && !onSegment(p[0], p[1], a);
538     else if (ans == n - 2) f = !onSegment(p[n - 2], p[n - 1], a) && !onSegment(p[
539         n - 1], p[0], a);
540     return inPolygon({p[0], p[ans], p[ans + 1]}, a, false) && f;
541 }
542
543 // =====
544 //                                ANTIPODAL POINTS
545 // =====
546
547 vector<pair<int, int>> all_anti_podal(int n, vector<pt> &p) {
548     vector<pair<int, int>> result;
549     auto nx = [&](int i) { return (i + 1) % n; };
550     auto pv = [&](int i) { return (i - 1 + n) % n; };
551     vector<bool> vis(n, false);
552
553     for (int p1 = 0, p2 = 0; p1 < n; ++p1) {
554         pt base = p[nx(p1)] - p[p1];
555         while (p2 == p1 || p2 == nx(p1) || sgn(cross(base, p[nx(p2)] - p[p2])) ==
556             sgn(cross(base, p[p2] - p[pv(p2)]))) {
557             p2 = nx(p2);
558         }
559         if (vis[p1]) continue;
560         vis[p1] = true;

```

```

560     result.push_back({p1, p2});
561     result.push_back({nx(p1), p2});
562
563     if (sgn(cross(base, p[nx(p2)] - p[p2])) == 0) {
564         result.push_back({p1, nx(p2)});
565         result.push_back({nx(p1), nx(p2)});
566         vis[p2] = true;
567     }
568 }
569 return result;
570 }

```

7.3 geometryReferenceSheet

```

1 // =====
2 //                                     BASICS & TYPES
3 // =====
4 int sgn(T val);           // Returns -1 (negative), 1 (positive), or 0.
5 T sq(pt p);              // Squared length of vector p.
6 ld abs(pt p);            // Length of vector p.
7 pt perp(pt p);           // Returns vector rotated 90 degrees CCW.
8 T dot(pt v, pt w);       // Dot product.
9 T cross(pt v, pt w);     // Cross product (v X w).
10 bool isPerp(pt v, pt w); // True if dot product is 0.
11 pt unitVector(pt p);     // Vector of length 1 in the same direction.
12
13 // =====
14 //                                     TRANSFORMATIONS
15 // =====
16 pt translate(pt v, pt p); // Moves point p by vector v.
17 pt scale(pt c, T factor, pt p); // Scales p away from center c by factor.
18 pt rot(pt p, pt c, ld a); // Rotates p around c by angle a (in radians).
19 pt linearTransfo(pt p, pt q, pt r, pt fp, pt fq); // Maps r given that p->fp and
    q->fq.
20
21 // =====
22 //                                     ANGLES
23 // =====
24 T orient(pt a, pt b, pt c); // Relative to AB: > 0 (C is left), < 0 (C is
    right), 0 (collinear).
25 bool inAngle(pt a, pt b, pt c, pt p); // True if p is inside CCW angle BAC.
26 T angle(pt v, pt w);           // Shortest positive non-oriented angle [0, PI].
27 ld orientedAngle(pt a, pt b, pt c); // CCW angle BAC in range [0, 2*PI].
28 ld angleTravelled(pt a, pt p, pt q); // Signed angle travelled from P to Q around
    A.
29
30 // =====
31 //                                     LINES (struct line)
32 // =====
33 line(pt v, T c);           // From direction vector and offset.
34 line(T a, T b, T _c);      // From equation ax + by = c.
35 line(pt p, pt q);          // From two points P and Q.
36 T side(pt p);              // > 0 (left of line), < 0 (right of line), 0 (
    on line).
37 double dist(pt p);         // Shortest distance from p to line.
38 line perpThrough(pt p);    // Returns perpendicular line passing through p.
39 line translate(pt t);      // Translates line by vector t.
40 line shiftLeft(double dist); // Shifts line to the left by given distance.
41 pt proj(pt p);             // Projection of point p onto the line.
42 pt refl(pt p);             // Reflection of point p across the line.
43
44 bool inter(line l1, line l2, pt &out); // Line-line intersection. Returns false
    if parallel.
45 line bisector(line l1, line l2, bool interior); // Angle bisector of l1 and l2.
46
47 // =====
48 //                                     SEGMENTS & RAYS
49 // =====
50 bool onSegment(pt a, pt b, pt p); // True if p lies on segment AB.
51 bool properInter(pt a, pt b, pt c, pt d, pt &out); // True if segments AB and CD
    intersect at exactly one interior point.
52 set<pair<ld, ld>> inters(pt a, pt b, pt c, pt d); // Returns all intersection
    points of segments AB and CD.
53 ld segPoint(pt a, pt b, pt p); // Shortest distance from p to segment AB.
54 ld rayPoint(pt a, pt b, pt p); // Shortest distance from p to ray AB.
55 ld segSeg(pt a, pt b, pt c, pt d); // Shortest distance between segment AB and
    segment CD.
56
57 // =====
58 //                                     CIRCLES
59 // =====
60 pair<pt, T> circumCircle(pt a, pt b, pt c); // Returns {center, radius} of
    triangle ABC.
61 int circleLine(pt o, double r, line l, pair<pt, pt> &out); // Intersections of
    line and circle. Returns count (0, 1, or 2).
62 int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt> &out); // Intersections
    of 2 circles. Returns count.
63 int tangents(pt o1, T r1, pt o2, T r2, bool inner, vector<pair<pt, pt>> &out); //
    Tangent lines between 2 circles.
64
65 // =====
66 //                                     BASIC POLYGONS
67 // =====
68 ld areaPolygon(vector<pt> p); // Area of any simple polygon.
69 bool isConvex(vector<pt> p); // True if polygon is strictly convex.
70 bool inPolygon(vector<pt> p, pt a, bool strict = true); // O(N). True if a is
    inside. strict=true excludes boundaries.
71 pt polygonCentroid(const vector<pt>& p); // Center of gravity. Vertices must be
    ordered (CW/CCW).
72
73 // =====
74 //                                     PICK'S THEOREM (Integer Coordinates)
75 // =====
76 int boundaryPoints(const vector<pt>& p); // Number of integer points on
    boundary.
77 int doubleAreaPolygon(const vector<pt>& p); // 2 * Area (exact integer).
78 int interiorPoints(const vector<pt>& p); // Number of strictly interior integer

```

```

79     points.
80 // =====
81 // ADVANCED POLYGONS
82 // =====
83 void convex_hull(vector<pt>& a, bool include_collinear = false); //  $O(N \log N)$ .
    Modifies vector in-place.
84 vector<pt> minkowski(vector<pt> P, vector<pt> Q); //  $O(N+M)$ . Minkowski sum. P and
    Q must be CCW convex.
85 double maximum_dist_from_polygon_to_polygon(vector<pt> &u, vector<pt> &v); //  $O(N^2+M^2)$ . Max distance between two convex polygons.
86
87 // =====
88 // HALF-PLANE INTERSECTION
89 // =====
90 Halfplane(const pt& a, const pt& b); // Halfplane to the LEFT of directed line AB
    .
91 vector<pt> hp_intersect(vector<Halfplane>& H); //  $O(N \log N)$ . Returns CCW convex
    polygon of the intersection. Empty vector if no intersection.
92
93 // =====
94 //  $O(\log N)$  CONVEX INSIDE
95 // =====
96 bool inConvex(vector<pt> &p, pt a); //  $O(\log N)$ . True if point inside convex
    polygon (including boundary).
97 bool StrictInConvex(vector<pt> &p, pt a); //  $O(\log N)$ . True if point strictly
    inside convex polygon.
98
99 // =====
100 // ANTIPODAL POINTS
101 // =====
102 vector<pair<int, int>> all_anti_podal(int n, vector<pt> &p); //  $O(N)$ . Returns
    pairs of indices of antipodal points in a convex polygon.

```

8 String

8.1 Aho Corasick

8.1.1 Aho corasick

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: AHO-CORASICK
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Multiple patterns", "Dictionary matching", "String DP".
6  * - Classic Scenarios: Find all occurrences of an array of strings inside a
    massive text,
7  * or count how many strings of length N don't contain any of the dictionary
    words.
8  * - The Magic: Builds a Trie and connects nodes via "fail" links. If you
    mismatch,

```

```

9  * the fail link teleports you to the longest proper suffix that exists in the
    Trie.
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Initialization:
13 *     AhoCorasick ac;
14 *     for (string s : patterns) ac.insert(s);
15 *     ac.build();
16 * - Query:
17 *     int total_matches = ac.search(text);
18 *
19 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
20 * - DP Transitions: The `next_state(u, c)` transitions are precalculated during
    `build()`.
21 * You can use `ac.trie[u].next[c]` to transition seamlessly in Dynamic
    Programming.
22 * - Exit Link (dict_link): Fast-forwards to the next valid dictionary word
    ending to
23 * avoid  $O(N)$  worst-case walking up the failure tree.
24 */
25
26 const int K = 26;
27
28 struct Vertex
29 {
30     int next[K];
31     bool leaf = false;
32     int p = -1;
33     char pch;
34     int link = -1;
35     int exit_link = -1;
36     int matches = 0; // Count of dictionary words ending here
37
38     Vertex(int p = -1, char ch = '$') : p(p), pch(ch)
39     {
40         fill(begin(next), end(next), -1);
41     }
42 };
43
44 struct AhoCorasick
45 {
46     vector<Vertex> t;
47
48     AhoCorasick()
49     {
50         t.emplace_back();
51     }
52
53     void insert(const string &s)
54     {
55         int v = 0;
56         for (char ch : s)
57         {
58             int c = ch - 'a';
59             if (t[v].next[c] == -1)

```

```

60     {
61         t[v].next[c] = t.size();
62         t.emplace_back(v, ch);
63     }
64     v = t[v].next[c];
65 }
66 t[v].leaf = true;
67 t[v].matches++;
68 }
69
70 void build()
71 {
72     queue<int> q;
73     q.push(0);
74     t[0].link = 0;
75     t[0].exit_link = 0;
76
77     while (!q.empty())
78     {
79         int v = q.front();
80         q.pop();
81
82         for (int c = 0; c < K; c++)
83         {
84             int u = t[v].next[c];
85             if (u != -1)
86             {
87                 t[u].link = (v == 0) ? 0 : t[t[v].link].next[c];
88                 t[u].exit_link = t[t[u].link].leaf ? t[u].link : t[t[u].link]
89                     .exit_link;
90                 t[u].matches += t[t[u].link].matches;
91                 q.push(u);
92             }
93             else
94             {
95                 t[v].next[c] = (v == 0) ? 0 : t[t[v].link].next[c];
96             }
97         }
98     }
99
100 long long search(const string &text)
101 {
102     long long total = 0;
103     int v = 0;
104     for (char ch : text)
105     {
106         v = t[v].next[ch - 'a'];
107         total += t[v].matches;
108     }
109     return total;
110 }
111 long long search_non_overlapping(const string &text)
112 {
113     long long total = 0;

```

```

114     int v = 0;
115     for (char ch : text)
116     {
117         v = t[v].next[ch - 'a'];
118
119         if (t[v].matches > 0)
120         {
121             total++;
122             v = 0;
123         }
124     }
125     return total;
126 }
127 };

```

8.2 Hashing

8.2.1 2D string hashing

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 2D STRING HASHING
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Subgrid matching", "2D Pattern matching".
6  * - Classic Scenarios: Find how many times an R x C pattern appears inside
7  *   an N x M grid.
8  * - The Magic: Extends the rolling hash to two dimensions using two different
9  *   bases (one for rows, one for columns). Calculates 2D prefix sums of the hash
10  *
11  * 2. HOW TO USE IT (THE BLACK BOX)
12  * - Initialization:
13  *   Hash2D h(grid);
14  * - Query:
15  *   ll val = h.get(r1, c1, r2, c2); // 0-indexed, inclusive boundaries
16  */
17
18 using ll = long long;
19
20 struct Hash2D
21 {
22     const ll MOD = 1e9 + 7;
23     const ll BASE_R = 313, BASE_C = 317;
24     vector<vector<ll>> hash;
25     vector<ll> pR, pC;
26
27     Hash2D(const vector<string> &grid)
28     {
29         if (grid.empty())
30             return;
31         int n = grid.size();
32         int m = grid[0].size();
33

```

```

34 hash.assign(n + 1, vector<ll>(m + 1, 0));
35 pR.assign(n + 1, 1);
36 pC.assign(m + 1, 1);
37
38 for (int i = 1; i <= n; i++)
39     pR[i] = (pR[i - 1] * BASE_R) % MOD;
40 for (int i = 1; i <= m; i++)
41     pC[i] = (pC[i - 1] * BASE_C) % MOD;
42
43 for (int i = 1; i <= n; i++)
44 {
45     for (int j = 1; j <= m; j++)
46     {
47         hash[i][j] = (hash[i - 1][j] * BASE_R + hash[i][j - 1] * BASE_C
48             - hash[i - 1][j - 1] * BASE_R % MOD * BASE_C % MOD + grid[i]
49             - 1)[j - 1] % MOD;
50         if (hash[i][j] < 0)
51             hash[i][j] += MOD;
52     }
53 }
54 ll get(int r1, int c1, int r2, int c2)
55 {
56     r1++;
57     c1++;
58     r2++;
59     c2++;
60     ll h = hash[r2][c2];
61     h = (h - hash[r1 - 1][c2] * pR[r2 - r1 + 1]) % MOD;
62     h = (h - hash[r2][c1 - 1] * pC[c2 - c1 + 1]) % MOD;
63     h = (h + hash[r1 - 1][c1 - 1] * pR[r2 - r1 + 1] % MOD * pC[c2 - c1 + 1])
64         % MOD;
65     if (h < 0)
66         h += MOD;
67     return h;
68 }

```

8.2.2 Hash(Hassan)

```

1 const int mod1 = 1e9 + 7;
2 const int mod2 = 1e9 + 9;
3 const int base1 = 313;
4 const int base2 = 317;
5
6 const int MAXN = 3e4 + 100;
7 int pw1[MAXN], pw2[MAXN];
8 bool isCalPower = false;
9
10 void pre() {
11     if (isCalPower) return;
12

```

```

13     pw1[0] = pw2[0] = 1;
14     for (int i = 1; i < MAXN; i++) {
15         pw1[i] = (pw1[i - 1] * base1) % mod1;
16         pw2[i] = (pw2[i - 1] * base2) % mod2;
17     }
18     isCalPower = true;
19 }
20
21 struct Hash {
22     int n;
23     vector<pair<int, int>> pref;
24     vector<pair<int, int>> suf; // Used only if palindrome checking is needed
25
26     // Pass `compute_rev = true` if you need O(1) palindrome checks
27     Hash(const string& s, bool compute_rev = false) {
28         pre();
29         n = s.size();
30         pref.assign(n + 1, {0, 0});
31
32         for (int i = 0; i < n; i++) {
33             pref[i + 1].first = (pref[i].first * base1 + s[i]) % mod1;
34             pref[i + 1].second = (pref[i].second * base2 + s[i]) % mod2;
35         }
36
37         if (compute_rev) {
38             suf.assign(n + 1, {0, 0});
39             for (int i = n - 1; i >= 0; i--) {
40                 suf[i].first = (suf[i + 1].first * base1 + s[i]) % mod1;
41                 suf[i].second = (suf[i + 1].second * base2 + s[i]) % mod2;
42             }
43         }
44     }
45
46     // Get hash of substring s[l...r] (0-indexed) in O(1)
47     pair<int, int> get(int l, int r) {
48         if (l > r) return {0, 0};
49         int len = r - l + 1;
50
51         int h1 = (pref[r + 1].first - (pref[l].first * pw1[len]) % mod1) % mod1;
52         if (h1 < 0) h1 += mod1;
53
54         int h2 = (pref[r + 1].second - (pref[l].second * pw2[len]) % mod2) % mod2;
55         if (h2 < 0) h2 += mod2;
56
57         return {h1, h2};
58     }
59
60     // Get reverse hash of substring s[l...r] in O(1)
61     pair<int, int> get_rev(int l, int r) {
62         if (l > r || suf.empty()) return {0, 0};
63         int len = r - l + 1;
64         int h1 = (suf[l].first - suf[r + 1].first * pw1[len]) % mod1;
65         if (h1 < 0) h1 += mod1;
66
67         int h2 = (suf[l].second - suf[r + 1].second * pw2[len]) % mod2;

```

```

68     if (h2 < 0) h2 += mod2;
69
70     return {h1, h2};
71 }
72
73 // Check if s[l...r] is a palindrome in O(1)
74 bool is_pal(int l, int r) {
75     return get(l, r) == get_rev(l, r);
76 }
77 };
78
79
80 // --- USEFUL HASHING UTILITY FUNCTIONS ---
81
82 // Concatenate two hashes (h1 followed by h2 of length len2) in O(1)
83 pair<int, int> concat_hash(pair<int, int> h1, pair<int, int> h2, int len2) {
84     int ans1 = (h1.first * pw1[len2] + h2.first) % mod1;
85     int ans2 = (h1.second * pw2[len2] + h2.second) % mod2;
86     return {ans1, ans2};
87 }
88
89 // Find Longest Common Prefix (LCP) length of s1[l1...r1] and s2[l2...r2] in O(
    log N)
90 int get_lcp(Hash& sh1, int l1, int r1, Hash& sh2, int l2, int r2) {
91     int low = 1, high = min(r1 - l1 + 1, r2 - l2 + 1), ans = 0;
92     while (low <= high) {
93         int mid = low + (high - low) / 2;
94         if (sh1.get(l1, l1 + mid - 1) == sh2.get(l2, l2 + mid - 1)) {
95             ans = mid;
96             low = mid + 1;
97         } else {
98             high = mid - 1;
99         }
100     }
101     return ans;
102 }
103
104 // Compare s1[l1...r1] and s2[l2...r2] lexicographically in O(log N)
105 // Returns: -1 if s1 < s2, 0 if s1 == s2, 1 if s1 > s2
106 int compare_substrings(const string& s1, Hash& sh1, int l1, int r1,
107     const string& s2, Hash& sh2, int l2, int r2) {
108     int lcp_len = get_lcp(sh1, l1, r1, sh2, l2, r2);
109     int len1 = r1 - l1 + 1;
110     int len2 = r2 - l2 + 1;
111
112     // If one is completely a prefix of another
113     if (lcp_len == len1 && lcp_len == len2) return 0;
114     if (lcp_len == len1) return -1;
115     if (lcp_len == len2) return 1;
116
117     // Compare the first differing character
118     return s1[l1 + lcp_len] < s2[l2 + lcp_len] ? -1 : 1;
119 }

```

8.2.3 Hash

```

1 struct Hash
2 {
3     int base, mod1, mod2, inv1, inv2, n;
4     vector<int> pw1, pw2, invpw1, invpw2, pre1, pre2;
5
6     int power(int b, int p, int m)
7     {
8         int res{1};
9         b %= m;
10        while (p)
11        {
12            if (p & 1)
13                res = res * b % m;
14            b = b * b % m;
15            p >>= 1;
16        }
17        return res;
18    }
19
20    Hash(const string &s, int b)
21    {
22        base = b;
23        mod1 = 1e9 + 7;
24        mod2 = 1e9 + 9;
25        n = int(s.size());
26
27        inv1 = power(base, mod1 - 2, mod1);
28        inv2 = power(base, mod2 - 2, mod2);
29
30        pw1.assign(n + 1, 1), pw2.assign(n + 1, 1);
31        invpw1.assign(n + 1, 1), invpw2.assign(n + 1, 1);
32        pre1.assign(n + 1, {}), pre2.assign(n + 1, {});
33
34        for (int i = 0; i < n; i++)
35        {
36            pw1[i + 1] = pw1[i] * base % mod1;
37            pw2[i + 1] = pw2[i] * base % mod2;
38            invpw1[i + 1] = invpw1[i] * inv1 % mod1;
39            invpw2[i + 1] = invpw2[i] * inv2 % mod2;
40            pre1[i + 1] = (pre1[i] + pw1[i] * s[i]) % mod1;
41            pre2[i + 1] = (pre2[i] + pw2[i] * s[i]) % mod2;
42        }
43    }
44
45    pair<int, int> get(int l, int r)
46    {
47        return {(pre1[r + 1] - pre1[l] + mod1) * invpw1[l] % mod1, (pre2[r + 1] -
            pre2[l] + mod2) * invpw2[l] % mod2};
48    }
49 };

```

8.2.4 MultisetHashing

```
1 struct MultisetHashing
2 {
3     int base, m, n;
4     vector<int> pre;
5
6     int power(int b, int p)
7     {
8         b %= m;
9         int res{1};
10        while (p)
11        {
12            if (p & 1)
13                res = res * b % m;
14            b = b * b % m;
15            p >>= 1;
16        }
17        return res;
18    }
19
20    MultisetHashing(vector<int> &arr, int b)
21    {
22        n = int(arr.size());
23        m = 1e9 + 7;
24        base = b;
25        pre.assign(n + 1, {});
26        for (int i = 0; i < n; i++)
27            pre[i + 1] = (pre[i] + power(base, arr[i])) % m;
28    }
29
30    int get(int l, int r)
31    {
32        // 0-based
33        return (pre[r + 1] - pre[l] + m) % m;
34    }
35};
```

8.2.5 XOR hashing

```
1 struct XorHashing
2 {
3     int n;
4     map<int, int> h;
5     vector<int> pre;
6
7     XorHashing(vector<int> &arr)
8     {
9         n = int(arr.size());
10        pre.assign(n + 1, {});
11        for (int i = 0; i < n; i++)
12        {
13            if (!h.count(arr[i]))
```

```
14                h[arr[i]] = getRandom(1, 1e18);
15                pre[i + 1] = pre[i] ^ h[arr[i]];
16        }
17    }
18
19    int get(int l, int r)
20    {
21        // 0-based
22        return pre[r + 1] ^ pre[l];
23    }
24};
```

8.3 LCS

8.3.1 Bitset LCS

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: BITSET LCS
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "LCS of massive strings", " $O(N^2)$  TLE".
6  * - Classic Scenarios: Find the Longest Common Subsequence of two strings where
7  *    $N$  and  $M$  are up to 50,000.
8  * - The Magic: C++ `std::bitset` does not support subtraction, which is required
9  *   for the classic bitwise LCS formula. This custom block engine maps the DP
10  *   states
11  *   directly onto native 64-bit unsigned integers (uint64_t). It computes 64 DP
12  *   cells simultaneously using carry-flag subtraction, slashing the  $O(N * M)$ 
13  *   time
14  *   by a factor of 64.
15  *
16  * 2. HOW TO USE IT (THE BLACK BOX)
17  * - Query:
18  *   int length = bitset_lcs(string_A, string_B);
19  *
20  * - Complexity:
21  *   Time:  $O(N * M / 64)$ 
22  *   Space:  $O(M / 64)$ 
23  */
24
25 #include <cstdint>
26
27 int bitset_lcs(const string &a, const string &b)
28 {
29     int n = a.length(), m = b.length();
30     if (n == 0 || m == 0)
31         return 0;
32
33     int words = (m >> 6) + 1;
34     vector<vector<uint64_t>> pos(256, vector<uint64_t>(words, 0));
35
36     for (int i = 0; i < m; i++)
37     {
38         int b_char = b[i];
39         for (int j = 0; j < n; j++)
40         {
41             int a_char = a[j];
42             pos[a_char][i >> 6] |= (pos[b_char][i >> 6] << 1) & (1 << (j >> 6));
43         }
44     }
45 }
```

```

36     pos[b[i]][i >> 6] |= (1ULL << (i & 63));
37 }
38
39 vector<uint64_t> D(words, 0);
40
41 for (int i = 0; i < n; i++)
42 {
43     uint64_t b_carry = 1;
44     uint64_t shift_carry = 0;
45
46     for (int j = 0; j < words; j++)
47     {
48         uint64_t P = D[j] | pos[a[i]][j];
49         uint64_t d_shl = (D[j] << 1) | shift_carry;
50         shift_carry = D[j] >> 63;
51
52         uint64_t sub = d_shl + b_carry;
53         uint64_t next_carry = (P < sub) || (sub < d_shl);
54         uint64_t diff = P - sub;
55
56         D[j] = P & (~P ^ diff);
57         b_carry = next_carry;
58     }
59 }
60
61 int lcs_length = 0;
62 for (int j = 0; j < words; j++)
63 {
64     uint64_t val = D[j];
65     while (val)
66     {
67         val &= (val - 1);
68         lcs_length++;
69     }
70 }
71
72 return lcs_length;
73 }

```

8.4 Matching

8.4.1 FFT wildcard matching

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FFT WILDCARD MATCHING
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Wildcards in both text and pattern", "Massive strings".
6  * - Classic Scenarios: Both strings have '?' wildcards. Bitsets will TLE if
7  *   N = 200,000.
8  * - The Magic: Uses the algebraic identity  $(T - P)^2 * T * P = 0$ . Maps wildcards
9  *   to 0 and letters to 1-26. Expands the polynomial into three separate NTT
10  *   convolutions. A result of 0 at index i means a perfect match.

```

```

11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *   vector<int> matches = fft_wildcard_match(text, pattern);
15 *
16 * 3. HOW TO ADAPT IT
17 * - Dependencies: This heavily relies on your `multiply(A, B)` function from
18 *   your NTT template. Make sure it is included above this.
19 */
20
21 using ll = long long;
22
23 vector<ll> multiply(vector<ll> const &a, vector<ll> const &b);
24
25 vector<int> fft_wildcard_match(const string &text, const string &pattern)
26 {
27     int n = text.length(), m = pattern.length();
28     if (m > n)
29         return {};
30
31     vector<ll> t1(n), t2(n), t3(n);
32     vector<ll> p1(m), p2(m), p3(m);
33
34     for (int i = 0; i < n; i++)
35     {
36         ll val = (text[i] == '?' ? 0 : text[i] - 'a' + 1);
37         t1[i] = val;
38         t2[i] = val * val;
39         t3[i] = val * val * val;
40     }
41
42     for (int i = 0; i < m; i++)
43     {
44         ll val = (pattern[i] == '?' ? 0 : pattern[i] - 'a' + 1);
45         p1[m - 1 - i] = val;
46         p2[m - 1 - i] = val * val;
47         p3[m - 1 - i] = val * val * val;
48     }
49
50     vector<ll> res1 = multiply(t3, p1);
51     vector<ll> res2 = multiply(t2, p2);
52     vector<ll> res3 = multiply(t1, p3);
53
54     vector<int> matches;
55     for (int i = m - 1; i < n; i++)
56     {
57         ll diff = res1[i] - 2 * res2[i] + res3[i];
58         if (diff == 0)
59         {
60             matches.push_back(i - (m - 1));
61         }
62     }
63     return matches;
64 }

```

8.4.2 KMP

```
1 vector<int> KMP(string s, string p = "")
2 {
3     if (!p.empty())
4         s = p + "$" + s;
5     int n = int(s.size());
6     vector<int> LPS(n);
7     int i{1}, j{};
8     while (i < n)
9     {
10         if (s[i] == s[j])
11         {
12             j++;
13             LPS[i] = j;
14             i++;
15         }
16         else
17         {
18             if (j == 0)
19             {
20                 LPS[i] = 0;
21                 i++;
22             }
23             else
24                 j = LPS[j - 1];
25         }
26     }
27     return LPS;
28 }
29
30 int numberOfMatches(string &s, string &p)
31 {
32     vector<int> pi = KMP(s, p);
33     int ans{};
34     for (auto &val : pi)
35         ans += val == int(p.size());
36     return ans;
37 }
38
39 vector<int> countPrefixMatching(string &s)
40 {
41     int n = int(s.size());
42     vector<int> pi = KMP(s);
43     vector<int> frq(n);
44     for (auto &x : pi)
45         frq[x]++;
46     for (int i = n - 1; i; i--)
47         frq[pi[i - 1]] += frq[i];
48     return frq;
49 }
50
51 vector<vector<int>> KMPAutomaton(string &s)
52 {
53     vector<int> pi = KMP(s);
```

```
54     int n = int(s.size());
55     vector aut(n + 1, vector<int>(26));
56     aut[0][s[0] - 'a'] = 1;
57     for (int i = 1; i <= n; i++)
58     {
59         for (int j = 0; j < 26; j++)
60         {
61             if (i < n && s[i] - 'a' == j)
62                 aut[i][j] = i + 1;
63             else
64                 aut[i][j] = aut[pi[i - 1]][j];
65         }
66     }
67     return aut;
68 }
```

8.4.3 String matching using bitset

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: BITSET STRING MATCHING
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Wildcards in text", "Small alphabet", "Multiple patterns".
6  * - Classic Scenarios: Text has wildcards. Find all occurrences of pattern.
7  * - The Magic: Represents occurrences of each character as a bitset. By shifting
8  *   and ANDing the bitsets corresponding to the pattern's characters, you
9  *   instantly
10  *   identify valid shift offsets in  $O(N * M / 64)$  time.
11  *
12  * 2. HOW TO USE IT (THE BLACK BOX)
13  * - Query:
14  *   bitset<MAX_N> matches = bitset_match(text, pattern);
15  *   // If matches[i] is 1, the pattern exists starting at text[i].
16  *
17  * 3. HOW TO ADAPT IT
18  * - MAX_N must be a constant (e.g., 100005) because bitsets require compile-time
19  *   sizes.
20  */
21
22 const int MAX_N = 100005;
23
24 bitset<MAX_N> bitset_match(const string &text, const string &pattern)
25 {
26     vector<bitset<MAX_N>> mask(26);
27     for (int i = 0; i < text.length(); i++)
28     {
29         if (text[i] != '?')
30         {
31             mask[text[i] - 'a'].set(i);
32         }
33         else
34         {
35             for (int c = 0; c < 26; c++)
```

```

34         mask[c].set(i);
35     }
36 }
37
38 bitset<MAX_N> ans;
39 ans.set();
40
41 for (int i = 0; i < pattern.length(); i++)
42 {
43     if (pattern[i] != '?')
44     {
45         ans &= (mask[pattern[i] - 'a'] >> i);
46     }
47 }
48 return ans;
49 }

```

8.4.4 Z

```

1 struct ZAlgorithm
2 {
3     vector<int> build(const string &s)
4     {
5         int n = s.size();
6         vector<int> z(n, 0);
7         int l = 0, r = 0;
8         for (int i = 1; i < n; i++)
9         {
10             if (i < r)
11                 z[i] = min(r - i, z[i - l]);
12             while (i + z[i] < n && s[z[i]] == s[i + z[i]])
13                 z[i]++;
14             if (i + z[i] > r)
15             {
16                 l = i;
17                 r = i + z[i];
18             }
19         }
20         return z;
21     }
22
23     int numberOfMatches(const string &s, const string &p)
24     {
25         if (p.empty() || s.empty())
26             return 0;
27         string combined = p + "$" + s;
28         vector<int> z = build(combined);
29         int ans = 0;
30         int p_len = p.size();
31         for (int i = p_len + 1; i < z.size(); i++)
32         {
33             if (z[i] == p_len)
34                 ans++;

```

```

35     }
36     return ans;
37 }
38
39 vector<int> findOccurrences(const string &s, const string &p)
40 {
41     if (p.empty() || s.empty())
42         return {};
43     string combined = p + "$" + s;
44     vector<int> z = build(combined);
45     vector<int> occ;
46     int p_len = p.size();
47     for (int i = p_len + 1; i < z.size(); i++)
48     {
49         if (z[i] == p_len)
50             occ.push_back(i - p_len - 1);
51     }
52     return occ;
53 }
54 };

```

8.5 Misc

8.5.1 Debruijn sequence

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DE BRUIJN SEQUENCE
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Contains all permutations", "Shortest string with all substrings".
6  * - Classic Scenarios: Generate the shortest possible string that contains every
7  * possible sequence of length N using an alphabet of size K.
8  * - The Magic: Mathematical guarantees state this string will always have length
9  *  $K^N + N - 1$ . This uses a recursive DFS (essentially finding an Eulerian path).
10 * to generate the lexicographically smallest De Bruijn sequence in optimal
    time.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     string sequence = de_bruijn(K, N);
15 * - Example:
16 *     de_bruijn(2, 3) generates "00010111".
17 *     (Note: To act as a true cyclic sequence, you conceptually wrap the ends.
18 * If you need it strictly linear, append the first N-1 characters to the
    end).
19 *
20 * - Complexity:
21 *     Time:  $O(K^N)$ 
22 *     Space:  $O(K * N)$ 
23 */
24

```

```

25 string de_bruijn(int k, int n)
26 {
27     if (k == 1)
28         return string(n, '0');
29     if (n == 1)
30     {
31         string res = "";
32         for (int i = 0; i < k; i++)
33             res += (char)('0' + i);
34         return res;
35     }
36
37     vector<int> a(k * n, 0);
38     string sequence;
39
40     auto db = [&](auto &self, int t, int p) -> void
41     {
42         if (t > n)
43         {
44             if (n % p == 0)
45             {
46                 for (int i = 1; i <= p; i++)
47                 {
48                     sequence += (char)('0' + a[i]);
49                 }
50             }
51         }
52         else
53         {
54             a[t] = a[t - p];
55             self(self, t + 1, p);
56             for (int j = a[t - p] + 1; j < k; j++)
57             {
58                 a[t] = j;
59                 self(self, t + 1, t);
60             }
61         }
62     };
63
64     db(db, 1, 1);
65     return sequence;
66 }

```

8.5.2 Expression parser

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: EXPRESSION PARSER
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Evaluate expression", "Calculator", "Infix to Postfix".
6  * - Classic Scenarios: You are given a raw string like "3 + 5 * (2 - 8)" and
7  *   need the exact mathematical result.
8  * - The Magic: Uses Recursive Descent to strictly enforce PEMDAS (Parentheses,

```

```

9  *   Exponents, Multiplication/Division, Addition/Subtraction) in pure O(N) time.
10 *
11 * 2. HOW TO USE IT (THE BLACK BOX)
12 * - Query:
13 *     string expr = "10 + 2 * (6 - 3)";
14 *     int pos = 0;
15 *     long long result = parse_expr(expr, pos);
16 *
17 * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
18 * - Division by Zero: Currently, it performs integer division. If a problem
19 *   tests division by zero, add a check inside `parse_term`.
20 * - Modulo: If the expression must be evaluated modulo M, wrap all additions,
21 *   subtractions, and multiplications in the respective functions with `% MOD`.
22 */
23
24 using ll = long long;
25
26 ll parse_expr(const string &s, int &i);
27 ll parse_term(const string &s, int &i);
28 ll parse_factor(const string &s, int &i);
29 ll parse_number(const string &s, int &i);
30
31 void skip_spaces(const string &s, int &i)
32 {
33     while (i < s.length() && isspace(s[i]))
34         i++;
35 }
36
37 ll parse_expr(const string &s, int &i)
38 {
39     ll res = parse_term(s, i);
40     while (i < s.length())
41     {
42         skip_spaces(s, i);
43         if (i < s.length() && (s[i] == '+' || s[i] == '-'))
44         {
45             char op = s[i++];
46             ll next_term = parse_term(s, i);
47             if (op == '+')
48                 res += next_term;
49             else
50                 res -= next_term;
51         }
52         else
53         {
54             break;
55         }
56     }
57     return res;
58 }
59
60 ll parse_term(const string &s, int &i)
61 {
62     ll res = parse_factor(s, i);
63     while (i < s.length())

```

```

64 {
65     skip_spaces(s, i);
66     if (i < s.length() && (s[i] == '*' || s[i] == '/'))
67     {
68         char op = s[i++];
69         ll next_factor = parse_factor(s, i);
70         if (op == '*')
71             res *= next_factor;
72         else
73             res /= next_factor;
74     }
75     else
76     {
77         break;
78     }
79 }
80 return res;
81 }
82
83 ll parse_factor(const string &s, int &i)
84 {
85     skip_spaces(s, i);
86     if (i < s.length() && s[i] == '(')
87     {
88         i++;
89         ll res = parse_expr(s, i);
90         skip_spaces(s, i);
91         if (i < s.length() && s[i] == ')')
92             i++;
93         return res;
94     }
95     return parse_number(s, i);
96 }
97
98 ll parse_number(const string &s, int &i)
99 {
100     skip_spaces(s, i);
101     ll res = 0;
102     while (i < s.length() && isdigit(s[i]))
103     {
104         res = res * 10 + (s[i++] - '0');
105     }
106     return res;
107 }

```

8.5.3 Min cyclic shift

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DUVAL'S & MIN CYCLIC SHIFT
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Lexicographically smallest rotation", "Lyndon words".
6  * - Classic Scenarios: You can rotate a string S any number of times. Find the

```

```

7  * rotation that comes first in the dictionary.
8  * - The Magic: Duval's algorithm processes the string in strict  $O(N)$  time and
9  *  $O(1)$  auxiliary space, bypassing the need to build a heavy Suffix Automaton
10 * on the concatenated string `S + S`.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     int best_index = min_cyclic_shift(S);
15 *     string smallest_rotation = S.substr(best_index) + S.substr(0, best_index
16 * );
17 *     vector<string> factorization = duval_factorization(S);
18 * - Complexity:
19 *     Time:  $O(N)$ 
20 *     Space:  $O(1)$  for shift,  $O(N)$  for factorization return.
21 */
22
23 int min_cyclic_shift(string s)
24 {
25     s += s;
26     int n = s.length();
27     int i = 0, ans = 0;
28     while (i < n / 2)
29     {
30         ans = i;
31         int j = i + 1, k = i;
32         while (j < n && s[k] <= s[j])
33         {
34             if (s[k] < s[j])
35                 k = i;
36             else
37                 k++;
38             j++;
39         }
40         while (i <= k)
41             i += j - k;
42     }
43     return ans;
44 }
45
46 vector<string> duval_factorization(const string &s)
47 {
48     int n = s.length();
49     int i = 0;
50     vector<string> factorization;
51     while (i < n)
52     {
53         int j = i + 1, k = i;
54         while (j < n && s[k] <= s[j])
55         {
56             if (s[k] < s[j])
57                 k = i;
58             else
59                 k++;
60             j++;

```

```

61     }
62     while (i <= k)
63     {
64         factorization.push_back(s.substr(i, j - k));
65         i += j - k;
66     }
67 }
68 return factorization;
69 }

```

8.6 Palindromes

8.6.1 Manacher

```

1 struct Manacher
2 {
3     vector<int> p;
4     string s;
5
6     Manacher(const string &str)
7     {
8         s = str;
9         string t = "";
10        for (char c : s)
11        {
12            t += "#";
13            t += c;
14        }
15        t += "#";
16
17        int n = t.size();
18        string temp = "$" + t + "^";
19        vector<int> p_temp(n + 2, 0);
20        int l = 0, r = 1;
21
22        for (int i = 1; i <= n; i++)
23        {
24            if (i <= r)
25                p_temp[i] = min(r - i, p_temp[l + (r - i)]);
26            while (temp[i - p_temp[i]] == temp[i + p_temp[i]])
27                p_temp[i]++;
28            if (i + p_temp[i] > r)
29            {
30                l = i - p_temp[i];
31                r = i + p_temp[i];
32            }
33        }
34
35        p.assign(n, 0);
36        for (int i = 1; i <= n; i++)
37        {
38            p[i - 1] = p_temp[i] - 1;
39        }

```

```

40     }
41
42     bool is_palindrome(int l, int r)
43     {
44         if (l > r)
45             swap(l, r);
46         int len = r - l + 1;
47         int idx = l + r + 1;
48         return p[idx] >= len;
49     }
50
51     string longest_palindrome()
52     {
53         int max_len = 0, center_idx = 0;
54         for (int i = 0; i < p.size(); i++)
55         {
56             if (p[i] > max_len)
57             {
58                 max_len = p[i];
59                 center_idx = i;
60             }
61         }
62         int start = (center_idx - max_len) / 2;
63         return s.substr(start, max_len);
64     }
65 };

```

8.6.2 Palindromic tree

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PALINDROMIC TREE (EERTREE)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Distinct palindromes", "Occurrences of palindromes".
6  * - Classic Scenarios: Find the number of distinct palindromic substrings, or
7  * find the palindrome P that maximizes length(P) * occurrences(P).
8  * - The Magic: It builds a graph containing exactly two roots: one for even-
9  length
10 * palindromes and one for odd-length. Every node represents a distinct
11 * palindrome,
12 * and fail links point to the longest proper palindromic suffix.
13 *
14 * 2. HOW TO USE IT (THE BLACK BOX)
15 * - Initialization:
16 *     Eertree pt;
17 *     for (char c : S) pt.add_char(c);
18 *     pt.count_occurrences(); // Must call this after building!
19 *
20 * - Complexity:
21 *     Time: Amortized O(N)
22 *     Space: O(N * ALPHABET_SIZE)
23 */

```

```

23 struct Eertree
24 {
25     static const int K = 26;
26     struct Node
27     {
28         int next[K];
29         int len;
30         int link;
31         long long cnt;
32
33         Node(int len = 0, int link = 0) : len(len), link(link), cnt(0)
34         {
35             fill(begin(next), end(next), 0);
36         }
37     };
38
39     vector<Node> tree;
40     string s;
41     int last;
42
43     Eertree()
44     {
45         tree.emplace_back(0, 1); // Node 0: Even length root
46         tree.emplace_back(-1, 1); // Node 1: Odd length root
47         last = 0;
48     }
49
50     void add_char(char ch)
51     {
52         s += ch;
53         int c = ch - 'a';
54         int curr = last;
55         int pos = s.length() - 1;
56
57         while (pos - 1 - tree[curr].len < 0 || s[pos - 1 - tree[curr].len] != ch)
58         {
59             curr = tree[curr].link;
60         }
61
62         if (tree[curr].next[c])
63         {
64             last = tree[curr].next[c];
65             tree[last].cnt++;
66             return;
67         }
68
69         int now = tree.size();
70         tree.emplace_back(tree[curr].len + 2);
71         tree[curr].next[c] = now;
72
73         if (tree[now].len == 1)
74         {
75             tree[now].link = 0;
76         }
77         else

```

```

78     {
79         int fail = tree[curr].link;
80         while (pos - 1 - tree[fail].len < 0 || s[pos - 1 - tree[fail].len] !=
            ch)
81         {
82             fail = tree[fail].link;
83         }
84         tree[now].link = tree[fail].next[c];
85     }
86
87     last = now;
88     tree[last].cnt++;
89 }
90
91 void count_occurrences()
92 {
93     for (int i = tree.size() - 1; i > 1; i--)
94     {
95         tree[tree[i].link].cnt += tree[i].cnt;
96     }
97 }
98 };

```

8.7 Suffix Structures

8.7.1 Suffix array

```

1 struct SuffixArray
2 {
3     string s;
4     int n;
5     vector<int> p;
6     vector<vector<int>>> C;
7     vector<int> lcp;
8     bool is_cyclic;
9
10    SuffixArray(string str, bool cyclic = false)
11    {
12        is_cyclic = cyclic;
13        s = str;
14        if (!is_cyclic)
15            s += '$';
16        n = s.size();
17        const int alphapet = 256;
18        p.assign(n, 0);
19        vector<int> c(n), cnt(max(n, alphapet), 0);
20
21        for (int i = 0; i < n; i++)
22            cnt[s[i]]++;
23        for (int i = 1; i < alphapet; i++)
24            cnt[i] += cnt[i - 1];
25        for (int i = 0; i < n; i++)
26            p[--cnt[s[i]]] = i;

```

```

27
28 c[p[0]] = 0;
29 int classes = 1;
30 for (int i = 1; i < n; i++)
31 {
32     if (s[p[i]] != s[p[i - 1]])
33         classes++;
34     c[p[i]] = classes - 1;
35 }
36 C.push_back(c);
37
38 vector<int> pn(n), cn(n);
39 for (int h = 0; (1 << h) < n; h++)
40 {
41     for (int i = 0; i < n; i++)
42     {
43         pn[i] = p[i] - (1 << h);
44         if (pn[i] < 0)
45             pn[i] += n;
46     }
47     fill(cnt.begin(), cnt.begin() + classes, 0ll);
48     for (int i = 0; i < n; i++)
49         cnt[C.back()[pn[i]]]++;
50     for (int i = 1; i < classes; i++)
51         cnt[i] += cnt[i - 1];
52     for (int i = n - 1; i >= 0; i--)
53         p[--cnt[C.back()[pn[i]]]] = pn[i];
54
55     cn[p[0]] = 0;
56     classes = 1;
57     for (int i = 1; i < n; i++)
58     {
59         pair<int, int> curr = {C.back()[p[i]], C.back()[p[i] + (1 << h)] % n};
60         pair<int, int> prev = {C.back()[p[i - 1]], C.back()[p[i - 1] + (1 << h)] % n};
61         if (curr != prev)
62             classes++;
63         cn[p[i]] = classes - 1;
64     }
65     C.push_back(cn);
66 }
67
68 if (!is_cyclic)
69 {
70     p.erase(p.begin());
71     int orig_n = n - 1;
72     vector<int> rank(orig_n, 0);
73     for (int i = 0; i < orig_n; i++)
74         rank[p[i]] = i;
75     int k = 0;
76     lcp.assign(orig_n - 1, 0);
77     for (int i = 0; i < orig_n; i++)
78     {
79         if (rank[i] == orig_n - 1)

```

```

80
81         k = 0;
82         continue;
83     }
84     int j = p[rank[i] + 1];
85     while (i + k < orig_n && j + k < orig_n && s[i + k] == s[j + k])
86         k++;
87     lcp[rank[i]] = k;
88     if (k)
89         k--;
90 }
91 }
92 }
93
94 string smallestCyclicShift()
95 {
96     return s.substr(p[0]) + s.substr(0, p[0]);
97 }
98
99 vector<int> subStringInString(string t)
100 {
101     int m = t.size();
102     int orig_n = n - 1;
103     int l = 0, r = orig_n - 1;
104     int first = -1;
105     while (l <= r)
106     {
107         int mid = l + (r - l) / 2;
108         auto can = [&]()
109         {
110             for (int i = 0; i < m; i++)
111             {
112                 if (p[mid] + i >= orig_n)
113                     return 1LL;
114                 if (s[p[mid] + i] != t[i])
115                 {
116                     if (s[p[mid] + i] < t[i])
117                         return 1LL;
118                     if (s[p[mid] + i] > t[i])
119                         return 2LL;
120                 }
121             }
122             return 0LL;
123         };
124         int res = can();
125         if (res == 0)
126         {
127             first = mid;
128             r = mid - 1;
129         }
130         else if (res == 1)
131         {
132             l = mid + 1;
133         }
134         else

```

```

135     {
136         r = mid - 1;
137     }
138 }
139 if (first == -1)
140     return {};
141 l = 0, r = orig_n - 1;
142 int last = -1;
143 while (l <= r)
144 {
145     int mid = l + (r - l) / 2;
146     auto can = [&]()
147     {
148         for (int i = 0; i < m; i++)
149         {
150             if (p[mid] + i >= orig_n)
151                 return 1LL;
152             if (s[p[mid] + i] != t[i])
153             {
154                 if (s[p[mid] + i] < t[i])
155                     return 1LL;
156                 if (s[p[mid] + i] > t[i])
157                     return 2LL;
158             }
159         }
160         return 0LL;
161     };
162     int res = can();
163     if (res == 0)
164     {
165         last = mid;
166         l = mid + 1;
167     }
168     else if (res == 1)
169     {
170         l = mid + 1;
171     }
172     else
173     {
174         r = mid - 1;
175     }
176 }
177 vector<int> ans;
178 for (int i = first; i <= last; i++)
179 {
180     ans.push_back(p[i]);
181 }
182 sort(ans.begin(), ans.end());
183 return ans;
184 }
185
186 int compareSubstrings(int i, int len1, int j, int len2)
187 {
188     int l = min(len1, len2);
189     if (l > 0)

```

```

190 {
191     int k = __lg(l);
192     pair<int, int> a = {C[k][i], C[k][i + 1 - (1 << k)]};
193     pair<int, int> b = {C[k][j], C[k][j + 1 - (1 << k)]};
194     if (a != b)
195         return a < b ? -1 : 1;
196 }
197 if (len1 == len2)
198     return 0;
199 return len1 < len2 ? -1 : 1;
200 }
201
202 int numberOfDifferentSubStrings()
203 {
204     int orig_n = n - 1;
205     return (orig_n * orig_n + orig_n) / 2 - accumulate(lcp.begin(), lcp.end(), 0ll);
206 }
207
208 string longestCommonSubstring(string t)
209 {
210     string orig_s = s;
211     if (!is_cyclic)
212         orig_s.pop_back();
213     int len1 = orig_s.size();
214     SuffixArray sa(orig_s + char(1) + t);
215     int max_len = 0;
216     int start_idx = -1;
217     for (int i = 0; i < (int)sa.lcp.size(); i++)
218     {
219         if ((sa.p[i] < len1 && sa.p[i + 1] > len1) || (sa.p[i] > len1 && sa.p[i + 1] < len1))
220         {
221             if (sa.lcp[i] > max_len)
222             {
223                 max_len = sa.lcp[i];
224                 start_idx = sa.p[i];
225             }
226         }
227     }
228     if (max_len == 0)
229         return "";
230     return sa.s.substr(start_idx, max_len);
231 }
232
233 string longestRepeatedSubstring()
234 {
235     int max_len = 0;
236     int start_idx = -1;
237     for (int i = 0; i < (int)lcp.size(); i++)
238     {
239         if (lcp[i] > max_len)
240         {
241             max_len = lcp[i];
242             start_idx = p[i];

```

```

243     }
244 }
245 if (max_len == 0)
246     return "";
247 return s.substr(start_idx, max_len);
248 }
249 };

```

8.7.2 Suffix automaton

```

1 struct SuffixAutomaton
2 {
3     struct State
4     {
5         int len, link;
6         int first_pos;
7         bool is_clone;
8         int cnt;
9         long long paths;
10        map<char, int> next;
11        vector<int> inv_link;
12    };
13
14    vector<State> st;
15    int sz, last;
16    string s;
17
18    SuffixAutomaton()
19    {
20        st.assign(2, State());
21        st[0].len = 0;
22        st[0].link = -1;
23        st[0].cnt = 0;
24        sz = 1;
25        last = 0;
26    }
27
28    void extend(char c)
29    {
30        int cur = sz++;
31        if (cur >= st.size())
32            st.resize(st.size() * 2);
33        st[cur].len = st[last].len + 1;
34        st[cur].first_pos = st[cur].len - 1;
35        st[cur].is_clone = false;
36        st[cur].cnt = 1;
37
38        s += c;
39
40        int p = last;
41        while (p != -1 && !st[p].next.count(c))
42        {
43            st[p].next[c] = cur;

```

```

44            p = st[p].link;
45        }
46        if (p == -1)
47        {
48            st[cur].link = 0;
49        }
50        else
51        {
52            int q = st[p].next[c];
53            if (st[p].len + 1 == st[q].len)
54            {
55                st[cur].link = q;
56            }
57            else
58            {
59                int clone = sz++;
60                if (clone >= st.size())
61                    st.resize(st.size() * 2);
62                st[clone].len = st[p].len + 1;
63                st[clone].next = st[q].next;
64                st[clone].link = st[q].link;
65                st[clone].first_pos = st[q].first_pos;
66                st[clone].is_clone = true;
67                st[clone].cnt = 0;
68
69                while (p != -1 && st[p].next[c] == q)
70                {
71                    st[p].next[c] = clone;
72                    p = st[p].link;
73                }
74                st[q].link = st[cur].link = clone;
75            }
76        }
77        last = cur;
78    }
79
80    void build(const string &text)
81    {
82        for (char c : text)
83            extend(c);
84        precompute();
85    }
86
87    void precompute()
88    {
89        for (int v = 1; v < sz; v++)
90        {
91            st[st[v].link].inv_link.push_back(v);
92        }
93        vector<pair<int, int>> order(sz);
94        for (int i = 0; i < sz; i++)
95            order[i] = {st[i].len, i};
96        sort(order.rbegin(), order.rend());
97        for (int i = 0; i < sz - 1; i++)
98        {

```

```

99         int v = order[i].second;
100         if (st[v].link != -1)
101         {
102             st[st[v].link].cnt += st[v].cnt;
103         }
104     }
105 }
106
107 bool checkOccurrence(string P)
108 {
109     int v = 0;
110     for (char c : P)
111     {
112         if (!st[v].next.count(c))
113             return false;
114         v = st[v].next[c];
115     }
116     return true;
117 }
118
119 long long numberOfDifferentSubstrings()
120 {
121     long long tot = 0;
122     for (int i = 1; i < sz; i++)
123     {
124         tot += st[i].len - st[st[i].link].len;
125     }
126     return tot;
127 }
128
129 long long totalLengthOfDifferentSubstrings()
130 {
131     long long tot = 0;
132     for (int i = 1; i < sz; i++)
133     {
134         long long shortest = st[st[i].link].len + 1;
135         long long longest = st[i].len;
136         long long num_strings = longest - shortest + 1;
137         long long cur = num_strings * (longest + shortest) / 2;
138         tot += cur;
139     }
140     return tot;
141 }
142
143 void precomputePaths()
144 {
145     vector<pair<int, int>> order(sz);
146     for (int i = 0; i < sz; i++)
147         order[i] = {st[i].len, i};
148     sort(order.rbegin(), order.rend());
149     for (int i = 0; i < sz; i++)
150     {
151         int v = order[i].second;
152         st[v].paths = st[v].cnt;
153         for (auto &edge : st[v].next)

```

```

154         {
155             st[v].paths += st[edge.second].paths;
156         }
157     }
158 }
159
160 string kthSubstring(long long k)
161 {
162     precomputePaths();
163     int v = 0;
164     string ans = "";
165     while (k > 0)
166     {
167         for (auto &edge : st[v].next)
168         {
169             int w = edge.second;
170             if (st[w].paths >= k)
171             {
172                 ans += edge.first;
173                 k -= st[w].cnt;
174                 v = w;
175                 break;
176             }
177             else
178             {
179                 k -= st[w].paths;
180             }
181         }
182     }
183     return ans;
184 }
185
186 static string smallestCyclicShift(string S)
187 {
188     SuffixAutomaton sa;
189     sa.build(S + S);
190     int v = 0;
191     string ans = "";
192     for (int i = 0; i < S.size(); i++)
193     {
194         auto edge = *sa.st[v].next.begin();
195         ans += edge.first;
196         v = edge.second;
197     }
198     return ans;
199 }
200
201 int numberOfOccurrences(string P)
202 {
203     int v = 0;
204     for (char c : P)
205     {
206         if (!st[v].next.count(c))
207             return 0;
208         v = st[v].next[c];

```

```

209     }
210     return st[v].cnt;
211 }
212
213 int firstOccurrencePosition(string P)
214 {
215     int v = 0;
216     for (char c : P)
217     {
218         if (!st[v].next.count(c))
219             return -1;
220         v = st[v].next[c];
221     }
222     return st[v].first_pos - P.length() + 1;
223 }
224
225 void dfsOccurrences(int v, int P_length, vector<int> &res)
226 {
227     if (!st[v].is_clone)
228     {
229         res.push_back(st[v].first_pos - P_length + 1);
230     }
231     for (int u : st[v].inv_link)
232     {
233         dfsOccurrences(u, P_length, res);
234     }
235 }
236
237 vector<int> allOccurrencePositions(string P)
238 {
239     int v = 0;
240     for (char c : P)
241     {
242         if (!st[v].next.count(c))
243             return {};
244         v = st[v].next[c];
245     }
246     vector<int> res;
247     dfsOccurrences(v, P.length(), res);
248     sort(res.begin(), res.end());
249     return res;
250 }
251
252 string shortestNonAppearingString(char min_char = 'a', char max_char = 'z')
253 {
254     vector<int> d(sz, -1);
255     auto dfs = [&](auto &self, int v) -> int
256     {
257         if (d[v] != -1)
258             return d[v];
259         d[v] = 1e9;
260         for (char c = min_char; c <= max_char; c++)
261         {
262             if (!st[v].next.count(c))
263

```

```

264             d[v] = 1;
265             break;
266         }
267         else
268         {
269             d[v] = min(d[v], 1 + self(self, st[v].next[c]));
270         }
271     }
272     return d[v];
273 };
274 dfs(dfs, 0);
275 string ans = "";
276 int v = 0;
277 while (true)
278 {
279     for (char c = min_char; c <= max_char; c++)
280     {
281         if (!st[v].next.count(c))
282         {
283             ans += c;
284             return ans;
285         }
286     }
287     for (char c = min_char; c <= max_char; c++)
288     {
289         if (d[v] == 1 + d[st[v].next[c]])
290         {
291             ans += c;
292             v = st[v].next[c];
293             break;
294         }
295     }
296 }
297
298
299 string longestCommonSubstringTwo(string T)
300 {
301     int v = 0, l = 0, best = 0, bestpos = 0;
302     for (int i = 0; i < T.size(); i++)
303     {
304         while (v && !st[v].next.count(T[i]))
305         {
306             v = st[v].link;
307             l = st[v].len;
308         }
309         if (st[v].next.count(T[i]))
310         {
311             v = st[v].next[T[i]];
312             l++;
313         }
314         if (l > best)
315         {
316             best = l;
317             bestpos = i;
318

```

```

319     }
320     return T.substr(bestpos - best + 1, best);
321 }
322
323 static string longestCommonSubstringMultiple(vector<string> strings)
324 {
325     if (strings.empty())
326         return "";
327     if (strings.size() == 1)
328         return strings[0];
329
330     int min_idx = 0;
331     for (int i = 1; i < strings.size(); i++)
332     {
333         if (strings[i].size() < strings[min_idx].size())
334         {
335             min_idx = i;
336         }
337     }
338     swap(strings[0], strings[min_idx]);
339
340     SuffixAutomaton sa;
341     sa.build(strings[0]);
342
343     vector<int> order(sa.sz);
344     iota(order.begin(), order.end(), 0);
345     sort(order.begin(), order.end(), [&](int a, int b)
346         { return sa.st[a].len > sa.st[b].len; });
347
348     vector<int> min_max_l(sa.sz);
349     for (int i = 0; i < sa.sz; i++)
350     {
351         min_max_l[i] = sa.st[i].len;
352     }
353
354     for (int i = 1; i < strings.size(); i++)
355     {
356         vector<int> max_l(sa.sz, 0);
357         int v = 0, l = 0;
358         for (char c : strings[i])
359         {
360             while (v > 0 && !sa.st[v].next.count(c))
361             {
362                 v = sa.st[v].link;
363                 l = sa.st[v].len;
364             }
365             if (sa.st[v].next.count(c))
366             {
367                 v = sa.st[v].next[c];
368                 l++;
369             }
370             max_l[v] = max(max_l[v], l);
371         }
372
373         for (int u : order)

```

```

374     {
375         min_max_l[u] = min(min_max_l[u], max_l[u]);
376         int lnk = sa.st[u].link;
377         if (lnk != -1)
378         {
379             max_l[lnk] = max(max_l[lnk], min(max_l[u], sa.st[lnk].len));
380         }
381     }
382 }
383
384 int best_len = 0;
385 int best_v = 0;
386 for (int i = 1; i < sa.sz; i++)
387 {
388     if (min_max_l[i] > best_len)
389     {
390         best_len = min_max_l[i];
391         best_v = i;
392     }
393 }
394 if (best_len == 0)
395     return "";
396 return strings[0].substr(sa.st[best_v].first_pos - best_len + 1, best_len);
397 }
398 };

```

9 DP

9.1 DP Bitmask

9.1.1 Bitmask

```

1 int n;
2 vector<vector<int>> v;
3 int dp[22][1 << 22];
4
5 int sol(int man, int mask)
6 {
7     if (man >= n)
8         return 1;
9     auto &ret = dp[man][mask];
10    if (~ret)
11        return ret;
12    ret = 0;
13    for (int i = 0; i < n; i++)
14    {
15        if (mask >> i & 1)
16            continue;
17        if (v[man][i])
18        {
19            ret += sol(man + 1, mask | 1 << i);

```

```

20         ret %= MOD;
21     }
22 }
23 return ret;
24 }

```

9.1.2 SOS

```

1 vector<int> SOS(int n, vector<int> &a)
2 {
3     vector<int> p = a;
4     for (int dim = 0; dim < n; dim++)
5     {
6         for (int i = 0; i < 1 << n; i++)
7         {
8             if (i >> dim & 1)
9                 p[i] += p[i - (1 << dim)];
10        }
11    }
12    return p;
13 }

```

9.1.3 Submask

```

1 int dp[1<<16];
2 int cost[1<<16];
3 int memo (int mask) {
4     if (mask == 0) return 0;
5     auto &ret = dp[mask];
6     if (~ret) return ret;
7     for (int i = mask; i > 0; i = (i-1) & mask) {
8         ret = max(ret, cost[i] + memo(mask ^ i));
9     }
10    return ret;
11 }
12 void hassan() {
13     int n; cin >> n;
14     int a[n][n];
15     for (int i = 0; i < n; i++) {
16         for (int j = 0; j < n; j++) {
17             cin >> a[i][j];
18         }
19     }
20     for (int mask = 0; mask < (1<<n); mask++) {
21         for (int i = 0; i < n; i++) {
22             if (mask & (1<<i)) {
23                 for (int j = 0; j < i; j++) {
24                     if (mask & (1<<j)) {
25                         cost[mask] += a[i][j];
26                     }
27                 }
28             }
29         }
30     }
31 }

```

```

29     }
30     // cout << mask << " >> " << cost[mask] << endl;
31 }
32 memset(dp, -1, sizeof(dp));
33 cout << memo((1<<n)-1);
34
35 }

```

9.1.4 subset convolution

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SUBSET CONVOLUTION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Disjoint subset sum", " $i \text{ OR } j = k \text{ AND } i \text{ AND } j = 0$ ".
6  * - Classic Scenarios: Combining two DP states where the masks must perfectly
7  *   interlock (share no common set bits) to form a target mask.
8  * - The Magic: By adding an extra dimension for the `__builtin_popcount`, we
9  *   guarantee that if two masks combine to form a mask of size  $K$ , their
10  *   individual
11  *   set bits must sum to exactly  $K$ , implicitly enforcing that they are disjoint.
12  *   It runs SOS DP on each popcount layer.
13  *
14  * 2. HOW TO USE IT (THE BLACK BOX)
15  * - Query:
16  *   vector<long long> C = subset_convolution(A, B, N, MOD);
17  * - Complexity:
18  *   Time:  $O(N^2 * 2^N)$ 
19  *   Space:  $O(N * 2^N)$ 
20  */
21
22 using ll = long long;
23
24 vector<ll> subset_convolution(const vector<ll> &a, const vector<ll> &b, int n, ll
    mod)
25 {
26     vector<vector<ll>> f_hat(n + 1, vector<ll>(1 << n, 0));
27     vector<vector<ll>> g_hat(n + 1, vector<ll>(1 << n, 0));
28     vector<vector<ll>> h_hat(n + 1, vector<ll>(1 << n, 0));
29
30     for (int mask = 0; mask < (1 << n); mask++)
31     {
32         int pc = __builtin_popcount(mask);
33         f_hat[pc][mask] = a[mask] % mod;
34         g_hat[pc][mask] = b[mask] % mod;
35     }
36
37     for (int i = 0; i <= n; i++)
38     {
39         for (int j = 0; j < n; j++)
40         {
41             for (int mask = 0; mask < (1 << n); mask++)

```

```

42     {
43         if (mask & (1 << j))
44         {
45             f_hat[i][mask] = (f_hat[i][mask] + f_hat[i][mask ^ (1 << j)])
46                 % mod;
47             g_hat[i][mask] = (g_hat[i][mask] + g_hat[i][mask ^ (1 << j)])
48                 % mod;
49         }
50     }
51 }
52 for (int mask = 0; mask < (1 << n); mask++)
53 {
54     for (int i = 0; i <= n; i++)
55     {
56         ll sum = 0;
57         for (int j = 0; j <= i; j++)
58         {
59             sum = (sum + f_hat[j][mask] * g_hat[i - j][mask]) % mod;
60         }
61         h_hat[i][mask] = sum;
62     }
63 }
64
65 for (int i = 0; i <= n; i++)
66 {
67     for (int j = 0; j < n; j++)
68     {
69         for (int mask = 0; mask < (1 << n); mask++)
70         {
71             if (mask & (1 << j))
72             {
73                 h_hat[i][mask] = (h_hat[i][mask] - h_hat[i][mask ^ (1 << j)]
74                     + mod) % mod;
75             }
76         }
77     }
78 }
79 vector<ll> c(1 << n);
80 for (int mask = 0; mask < (1 << n); mask++)
81 {
82     c[mask] = h_hat[__builtin_popcount(mask)][mask];
83 }
84 return c;
85 }

```

9.2 Digit Dp

9.2.1 main

```
1 string s;
```

```

2 int d, k;
3 int dp[20][20][2][2];
4
5 int sol(int idx, int cnt, int tight, int num)
6 {
7     if (idx >= int(s.size()))
8     {
9         if (cnt == k)
10             return 1;
11         return 0;
12     }
13     auto &ret = dp[idx][cnt][tight][num];
14     if (~ret)
15         return ret;
16     ret = 0;
17     int limit = tight ? s[idx] - '0' : 9;
18     for (int i = 0; i <= limit; i++)
19     {
20         int nn = num || i;
21         int nt = tight && i == limit;
22         int nc = cnt + (d == i && nn);
23         ret += sol(idx + 1, nc, nt, nn);
24     }
25     return ret;
26 }
27
28 void solve()
29 {
30     cin >> s >> d >> k;
31     memset(dp, -1, sizeof dp);
32     cout << sol(0, 0, 1, 0);
33 }

```

9.3 Iterative

9.3.1 main

```

1 int answer(int n)
2 {
3     if (n == 0 || n == 1)
4         return n;
5     return dp[n];
6 }
7
8 void solve()
9 {
10     int n;
11     cin >> n;
12     for (int i = 2; i <= n; i++)
13         dp[i] = answer(i - 1) + answer(i - 2);
14     cout << answer(n);
15 }

```

9.4 Knapsack

9.4.1 main

```
1 int n, W;
2 int dp[1001][31];
3 vector<pair<int, int>> a;
4
5 int sol(int i, int w)
6 {
7     if (i >= n)
8         return 0;
9     auto &ret = dp[i][w];
10    if (~ret)
11        return ret;
12    ret = sol(i + 1, w);
13    if (w + a[i].second <= W)
14        ret = max(ret, a[i].first + sol(i + 1, w + a[i].second));
15    return ret;
16 }
```

9.5 LCS

9.5.1 LCS

```
1 vector<vector<int>> dp;
2 int n1, n2;
3 string s1, s2;
4
5 int LCS(int i1, int i2)
6 {
7     if (i1 >= n1 || i2 >= n2)
8         return 0;
9     auto &ret = dp[i1][i2];
10    if (ret != -1)
11        return ret;
12    int choice1 = LCS(i1 + 1, i2);
13    int choice2 = LCS(i1, i2 + 1);
14    int choice3{};
15    if (s1[i1] == s2[i2])
16        choice3 = 1 + LCS(i1 + 1, i2 + 1);
17    return ret = max({choice1, choice2, choice3});
18 }
```

9.6 LIS

9.6.1 Basic 2D

```
1 int dp1[2010][2010];
2 int dp2[2010][2010];
3
```

```
4 int LDS(int i, int prev)
5 {
6     if (i > n)
7         return 0;
8     auto &ret = dp1[i][prev];
9     if (~ret)
10        return ret;
11    ret = LDS(i + 1, prev);
12    if (prev == 0 || a[i] < a[prev])
13        ret = max(ret, 1 + LDS(i + 1, i));
14    return ret;
15 }
16
17 int LIS(int i, int prev)
18 {
19     if (i > n)
20         return 0;
21     auto &ret = dp2[i][prev];
22     if (~ret)
23         return ret;
24    ret = LIS(i + 1, prev);
25    if (prev == 0 || a[i] > a[prev])
26        ret = max(ret, 1 + LIS(i + 1, i));
27    return ret;
28 }
```

9.6.2 LIS1D

```
1 int LIS(int i)
2 {
3     if (i > n)
4         return 0;
5     auto &ret = dp[i];
6     if (dp[i] == -1)
7         return ret;
8     ret = 0;
9     for (int j = i + 1; j <= n; j++)
10    {
11        if (v[j] > v[i])
12            ret = max(ret, LIS(j));
13    }
14    ret++;
15    return ret;
16 }
17
18 // int res{};
19 // for (int i = 1; i <= n; i++)
20 // {
21 //     int num = LIS(i);
22 //     res = max(num, res);
23 // }
```

9.6.3 LIS BS

```
1 vector<int> LIS;
2 for (int i = 0; i < n; i++)
3 {
4     auto it = lower_bound(LIS.begin(), LIS.end(), ans[i]);
5     if (it == LIS.end())
6         LIS.push_back(ans[i]);
7     else
8         *it = ans[i];
9 }
```

9.6.4 LIS seg

```
1 struct Node
2 {
3     int val;
4
5     Node()
6     {
7         val = 0;
8     }
9
10    Node(int val)
11    : val(val)
12    {
13    }
14
15    void change(int x)
16    {
17        val = max(x, val);
18    }
19 };
20
21 struct segmentTree
22 {
23     int sz;
24     vector<Node> seg;
25
26     segmentTree(int n)
27     {
28         sz = 1;
29         while (sz < n)
30             sz *= 2;
31         seg.assign(2 * sz, Node());
32     }
33
34     void init(vector<int> &arr, int node, int lx, int rx)
35     {
36         if (rx - lx == 1)
37         {
38             if (lx < int(arr.size()))
39                 seg[node] = Node(arr[lx]);
```

```
40         return;
41     }
42     int mid = (lx + rx) / 2;
43     init(arr, 2 * node + 1, lx, mid);
44     init(arr, 2 * node + 2, mid, rx);
45     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
46 }
47
48 void init(vector<int> &arr)
49 {
50     init(arr, 0, 0, sz);
51 }
52
53 Node merge(Node &lf, Node &rt)
54 {
55     Node ans = Node();
56     ans.val = max(lf.val, rt.val);
57     return ans;
58 }
59
60 void set(int idx, int val, int node, int lx, int rx)
61 {
62     if (rx - lx == 1)
63     {
64         seg[node].change(val);
65         return;
66     }
67     int mid = (lx + rx) / 2;
68     if (idx < mid)
69         set(idx, val, 2 * node + 1, lx, mid);
70     else
71         set(idx, val, 2 * node + 2, mid, rx);
72     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
73 }
74
75 void set(int idx, int val)
76 {
77     set(idx, val, 0, 0, sz);
78 }
79
80 Node get(int l, int r, int node, int lx, int rx)
81 {
82     if (lx >= l && rx <= r)
83         return seg[node];
84     if (rx <= l || lx >= r)
85         return Node();
86     int mid = (lx + rx) / 2;
87     Node lf = get(l, r, 2 * node + 1, lx, mid);
88     Node ri = get(l, r, 2 * node + 2, mid, rx);
89     return merge(lf, ri);
90 }
91
92 int get(int l, int r)
93 {
94     return get(l, r, 0, 0, sz).val;
```

```

95     }
96 };
97
98 int LIS(int n, vector<int> &p)
99 {
100     segmentTree seg(n + 5);
101     vector<int> dp(n + 1);
102     int ans{};
103     for (int i = 0; i < n; i++)
104     {
105         dp[i] = seg.get(0, p[i]) + 1;
106         seg.set(p[i], dp[i]);
107         ans = max(ans, dp[i]);
108     }
109     return ans;
110 }

```

9.7 Lichao tree

9.7.1 lichao tree

```

1  /**
2   *  THE ULTIMATE CP TEMPLATE BLUEPRINT: LI CHAO TREE (EXTENDED)
3   *
4   *  1.  WHAT PROBLEM DOES THIS SOLVE?
5   *  - Keywords: "Minimum of lines", "Linear cost over a range", " $y = mx + c$ ".
6   *  - Classic Scenarios: You need to minimize a linear DP transition cost, but
7   *  certain transitions/lines are only valid within a specific interval  $[L, R]$ .
8   *  - The Magic: A sparse segment tree where each node holds the line that
9   *  provides the optimal value at the midpoint of its interval. It dynamically
10  *  allocates memory (sparse) so it can comfortably handle  $x$ -coordinates up to
11  *   $10^{18}$ .
12  *
13  *  2.  HOW TO USE IT (THE BLACK BOX)
14  *  - Initialization:
15  *      LiChaoTree lct(-1e18, 1e18); // Pass the minimum and maximum possible  $x$ 
16  *      queries
17  *  - Operations:
18  *      lct.add_line(M, C);           // Adds an infinite line  $y = Mx + C$ 
19  *      lct.add_segment(M, C, L, R); // Adds line  $y = Mx + C$  strictly
20  *      for  $x$  in  $[L, R]$ 
21  *      long long min_y = lct.query(X); // Gets the minimum  $y$  at point  $X$ 
22  *
23  *  3.  HOW TO ADAPT IT (THE DIALS & SWITCHES)
24  *  - Maximization: This template is strictly built to find the MINIMUM value.
25  *  If you need the MAXIMUM, insert your lines with inverted signs:
26  *  `lct.add_line(-M, -C)` and then negate the result:  $-lct.query(X)$ .
27  */
28
29 using ll = long long;
30 const ll INF = 4e18; // Safe upper bound to prevent overflow during additions
31
32 struct Line

```

```

30 {
31     ll m, c;
32     ll eval(ll x) const { return m * x + c; }
33 };
34
35 struct Node
36 {
37     Line line;
38     int lc = -1, rc = -1;
39     Node() : line({0, INF}) {}
40     Node(Line l) : line(l) {}
41 };
42
43 struct LiChaoTree
44 {
45     vector<Node> tree;
46     ll MIN_X, MAX_X;
47
48     LiChaoTree(ll min_x, ll max_x) : MIN_X(min_x), MAX_X(max_x)
49     {
50         tree.emplace_back();
51     }
52
53     void add_line(Line nw, int u, ll l, ll r)
54     {
55         ll mid = l + (r - l) / 2;
56         bool left = nw.eval(l) < tree[u].line.eval(l);
57         bool middle = nw.eval(mid) < tree[u].line.eval(mid);
58
59         if (middle)
60             swap(tree[u].line, nw);
61         if (l == r)
62             return;
63
64         if (left != middle)
65         {
66             if (tree[u].lc == -1)
67             {
68                 tree[u].lc = tree.size();
69                 tree.emplace_back();
70             }
71             add_line(nw, tree[u].lc, l, mid);
72         }
73         else
74         {
75             if (tree[u].rc == -1)
76             {
77                 tree[u].rc = tree.size();
78                 tree.emplace_back();
79             }
80             add_line(nw, tree[u].rc, mid + 1, r);
81         }
82     }
83
84     void add_line(ll m, ll c)

```

```

85 {
86     add_line({m, c}, 0, MIN_X, MAX_X);
87 }
88
89 void add_segment(Line nw, ll ql, ll qr, int u, ll l, ll r)
90 {
91     if (ql <= l && r <= qr)
92     {
93         add_line(nw, u, l, r);
94         return;
95     }
96     ll mid = l + (r - l) / 2;
97     if (ql <= mid)
98     {
99         if (tree[u].lc == -1)
100         {
101             tree[u].lc = tree.size();
102             tree.emplace_back();
103         }
104         add_segment(nw, ql, qr, tree[u].lc, l, mid);
105     }
106     if (qr > mid)
107     {
108         if (tree[u].rc == -1)
109         {
110             tree[u].rc = tree.size();
111             tree.emplace_back();
112         }
113         add_segment(nw, ql, qr, tree[u].rc, mid + 1, r);
114     }
115 }
116
117 void add_segment(ll m, ll c, ll ql, ll qr)
118 {
119     add_segment({m, c}, ql, qr, 0, MIN_X, MAX_X);
120 }
121
122 ll query(ll x, int u, ll l, ll r)
123 {
124     if (u == -1)
125         return INF;
126     ll res = tree[u].line.eval(x);
127     if (l == r)
128         return res;
129
130     ll mid = l + (r - l) / 2;
131     if (x <= mid)
132         res = min(res, query(x, tree[u].lc, l, mid));
133     else
134         res = min(res, query(x, tree[u].rc, mid + 1, r));
135
136     return res;
137 }
138
139 ll query(ll x)

```

```

140 {
141     return query(x, 0, MIN_X, MAX_X);
142 }
143 };

```

9.8 Misc

9.8.1 Broken profile

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: BROKEN PROFILE DP (PLUG DP)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Tile the grid", "Dominoes", "Hamiltonian Path on Grid".
6  * - Classic Scenarios: Find the number of ways to completely cover an  $N \times M$  grid
7  *   using  $1 \times 2$  dominoes where  $N$  is huge ( $10^5$ ) and  $M$  is very small ( $\leq 15$ ).
8  * - The Magic: It sweeps through the grid cell by cell (not row by row).
9  *   It maintains a bitmask of length  $M$  representing the "profile" (the boundary
10  *   of cells protruding into the current row). As we move to the next cell, we
11  *   shift the mask and try placing dominoes horizontally or vertically.
12  *
13  * 2. HOW TO USE IT (THE BLACK BOX)
14  * - Query:
15  *   long long ways = plug_dp_dominoes(N, M, MOD);
16  *
17  * - Complexity:
18  *   Time:  $O(N * M * 2^M)$ 
19  *   Space:  $O(2^M)$  using rolling arrays.
20  *
21  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22  * - Obstacles: If the grid has blocked cells, simply check if the current cell
23  *   ( $i, j$ ) is blocked. If it is, ensure the current bit in the mask is 0, shift
24  *   it to 1 (meaning it's "filled"), and move to the next cell without placing
25  *   dominoes.
26  */
27 using ll = long long;
28
29 ll plug_dp_dominoes(int n, int m, ll mod)
30 {
31     if ((n * m) % 2 != 0)
32         return 0; // Odd number of cells cannot be tiled by  $1 \times 2$ 
33
34     // We only need the current and next state, so we use a rolling array of size
35     // 2
36     vector<vector<ll>> dp(2, vector<ll>(1 << m, 0));
37
38     int cur = 0, next = 1;
39     dp[cur][0] = 1; // Base case: completely empty profile
40
41     for (int i = 0; i < n; i++)
42     {

```

```

42 for (int j = 0; j < m; j++)
43 {
44     fill(dp[next].begin(), dp[next].end(), 0);
45
46     for (int mask = 0; mask < (1 << m); mask++)
47     {
48         if (dp[cur][mask] == 0)
49             continue;
50
51         // Case 1: The current cell is already filled by a vertical
52         // domino
53         // from the previous row. We must leave it alone. The bit shifts
54         // out.
55         if (mask & (1 << (m - 1)))
56         {
57             int next_mask = (mask ^ (1 << (m - 1))) << 1;
58             dp[next][next_mask] = (dp[next][next_mask] + dp[cur][mask]) %
59             mod;
60         }
61         // Case 2: The cell is empty. We must place a domino.
62         else
63         {
64             // Option A: Place a vertical domino (protrudes into the next
65             // row).
66             // We set the 0-th bit of the shifted mask to 1.
67             int next_mask_vert = (mask << 1) | 1;
68             dp[next][next_mask_vert] = (dp[next][next_mask_vert] + dp[cur]
69             ][mask]) % mod;
70
71             // Option B: Place a horizontal domino.
72             // Only valid if we aren't at the right edge AND the next
73             // cell is empty.
74             if (j + 1 < m && !(mask & 1))
75             {
76                 // The horizontal domino fills the current and next cell
77                 // in THIS row.
78                 // We shift the mask by 1, but we don't protrude into the
79                 // next row,
80                 // so the 0-th bit remains 0. However, the cell to our
81                 // right is now filled,
82                 // which we must account for in our transition logic by
83                 // skipping it or
84                 // setting its bit. In this cell-by-cell state machine,
85                 // placing a horizontal
86                 // domino effectively sets the bit for the *next* cell in
87                 // the current row.
88                 int next_mask_horiz = (mask << 1) | 2;
89                 dp[next][next_mask_horiz] = (dp[next][next_mask_horiz] +
90                 dp[cur][mask]) % mod;
91             }
92         }
93     }
94     swap(cur, next);
95 }
96 }

```

```

84
85 // The answer is the number of ways to have a completely flat profile (mask
86 // == 0) at the end.
87 return dp[cur][0];
88 }

```

9.9 More Tricks

9.9.1 Slope trick

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SLOPE TRICK
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Minimum operations to make array non-decreasing".
6  * - Classic Scenarios: You can increment or decrement elements of an array.
7  * Find the minimum total cost to make the array sorted (non-decreasing).
8  * - The Magic: A 2D DP takes  $O(N * MAX\_VAL)$  which TLEs. By observing that the
9  * DP cost function is convex and piecewise linear, we can just maintain the
10  * points where the slope changes using a max-heap.
11  *
12  * 2. HOW TO USE IT (THE BLACK BOX)
13  * - Query:
14  *     long long min_cost = make_non_decreasing(A);
15  *
16  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
17  * - Strictly Increasing: If the problem requires strictly increasing ( $A[i] < A[i$ 
18  * +1]),
19  * subtract `i` from every element initially:  $A[i] -= i$ , then run this exact
20  * function. It maps the strictly increasing problem directly to a non-
21  * decreasing one.
22  *
23  * - Complexity:
24  *     Time:  $O(N \log N)$ 
25  *     Space:  $O(N)$ 
26  */
27 using ll = long long;
28 ll make_non_decreasing(const vector<ll> &a)
29 {
30     priority_queue<ll> pq;
31     ll total_cost = 0;
32
33     for (int i = 0; i < a.size(); i++)
34     {
35         pq.push(a[i]);
36
37         // If the max element so far is strictly greater than the current element
38         // ,
39         // it means we have a slope violation. We "flatten" it to match  $A[i]$ .
40         if (pq.top() > a[i])
41         {

```

```

41         total_cost += pq.top() - a[i];
42         pq.pop();
43         pq.push(a[i]);
44     }
45 }
46
47 return total_cost;
48 }

```

9.10 Optimization

9.10.1 1D1D

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: 1D1D OPTIMIZATION (LINE CONTAINER)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Convex Hull Trick", "y = mx + c", "Li Chao Tree alternative".
6  * - Classic Scenarios: dp[i] = min_{j < i} (dp[j] + M[j] * X[i] + C[j]).
7  * - The Magic: Maintains the upper/lower envelope of a set of lines. It supports
8  *   adding lines dynamically and querying the extreme value at any X in O(log N)
9  *   .
10  * This specific implementation (KACTL's LineContainer) handles lines added
11  * with NON-MONOTONIC slopes, making it universally safe.
12  *
13  * 2. HOW TO USE IT (THE BLACK BOX)
14  * - Initialization:
15  *   LineContainer cht;
16  * - Operations:
17  *   cht.add(m, c); // Adds line y = mx + c
18  *   ll max_y = cht.query(x);
19  *
20  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
21  * - Minimization: This natively computes the MAXIMUM value at x. If you need the
22  *   MINIMUM value, insert lines with inverted signs: `cht.add(-m, -c)` and
23  *   negate the queried result: `-cht.query(x)`.
24  */
25 using ll = long long;
26
27 struct Line
28 {
29     mutable ll k, m, p;
30     bool operator<(const Line &o) const { return k < o.k; }
31     bool operator<(ll x) const { return p < x; }
32 };
33
34 struct LineContainer : multiset<Line, less<>>
35 {
36     static const ll inf = LLONG_MAX;
37     ll div(ll a, ll b)

```

```

38 {
39     return a / b - ((a ^ b) < 0 && a % b);
40 }
41 bool isect(iterator x, iterator y)
42 {
43     if (y == end())
44         return x->p = inf, 0;
45     if (x->k == y->k)
46         x->p = x->m > y->m ? inf : -inf;
47     else
48         x->p = div(y->m - x->m, x->k - y->k);
49     return x->p >= y->p;
50 }
51 void add(ll k, ll m)
52 {
53     auto z = insert({k, m, 0}), y = z++, x = y;
54     while (isect(y, z))
55         z = erase(z);
56     if (x != begin() && isect(--x, y))
57         isect(x, y = erase(y));
58     while ((y = x) != begin() && (--x)->p >= y->p)
59         isect(x, erase(y));
60 }
61 ll query(ll x)
62 {
63     assert(!empty());
64     auto l = *lower_bound(x);
65     return l.k * x + l.m;
66 }
67 };

```

9.10.2 Aliens

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define El_Hefnawys \
6 ios_base::sync_with_stdio(false); \
7 cin.tie(nullptr); \
8 cout.tie(nullptr)
9
10 typedef long long ll;
11 #define int ll
12
13 const int OO = 1e15;
14
15 // choose k subarrays from the array and trying to maximize the sum
16 // dp[i][0/1]
17
18 // The DP function returns a pair:
19 // 1. The maximum sum (including the penalties applied)
20 // 2. The NEGATIVE number of subarrays used (to break ties by preferring fewer
    subarrays)

```

```

21 pair<int, int> check(int penalty, int n, const vector<int> &a)
22 {
23     // dp[i][0] = max sum up to index i, currently NOT in a subarray
24     // dp[i][1] = max sum up to index i, currently INSIDE a subarray
25     vector<vector<pair<int, int>>> dp(n, vector<pair<int, int>>(2));
26
27     dp[0][0] = {0, 0};
28     dp[0][1] = {a[0] - penalty, -1}; // Starting an array costs 'penalty'
29
30     for (int i = 1; i < n; i++)
31     {
32         // Option 0: We are not in a subarray.
33         // We either were already not in one, or we just closed the previous one
34         dp[i][0] = max(dp[i - 1][0], dp[i - 1][1]);
35
36         // Option 1: We are inside a subarray.
37         // Transition A: Start a brand NEW subarray today (costs penalty, adds
38         // to count)
39         pair<int, int> start_new = make_pair(
40             dp[i - 1][0].first - penalty + a[i],
41             dp[i - 1][0].second - 1);
42
43         // Transition B: Extend the EXISTING subarray (no penalty, count doesn't
44         // change)
45         pair<int, int> extend_existing = make_pair(
46             dp[i - 1][1].first + a[i],
47             dp[i - 1][1].second);
48
49         dp[i][1] = max(start_new, extend_existing);
50     }
51
52     return max(dp[n - 1][0], dp[n - 1][1]);
53 }
54
55 void solve()
56 {
57     int n, k;
58     if (!(cin >> n >> k))
59         return;
60
61     vector<int> a(n);
62     for (int i = 0; i < n; i++)
63     {
64         cin >> a[i];
65     }
66
67     // The penalty bounds. Can be negative depending on the problem!
68     int l = -1e14, r = 1e14;
69     int best_ans = 0;
70
71     while (l <= r)
72     {
73         int mid_penalty = l + (r - l) / 2;
74
75         pair<int, int> res = check(mid_penalty, n, a);

```

```

74         int arrays_used = -res.second; // Flip back to positive
75
76         // If we used at least K arrays, this penalty is valid to consider.
77         // We might have tied and used exactly K, or more.
78         if (arrays_used >= k)
79         {
80             // Restore the true sum by adding back the penalty for EXACTLY K
81             // arrays.
82             // Even if the DP used more than K due to ties (0-sum arrays),
83             // the mathematical projection at the hull tangent guarantees this
84             // formula is exact!
85             best_ans = res.first + (k * mid_penalty);
86
87             // Try to force it to use FEWER arrays by INCREASING the penalty
88             l = mid_penalty + 1;
89         }
90         else
91         {
92             // We used too few arrays. The penalty is too harsh. Decrease it.
93             r = mid_penalty - 1;
94         }
95     }
96
97     cout << best_ans << "\n";
98 }
99
100 signed main()
101 {
102     El_Hefnawys;
103     int t{1};
104     // cin >> t;
105     while (t--)
106     {
107         solve();
108     }
109     return 0;
110 }

```

9.10.3 DC

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DIVIDE AND CONQUER OPTIMIZATION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Partition array into K contiguous segments", "Minimize maximum
6  *   cost".
7  * - Classic Scenarios:  $dp[i][j] = \min_{k < j} (dp[i-1][k] + cost(k, j))$ .
8  * - The Magic: If the optimal split point opt(i, j) satisfies the condition
9  *    $opt(i, j) \leq opt(i, j+1)$ , we don't need to check all k. We recursively
10  *   compute the middle state and restrict the search bounds for the left and
11  *   right halves, dropping  $O(K * N^2)$  to  $O(K * N \log N)$ .
12  *
13  * 2. HOW TO USE IT (THE BLACK BOX)

```

```

13 * - Setup: You need two 1D arrays: `dp_prev` (results from i-1 segments)
14 *   and `dp_cur` (results for i segments).
15 * - Query:
16 *     for (int i = 2; i <= K; i++) {
17 *         compute_dc(1, N, 1, N, dp_prev, dp_cur);
18 *         dp_prev = dp_cur;
19 *     }
20 */
21
22 using ll = long long;
23 const ll INF = 1e18;
24
25 // Replace this with your actual O(1) cost function (usually via prefix sums)
26 ll cost(int l, int r)
27 {
28     if (l > r)
29         return 0;
30     return 0; // return actual cost
31 }
32
33 void compute_dc(int l, int r, int opt_l, int opt_r, const vector<ll> &dp_prev,
34                 vector<ll> &dp_cur)
35 {
36     if (l > r)
37         return;
38     int mid = l + (r - l) / 2;
39     pair<ll, int> best = {INF, -1};
40     for (int k = opt_l; k <= min(mid, opt_r); k++)
41     {
42         ll current_cost = dp_prev[k - 1] + cost(k, mid);
43         if (current_cost < best.first)
44         {
45             best = {current_cost, k};
46         }
47     }
48     dp_cur[mid] = best.first;
49     int opt = best.second;
50
51     compute_dc(l, mid - 1, opt_l, opt, dp_prev, dp_cur);
52     compute_dc(mid + 1, r, opt, opt_r, dp_prev, dp_cur);
53 }
54 }

```

9.10.4 Knuth

```

1 /**
2 * THE ULTIMATE CP TEMPLATE BLUEPRINT: DIVIDE AND CONQUER OPTIMIZATION
3 *
4 * 1. WHAT PROBLEM DOES THIS SOLVE?
5 * - Keywords: "Partition array into K contiguous segments", "Minimize maximum
6 *   cost".
7 * - Classic Scenarios:  $dp[i][j] = \min_{k < j} (dp[i-1][k] + cost(k, j))$ .

```

```

7 * - The Magic: If the optimal split point `opt(i, j)` satisfies the condition
8 *    $opt(i, j) \leq opt(i, j+1)$ , we don't need to check all `k`. We recursively
9 *   compute the middle state and restrict the search bounds for the left and
10 *   right halves, dropping  $O(K * N^2)$  to  $O(K * N \log N)$ .
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Setup: You need two 1D arrays: `dp_prev` (results from i-1 segments)
14 *   and `dp_cur` (results for i segments).
15 * - Query:
16 *     for (int i = 2; i <= K; i++) {
17 *         compute_dc(1, N, 1, N, dp_prev, dp_cur);
18 *         dp_prev = dp_cur;
19 *     }
20 */
21
22 using ll = long long;
23 const ll INF = 1e18;
24
25 // Replace this with your actual O(1) cost function (usually via prefix sums)
26 ll cost(int l, int r)
27 {
28     if (l > r)
29         return 0;
30     return 0; // return actual cost
31 }
32
33 void compute_dc(int l, int r, int opt_l, int opt_r, const vector<ll> &dp_prev,
34                 vector<ll> &dp_cur)
35 {
36     if (l > r)
37         return;
38     int mid = l + (r - l) / 2;
39     pair<ll, int> best = {INF, -1};
40     for (int k = opt_l; k <= min(mid, opt_r); k++)
41     {
42         ll current_cost = dp_prev[k - 1] + cost(k, mid);
43         if (current_cost < best.first)
44         {
45             best = {current_cost, k};
46         }
47     }
48     dp_cur[mid] = best.first;
49     int opt = best.second;
50
51     compute_dc(l, mid - 1, opt_l, opt, dp_prev, dp_cur);
52     compute_dc(mid + 1, r, opt, opt_r, dp_prev, dp_cur);
53 }
54 }

```

9.10.5 Using data structures

```

1 struct Node

```

```

2 {
3     int val;
4
5     Node()
6     {
7         val = 0;
8     }
9
10    Node(int val)
11    : val(val)
12    {
13    }
14
15    void change(int x)
16    {
17        val = max(x, val);
18    }
19 };
20
21 struct segmentTree
22 {
23     int sz;
24     vector<Node> seg;
25
26     segmentTree(int n)
27     {
28         sz = 1;
29         while (sz < n)
30             sz *= 2;
31         seg.assign(2 * sz, Node());
32     }
33
34     void init(vector<int> &arr, int node, int lx, int rx)
35     {
36         if (rx - lx == 1)
37         {
38             if (lx < int(arr.size()))
39                 seg[node] = Node(arr[lx]);
40             return;
41         }
42         int mid = (lx + rx) / 2;
43         init(arr, 2 * node + 1, lx, mid);
44         init(arr, 2 * node + 2, mid, rx);
45         seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
46     }
47
48     void init(vector<int> &arr)
49     {
50         init(arr, 0, 0, sz);
51     }
52
53     Node merge(Node &lf, Node &rt)
54     {
55         Node ans = Node();
56         ans.val = max(lf.val, rt.val);

```

```

57         return ans;
58     }
59
60     void set(int idx, int val, int node, int lx, int rx)
61     {
62         if (rx - lx == 1)
63         {
64             seg[node].change(val);
65             return;
66         }
67         int mid = (lx + rx) / 2;
68         if (idx < mid)
69             set(idx, val, 2 * node + 1, lx, mid);
70         else
71             set(idx, val, 2 * node + 2, mid, rx);
72         seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
73     }
74
75     void set(int idx, int val)
76     {
77         set(idx, val, 0, 0, sz);
78     }
79
80     Node get(int l, int r, int node, int lx, int rx)
81     {
82         if (lx >= l && rx <= r)
83             return seg[node];
84         if (rx <= l || lx >= r)
85             return Node();
86         int mid = (lx + rx) / 2;
87         Node lf = get(l, r, 2 * node + 1, lx, mid);
88         Node ri = get(l, r, 2 * node + 2, mid, rx);
89         return merge(lf, ri);
90     }
91
92     int get(int l, int r)
93     {
94         return get(l, r, 0, 0, sz).val;
95     }
96 };
97
98 int LIS(int n, vector<int> &p)
99 {
100     segmentTree seg(n + 5);
101     vector<int> dp(n + 1);
102     int ans{};
103     for (int i = 0; i < n; i++)
104     {
105         dp[i] = seg.get(0, p[i]) + 1;
106         seg.set(p[i], dp[i]);
107         ans = max(ans, dp[i]);
108     }
109     return ans;
110 }

```

10 Misc

10.1 Big Integer

10.1.1 Big integers

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: COMPRESSED BIGINT
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "No modulo given", "Massive addition/multiplication", "1000 digits"
6  * - Classic Scenarios: DP state sums or combinatorics where the answer easily
7  * exceeds  $2^{64}$ , but no modulo is provided.
8  * - The Magic: C++ has no native BigInt. Using strings character-by-character
9  * is  $O(N^2)$  with a massive constant factor. This struct bundles 9 decimal
10 * into a single integer, processing data 9x faster and using significantly
11 * less memory.
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Initialization:
14 *     BigInt a = "12345678901234567890";
15 *     BigInt b = 987654321;
16 * - Operations:
17 *     BigInt c = a + b;
18 *     c.print();
19 */
```

```
21 const int BASE = 1e9;
22
23 struct BigInt
24 {
25     vector<int> a;
26
27     BigInt() {}
28     BigInt(long long v)
29     {
30         while (v > 0)
31         {
32             a.push_back(v % BASE);
33             v /= BASE;
34         }
35     }
36     BigInt(string s)
37     {
38         if (s.empty())
39             return;
40         for (int i = s.length(); i > 0; i -= 9)
41         {
42             if (i < 9)
43                 a.push_back(stoll(s.substr(0, i)));
44             else
45                 a.push_back(stoll(s.substr(i - 9, 9)));
```

```
46         }
47         trim();
48     }
49
50     void trim()
51     {
52         while (a.size() > 1 && a.back() == 0)
53             a.pop_back();
54     }
55
56     BigInt operator+(const BigInt &b) const
57     {
58         BigInt res;
59         int carry = 0;
60         for (int i = 0; i < max(a.size(), b.a.size()) || carry; i++)
61         {
62             long long cur = carry;
63             if (i < a.size())
64                 cur += a[i];
65             if (i < b.a.size())
66                 cur += b.a[i];
67             res.a.push_back(cur % BASE);
68             carry = cur / BASE;
69         }
70         return res;
71     }
72
73     BigInt operator*(const BigInt &b) const
74     {
75         BigInt res;
76         if (a.empty() || b.a.empty())
77             return res;
78         res.a.assign(a.size() + b.a.size(), 0);
79         for (int i = 0; i < a.size(); i++)
80         {
81             long long carry = 0;
82             for (int j = 0; j < b.a.size() || carry > 0; j++)
83             {
84                 long long cur = res.a[i + j] + a[i] * 1LL * (j < b.a.size() ? b.a[j] : 0) + carry;
85                 res.a[i + j] = cur % BASE;
86                 carry = cur / BASE;
87             }
88         }
89         res.trim();
90         return res;
91     }
92
93     bool operator<(const BigInt &b) const
94     {
95         if (a.size() != b.a.size())
96             return a.size() < b.a.size();
97         for (int i = a.size() - 1; i >= 0; i--)
98         {
99             if (a[i] != b.a[i])
```

```

100         return a[i] < b.a[i];
101     }
102     return false;
103 }
104
105 void print() const
106 {
107     if (a.empty())
108     {
109         cout << 0 << "\n";
110         return;
111     }
112     cout << a.back();
113     for (int i = (int)a.size() - 2; i >= 0; i--)
114     {
115         cout << setfill('0') << setw(9) << a[i];
116     }
117     cout << "\n";
118 }
119 };

```

10.2 Chess

10.2.1 Knight's tour

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: WARNSDORFF'S HEURISTIC
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Visit every square", "Knight's Tour", "Hamiltonian path on grid"
6  * - Classic Scenarios: Output a valid sequence of knight moves to visit all  $N \times N$ 
7  * cells exactly once without repeating.
8  * - The Magic: Backtracking has a massive branching factor and TLEs immediately
9  * Warnsdorff's Rule states: always prioritize the next valid move that has the
10 * LEAST number of subsequent valid moves. This local greedy choice finds the
11 * global Hamiltonian path almost instantly.
12 *
13 * 2. HOW TO USE IT (THE BLACK BOX)
14 * - Query:
15 *     vector<pair<int, int>> tour;
16 *     bool success = knight_tour(START_X, START_Y, BOARD_SIZE, tour);
17 */
18
19 const int dx[] = {1, 1, 2, 2, -1, -1, -2, -2};
20 const int dy[] = {2, -2, 1, -1, 2, -2, 1, -1};
21
22 int get_degree(int x, int y, int n, const vector<vector<int>> &board)
23 {
24     int count = 0;
25     for (int i = 0; i < 8; i++)
26     {
27         int nx = x + dx[i], ny = y + dy[i];

```

```

28         if (nx >= 0 && ny >= 0 && nx < n && ny < n && board[nx][ny] == -1)
29         {
30             count++;
31         }
32     }
33     return count;
34 }
35
36 bool solve_kt(int x, int y, int move_i, int n, vector<vector<int>> &board, vector<pair<int, int>> &tour)
37 {
38     board[x][y] = move_i;
39     tour.push_back({x, y});
40
41     if (move_i == n * n - 1)
42         return true;
43
44     vector<pair<int, int>> next_moves;
45     for (int i = 0; i < 8; i++)
46     {
47         int nx = x + dx[i], ny = y + dy[i];
48         if (nx >= 0 && ny >= 0 && nx < n && ny < n && board[nx][ny] == -1)
49         {
50             next_moves.push_back({get_degree(nx, ny, n, board), i});
51         }
52     }
53
54     // Warnsdorff's Heuristic: Sort by degree ascending
55     sort(next_moves.begin(), next_moves.end());
56
57     for (auto move : next_moves)
58     {
59         int i = move.second;
60         int nx = x + dx[i], ny = y + dy[i];
61         if (solve_kt(nx, ny, move_i + 1, n, board, tour))
62         {
63             return true;
64         }
65     }
66
67     // Backtrack
68     board[x][y] = -1;
69     tour.pop_back();
70     return false;
71 }
72
73 bool knight_tour(int start_x, int start_y, int n, vector<pair<int, int>> &tour)
74 {
75     vector<vector<int>> board(n, vector<int>(n, -1));
76     return solve_kt(start_x, start_y, 0, n, board, tour);
77 }

```

10.3 Matroids

10.3.1 Matroid intersection

```
1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: MATROID INTERSECTION
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Spanning tree with unique colors", "Maximize intersection of
6  *   rules".
7  * - Classic Scenarios: Find the maximum number of elements from a set that
8  *   satisfy
9  *   TWO independent structural rules simultaneously (e.g., forms a forest AND
10  *   has
11  *   at most K edges from any specific set).
12  * - The Magic: It builds a bipartite graph between the currently picked elements
13  *   (IN)
14  *   and unpicked elements (OUT). It searches for an augmenting path that swaps
15  *   elements
16  *   while maintaining independence in BOTH matroids.
17  *
18  * 2. HOW TO USE IT (THE BLACK BOX)
19  * - Setup: You must define what "independence" means for your two rules by
20  *   writing
21  *   the `oracle1_can_add` and `oracle2_can_add` logic.
22  * - Query:
23  *   vector<int> independent_set = matroid_intersection(N);
24  *
25  * 3. ENGINEERING NOTE
26  * - The robust, modular design here perfectly mirrors the isolation of concerns
27  *   required in high-tier backend systems engineering. The core augmenting
28  *   engine
29  *   never cares about the underlying rules, only the true/false oracle responses
30  *   .
31  */
32 vector<int> matroid_intersection(int n)
33 {
34     vector<bool> in_set(n, false);
35
36     // Abstract augmenting path loop
37     while (true)
38     {
39         // Build the bipartite exchange graph
40         vector<int> X1, X2; // X1: valid starts, X2: valid ends
41         vector<vector<int>> adj(n);
42
43         // --- ORACLE 1 (e.g., Graphic Matroid) ---
44         // For every out_node 'v', if {in_set U v} is independent, add 'v' to X1
45         // If dependent, find which in_node 'u' forms the circuit and add edge v
46         //   -> u.
47
48         // --- ORACLE 2 (e.g., Colorful Matroid) ---
49         // For every out_node 'v', if {in_set U v} is independent, add 'v' to X2
50         // If dependent, find which in_node 'u' forms the circuit and add edge v
51
52         // Example placeholders:
53         /*
54         for (int i = 0; i < n; i++) {
55             if (!in_set[i]) {
56                 if (oracle1_can_add(i)) X1.push_back(i);
57                 if (oracle2_can_add(i)) X2.push_back(i);
58             }
59         }
60         */
61
62         // BFS to find the shortest augmenting path from X1 to X2
63         queue<int> q;
64         vector<int> dist(n, 1e9), parent(n, -1);
65
66         for (int x : X1)
67         {
68             dist[x] = 0;
69             q.push(x);
70         }
71
72         int end_node = -1;
73         while (!q.empty())
74         {
75             int u = q.front();
76             q.pop();
77
78             // Check if we hit the target set
79             if (find(X2.begin(), X2.end(), u) != X2.end())
80             {
81                 end_node = u;
82                 break;
83             }
84
85             for (int v : adj[u])
86             {
87                 if (dist[v] > dist[u] + 1)
88                 {
89                     dist[v] = dist[u] + 1;
90                     parent[v] = u;
91                     q.push(v);
92                 }
93             }
94         }
95
96         // If no augmenting path exists, we have reached the maximum independent
97         // set
98         if (end_node == -1)
99             break;
100
101         // Augment the independent set by flipping states along the path
102         int curr = end_node;
103         while (curr != -1)
104         {
105             in_set[curr] = !in_set[curr];
106             curr = parent[curr];
107         }
108     }
109 }
```

```

96         in_set[curr] = !in_set[curr];
97         curr = parent[curr];
98     }
99 }
100
101 vector<int> result;
102 for (int i = 0; i < n; i++)
103 {
104     if (in_set[i])
105         result.push_back(i);
106 }
107 return result;
108 }

```

10.4 Search

10.4.1 Ternary search

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // The function we want to minimize (e.g., total cost, distance, time)
5 // For this example, let's pretend it's a parabola:  $f(x) = (x - 5)^2 + 10$ 
6 // The minimum should clearly be at  $x = 5$ .
7 double f_double(double x)
8 {
9     return (x - 5.0) * (x - 5.0) + 10.0;
10 }
11
12 long long f_int(long long x)
13 {
14     return (x - 5) * (x - 5) + 10;
15 }
16
17 // -----
18 // TEMPLATE 1: Continuous Ternary Search (Floating Point)
19 // -----
20 double ternary_search_double(double l, double r)
21 {
22     // THE CP TRICK: Never use `while (r - l > eps)`. Floating point precision
23     // errors can cause an infinite loop and TLE your submission.
24     // Instead, looping exactly 200 times guarantees immense precision
25     // (the range shrinks by  $2/3^{200}$ , which is basically 0).
26     for (int i = 0; i < 200; i++)
27     {
28         double m1 = l + (r - l) / 3.0;
29         double m2 = r - (r - l) / 3.0;
30
31         if (f_double(m1) < f_double(m2))
32         {
33             r = m2; // We are minimizing, so throw away the higher right side
34         }
35         else

```

```

36     {
37         l = m1; // Throw away the higher left side
38     }
39 }
40 return l; // l and r are essentially identical now
41 }
42
43 // -----
44 // TEMPLATE 2: Discrete Ternary Search (Integers)
45 // -----
46 long long ternary_search_int(long long l, long long r)
47 {
48     // We stop when the search space shrinks to 3 elements or less.
49     // If we try to split a range of size 2 into thirds with integers,
50     // m1 and m2 will evaluate to the same number and cause an infinite loop!
51     while (r - l > 2)
52     {
53         long long m1 = l + (r - l) / 3;
54         long long m2 = r - (r - l) / 3;
55
56         if (f_int(m1) < f_int(m2))
57         {
58             r = m2;
59         }
60         else
61         {
62             l = m1;
63         }
64     }
65
66     // Now we have a tiny range (l, l+1, r). Just brute force the answer!
67     long long best_x = l;
68     long long min_val = f_int(l);
69
70     for (long long i = l + 1; i <= r; i++)
71     {
72         if (f_int(i) < min_val)
73         {
74             min_val = f_int(i);
75             best_x = i;
76         }
77     }
78
79     return best_x;
80 }
81
82 void solve()
83 {
84     cout << "Continuous_Minimum_at_X= " << ternary_search_double(-1e9, 1e9) << "
85         \n";
86     cout << "Discrete_Minimum_at_X= " << ternary_search_int(-1000000, 1000000)
87         << "\n";
88 }
89
90 int main()

```

```

89 {
90     ios_base::sync_with_stdio(false);
91     cin.tie(NULL);
92     solve();
93     return 0;
94 }

```

10.4.2 fraction binary search

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: FRACTION BINARY SEARCH
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Best rational approximation", "Fraction with bounded denominator".
6  * - Classic Scenarios: You are given a target floating point value and must find
7  *   a fraction P/Q exactly equal to it (or closest to it) such that Q <= MAX_D.
8  * - The Magic: The Stern-Brocot tree generates all positive rational numbers in
9  *   simplest form. Given bounds a/b and c/d, the mediant (a+c)/(b+d) is strictly
10  *   between them. We binary search down this tree.
11  *
12  * 2. HOW TO USE IT (THE BLACK BOX)
13  * - Query:
14  *     pair<ll, ll> best_fraction = fraction_search(target, MAX_D);
15  *
16  * - Complexity:
17  *     Time: O(log(MAX_D))
18  */
19
20 using ll = long long;
21
22 // Returns {numerator, denominator}
23 pair<ll, ll> fraction_search(double target, ll max_d)
24 {
25     ll a = 0, b = 1; // Lower bound: 0/1
26     ll c = 1, d = 0; // Upper bound: 1/0 (Infinity)
27
28     ll best_num = 0, best_den = 1;
29     double min_diff = 1e18;
30
31     while (b + d <= max_d)
32     {
33         ll mid_num = a + c;
34         ll mid_den = b + d;
35
36         double mid_val = (double)mid_num / mid_den;
37         double diff = abs(mid_val - target);
38
39         if (diff < min_diff)
40         {
41             min_diff = diff;
42             best_num = mid_num;
43             best_den = mid_den;

```

```

44     }
45
46     if (mid_val == target)
47     {
48         return {mid_num, mid_den};
49     }
50     else if (mid_val < target)
51     {
52         a = mid_num;
53         b = mid_den;
54     }
55     else
56     {
57         c = mid_num;
58         d = mid_den;
59     }
60 }
61
62 return {best_num, best_den};
63 }

```

10.4.3 parallel binary search

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: PARALLEL BINARY SEARCH
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Earliest time all components connect", "Minimum weight to satisfy
6  *   Q queries".
7  * - Classic Scenarios: You are given an empty graph and an array of edges added
8  *   over time.
9  *   You have Q queries asking: "At what time do nodes U and V finally connect?"
10  * - The Magic: It runs a binary search for ALL queries simultaneously. It sweeps
11  *   through the timeline 1 to T exactly log(T) times. At each step, it checks if
12  *   the current timeline state satisfies the query's midpoint.
13  *
14  * 2. HOW TO USE IT (THE BLACK BOX)
15  * - Array Setup:
16  *     L[i] = 1, R[i] = T for all queries i.
17  *     ans[i] = -1.
18  * - Loop:
19  *     Run the `while` loop until no queries need updating.
20  *
21  * 3. HOW TO ADAPT IT (THE DIALS & SWITCHES)
22  * - State Machine: The `DSU` and `apply_event` logic inside the timeline sweep
23  *   MUST
24  *   be replaced with whatever data structure the specific problem demands
25  *   (e.g., Fenwick Tree for range sums, Segment Tree, etc.).
26  */
27
28 struct Query
29 {
30     int u, v; // Replace with problem-specific query parameters

```

```

28 };
29
30 void parallel_binary_search(int num_events, const vector<Query> &queries)
31 {
32     int q = queries.size();
33     vector<int> L(q, 1), R(q, num_events), ans(q, -1);
34     bool changed = true;
35
36     while (changed)
37     {
38         changed = false;
39         // check_at[t] stores a list of query indices that need to be evaluated
40         // at time t
41         vector<vector<int>> check_at(num_events + 1);
42
43         for (int i = 0; i < q; i++)
44         {
45             if (L[i] <= R[i])
46             {
47                 int mid = L[i] + (R[i] - L[i]) / 2;
48                 check_at[mid].push_back(i);
49                 changed = true;
50             }
51         }
52
53         if (!changed)
54             break;
55
56         // --- RESET DATA STRUCTURE HERE ---
57         // e.g., DSU dsu(N);
58         // -----
59         for (int t = 1; t <= num_events; t++)
60         {
61             // --- APPLY EVENT 't' TO DATA STRUCTURE ---
62             // e.g., dsu.unite(edges[t].u, edges[t].v);
63             // -----
64
65             for (int query_idx : check_at[t])
66             {
67                 // --- EVALUATE QUERY CONDITION ---
68                 // e.g., bool ok = dsu.same(queries[query_idx].u, queries[
69                 // query_idx].v);
70                 bool ok = true; // Replace with actual condition
71                 // -----
72
73                 if (ok)
74                 {
75                     ans[query_idx] = t;
76                     R[query_idx] = t - 1; // Try to find an earlier time
77                 }
78                 else
79                 {
80                     L[query_idx] = t + 1; // Need more time/events

```

```

81     }
82 }
83 }
84
85 // 'ans' now contains the exact minimum time for every query in O((N+Q) log T)
86 }

```

10.5 Techniques

10.5.1 MANHATTAN CHEBYSHEV

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: MANHATTAN <-> CHEBYSHEV
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Rotate coordinate system", "Simplify absolute value distance".
6  * - Classic Scenarios: A problem asks you to find the number of points within
7  *   Manhattan distance D from (X, Y). This is an "octagonal" area, which is
8  *   extremely hard to query.
9  * - The Magic:
10 *   Transformation: (x, y) -> (x+y, x-y)
11 *   Manhattan distance in the original plane is equivalent to Chebyshev distance
12 *   in the transformed plane.
13 *   Chebyshev distance: max(|x1 - x2|, |y1 - y2|) creates a *square* area, which
14 *   is trivially solvable using standard 2D Segment Trees or Fenwick Trees.
15 *
16 * 2. HOW TO USE IT
17 * - Transform:
18 *     new_x = x + y;
19 *     new_y = x - y;
20 */

```

10.5.2 Meet in the middle

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // This template solves the classic "Find if any subset sums to Target"
5 // Complexity: O(2^(N/2) * N)
6
7 void generate_sums(const vector<long long> &arr, vector<long long> &sums)
8 {
9     int n = arr.size();
10    // Use bitmasking to generate all 2^n subsets
11    for (int i = 0; i < (1 << n); i++)
12    {
13        long long current_sum = 0;
14        for (int j = 0; j < n; j++)
15        {
16            if ((i >> j) & 1)
17            {

```

```

18         current_sum += arr[j];
19     }
20 }
21 sums.push_back(current_sum);
22 }
23 }
24
25 void solve()
26 {
27     int n;
28     long long target;
29     cin >> n >> target;
30
31     vector<long long> arr(n);
32     for (int i = 0; i < n; i++)
33         cin >> arr[i];
34
35     // 1. Split into two halves
36     vector<long long> left_part, right_part;
37     for (int i = 0; i < n; i++)
38     {
39         if (i < n / 2)
40             left_part.push_back(arr[i]);
41         else
42             right_part.push_back(arr[i]);
43     }
44
45     // 2. Generate all subset sums for both halves
46     vector<long long> left_sums, right_sums;
47     generate_sums(left_part, left_sums);
48     generate_sums(right_part, right_sums);
49
50     // 3. Sort one half for binary searching
51     sort(left_sums.begin(), left_sums.end());
52
53     // 4. Search for the complement in the other half
54     bool found = false;
55     for (long long s : right_sums)
56     {
57         long long needed = target - s;
58         // Check if 'needed' exists in left_sums
59         if (binary_search(left_sums.begin(), left_sums.end(), needed))
60         {
61             found = true;
62             break;
63         }
64     }
65
66     if (found)
67         cout << "YES\n";
68     else
69         cout << "NO\n";
70 }
71
72 int main()

```

```

73 {
74     ios_base::sync_with_stdio(false);
75     cin.tie(NULL);
76     solve();
77     return 0;
78 }

```

10.5.3 Sparse table lsa trick

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SPARSE TABLE (LSA TRICK)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Range Maximum Query", "Immutable range query".
6  * - Classic Scenarios: You have an array that never changes, and you need to
7  *   query the maximum/minimum in any range [L, R] in O(1).
8  * - The Magic: Precomputes logs for all ranges of length 2^k. Since `max` is
9  *   idempotent (max(a, a) = a), we can answer queries by taking the max of two
10  *   overlapping 2^k blocks that cover the entire [L, R].
11  */
12
13 const int K = 20; // Enough for 10^6 elements (2^20 > 10^6)
14 int st[K][100005];
15 int lg[100005];
16
17 void build_sparse_table(const vector<int> &a)
18 {
19     int n = a.size();
20     lg[1] = 0;
21     for (int i = 2; i <= n; i++)
22         lg[i] = lg[i / 2] + 1;
23     for (int i = 0; i < n; i++)
24         st[0][i] = a[i];
25
26     for (int j = 1; j < K; j++)
27     {
28         for (int i = 0; i + (1 << j) <= n; i++)
29         {
30             st[j][i] = max(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
31         }
32     }
33 }
34
35 int query(int L, int R)
36 {
37     int j = lg[R - L + 1];
38     return max(st[j][L], st[j][R - (1 << j) + 1]);
39 }

```

10.6 min plus convolution.cpp

10.6.1 shawk

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: SMAWK ALGORITHM
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Totally monotone matrix", "Min-plus convolution", " $O(N)$ 
6  * - Classic Scenarios: You need to find the minimum value in every row of an  $N \times M$ 
7  * matrix where the matrix satisfies the Monge property (or is totally monotone).
8  * - The Magic: Bypasses standard  $O(NM)$  brute force and even  $O(N \log M)$  Divide & Conquer. It recursively filters out columns that can never contain a row
9  * minimum,
10 * solving the entire problem in strictly  $O(N + M)$  time.
11 *
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Query:
14 *     vector<int> min_indices = smawk(R, C, cost_function);
15 * - Result:
16 *     Returns an array `ans` of size R, where `ans[i]` is the column index
17 *     containing the minimum value for row i.
18 */
19
20 using ll = long long;
21
22 template <class F>
23 vector<int> smawk(int r, int c, F cost)
24 {
25     auto solve = [&](auto &self, const vector<int> &rows, const vector<int> &cols)
26         -> vector<int>
27     {
28         if (rows.empty())
29             return {};
30
31         // Reduce: filter columns
32         vector<int> st;
33         for (int c : cols)
34         {
35             while (!st.empty())
36             {
37                 if (cost(rows[st.size() - 1], st.back()) > cost(rows[st.size() - 1], c))
38                     st.pop_back();
39             }
40             else if (st.size() < rows.size())
41             {
42                 break;
43             }
44             else
45             {
46                 goto next_col;
47             }
48         }
49         st.push_back(c);
50
51         next_col;
52     }
53
54     vector<int> odd_rows;
55     for (int i = 1; i < rows.size(); i += 2)
56         odd_rows.push_back(rows[i]);
57
58     // Solve recursively for odd-indexed rows
59     vector<int> odd_ans = self(self, odd_rows, st);
60     vector<int> ans(rows.size());
61     for (int i = 0; i < odd_ans.size(); i++)
62         ans[2 * i + 1] = odd_ans[i];
63
64     // Interpolate for even-indexed rows
65     int c_idx = 0;
66     for (int i = 0; i < rows.size(); i += 2)
67     {
68         int end_c = (i + 1 == rows.size()) ? st.back() : ans[i + 1];
69         int best_c = st[c_idx];
70         ll best_val = cost(rows[i], best_c);
71
72         while (st[c_idx] != end_c)
73         {
74             c_idx++;
75             ll val = cost(rows[i], st[c_idx]);
76             if (val < best_val)
77             {
78                 best_val = val;
79                 best_c = st[c_idx];
80             }
81         }
82         ans[i] = best_c;
83     }
84     return ans;
85 };
86
87 vector<int> rows(r), cols(c);
88 iota(rows.begin(), rows.end(), 0);
89 iota(cols.begin(), cols.end(), 0);
90 return solve(solve, rows, cols);
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

10.7 misc

10.7.1 Dynamic bitset

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: DYNAMIC BITSET
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Bitset DP with variable size", "Memory Limit Exceeded".
6  * - Classic Scenarios: You need the  $O(N / 64)$  speedup of std::bitset, but the
7  * required size of the bitset is determined by the input (e.g., sum of array),

```

```

8  *   meaning you cannot use std::bitset<MAX_SIZE> without hitting Memory Limits.
9  * - The Magic: Wraps a std::vector<uint64_t> and manually implements the
10 *   left-shift (<<) and bitwise OR (|) operators with hardware-level carry logic.
11
12 * 2. HOW TO USE IT (THE BLACK BOX)
13 * - Initialization:
14 *   DynamicBitset dp(total_sum + 1);
15 *   dp.set(0);
16 * - Transitions:
17 *   dp |= (dp << item_weight);
18 */
19
20 #include <stdint>
21
22 struct DynamicBitset
23 {
24     vector<uint64_t> blocks;
25     int bits;
26
27     DynamicBitset(int n = 0)
28     {
29         bits = n;
30         blocks.assign((n + 63) / 64, 0);
31     }
32
33     void set(int i)
34     {
35         if (i < bits)
36             blocks[i / 64] |= (1ULL << (i % 64));
37     }
38
39     bool test(int i) const
40     {
41         if (i >= bits)
42             return false;
43         return (blocks[i / 64] >> (i % 64)) & 1;
44     }
45
46     // Crucial for Knapsack / Reachability DP
47     DynamicBitset operator<<(int shift) const
48     {
49         DynamicBitset res(bits + shift);
50         int block_shift = shift / 64;
51         int bit_shift = shift % 64;
52         uint64_t carry = 0;
53
54         for (int i = 0; i < blocks.size(); i++)
55         {
56             res.blocks[i + block_shift] = (blocks[i] << bit_shift) | carry;
57             if (bit_shift > 0)
58             {
59                 carry = blocks[i] >> (64 - bit_shift);
60             }
61             else

```

```

        {
            carry = 0;
        }
    }
    if (carry && (blocks.size() + block_shift < res.blocks.size()))
    {
        res.blocks[blocks.size() + block_shift] = carry;
    }
    // Restore original logical size constraints if necessary
    res.bits = bits + shift;
    return res;
}

void operator|=(const DynamicBitset &other)
{
    for (int i = 0; i < min(blocks.size(), other.blocks.size()); i++)
    {
        blocks[i] |= other.blocks[i];
    }
}
};

```

10.7.2 LGV lemma

```

1 /**
2  * THE ULTIMATE CP TEMPLATE BLUEPRINT: LGV LEMMA (NON-INTERSECTING PATHS)
3  *
4  * 1. WHAT PROBLEM DOES THIS SOLVE?
5  * - Keywords: "Non-intersecting paths", "Multiple robots on a grid".
6  * - Classic Scenarios: You have K robots starting at points A_1...A_K and they
7  *   must reach B_1...B_K on a DAG (like a grid with obstacles). They cannot
8  *   share
9  *   any vertices. How many valid routing configurations exist?
10 * - The Magic: You only need to calculate the standard number of paths from
11 *   every A_i to every B_j. You construct a K x K matrix of these path counts.
12 *   The determinant of this matrix magically cancels out all intersecting path
13 *   configurations via parity inversions.
14
15 * 2. HOW TO USE IT (THE BLACK BOX)
16 * - Dependencies: You MUST have your `det_prime` function (Template 371)
17 *   included
18 *   above this from your Linear Algebra section.
19 * - Logic: Replace `calc_ways(start, end)` with a standard O(V+E) DP or nCr math
20 *
21 */
22 using ll = long long;
23
24 // Forward declaration of your existing determinant template
25 ll det_prime(vector<vector<ll>> a, ll mod);
26
27 // Replace this with your specific problem's path counting logic
28 ll calc_ways(pair<int, int> start, pair<int, int> end, ll mod)

```

```

27 {
28     // Example: Grid paths without obstacles using Combinatorics:  $C(dx + dy, dx)$ 
29     // Example: Grid paths with obstacles using  $O(N^2)$  DP
30     return 0;
31 }
32
33 ll lgv_lemma(const vector<pair<int, int>> &starts, const vector<pair<int, int>> &
    ends, ll mod)
34 {
35     int k = starts.size();
36     if (k != ends.size())
37         return 0;
38
39     vector<vector<ll>> M(k, vector<ll>(k, 0));
40
41     // Build the path matrix
42     for (int i = 0; i < k; i++)
43     {
44         for (int j = 0; j < k; j++)
45         {
46             M[i][j] = calc_ways(starts[i], ends[j], mod);
47         }
48     }
49
50     // The answer is literally just the determinant of the path matrix
51     return det_prime(M, mod);
52 }

```

11 Helpers

11.1 Adhocs

```

1 void PratialSum()
2 {
3     int n; cin >> n;
4     vector<int>arr(n + 1);
5     vector<ll> pratial(n + 1, 0);
6
7     for (int i = 1; i <= n; i++) cin >> arr[i];
8
9     int l, r, value; cin >> l >> r >> value;    // sure  $l < r$ 
10
11     pratial[l] += value;
12     pratial[r + 1] -= value;
13
14     for (int i = 1; i <= n; i++)
15         pratial[i] += pratial[i - 1];
16
17     for (int i = 1; i <= n; i++)
18         pratial[i] += arr[i];
19
20     for (int i = 1; i <= n; i++)

```

```

21         cout << pratial[i] << "uuu";
22
23 }
24 // Function to get the sum of elements in the subarray defined by (x1, y1) to (x2
    , y2)
25 long long getSum(const vector<vector<ll>>& Prefix, int x1, int y1, int x2, int y2
    ) {
26     return Prefix[x2][y2] - Prefix[x1 - 1][y2] - Prefix[x2][y1 - 1] + Prefix[x1 -
    1][y1 - 1];
27 }
28 void PrefixSum_2D() {
29     int n, m, q;
30     cin >> n >> m >> q;
31     vector<vector<ll>> arr(n + 1, vector<ll>(m + 1, 0));
32     vector<vector<ll>> Prefix(n + 1, vector<ll>(m + 1, 0));
33     for (int i = 1; i <= n; i++) {
34         for (int j = 1; j <= m; j++) {
35             cin >> arr[i][j];
36         }
37     }
38     for (int i = 1; i <= n; i++) {
39         for (int j = 1; j <= m; j++) {
40             Prefix[i][j] = arr[i][j]
41                 + Prefix[i - 1][j]
42                 + Prefix[i][j - 1]
43                 - Prefix[i - 1][j - 1];
44         }
45     }
46     while (q--) {
47         int x1, y1, x2, y2;
48         cin >> x1 >> y1 >> x2 >> y2; // sure  $x1 < x2$  and  $y1 < y2$ 
49         long long result = getSum(Prefix, x1, y1, x2, y2);
50         cout << result << endl;
51     }
52 }
53
54 void PratialSum_2D()
55 {
56     int n, m, q;
57     cin >> n >> m >> q;
58     vector<vector<ll>> arr(n + 1, vector<ll>(m + 1, 0));
59     vector<vector<ll>> Pratial(n + 2, vector<ll>(m + 2, 0));
60     while (q--) {
61         int x1, y1, x2, y2, val;
62         cin >> x1 >> y1 >> x2 >> y2 >> val;
63         Pratial[x1][y1] += val;
64         Pratial[x1][y2 + 1] -= val;
65         Pratial[x2 + 1][y1] -= val;
66         Pratial[x2 + 1][y2 + 1] += val;
67     }
68     for (int i = 1; i <= n; i++) {
69         for (int j = 1; j <= m; j++) {
70             Pratial[i][j] = Pratial[i][j]
71                 + Pratial[i - 1][j]
72                 + Pratial[i][j - 1]

```

```

73         - Pratial[i - 1][j - 1];
74     }
75 }
76
77 for (int i = 1; i <= n; i++) {
78     for (int j = 1; j <= m; j++) {
79         cout << Pratial[i][j] << " ";
80     }
81     cout << '\n';
82 }
83 }
84
85 }

```

11.2 BitMask&Complete Search

```

1
2 vector<ll> getRepresentation(ll num, ll base) // convert any number to (any)base
3 {
4     vector<ll>ans;
5     while (num)
6     {
7         ans.push_back(num % base);
8         num /= base;
9     }
10    reverse(ans.begin(), ans.end());
11    return ans;
12 }
13
14 // 1. Check if the i-th bit is set (1) or not (0)
15 bool checkBit(long long mask, int i) {
16     return (mask & (1LL << i)) != 0;
17 }
18
19 // 2. Set the i-th bit to 1 (leave other bits unchanged)
20 long long setBit(long long mask, int i) {
21     return mask | (1LL << i);
22 }
23
24 // 3. Clear the i-th bit to 0 (leave other bits unchanged)
25 long long clearBit(long long mask, int i) {
26     return mask & ~(1LL << i);
27 }
28
29 // 4. Toggle the i-th bit (flip 1 to 0, or 0 to 1)
30 long long toggleBit(long long mask, int i) {
31     return mask ^ (1LL << i);
32 }
33
34 // 5. Get the value of the lowest set bit (e.g., 1010 -> 0010)
35 // Useful for Fenwick trees (Binary Indexed Trees)
36 long long lowestSetBit(long long mask) {
37     return mask & -mask;

```

```

38 }
39
40 // 6. Check if a number is a perfect power of 2
41 // A power of 2 has exactly one bit set (e.g., 8 is 1000 in binary)
42 bool isPowerOfTwo(long long n) {
43     return n > 0 && (n & (n - 1)) == 0;
44 }
45
46 // 7. Strip the lowest set bit (change the rightmost 1 to a 0)
47 long long stripLowestSetBit(long long mask) {
48     return mask & (mask - 1);
49 }
50 int LastBitValue(ll n)
51 {
52     return n & ~(n - 1);
53 }
54
55 // 1. Count the number of set bits (1s) in a 32-bit integer
56 int count = __builtin_popcount(mask);
57
58 // 2. Count the number of set bits (1s) in a 64-bit integer (long long)
59 int countLL = __builtin_popcountll(mask);
60
61 // 3. Count the number of leading zeros
62 int leadingZeros = __builtin_clz(mask);
63
64 // 4. Count the number of trailing zeros (useful to find the index of the first
65     set bit)
66 int trailingZeros = __builtin_ctz(mask);
67
68 // Ranges
69 void prefix_Xor()
70 {
71     int n; cin >> n; vector<ll>v(n + 1);
72     for (int i = 1; i <= n; i++) cin >> v[i];
73     vector<ll>pre_Xor(n + 1);
74     for (int i = 1; i <= n; i++)
75         pre_Xor[i] = pre_Xor[i - 1] ^ v[i];
76     int q; cin >> q; while (q--) {
77         int l, r; cin >> l >> r;
78         cout << (pre_Xor[r] ^ pre_Xor[l - 1]) << '\n';
79     }
80 }
81 void prefix_OR()
82 {
83     int n; cin >> n; vector<ll>v(n + 1);
84     vector<vector<ll>>prefix_bit(64, vector<ll>(n + 1));
85     for (int i = 1; i <= n; i++) cin >> v[i];
86
87     for (int bit = 0; bit < 64; bit++)
88     {
89         for (int i = 1; i <= n; i++)
90         {
91             prefix_bit[bit][i] = prefix_bit[bit][i - 1] + ((v[i] >> bit) & 1);

```

```

92     }
93 }
94 int q; cin >> q; while (q--) {
95     int l, r; cin >> l >> r;
96     ll OR = 0;
97     for (int bit = 0; bit < 64; bit++)
98     {
99         int on_bits = prefix_bit[bit][r] - prefix_bit[bit][l - 1];
100         if (on_bits > 0)
101         {
102             OR += (1LL << bit);
103         }
104     }
105     cout << OR << '\n';
106 }
107 }
108 void prefix_AND()
109 {
110     int n; cin >> n; vector<ll>v(n + 1);
111     vector<vector<ll>>prefix_bit(64, vector<ll>(n + 1));
112     for (int i = 1; i <= n; i++) cin >> v[i];
113
114     for (int bit = 0; bit < 64; bit++)
115     {
116         for (int i = 1; i <= n; i++)
117         {
118             prefix_bit[bit][i] = prefix_bit[bit][i - 1] + ((v[i] >> bit) & 1);
119         }
120     }
121     int q; cin >> q; while (q--) {
122         int l, r; cin >> l >> r;
123         ll AND = 0;
124         for (int bit = 0; bit < 64; bit++)
125         {
126             int on_bits = prefix_bit[bit][r] - prefix_bit[bit][l - 1];
127             if (on_bits == r-l+1)
128             {
129                 AND += (1LL << bit);
130             }
131         }
132         cout << AND << '\n';
133     }
134 }
135
136
137 void CompleteSearch()
138 {
139     int n; cin >> n;
140     for (int mask = 0; mask < (1 << n); mask++)
141     {
142         for (int i = 0; i < n; i++)
143         {
144             if ((mask >> i) & 1)
145                 //if (getBit(mask, i))
146                 cout << 1;

```

```

147         else
148             cout << 0;
149     }
150     cout << '\n';
151 }
152 }
153
154 void iterateSubsets(int mask) {
155     // Start with the full mask, and keep stripping away bits
156     for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
157         // 'sub' is a valid subset of 'mask'
158         std::cout << sub << "\n";
159     }
160     // Note: The loop above misses the empty subset (0).
161     // If you need to process 0 as well, you can do it after the loop.
162 }
163
164
165 void BitSet() {
166     bitset<32> b = 15;
167
168     b[3] = 0;
169     cout << b.to_ullong() << '\n'; // 7
170     cout << b << '\n'; // 0000000000000000000000000000111
171     cout << b.size() << "\n"; //32 bit
172     cout << b.test(2) << '\n'; // 1 >> true
173     cout << b.test(3) << '\n'; // 0 >> false
174     cout << b.count() << '\n'; // Count number of 1 == 3
175     cout << b.flip() << '\n'; // 1111111111111111111111111111000
176     cout << b.set() << '\n'; //1111111111111111111111111111111
177     cout << b.reset() << '\n'; //0000000000000000000000000000000
178     b[4] = 1;
179     cout << b.to_string() << '\n'; //000000000000000000000000000010000
180 }

```

11.3 Bitmasks(AMR)

```

1 //
2 // Created by AMR on 4/16/2025.
3 //
4
5 #include <bits/stdc++.h>
6 using namespace std;
7 typedef long long ll;
8 #define El_Hefnawys ios_base::sync_with_stdio(false); cin.tie(nullptr); cout.tie(
    nullptr)
9
10 const int MOD = 1e9 + 7;
11
12
13
14 //  $A \sim B = (A/B) - (A \oslash B)$ 
15 //  $A \sim B = (A+B) - 2*(A \oslash B)$ 

```

```

16 //  $A^B = 2^{(A/B)} - (A+B)$ 
17
18 bool is_power_2(int x) {
19     if (x == 0)
20         return 0;
21     return x&(x-1) == 0;
22 }
23
24 int MSB_value(int n) {
25     if (n == 0)
26         return 0;
27     int index = 31 - __builtin_clz(n);
28     return 1 << index;
29 }
30
31 int MSB_index(int n) {
32     if (n == 0)
33         return -1;
34     return 31 - __builtin_clz(n);
35 }
36
37 // you can use __builtin_ctz(n) to get the LSB also directly
38 int LSB_value(int n) {
39     if (n == 0)
40         return 0;
41     return n&(-n);
42 }
43
44 int LSB_index(int n) {
45     if (n == 0)
46         return -1;
47     int value = n&(-n);
48     return 31 - __builtin_clz(value);
49 }
50
51 bool check_bit(ll n, ll i) {
52     return n&(1ll<<i);
53 }
54
55 // Another way
56 bool check_bit_2(ll n, ll i) {
57     return 1ll&(n>>i);
58 }
59
60 ll toggle_bit(ll n, ll i) {
61     return n^(1ll<<i);
62 }
63
64 ll set_bit(ll n, ll i) {
65     return n | (1ll<<i);
66 }
67
68 ll clear_bit(ll n, ll i) {
69     return n & ~(1ll<<i);
70 }

```

```

71
72 bool check_pow_2(ll n) {
73     return __builtin_popcountll(n) == 1; // function to count the number of 1s in
        a number
74     // O(1) function
75 }
76
77 ll num_modulo_pow_2(ll n, ll k) {
78     return n & ((1ll << k) - 1);
79 }
80
81 int main() {
82     El_Hefnawys;
83     cout << num_modulo_pow_2(25,4);
84     return 0;
85 }
86
87 //////////////////////////////////////
88 //
89 // Created by AMR on 4/17/2025.
90 //
91
92 #include <bits/stdc++.h>
93 using namespace std;
94 typedef long long ll;
95 #define El_Hefnawys \
96     ios_base::sync_with_stdio(false); \
97     cin.tie(nullptr); \
98     cout.tie(nullptr)
99
100
101 const int MOD = 1e9 + 7;
102
103 int Xor_nums_from_1_to_n(int n)
104 {
105     if (!(n % 4))
106         return n;
107     if (n % 4 == 1)
108         return 1;
109     if (n % 4 == 2)
110         return n + 1;
111     return 0;
112 }

```

11.4 Compression

```

1 vector<int> compress(vector<int> &v)
2 {
3     vector<int> help = v;
4     sort(help.begin(), help.end());
5     help.erase(unique(help.begin(), help.end()), help.end());
6     // 1-based
7     // to get original value -> help[compressed value - 1];

```

```

8     for (auto &val : v)
9         val = distance(help.begin(), lower_bound(help.begin(), help.end(), val))
            + 1;
10    return help;
11 }

```

11.5 Hashing segment tree

```

1 #define int long long
2 const int N = 1e5 + 5, mod1 = 1e9 + 7, mod2 = 2e9 + 11;
3 ll base1 = 31, base2 = 37, pw1[N + 1], pw2[N + 1], inv1[N + 1], inv2[N + 1];
4
5 ll powmod(ll a, ll b, ll m)
6 {
7     ll ans = 1;
8     while (b > 0)
9     {
10         if (b & 1)
11         {
12             ans = (ans * a) % m;
13         }
14         a = (a * a) % m;
15         b >>= 1;
16     }
17     return ans;
18 }
19
20 void init()
21 {
22     pw1[0] = pw2[0] = inv1[0] = inv2[0] = 1;
23     int temp1 = powmod(base1, mod1 - 2, mod1);
24     int temp2 = powmod(base2, mod2 - 2, mod2);
25     for (int i = 1; i < N; i++)
26     {
27         pw1[i] = (base1 * pw1[i - 1]) % mod1;
28         pw2[i] = (base2 * pw2[i - 1]) % mod2;
29         inv1[i] = (inv1[i - 1] * temp1) % mod1;
30         inv2[i] = (inv2[i - 1] * temp2) % mod2;
31     }
32 }
33
34 struct HashingSegmentTree
35 {
36 private:
37     vector<pair<int, int>> seg;
38     int sz;
39
40     pair<int, int> merge(pair<int, int> l, pair<int, int> r)
41     {
42         pair<int, int> ret = l;
43         ret.first = (ret.first + r.first) % mod1;
44         ret.second = (ret.second + r.second) % mod2;
45         return ret;

```

```

46     }
47
48 void update(int l, int r, int node, int idx, int ch)
49 {
50     if (l == r)
51     {
52         seg[node] = {(ch * pw1[idx]) % mod1, (ch * pw2[idx]) % mod2};
53         return;
54     }
55     int mid = l + r >> 1;
56     if (idx <= mid)
57         update(l, mid, 2 * node + 1, idx, ch);
58     else
59         update(mid + 1, r, 2 * node + 2, idx, ch);
60     seg[node] = merge(seg[2 * node + 1], seg[2 * node + 2]);
61 }
62
63 pair<int, int> query(int l, int r, int node, int lx, int rx)
64 {
65     if (l >= lx && r <= rx)
66     {
67         return seg[node];
68     }
69     if (l > rx || r < lx)
70         return {0, 0};
71     int mid = l + r >> 1;
72     pair<int, int> lft = query(l, mid, 2 * node + 1, lx, rx);
73     pair<int, int> rgt = query(mid + 1, r, 2 * node + 2, lx, rx);
74     return merge(lft, rgt);
75 }
76
77 public:
78     HashingSegmentTree(int n)
79     {
80         sz = 1;
81         while (sz <= n)
82             sz *= 2;
83         seg = vector<pair<int, int>>(sz * 2);
84     }
85
86 void update(int idx, char ch)
87 {
88     update(0, sz - 1, 0, idx, ch - 'a' + 1);
89 }
90
91 pair<int, int> query(int l, int r)
92 {
93     pair<int, int> ret = query(0, sz - 1, 0, l, r);
94     ret.first = (ret.first * inv1[l - 1]) % mod1;
95     ret.second = (ret.second * inv2[l - 1]) % mod2;
96     return ret;
97 }
98 };
99
100 bool isPalindrome(HashingSegmentTree &a, HashingSegmentTree &b, int &l, int &r,

```

```

        int &n)
101 {
102     return (a.query(l, r) == b.query(n - r + 1, n - l + 1));
103 }

```

11.6 Hashing treap

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4 #define int long long
5 const int N = 1e6 + 5, mod = 1e9 + 7;
6 ll base = 31, pw[N + 1];
7
8 void init()
9 {
10     pw[0] = 1;
11     for (int i = 1; i < N; i++)
12     {
13         pw[i] = (base * pw[i - 1]) % mod;
14     }
15 }
16
17 std::mt19937_64 rnd(std::chrono::system_clock::now().time_since_epoch().count());
18
19 struct Treap
20 {
21     struct Node
22     {
23         int val, h = 0;
24         int pri = rnd(), sz = 1;
25         array<Node *, 2> c = {0, 0};
26
27         Node()
28         {
29         }
30
31         Node(int k)
32         {
33             val = h = k;
34         }
35     };
36
37     Node *root = 0;
38     int getSize(Node *t)
39     {
40         return t ? t->sz : 0;
41     }
42
43     int getHash(Node *t)
44     {
45         return t ? t->h : 0;
46     }

```

```

47
48     Treap(string &s)
49     {
50         for (int i = 0; i < s.size(); i++)
51         {
52             root = merge(root, new Node(s[i] - 'a' + 1));
53         }
54     }
55
56     Node *fix(Node *t)
57     {
58         t->sz = getSize(t->c[0]) + getSize(t->c[1]) + 1;
59         t->h = (((getHash(t->c[0]) * base + t->val) % mod) * pw[getSize(t->c[1])]
60             + getHash(t->c[1])) % mod;
61         return t;
62     }
63
64     array<Node *, 2> split(Node *t, int k)
65     {
66         if (!t)
67             return {0, 0};
68         if (getSize(t->c[0]) >= k)
69         {
70             auto ret = split(t->c[0], k);
71             t->c[0] = ret[1];
72             return {ret[0], fix(t)};
73         }
74         else
75         {
76             auto ret = split(t->c[1], k - getSize(t->c[0]) - 1);
77             t->c[1] = ret[0];
78             return {fix(t), ret[1]};
79         }
80     }
81
82     Node *merge(Node *u, Node *v)
83     {
84         if (!u || !v)
85             return u ? u : v;
86         if (u->pri > v->pri)
87         {
88             u->c[1] = merge(u->c[1], v);
89             return fix(u);
90         }
91         else
92         {
93             v->c[0] = merge(u, v->c[0]);
94             return fix(v);
95         }
96     }
97
98     void deleteRange(int l, int r)
99     {
100         auto a = split(root, l - 1);
101         auto b = split(a[1], r - l + 1);

```

```

101     root = merge(a[0], b[1]);
102 }
103
104 int getHash(int l, int r)
105 {
106     auto a = split(root, l - 1);
107     auto b = split(a[1], r - l + 1);
108     int ret = b[0]->h;
109     root = merge(a[0], merge(b[0], b[1]));
110     return ret;
111 }
112
113 void addChar(int pos, char c)
114 {
115     auto a = split(root, pos - 1);
116     root = merge(merge(a[0], new Node(c - 'a' + 1)), a[1]);
117 }
118
119 void print(Node *t)
120 {
121     if (!t)
122         return;
123     print(t->c[0]);
124     cout << t->h << '□';
125     print(t->c[1]);
126 }
127 };
128
129 int rev(int i, int n)
130 {
131     return n - i + 1;
132 }
133
134 signed main()
135 {
136     ios_base::sync_with_stdio(0);
137     cin.tie(0);
138     cout.tie(0);
139     init();
140     int n, q;
141     cin >> n >> q;
142     string s;
143     cin >> s;
144     string f = s;
145     reverse(f.begin(), f.end());
146     Treap a(s), b(f);
147     while (q--)
148     {
149         int op;
150         cin >> op;
151         if (op == 1)
152         {
153             int l, r;
154             cin >> l >> r;
155             int l2 = rev(l, n), r2 = rev(r, n);

```

```

156         n -= (r - l + 1);
157         a.deleteRange(l, r);
158         b.deleteRange(r2, l2);
159     }
160     else if (op == 2)
161     {
162         n++;
163         int pos;
164         char c;
165         cin >> c >> pos;
166         a.addChar(pos, c);
167         b.addChar(rev(pos, n), c);
168     }
169     else
170     {
171         int l, r;
172         cin >> l >> r;
173         int l2 = rev(l, n), r2 = rev(r, n);
174         if (a.getHash(l, r) == b.getHash(r2, l2))
175         {
176             cout << "yes\n";
177         }
178         else
179         {
180             cout << "no\n";
181         }
182     }
183 }
184 return 0;
185 }

```

11.7 INT 128

```

1 __int128 read() {
2     __int128 x = 0, f = 1;
3     char ch = getchar();
4     while (ch < '0' || ch > '9') {
5         if (ch == '-') f = -1;
6         ch = getchar();
7     }
8     while (ch >= '0' && ch <= '9') {
9         x = x * 10 + ch - '0';
10        ch = getchar();
11    }
12    return x * f;
13 }
14 void print(__int128 x) {
15     if (x < 0) {
16         putchar('-');
17         x = -x;
18     }
19     if (x > 9) print(x / 10);
20     putchar(x % 10 + '0');

```

```

21 }
22 bool cmp(__int128 x, __int128 y) { return x > y; }

```

11.8 MEX

```

1 int MEX(vector<int> &v)
2 {
3     int n = int(v.size());
4     vector<int> frq(n);
5     for (int i = 0; i < n; i++)
6     {
7         if (v[i] < n)
8             frq[v[i]]++;
9     }
10    for (int i = 0; i < n; i++)
11    {
12        if (frq[i] == 0)
13            return i;
14    }
15    return n;
16 }

```

11.9 Ordered set

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 typedef tree<int, null_type, less<>, rb_tree_tag,
6             tree_order_statistics_node_update> ordered_set;
7
8 void myerase(ordered_set &t, int v)
9 {
10    int rank = t.order_of_key(v);
11    auto it = t.find_by_order(rank);
12    t.erase(it);
13 }
14 /*
15  * t.order_of_key(v);           // Returns the NUMBER of elements strictly less
16  *                               than v (the rank)
17  * *t.find_by_order(k);         // Returns the value at index k (0-indexed)
18  * myerase(t, v);               // Safely erases exactly ONE instance of v
19 */

```

11.10 Power

```

1 ll power(ll base, ll pow)
2 {
3     base %= MOD;
4     ll res{1};

```

```

5     while (pow)
6     {
7         if (pow & 1)
8             res = res * base % MOD;
9         base = base * base % MOD;
10        pow >>= 1;
11    }
12    return res;
13 }

```

11.11 Query

```

1 struct Query
2 {
3     int x, a, id;
4
5     bool operator<(const Query &other) const
6     {
7         return a < other.a;
8     }
9 };

```

11.12 Solve Equations

```

1 pair<ll,ll> equ(ll a,ll b,ll c)
2 {
3     ll x1 = (-b + sqrt(b * b - 4 * a * c)) / (2 * a);
4     ll x2 = (-b - sqrt(b * b - 4 * a * c)) / (2 * a);
5     return {x1,x2};
6 }

```

11.13 Split mix

```

1 struct custom_hash
2 {
3     static uint64_t splitmix64(uint64_t x)
4     {
5         x += 0x9e3779b97f4a7c15;
6         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
7         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
8         return x ^ (x >> 31);
9     }
10
11    size_t operator()(uint64_t x) const
12    {
13        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().
14            time_since_epoch().count();
15        return splitmix64(x + FIXED_RANDOM);
16    }
17 };

```

11.14 debug

```
1 #define LOCAL ;
2
3 template <typename A, typename B>
4 string to_string(pair<A, B> p);
5
6 template <typename A, typename B, typename C>
7 string to_string(tuple<A, B, C> p);
8
9 template <typename A, typename B, typename C, typename D>
10 string to_string(tuple<A, B, C, D> p);
11
12 string to_string(const string &s)
13 {
14     return '"' + s + '"';
15 }
16
17 string to_string(const char *s)
18 {
19     return to_string((string)s);
20 }
21
22 string to_string(bool b)
23 {
24     return (b ? "true" : "false");
25 }
26
27 string to_string(vector<bool> v)
28 {
29     bool first = true;
30     string res = "{";
31     for (auto &&i : v)
32     {
33         if (!first)
34         {
35             res += ",";
36         }
37         first = false;
38         res += to_string(i);
39     }
40     res += "}";
41     return res;
42 }
43
44 template <size_t N>
45 string to_string(bitset<N> v)
46 {
47     string res = "";
48     for (size_t i = 0; i < N; i++)
49     {
50         res += static_cast<char>('0' + v[i]);
51     }
52     return res;
53 }
```

```
54
55 template <typename A>
56 string to_string(A v)
57 {
58     bool first = true;
59     string res = "{";
60     for (const auto &x : v)
61     {
62         if (!first)
63         {
64             res += ",";
65         }
66         first = false;
67         res += to_string(x);
68     }
69     res += "}";
70     return res;
71 }
72
73 template <typename A, typename B>
74 string to_string(pair<A, B> p)
75 {
76     return "(" + to_string(p.first) + "," + to_string(p.second) + ")";
77 }
78
79 template <typename A, typename B, typename C>
80 string to_string(tuple<A, B, C> p)
81 {
82     return "(" + to_string(get<0>(p)) + "," + to_string(get<1>(p)) + "," +
83         to_string(get<2>(p)) + ")";
84 }
85
86 template <typename A, typename B, typename C, typename D>
87 string to_string(tuple<A, B, C, D> p)
88 {
89     return "(" + to_string(get<0>(p)) + "," + to_string(get<1>(p)) + "," +
90         to_string(get<2>(p)) + "," + to_string(get<3>(p)) + ")";
91 }
92
93 template <typename T>
94 string to_string(queue<T> q)
95 {
96     string res = "{";
97     bool first = true;
98     while (!q.empty())
99     {
100         if (!first)
101             res += ",";
102         first = false;
103         res += to_string(q.front());
104         q.pop();
105     }
106     res += "}";
107     return res;
108 }
```

```

107 }
108
109 template <typename T>
110 string to_string(stack<T> s)
111 {
112     string res = "{";
113     bool first = true;
114     while (!s.empty())
115     {
116         if (!first)
117             res += ", ";
118         first = false;
119         res += to_string(s.top());
120         s.pop();
121     }
122     res += "}";
123     return res;
124 }
125
126 template <typename T, typename Container, typename Compare>
127 string to_string(std::priority_queue<T, Container, Compare> pq)
128 {
129     string res = "{";
130     bool first = true;
131     while (!pq.empty())
132     {
133         if (!first)
134             res += ", ";
135         first = false;
136         res += to_string(pq.top());
137         pq.pop();
138     }
139     res += "}";
140     return res;
141 }
142
143 void debug_out() { cerr << endl; }
144
145 template <typename Head, typename... Tail>
146 void debug_out(Head H, Tail... T)
147 {
148     cerr << " " << to_string(H);
149     debug_out(T...);
150 }
151
152 #ifdef LOCAL
153 #define debug(...) cerr << "[" << __VA_ARGS__ << "]:", debug_out(__VA_ARGS__)
154 #else
155 #define debug(...) 42
156 #endif

```

11.15 dist

```

1 double dist(double x1, double y1, double x2, double y2)
2 {
3     return hypot(x2 - x1, y2 - y1);
4 }

```

11.16 div

```

1 vector<int> div(int n)
2 {
3     vector<int> divs;
4     for (int i = 1; i * i <= n; i++)
5     {
6         if (n % i == 0)
7         {
8             divs.push_back(i);
9             if (i * i != n)
10                 divs.push_back(n / i);
11         }
12     }
13     sort(divs.begin(), divs.end());
14     return divs;
15 }

```

11.17 double compare

```

1 int dcomp(double a, double b)
2 {
3     return fabs(a - b) < EPS ? 0 : a > b ? 1
4                                     : -1;
5 }

```

11.18 edge

```

1 struct edge
2 {
3     ll from, to, w;
4
5     edge(ll from, ll to, ll w) : from(from), to(to), w(w)
6     {
7     }
8
9     bool operator<(const edge &e) const
10    {
11        return w < e.w;
12    }
13
14    bool operator==(const edge &e)
15    {
16        return from == e.from && to == e.to && w == e.w;
17    }

```

```
18 };
```

11.19 find last occurrence

```
1 find last occurrence for char to specific index
2 vector<vector<int>> nxt;
3 vector<int> last;
4 string s;
5 int n,k;
6 void pre() {
7     nxt.assign(n+5, vector<int>(k+5, 0));
8     last.assign(k+5, n);
9
10    for (int i = n-1; i >= 0; i--) {
11        for (int j = 0; j < k; j++) {
12            nxt[i][j] = last[j];
13        }
14        last[s[i] - 'a'] = i;
15    }
16
17 }
```

11.20 inv mod

```
1 ll power(ll base, ll pow)
2 {
3     base %= MOD;
4     ll res{1};
5     while (pow)
6     {
7         if (pow & 1)
8             res = res * base % MOD;
9         base = base * base % MOD;
10        pow >>= 1;
11    }
12    return res;
13 }
14
15 // MOD must be prime number
16 ll modInv(ll x)
17 {
18     return power(x, MOD - 2);
19 }
```

11.21 is prime

```
1 bool is_prime(int x)
2 {
3     if (x <= 1)
4         return false;
```

```
5     if (x % 2 == 0)
6         return x == 2;
7     for (int i = 3; i * i <= x; i += 2)
8     {
9         if (x % i == 0)
10            return false;
11    }
12    return true;
13 }
```

11.22 kadaness

```
1 int curr{a[0]};
2 int glob{a[0]};
3 for (int i = 1; i < n; i++)
4 {
5     curr = max(a[i], curr + a[i]);
6     glob = max(curr, glob);
7 }
8
9 //-----//
10
11 int mx{}, curr{};
12 pair<int, int> p{0, 0};
13 pair<int, int> best{};
14 for (int i = 0; i < n - 1; i++)
15 {
16     curr += a[i];
17     if (curr < 0)
18     {
19         curr = 0;
20         p.first = i + 1;
21     }
22     p.second = i;
23     if (curr > mx)
24     {
25         best = p;
26         mx = curr;
27     }
28     else if (mx == curr)
29     {
30         if (best.second - best.first < p.second - p.first)
31             best = p;
32     }
33 }
34
35 //-----//
36
37 int mx = -00;
38
39 for (int left = 0; left < n; left++)
40 {
41     vector<int> temp(n, 0);
```

```

42
43 for (int right = left; right < n; right++)
44 {
45     for (int i = 0; i < n; i++)
46     {
47         temp[i] += a[i][right];
48     }
49
50     int current_max = temp[0];
51     int current_sum = temp[0];
52
53     for (int i = 1; i < n; i++)
54     {
55         current_sum = max(temp[i], current_sum + temp[i]);
56         current_max = max(current_max, current_sum);
57     }
58
59     mx = max(mx, current_max);
60 }
61 }

```

11.23 nCr

```

1 int nCr(int n, int r)
2 {
3     if (r < 0 || n < 0 || r > n)
4         return 0;
5     int ret{1};
6     for (int i = 1; i <= min(r, n - r); i++)
7     {
8         ret *= n - i + 1;
9         ret /= i;
10    }
11    return ret;
12 }
13
14 int nPr(int n, int r)
15 {
16     if (r < 0 || n < 0 || r > n)
17         return 0;
18     int ret{1};
19     for (int i = 0; i < r; i++)
20         ret = mul(ret, n - i);
21     return ret;
22 }

```

11.24 next greater

```

1 vector<int> nextGreater(n, n);
2 for (int i = 0; i < n; i++)
3 {

```

```

4     while (!s.empty() && v[i] > v[s.top()])
5     {
6         nextGreater[s.top()] = i;
7         s.pop();
8     }
9     s.push(i);
10 }

```

11.25 permutation hashing

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define ll long long
4 const int N = 2e5 + 5;
5 std::mt19937_64 rng(std::chrono::system_clock::now().time_since_epoch().count());
6 ll rand(ll l, ll r)
7 {
8     return uniform_int_distribution<ll>(l, r)(rng);
9 }
10
11 struct node
12 {
13     ll a, b, c, d, e;
14
15     node() : a(0), b(0), c(0), d(0), e(0)
16     {
17     }
18
19     node(ll x, ll xx, ll xxx, ll xxxx, ll xxxxx) : a(x), b(xx), c(xxx), d(xxxx),
20                                                    e(xxxxx)
21     {
22     }
23
24     node operator+(const node &x)
25     {
26         node w;
27         w.a = a + x.a, w.b = b + x.b, w.c = c + x.c, w.d = d + x.d, w.e = e + x.e;
28         ;
29         return w;
30     }
31
32     bool operator==(const node &x)
33     {
34         return a == x.a && b == x.b && c == x.c && d == x.d && e == x.e;
35     }
36
37     void operator+=(const node &x)
38     {
39         a += x.a, b += x.b, c += x.c, d += x.d, e += x.e;
40     }
41
42     node operator=(const node &x)
43     {

```

```

42     a = x.a, b = x.b, c = x.c, d = x.d, e = x.e;
43     return node(a, b, c, d, e);
44 }
45 };
46
47 node f[N], per[N];
48
49 void init()
50 {
51     for (int i = 1; i < N; i++)
52         f[i] = per[i] = node(rand(1, 1e9), rand(1, 1e9), rand(1, 1e9), rand(1, 1e9),
53                               rand(1, 1e9));
54     for (int i = 2; i < N; i++)
55         per[i] += per[i - 1];
56 }
57
58 signed main()
59 {
60     ios_base::sync_with_stdio(0);
61     cin.tie(0);
62     cout.tie(0);
63     init();
64     int t;
65     cin >> t;
66     while (t--)
67     {
68         int n;
69         cin >> n;
70         ll s[n + 1] = {};
71         for (int i = 1; i <= n; i++)
72             cin >> s[i];
73         node pre[n + 5], suf[n + 5];
74         for (int i = 1; i <= n; i++)
75             pre[i] = suf[i] = f[s[i]];
76         for (int i = 2; i <= n; i++)
77             pre[i] += pre[i - 1];
78         for (int i = n - 1; i >= 1; i--)
79             suf[i] += suf[i + 1];
80     }
81     return 0;
82 }

```

11.26 random

```

1 int getRandom(int min, int max)
2 {
3     static std::mt19937 gen(chrono::steady_clock::now().time_since_epoch().count
4                               ());
5     std::uniform_int_distribution<int> distrib(min, max);
6     return distrib(gen);
7 }

```

11.27 valid

```

1 bool valid(int i, int j)
2 {
3     if (i >= r || i < 0 || j >= c || j < 0)
4         return false;
5     return true;
6 }

```